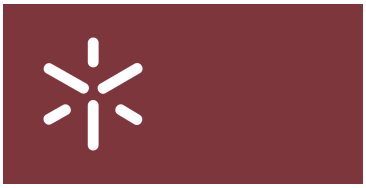


Universidade do Minho
Escola de Engenharia
Licenciatura em Engenharia Informática

Unidade Curricular de Laboratórios de Informática IV

Ano Letivo de 2025/2026

Sistema de Gestão da Taki - Grupo 17

Diogo Campos	**João Delgado**	**Nelson Mendes**	**Simão Mendes**	**Tomás Machado**
A106920	A106836	A106884	A106928	A104186

Maio, 2026

LI4

Data da Receção	
Responsável	
Avaliação	
Observações	

Sistema de Gestão da Taki - Grupo 17

Diogo Campos	**João Delgado**	**Nelson Mendes**	**Simão Mendes**	**Tomás Machado**
A106920	A106836	A106884	A106928	A104186

Maio, 2026

Resumo

O projeto desenvolvido consistiu na conceção e implementação do Taki, um sistema distribuído de gestão de retalho desenhado para operar num modelo híbrido de nós centrais (central) e locais (local). A motivação principal residiu na criação de uma plataforma capaz de garantir a autonomia operacional das lojas individuais, assegurando simultaneamente a consolidação administrativa e estatística centralizada de toda a cadeia.

O trabalho realizado envolveu a implementação de uma arquitetura modular em camadas, implantada em *containers* Docker, utilizando a *framework* Javalin (Java) para a exposição de APIs REST robustas. A API Local foi desenvolvida para gerir fluxos transacionais críticos, como o registo de vendas, movimentos de inventário e gestão de fornecedores. Paralelamente, a API Global foi implementada para permitir a monitorização de KPIs, gestão de catálogo unificado e exportação de dados fiscais (SAF-T). A segurança foi um pilar central, concretizada através de autenticação via JWT com assinatura RSA256 e um sistema rigoroso de controlo de acessos baseado em perfis.

No seu estado final, o sistema encontra-se totalmente funcional e validado. A infraestrutura demonstra elevada resiliência, comprovada por testes de desempenho que registaram uma taxa de sucesso de 100% sob cenários de stress intensivo (até 500 utilizadores simultâneos). Embora o sistema atinja o seu ponto de saturação ao nível da latência em cargas extremas, mantém a integridade dos dados e a estabilidade operacional. A Taki apresenta-se, assim, como uma solução escalável, segura e pronta para responder às exigências de conformidade e eficiência do setor do retalho moderno.

Área de Aplicação: Engenharia de *Software* e Arquitetura de Sistemas Distribuídos.

Esta área abrange o desenho de arquiteturas de *software* modulares, a implementação de sistemas de persistência relacionais (PostgreSQL), o desenvolvimento de APIs escaláveis e a integração de mecanismos de segurança criptográfica para a proteção de dados transacionais.

Palavras-Chave: Sistemas Distribuídos, Engenharia de *Software*, API REST, Javalin, Docker, JWT RSA256, PostgreSQL, Gestão de Retalho, SAF-T (PT), Testes de Performance.

Índice

1. Introdução 1
1.1. Contextualização 1
1.2. Apresentação do Caso de Estudo 1
1.3. Identidade do Projeto 2
1.4. Fundamentação 3
1.5. Motivação e Objetivos 3
1.6. Viabilidade 4
1.7. Recursos e Equipa de Trabalho 4
1.8. Plano de Execução 6
1.9. Revisão e Aprovação do Projeto 7
1.10. Estrutura do Relatório 8
1.11. Utilização de Inteligência Artificial 8

2. Conceção e Engenharia de Requisitos Assistida por LLM 11
2.1. Levantamento de Requisitos 11
2.1.1. Método de Levantamento e de Análise de Requisitos Adotado 11
2.1.2. Elicitação e Organização dos Requisitos 13
2.1.3. Validação dos Requisitos 32
2.2. Análise de Requisitos 33
2.2.1. Modelo de Domínio 33
2.2.2. *User Stories* 34
2.2.3. Casos de Uso 37
2.3. Utilização de Inteligência Artificial 97
2.3.1. Estratégia de Interação e Diretrizes 97
2.3.2. Interações e *Prompts* Chave na Engenharia de Requisitos 97
2.3.3. Impacto Operacional e Validação 98

3. Arquitetura e *Design* do *Software* utilizando LLM 99
3.1. *Data Layer* 99
3.1.1. Modelo Concetual do *Data Layer* 99
3.1.2. Decisões Arquiteturais e de Implementação 108
3.1.3. Modelo Lógico 110
3.1.4. Normalização da BD 113
3.2. Lógica de Negócio 114
3.2.1. Diagrama de Componentes 114
3.2.2. *Data Access Objects* (DAO) 114
3.2.3. Gestão da Sincronização 116
3.2.4. Gestão de Catálogo (Lojas) 126
3.2.5. Gestão de Inventário 136
3.2.6. Gestão de Fornecimentos e Encomendas 139
3.2.7. Ponto de Venda e Transações 149
3.2.8. Estatísticas e Relatórios 153
3.2.9. Sistema Local 155
3.2.10. Sistema Central 157
3.2.11. Arquitetura da API 158
3.3. *User-Interface* 161

3.4. Utilização de Inteligência Artificial . . . 169
3.4.1. Diretrizes de Sistema e a Ativação das Regras de Andrej Karpathy . . . 169
3.4.2. Interações e *Prompts* Chave na Arquitetura . . . 169
3.4.3. Prompts da LN . . . 171

4. Implementação e Desenvolvimento Assistido por LLM . . . 172
4.1. Configuração do Ambiente de Trabalho . . . 172
4.1.1. Orquestração com Docker Compose . . . 172
4.1.2. Isolamento e Replicabilidade do Ambiente . . . 172
4.1.3. Configuração da Aplicação por Variáveis de Ambiente . . . 173
4.1.4. Ferramentas e Controlo de Versões . . . 173
4.2. *Data Layer* . . . 173
4.2.1. Criação do Esquema Físico . . . 173
4.2.2. Gestão de Acessos e Utilizadores . . . 174
4.2.3. Povoamento da Base de Dados . . . 175
4.2.4. Cálculo do Espaço e Crescimento . . . 176
4.2.5. Lógica Ativa: Vistas, Funções e *Triggers* . . . 178
4.2.6. Indexação e Otimização . . . 180
4.2.7. Mecanismo de Sincronização e Consolidação . . . 181
4.3. Lógica de Negócio . . . 182
4.3.1. Camada de Acesso a Dados (DAO) . . . 182
4.3.2. Consumo da Lógica Ativa e Sincronização . . . 183
4.3.3. Implementação do Catálogo (Lojas) . . . 184
4.3.4. Implementação de Inventário . . . 185
4.3.5. Implementação de Fornecimentos e Encomendas . . . 187
4.3.6. Implementação do Ponto de Venda e Transações . . . 188
4.3.7. Implementação das Estatísticas e Relatórios . . . 189
4.3.8. Implementação do Sistema Local . . . 189
4.3.9. Implementação do Sistema Global . . . 190
4.3.10. Implementação da API . . . 191
4.4. *User-Interface* . . . 192
4.5. Utilização de Inteligência Artificial . . . 192
4.5.1. Transição Tecnológica: De Gemini e Copilot para Claude Code . . . 192
4.5.2. Metodologia de Implementação Assistida . . . 193

5. Verificação, Validação e Avaliação da Qualidade do *Software* Produzido . . . 194
5.1. Verificação . . . 194
5.1.1. Testes unitários (LN) . . . 194
5.1.2. Testes aos DAOs . . . 195
5.1.3. Testes E2E da UI . . . 195
5.1.4. Análise estática . . . 196
5.2. Validação . . . 198
5.2.1. Álgebra Relacional (DL) . . . 198
5.2.2. Testes unitários (cenários reais do cliente) . . . 199
5.2.3. Heurísticas de Nielsen . . . 200
5.2.4. Reunião de Aceitação . . . 201
5.3. Avaliação da Qualidade . . . 202
5.3.1. Testes de carga (eficiência) . . . 202
5.3.2. Testes de stress (fiabilidade) . . . 203
5.3.3. Testes de segurança . . . 203
5.3.4. Métricas de Qualidade (ISO/IEC 25010) . . . 204
5.3.5. Síntese dos Resultados da Verificação e Validação . . . 206
5.4. Utilização de Inteligência Artificial . . . 207
5.4.1. O Impacto de Modelos Avançados na Validação . . . 207
5.4.2. Metodologia de Validação e Prompts Chave . . . 207

6. Conclusões e Trabalho Futuro 209
6.1. Pontos Fortes 209
6.2. Pontos A Melhorar 209
6.3. Expansão de Funcionalidades 210

7. Referências Bibliográficas 211

8. Lista de Siglas e Acrónimos 213

9. Anexos 215

Anexo 1 Ata de Reunião de Aprovação do Projeto (*Kick-off*) 215
Anexo 2 Ata de Reunião de Levantamento de Requisitos 216
Anexo 3 Ata de Reunião de Validação dos Requisitos 217
Anexo 4 Inquérito de Avaliação Operacional e Levantamento de Requisitos 218
Anexo 5 Resultados do Inquérito de Avaliação Operacional e Levantamento de Requisitos 219
Anexo 6 Relatório de Pesquisa Externa 220
Anexo 7 Diretrizes de Sistema para o *Gemini* (`GEMINI.md`) 225
Anexo 8 Instruções de Contexto para o *Copilot* (`copilot-instructions.md`) 228
Anexo 9 Diretrizes de Sistema para o *Claude* (`CLAUDE.md`) 231
Anexo 10 Ata de Reunião de Aceitação 233
Anexo 11 Manual instalação 234

Lista de Figuras

Figura 1 Logótipo do SegFault. 2
Figura 2 Logótipo da Taki. 2
Figura 3 Diagrama de `Gantt` Previsto. 6
Figura 4 Diagrama de `Gantt` Executado. 7
Figura 5 Resumo da Ata de Reunião de Aprovação do Projeto. A versão integral e assinada do documento encontra-se documentada no **Anexo I**. 8
Figura 6 Resumo da Ata de Reunião de Recolha de Requisitos. A versão integral e assinada do documento encontra-se documentada no **Anexo II**. 12
Figura 7 Resumo da Ata de Reunião de Validação de Requisitos. A versão integral e assinada do documento encontra-se documentada no **Anexo III**. 32
Figura 8 Modelo de Domínio 34
Figura 9 Diagrama de Atividades do Caso de Uso: Registar Venda 39
Figura 10 Diagrama de Atividades do Caso de Uso: Processar Devolução 41
Figura 11 Diagrama de Atividades do Caso de Uso: Registar Entrada de Mercadoria 43
Figura 12 Diagrama de Atividades do Caso de Uso: Realizar Ajuste de Inventário 45
Figura 13 Diagrama de Atividades do Caso de Uso: Consultar Stock 47
Figura 14 Diagrama de Atividades do Caso de Uso: Criar Encomenda a Fornecedor 49
Figura 15 Diagrama de Atividades do Caso de Uso: Configurar Alerta de Stock 51
Figura 16 Diagrama de Atividades do Caso de Uso: Consultar Alertas de Stock 52
Figura 17 Diagrama de Atividades do Caso de Uso: Adicionar Produto 54
Figura 18 Diagrama de Atividades do Caso de Uso: Editar Produto 56
Figura 19 Diagrama de Atividades do Caso de Uso: Descontinuar Produto 58
Figura 20 Diagrama de Atividades do Caso de Uso: Consultar Produto 60
Figura 21 Diagrama de Atividades do Caso de Uso: Criar Categoria 62
Figura 22 Diagrama de Atividades do Caso de Uso: Editar Categoria 64
Figura 23 Diagrama de Atividades do Caso de Uso: Descontinuar Categoria 66
Figura 24 Diagrama de Atividades do Caso de Uso: Registar Fornecedor 68
Figura 25 Diagrama de Atividades do Caso de Uso: Editar Dados do Fornecedor 70
Figura 26 Diagrama de Atividades do Caso de Uso: Remover Fornecedor 72
Figura 27 Diagrama de Atividades do Caso de Uso: Associar Produto a Fornecedor 74
Figura 28 Diagrama de Atividades do Caso de Uso: Criar política de promoção 76
Figura 29 Diagrama de Atividades do Caso de Uso: Cancelar política de promoção 77
Figura 30 Diagrama de Atividades do Caso de Uso: Gerar Relatório de Vendas 79
Figura 31 Diagrama de Atividades do Caso de Uso: Gerar Relatório de Inventário 80
Figura 32 Diagrama de Atividades do Caso de Uso: Efetuar Autenticação 82
Figura 33 Diagrama de Atividades do Caso de Uso: Registar conta de utilizador 84
Figura 34 Diagrama de Atividades do Caso de Uso: Editar conta de utilizador 86
Figura 35 Diagrama de Atividades do Caso de Uso: Bloquear conta de utilizador 88
Figura 36 Diagrama de Atividades do Caso de Uso: Criar perfil de acesso 90
Figura 37 Diagrama de Atividades do Caso de Uso: Editar perfil de acesso 92
Figura 38 Diagrama de Atividades do Caso de Uso: Atribuir perfil a utilizador 94
Figura 39 Diagrama Casos de Uso 1 95
Figura 40 Diagrama Casos de Uso 2 96
Figura 41 Diagrama Casos de Uso 3 97
Figura 42 Modelo Conceptual de Dados (Diagrama ER) do Sistema 107

Figura 43 Modelo Lógico de Base de Dados (PostgreSQL) 112
Figura 44 Diagrama de Componentes — Macroarquitetura do Sistema Taki 114
Figura 45 Diagrama UML ilustrando a Generalização dos DAOs 116
Figura 46 Diagrama de Classes: Subdomínio de Sincronização 118
Figura 47 Diagrama de Sequência: Captura de Evento (*Outbox Trigger*) 119
Figura 48 Diagrama de Sequência: Iniciar Agendamento Diário 119
Figura 49 Diagrama de Sequência: Execução Agendada Diária 120
Figura 50 Diagrama de Sequência: Sincronização Manual de Dados 120
Figura 51 Diagrama de Sequência: Exportar Lote para a Sede 121
Figura 52 Diagrama de Sequência: Enviar Exportação para a Sede 122
Figura 53 Diagrama de Sequência: Processar Exportação na Central 122
Figura 54 Diagrama de Sequência: Configuração de *Endpoints* Seguros 123
Figura 55 Diagrama de Sequência: Importar Atualizações Globais 124
Figura 56 Diagrama de Sequência: Processar Importação na Central 124
Figura 57 Diagrama de Sequência: Verificar Disponibilidade Central 125
Figura 58 Diagrama de Sequência: Consultar Estado de Sincronização 125
Figura 59 Diagrama de Classes: Subdomínio de Catálogo 127
Figura 60 Diagrama de Sequência: Adicionar Produto 128
Figura 61 Diagrama de Sequência: Editar Produto 129
Figura 62 Diagrama de Sequência: Inativar Produto 130
Figura 63 Diagrama de Sequência: Pesquisar Produto por Código de Barras 130
Figura 64 Diagrama de Sequência: Listar Produtos 131
Figura 65 Diagrama de Sequência: Associar Categoria a Produto 131
Figura 66 Diagrama de Sequência: Remover Categoria de Produto 132
Figura 67 Diagrama de Sequência: Listar Produtos por Categoria 132
Figura 68 Diagrama de Sequência: Adicionar Categoria 133
Figura 69 Diagrama de Sequência: Editar Categoria 134
Figura 70 Diagrama de Sequência: Inativar Categoria 135
Figura 71 Diagrama de Sequência: Obter Caminho Hierárquico da Categoria 135
Figura 72 Diagrama de Classes: Subdomínio de Inventário 137
Figura 73 Diagrama de Sequência: Gestão Autónoma de Alertas de *Stock* 138
Figura 74 Diagrama de Sequência: Ajuste Manual de Inventário 138
Figura 75 Diagrama de Sequência:Listar Inventário 139
Figura 76 Diagrama de Sequência:Consultar Alertas 139
Figura 77 Diagrama de Máquina de Estados: Ciclo de Vida da Encomenda 140
Figura 78 Diagrama de Classes: Subdomínio de Fornecimentos 141
Figura 79 Diagrama de Sequência: Registar Fornecedor 142
Figura 80 Diagrama de Sequência: Editar Dados do Fornecedor 143
Figura 81 Diagrama de Sequência: Inativar Fornecedor 144
Figura 82 Diagrama de Sequência: Listar Fornecedores 144
Figura 83 Diagrama de Sequência: Criar Encomenda a Fornecedor 145
Figura 84 Diagrama de Sequência: Enviar Encomenda 146
Figura 85 Diagrama de Sequência: Concluir Encomenda 146
Figura 86 Diagrama de Sequência: Cancelar Encomenda 147
Figura 87 Diagrama de Sequência: Calcular Total da Guia 147
Figura 88 Diagrama de Sequência: Associar Produto a Fornecedor 148
Figura 89 Diagrama de Sequência: Remover Associação Produto-Fornecedor 149
Figura 90 Diagrama de Sequência: Consultar Produtos de um Fornecedor 149
Figura 91 Diagrama de Classes: Subdomínio de Vendas (POS) 150
Figura 92 Diagrama de Sequência: Registar Venda 151
Figura 93 Diagrama de Sequência: Emissão da Fatura (referido no diagrama anterior) 151
Figura 94 Diagrama de Sequência: Processar Devolução 152
Figura 95 Diagrama de Sequência: Criar Política de Promoção 152
Figura 96 Diagrama de Sequência: Cancelar Política de Promoção 153
Figura 97 Diagrama de Classes: Subdomínio de Estatísticas 154

Figura 98 Diagrama de Sequência: Gerar Relatório de Vendas 154
Figura 99 Diagrama de Sequência: Gerar Relatório de Inventário 155
Figura 100 Diagrama de Classes: Fachada Local e Subsistemas 156
Figura 101 Diagrama de Sequência: Efetuar Autenticação 157
Figura 102 Diagrama de Classes: Fachada Principal da Lógica de Negócio 158
Figura 103 Login à plataforma 162
Figura 104 Opções de lojas da empresa 162
Figura 105 Aba da Caixa de Vendas (POS) 163
Figura 106 DashBoard da plataforma 164
Figura 107 Aba da Gestão do Inventário da empresa 165
Figura 108 Aba da Gestão de Encomendas da Empresa 165
Figura 109 Aba da gestão de Faturas da Empresa 166
Figura 110 Aba da Gestão dos Fornecedores da empresa 167
Figura 111 Aba da Geração de Relatórios da empresa 167
Figura 112 Aba da Estatísticas da empresa 168
Figura 113 Aba da Gestão de Funcionários da empresa 168
Figura 114 Resumo da Ata de Reunião de Aceitação. A versão integral do documento encontra-se documentada no **Anexo 10**. 202

Lista de Tabelas

Tabela 1 Requisito funcional 1. 14
Tabela 2 Requisito funcional 2. 14
Tabela 3 Requisito funcional 3. 15
Tabela 4 Requisito funcional 4. 16
Tabela 5 Requisito funcional 5. 16
Tabela 6 Requisito funcional 6. 17
Tabela 7 Requisito funcional 7. 18
Tabela 8 Requisito funcional 8. 19
Tabela 9 Requisito funcional 9. 20
Tabela 10 Requisito funcional 10. 21
Tabela 11 Requisito funcional 11. 22
Tabela 12 Requisito funcional 12. 23
Tabela 13 Requisito funcional 13. 24
Tabela 14 Requisito funcional 14. 25
Tabela 15 Requisito funcional 15. 25
Tabela 16 Requisito funcional 16. 26
Tabela 17 Requisito funcional 17. 26
Tabela 18 Requisito funcional 18. 27
Tabela 19 Requisito funcional 19. 27
Tabela 20 Requisito funcional 20. 28
Tabela 21 Requisito não funcional 1. 29
Tabela 22 Requisito não funcional 2. 29
Tabela 23 Requisito não funcional 3. 30
Tabela 24 Requisito não funcional 4. 30
Tabela 25 Requisito não funcional 5. 30
Tabela 26 Requisito não funcional 6. 31
Tabela 27 Requisito não funcional 7. 31
Tabela 28 Requisito não funcional 8. 32
Tabela 29 Caso de Uso: Registar Venda 37
Tabela 30 Caso de Uso: Processar Devolução 40
Tabela 31 Caso de Uso: Registar Entrada de Mercadoria 42
Tabela 32 Caso de Uso: Realizar Ajuste de Inventário 44
Tabela 33 Caso de Uso: Consultar Stock 46
Tabela 34 Caso de Uso: Criar Encomenda a Fornecedor 48
Tabela 35 Caso de Uso: Configurar Alerta de Stock 50
Tabela 36 Caso de Uso: Consultar Alertas de Stock 52
Tabela 37 Caso de Uso: Adicionar Produto 53
Tabela 38 Caso de Uso: Editar Produto 55
Tabela 39 Caso de Uso: Descontinuar Produto 57
Tabela 40 Caso de Uso: Consultar Produto 59
Tabela 41 Caso de Uso: Criar Categoria 61
Tabela 42 Caso de Uso: Editar Categoria 63
Tabela 43 Caso de Uso: Descontinuar Categoria 65
Tabela 44 Caso de Uso: Registar Fornecedor 67
Tabela 45 Caso de Uso: Editar Dados do Fornecedor 69

Tabela 46 Caso de Uso: Remover Fornecedor 71
Tabela 47 Caso de Uso: Associar Produto a Fornecedor 73
Tabela 48 Caso de Uso: Criar política de promoção 75
Tabela 49 Caso de Uso: Cancelar política de promoção 76
Tabela 50 Caso de Uso: Gerar Relatório de Vendas 78
Tabela 51 Caso de Uso: Gerar Relatório de Inventário 80
Tabela 52 Caso de Uso: Efetuar Autenticação 81
Tabela 53 Caso de Uso: Registar conta de utilizador 83
Tabela 54 Caso de Uso: Editar conta de utilizador 85
Tabela 55 Caso de Uso: Bloquear conta de utilizador 87
Tabela 56 Caso de Uso: Criar perfil de acesso 89
Tabela 57 Caso de Uso: Editar perfil de acesso 91
Tabela 58 Caso de Uso: Atribuir perfil a utilizador 93
Tabela 59 Identificação e Caracterização das Entidades 100
Tabela 60 Identificação e Caracterização dos Relacionamentos 102
Tabela 61 Atributos - Loja 103
Tabela 62 Atributos - Funcionario 103
Tabela 63 Atributos - Categoria 103
Tabela 64 Atributos - Produto 103
Tabela 65 Atributos - Promocao 104
Tabela 66 Atributos - Fornecedor 104
Tabela 67 Atributos - Encomenda 104
Tabela 68 Atributos - Linha_Encomenda 105
Tabela 69 Atributos - Inventario 105
Tabela 70 Atributos - Movimento_Inventario 105
Tabela 71 Atributos - Alerta_Stock 105
Tabela 72 Atributos - Venda 105
Tabela 73 Atributos - Linha_Venda 106
Tabela 74 Atributos - Pagamento 106
Tabela 75 Atributos - Fatura 106
Tabela 76 Atributos - Devolucao 106
Tabela 77 Atributos - Linha_Devolucao 107
Tabela 78 Mapeamento de Domínios para PostgreSQL 110
Tabela 79 Principais Endpoints da API Local 159
Tabela 80 Principais Endpoints da API Global 160
Tabela 81 Dimensão inicial mínima da estrutura completa da base de dados. 176
Tabela 82 Crescimento anual estimado para a Base de Dados Central, incluindo todas as entidades. . 177

1. Introdução

Este capítulo introdutório estabelece as bases fundamentais para a compreensão do trabalho realizado. Neste sentido, é apresentado o enquadramento do projeto, incluindo a contextualização do domínio de aplicação e a **descrição do caso de estudo que serviu de base** ao desenvolvimento. Segue-se a fundamentação teórica que suporta as opções metodológicas adotadas, bem como a motivação e os **objetivos que orientaram a equipa** ao longo do projeto. São ainda analisados a viabilidade da solução proposta, os recursos envolvidos e a constituição da equipa de trabalho. Por fim, é apresentado o **plano de execução, que traça o cronograma previsto para as atividades**, e a presente descrição da estrutura do relatório, que funciona como guia de leitura para os restantes capítulos.

1.1. Contextualização

A **Taki** é uma cadeia de pequenas lojas de conveniência fundada e gerida pelo Sr. Manuel Pinto da Costa. Pautada por uma filosofia de negócio pragmática, eficiente e sem complicações operacionais — uma cultura perfeitamente encapsulada no seu célebre *slogan* "Não inventes, Taki!" —, a empresa tem-se consolidado no mercado através da oferta de produtos de primeira necessidade e de um atendimento célere e direto ao cliente.

Recentemente, a marca iniciou um ambicioso plano de expansão geográfica. Atualmente, a Taki conta com espaços comerciais em Felgueiras, na ilha da Madeira e em Cabeceiras de Basto, mantendo o seu servidor central e a coordenação estratégica na sede corporativa em Famalicão. No entanto, este crescimento estrutural trouxe novos desafios à gestão do negócio. A dispersão das filiais e o consequente aumento do volume transacional evidenciaram as limitações dos processos manuais e tradicionais de controlo. Dificuldades na gestão e reposição de *stock*, no acompanhamento diário das vendas e na coordenação com a rede de fornecedores começaram a colocar em risco a eficiência operacional que caracteriza a empresa.

Por forma a mitigar estas anomalias logísticas e garantir uma transição sustentada para a nova escala do negócio, a Taki reconheceu a urgência de uma reestruturação tecnológica. Com esse intuito, recorreu aos serviços da **SegFault**, uma *startup* de desenvolvimento de *software*. A equipa da SegFault foi contratada com a missão de conceber um sistema informático robusto e integrado, capaz de descentralizar a operação diária das lojas e, simultaneamente, assegurar a consolidação fiável da informação crítica em Famalicão no final de cada dia. Este sistema visa preservar a identidade prática da Taki, modernizando a sua infraestrutura para suportar uma tomada de decisão rápida e sustentada em dados concretos.

1.2. Apresentação do Caso de Estudo

O presente caso de estudo foca-se na conceção e desenvolvimento de um sistema de gestão integrada, criado especificamente para apoiar a administração de uma cadeia de pequenas lojas de conveniência. A motivação central deste projeto é resolver um desafio comum no retalho: a dificuldade em sincronizar a gestão central com as operações de lojas que funcionam de forma autónoma. O objetivo primordial é, portanto, otimizar tanto os processos operacionais do dia-a-dia em cada loja, como a tomada de decisão estratégica ao nível da cadeia. Para tal, o sistema irá consolidar diariamente a informação essencial de cada estabelecimento, incluindo dados de vendas, estado dos stocks e registos de fornecedores, num repositório centralizado. Este processo respeita a autonomia de cada loja, ao mesmo tempo que fornece à

gestão uma visão global e atualizada. As funcionalidades chave incluirão um controlo de inventário rigoroso para minimizar perdas e garantir a disponibilidade de produtos, a automatização da reposição de stocks, e um módulo de faturação. Adicionalmente, o sistema será capaz de gerar relatórios de gestão detalhados e flexíveis, segmentados por loja, período ou categoria de produto, de forma a fornecer o suporte analítico necessário para decisões operacionais e estratégicas eficazes.

1.3. Identidade do Projeto

O projeto para o desenvolvimento do Sistema de Gestão Integrada para a cadeia de lojas de conveniência 'Taki' representa uma colaboração entre a entidade requisitante, Taki, Lda., e a equipa de desenvolvimento, SegFault. A ficha de identidade que se segue formaliza os detalhes administrativos e o âmbito central desta iniciativa.

Figura 1: Logótipo do SegFault.

Figura 2: Logótipo da Taki.

Ficha de Identidade do Projeto

Nome do Projeto:	Sistema de Gestão Integrada 'Taki'
Acrónimo:	SGI-Taki
Referência do Contrato:	2026-LI4-TAKISF-01
Cliente:	Taki, Lda.
Equipa de Desenvolvimento:	SegFault
Versão do Documento:	1.0
Data de Aprovação:	02/02/2026
Data de Início:	05/02/2026
Data de Conclusão:	19/06/2026
Gestor do Projeto:	Simão Mendes

Sumário do Projeto:

Centralizar a informação da cadeia de lojas para otimizar a gestão de *stocks*, melhorar a eficiência operacional e fornecer dados analíticos que suportem a tomada de decisão estratégica.

1.4. Fundamentação

Tendo em conta as crescentes exigências operacionais da Taki e a necessidade de suportar o seu contínuo plano de expansão , a adoção desta solução tecnológica e da sua respetiva arquitetura foram fundamentadas pelo sócio-gerente, Sr. Manuel Pinto da Costa, com base nas seguintes premissas:

- **Autonomia operacional através de bases de dados locais:** A opção por uma arquitetura assente em bases de dados locais nas várias filiais garante a resiliência do sistema. Desta forma, assegura-se que contingências técnicas, como eventuais falhas de conectividade à rede, não bloqueiam a atividade comercial das lojas, protegendo a faturação diária e a confiança dos consumidores num serviço sem interrupções.
- **Interface minimalista e de resposta imediata:** A conceção de uma interface de utilização propositadamente minimalista e desenhada para uma resposta imediata reflete a filosofia da marca. Esta abordagem reduzirá drasticamente a curva de aprendizagem dos colaboradores, minimizando o tempo de formação de novas equipas e garantindo que o foco dos funcionários se mantém exclusivamente na rapidez de transação em caixa e no atendimento ao cliente.
- **Automatização do controlo de stock:** A transição dos atuais registos em folhas de cálculo e documentos de texto para um mecanismo automatizado e rigoroso de controlo de stock retira a carga de verificação constante aos funcionários. O sistema passará a permitir um planeamento atempado das reposições junto dos fornecedores , evitando o escoamento total dos produtos nas prateleiras e a consequente perda de oportunidades de venda.
- **Inteligência de negócio e consolidação central:** A consolidação diária da informação no servidor central sediado em Famalicão converte os dados operacionais brutos em inteligência de negócio. Em vez de analisar ficheiros fragmentados, a gerência passa a dispor de uma visão centralizada sobre o estado de toda a rede. Esta centralização revela-se um fator crítico para sustentar o crescimento orgânico da marca, pois permite suportar analiticamente a tomada de decisão , ajudando a identificar com precisão a rentabilidade de cada nova localização geográfica.

1.5. Motivação e Objetivos

O desenvolvimento deste sistema surge da necessidade de otimizar a gestão diária da cadeia de pequenas lojas de conveniência **Taki**. Com a expansão do negócio liderado pelo **`Sr. Manuel Pinto da Costa`** para diversas geografias, torna-se essencial garantir que cada espaço comercial funcione de forma perfeitamente autónoma durante o seu horário de expediente.

Para manter a simplicidade processual que caracteriza a marca e de forma a honrar o lema **`"Não inventes, Taki!"`**, a solução tecnológica adotará uma arquitetura robusta onde, ao final de cada dia, toda a informação local é consolidada num **sistema central de dados alojado em Famalicão**. Esta abordagem centralizada garantirá um atendimento mais ágil e focado no cliente durante o dia, enquanto a análise integrada dos dados sincronizados facilitará a identificação de padrões de consumo. Consequentemente, a direção terá as ferramentas necessárias para realizar **ajustes estratégicos rápidos**, aprimorando a competitividade da empresa no mercado.

Os principais objetivos da startup **`Seg Fault`** com a conceção e implementação deste *software* para a empresa **Taki** são:

- **Garantir Autonomia Operacional e Sincronização**: Assegurar que cada loja opera de forma independente ao longo do dia, implementando processos que garantam a consolidação fiável da informação no servidor central (Famalicão) ao final de cada dia;
- **Otimizar o Controlo de Inventário**: Implementar serviços dinâmicos para a gestão rigorosa de *stocks*, assegurando a monitorização centralizada dos produtos e facilitando os processos de reposição atempada em cada uma das lojas da cadeia;

- **Centralizar a Gestão da Faturação e Fornecedores**: Criar estruturas adequadas e seguras para o registo e processamento unificado dos dados relativos a vendas, catálogo de produtos e rede de fornecedores;
- **Apoiar a Tomada de Decisão Estratégica**: Facilitar a geração de relatórios de gestão detalhados segmentados por loja, por período de tempo e por categoria de produto de forma a suportar analiticamente as decisões operacionais e financeiras da gerência;
- **Assegurar a Escalabilidade do Sistema:** Desenvolver uma arquitetura escalável e modular que suporte facilmente a expansão da marca para novas localidades no futuro.

1.6. Viabilidade

A cadeia de pequenas lojas de conveniência **Taki**, com funcionamento baseado em folhas de cálculo *Excel* e documentos *Word*, apresenta várias **falhas de registo** e **estrangulamentos na produtividade** diária. Estes problemas foram documentados pelos gerentes de loja através da sua experiência empírica e da análise aos lentos processos de reposição atuais. Deste modo, a empresa estima que, ao transitar para um **sistema informático integral e dedicado** (estruturado em *frontend*, *backend* e base de dados), conseguirá:

- **Reduzir em cerca de 30% o tempo** despendido pelos colaboradores em tarefas administrativas, como o fecho de caixa e a verificação manual de artigos, através do processamento automático e imediato de cada venda no novo sistema;
- **Diminuir as discrepâncias de *stock* em pelo menos 15%**, eliminando os erros humanos associados à introdução manual de faturas e à falta de sincronização com os fornecedores;
- **Aumentar o volume de vendas efetivas em torno de 5%**, garantindo a reposição atempada dos produtos nas prateleiras e evitando roturas de *stock* nos artigos de maior rotação, suportado pela nova geração de relatórios de gestão;
- **Reduzir em 20% os custos** associados a quebras de validade e desperdício de produtos perecíveis, graças a um alerta e controlo de inventário muito mais rigoroso;
- **Assegurar a continuidade do negócio com total autonomia operacional** das frentes de loja durante o dia, blindando a segurança dos dados de transações até à sua **consolidação noturna** com o servidor central alojado em Famalicão.

Assim, o Sr. Manuel Pinto da Costa concluiu que a implementação deste sistema de gestão integrada é **vital e economicamente viável** para sustentar a expansão da cadeia de lojas. Para garantir que o projeto não acarreta custos iniciais incomportáveis, decidiu que o recurso a uma equipa de **engenheiros de *software* em formação** (a equipa **`Seg Fault`**) seria a solução ideal para conceber e implementar toda esta nova infraestrutura tecnológica.

1.7. Recursos e Equipa de Trabalho

A gestão eficaz dos recursos é um pilar fundamental para o sucesso de qualquer projeto de software. Esta secção do documento tem como objetivo delinear de forma clara e detalhada todos os recursos humanos e tecnológicos que serão alocados ao desenvolvimento do Sistema de Gestão Integrada para a Taki (SGI-Taki).

Equipa Interna (Cliente - Taki, Lda.)

Os *stakeholders* do lado do cliente são os principais interessados no produto final e a fonte primária de conhecimento do negócio. A sua experiência e *feedback* contínuo são cruciais para garantir que o sistema desenvolvido responde eficazmente às necessidades da organização. A estrutura da equipa do cliente é a seguinte:

Nome	Posição na Empresa	Papel no Projeto

Manuel Pinto da Costa	Sócio Fundador e Maioritário (60%)	**Patrocinador do Projeto (Sponsor):** Responsável pela visão estratégica e aprovação final das fases chave.
Tiago Rocha	Sócio (10%) e Gestor da Loja de Famalicão	**Especialista em Logística:** Ponto de contacto para requisitos de stock, fornecedores e sincronização de dados.
Sofia Alves	Sócia (10%) e Gestora da Loja de Felgueiras	**Especialista Operacional:** Fonte de requisitos para as operações diárias da loja (vendas, caixa).
Carolina Jesus	Sócia (10%) e Gestora da Loja da Madeira	**Especialista em Análise de Negócio:** Ponto de contacto para requisitos de relatórios e análise de dados.
David Marques	Sócio (10%) e Gestor da Loja de Cabeceiras de Basto	**Especialista Financeiro:** Fonte de requisitos para os módulos de faturação e relatórios financeiros.
Vários (N/A)	Operadores de Loja	**Utilizadores Finais (End-Users):** Funcionários da linha da frente responsáveis pelo atendimento ao cliente. Fonte primária para a validação da usabilidade e eficiência do Ponto de Venda (POS).

Equipa Externa (SegFault)

A equipa externa, da empresa SegFault, é responsável por todo o ciclo de vida do desenvolvimento do software. A equipa é liderada pelo Gestor de Projeto, Simão Mendes, que atua como ponto de contacto principal com a Taki, Lda. e supervisiona todo o trabalho. A equipa técnica é composta pelos engenheiros de software Diogo Campos, João Delgado, Nelson Mendes, Simão Mendes e Tomás Machado. Este grupo trabalha de forma coesa e polivalente, cobrindo colaborativamente todas as áreas necessárias ao projeto, desde a análise de requisitos e desenho da arquitetura, até à implementação, teste e entrega final do sistema.

Recursos Tecnológicos e Ambiente de Desenvolvimento

A infraestrutura tecnológica selecionada para o projeto foi escolhida com base em critérios de maturidade, desempenho, segurança e adequação ao domínio aplicacional.

Plataforma de Desenvolvimento - `Java` 21 (LTS): O sistema será desenvolvido sobre a plataforma Java, selecionada devido à sua robustez, portabilidade e ao vasto ecossistema de ferramentas que potenciam a produtividade no desenvolvimento de aplicações empresariais.

Linguagem e Ferramenta de Modelação - `UML` e `Visual Paradigm`: A análise e o desenho do sistema serão formalizados através da *Unified Modeling Language* (UML) $2.5.1$. Para a criação e gestão dos artefactos de modelação (diagramas de casos de uso, de classes, etc.), a equipa utilizará a ferramenta *Visual Paradigm* 18.0. Esta ferramenta irá suportar a criação de modelos visuais consistentes para descrever a estrutura e o comportamento do sistema.

Sistema de Gestão de Base de Dados - `PostgreSQL`: Para o armazenamento persistente dos dados, foi selecionado o PostgreSQL, um sistema de código aberto reconhecido pela sua fiabilidade, extensibilidade e estrita conformidade com os padrões *SQL*.

Ferramentas de Suporte ao Desenvolvimento: O ciclo de vida do desenvolvimento será apoiado por ferramentas standard da indústria. O controlo de versões será gerido através do *Git*, e a documentação formal do projeto será produzida utilizando *Typst*.

Plataforma de *Deployment* (Produção): A infraestrutura de *deployment* da aplicação assenta numa arquitetura inteiramente baseada em *containers* **Docker**. O sistema é orquestrado através do `docker-compose`,

que levanta simultaneamente o *backend* Java e as múltiplas instâncias da base de dados PostgreSQL (uma central e várias locais para cada loja). Esta abordagem assegura paridade total entre o ambiente de desenvolvimento e produção, permitindo uma escalabilidade horizontal simplificada e a simulação rigorosa do ambiente distribuído da Taki de forma independente e isolada.d

1.8. Plano de Execução

O cronograma apresentado na **Figura 3 ilustra o planeamento temporal** das principais fases do projeto, estruturado sob a forma de diagrama de `Gantt`. O horizonte temporal considerado compreende o período de fevereiro a maio de 2026, totalizando quatro meses de execução.

As atividades encontram-se organizadas em **quatro grandes fases**, que refletem as etapas fundamentais do ciclo de desenvolvimento de *software*:

- **Conceção e Engenharia de Requisitos:** fase inicial dedicada à definição do sistema, levantamento e análise de requisitos junto dos *stakeholders*;
- **Arquitetura e *Design* do *Software*:** etapa correspondente à conceção da solução técnica, incluindo a modelação dos dados, a lógica de negócio e o *design* da interface;
- **Implementação e Desenvolvimento:** período destinado à codificação e materialização do sistema de acordo com as especificações definidas;
- **Verificação, Validação e Avaliação da Qualidade:** fase final dedicada aos testes, validação dos requisitos e garantia da qualidade do produto desenvolvido.

O seguinte diagrama permite observar a **sequência prevista para a execução das atividades**, a respetiva duração e o eventual paralelismo entre fases, constituindo um **instrumento fundamental para o acompanhamento e controlo** do progresso do projeto.

Figura 3: Diagrama de `Gantt` Previsto.

No entanto, nem sempre a **execução acompanha na íntegra o que foi planeado**, e a **Figura 4** que se segue ilustra com exatidão o desenrolar real do projeto ao longo do tempo, refletindo a sequência real das tarefas executadas. O projeto teve início com a **Definição de Sistema**, seguida da **Conceção e Engenharia de Requisitos**, que englobou o Levantamento e a Análise e Validação de Requisitos. Posteriormente, avançou-se para a **Arquitetura e *Design* do *Software***, fase que se estendeu até ao início de abril, dando lugar à **Implementação e Desenvolvimento**, a tarefa de maior duração de todo o projeto. Em paralelo e na fase final, decorreu a **Verificação, Validação e Avaliação da Qualidade**, culminando na **Conclusão do Relatório**, finalizada a 16 de maio de 2026. Este diagrama representa fielmente o percurso real do projeto, evidenciando a ordem, a duração e a sobreposição temporal de cada etapa.

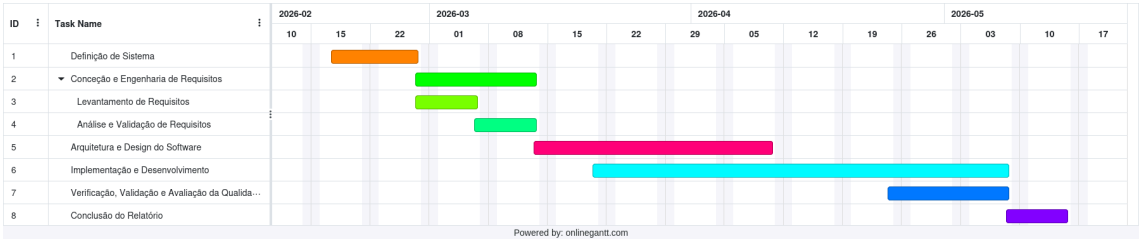


Figura 4: Diagrama de `Gantt` Executado.

Comparando os dois diagramas de `Gantt`, é possível identificar diferenças relevantes entre o planeamento inicial e a execução real do projeto. No **`Gantt` Previsto (Figura 3)**, o plano era mais simplificado, contemplando apenas quatro grandes fases, com uma distribuição temporal mais compacta e linear. Já no **`Gantt` Executado (Figura 4)**, o projeto revelou-se mais detalhado e extenso, incorporando tarefas adicionais como a Definição de Sistema e a Conclusão do Relatório, e com **algumas fases a prolongarem-se além do inicialmente previsto**, nomeadamente a Implementação e Desenvolvimento, que se estendeu consideravelmente para além de abril. Adicionalmente, verificou-se uma maior **sobreposição entre tarefas na execução real**, o que sugere uma gestão paralela de atividades não contemplada no plano original. Em suma, embora a estrutura geral do projeto tenha sido respeitada, a execução exigiu um esforço mais prolongado e uma decomposição mais granular das tarefas do que o inicialmente planeado.

1.9. Revisão e Aprovação do Projeto

Após a elaboração do plano de execução do projeto e do respetivo estudo de viabilidade, foi agendada uma reunião presencial entre a direção da Taki e a equipa de desenvolvimento da SegFault. O propósito principal deste encontro consistiu na apresentação detalhada da proposta de informatização, na revisão do cronograma estipulado e na obtenção de um consenso formal para o arranque dos trabalhos.

Durante a sessão, o Sr. Manuel Pinto da Costa pôde validar que as soluções tecnológicas propostas pela SegFault respeitavam a visão e os constrangimentos orçamentais da empresa, destacando o alinhamento da solução centralizada com o espírito descomplicado da marca. Concluídas as deliberações, o desenvolvimento do **Sistema de Gestão da Taki** foi formalmente aprovado, assinalando a transição oficial para a fase de levantamento e análise de requisitos.

Abaixo, apresenta-se o cabeçalho da ata elaborada na reunião de aprovação, que documenta a oficialização deste marco do projeto:

Sistema de Gestão da Taki - Reunião de Aprovação
Data: 26 de fevereiro de 2026
Início: 13h30 | **Término:** 15h15

Local: Sede da Taki (Famalicão)
Duração: 1h e 45 minutos

Participantes:
- Sr. Manuel Pinto da Costa (Dono e Gerente, Taki)
- Diogo Campos, João Delgado, Nelson Mendes, Simão Mendes, Tomás Machado (Equipa *SegFault*)

Pauta da reunião:
- Apresentação e discussão do paradigma de operação autónoma vs. servidor central;
- Revisão do plano de execução do projeto e respetivo cronograma;
- Aprovação formal da viabilidade do projeto e autorização para iniciar o levantamento de requisitos.

Figura 5: Resumo da Ata de Reunião de Aprovação do Projeto. A versão integral e assinada do documento encontra-se documentada no **Anexo I**.

1.10. Estrutura do Relatório

O presente relatório encontra-se estruturado em nove capítulos, organizados de acordo com as principais fases do ciclo de desenvolvimento de *software*.

Para além deste capítulo introdutório, que estabelece as bases do projeto, o documento organiza-se da seguinte forma:

- O segundo capítulo, **Conceção e Engenharia de Requisitos Assistida por LLM**, descreve o processo de definição do sistema e levantamento de requisitos, com recurso a modelos de linguagem de grande escala;
- O terceiro capítulo, **Arquitetura e *Design* do *Software* utilizando LLM**, apresenta a estrutura técnica do sistema, incluindo a modelação dos dados, a lógica de negócio e o *design* da interface, suportado por ferramentas de inteligência artificial;
- O quarto capítulo, **Implementação e Desenvolvimento Assistido por LLM**, documenta o processo de codificação e as decisões tomadas durante a construção do sistema, com o auxílio de modelos de linguagem;
- O quinto capítulo, **Verificação, Validação e Avaliação da Qualidade do *Software* Produzido**, aborda os testes realizados e a avaliação da qualidade do produto desenvolvido;
- O sexto capítulo, **Conclusões e Trabalho Futuro**, apresenta uma reflexão crítica sobre o trabalho realizado e sugere possíveis desenvolvimentos futuros;
- O sétimo capítulo reúne as **Referências Bibliográficas**, complementado pela **Lista de Siglas e Acrónimos** no oitavo capítulo e pelos **Anexos** no nono capítulo, que contêm informação adicional relevante para a compreensão do trabalho.

1.11. Utilização de Inteligência Artificial

A elaboração deste relatório, bem como o processo de engenharia de *software* subjacente ao projeto `Taki`, contou com o apoio de técnicas de Inteligência Artificial Generativa (IAG), em estrita conformidade com as diretrizes da unidade curricular. Para garantir consistência e qualidade ao longo de todo o ciclo

de vida do desenvolvimento, a equipa iniciou o trabalho com o modelo **Gemini 3.1 Pro**, integrado através do *Gemini CLI*, e, posteriormente, transitou para a utilização do **Claude Code** (com recurso aos modelos **Claude Sonnet 4.6** e **Claude Opus 4.7**), de modo a ultrapassar limitações de performance e qualidade das respostas.

Neste capítulo inicial ("Introdução"), a IA desempenhou um papel fundamental não na geração autónoma de ideias originais de negócio, mas sim na estruturação formal, refinação de texto e consolidação das visões da equipa para um formato académico e profissional.

Configuração do Agente e Diretrizes de Sistema

Para que os assistentes de IA operassem como verdadeiros parceiros técnicos, estabelecemos documentos de configuração de contexto rigorosos. Inicialmente criámos o ficheiro `GEMINI.md` e, aquando da transição para o ecossistema Anthropic, este foi integralmente adaptado para o formato `CLAUDE.md` (ambos documentados nos Anexos deste relatório). Estes ficheiros atuaram como as instruções de sistema (*system instructions*) contínuas e definiram os seguintes pilares de atuação:

1. **Papel e Persona:** O modelo foi instruído para atuar como um *Senior Software Engineering Lead*, com o intuito de garantir respostas focadas em padrões de qualidade e metodologias estruturadas;
2. **Contexto do Projeto:** Foi-lhe fornecido o resumo do caso de estudo (a rede de lojas de conveniência) e o objetivo de desenvolver um Sistema de Gestão Integrada;
3. **Padrões de Comunicação:** Definiu-se que a comunicação deveria ocorrer num tom profissional, na variante de Português de Portugal, e com a restrição gramatical de evitar ativamente a utilização do gerúndio. Esta premissa obrigou o modelo a privilegiar a voz ativa e estruturas de subordinação mais elegantes, para atingir uma maior clareza e assertividade no texto final;
4. **Formato de Saída:** Estabeleceu-se o `Typst` como a linguagem de formatação *standard* para qualquer bloco de texto ou tabela gerados.

Interações e *Prompts* Chave na Introdução

Durante a elaboração do capítulo introdutório, o fluxo de trabalho consistiu na introdução de informação de base por parte da equipa (ex: o que o negócio faz, quem são os intervenientes, a estrutura de gestão), seguida de pedidos ao *Gemini* para estruturar e formalizar essa mesma informação.

Abaixo detalham-se alguns dos *prompts* conceptuais mais relevantes utilizados na construção das subsecções anteriores, que demonstram a forma como a IA foi conduzida:

- **Para a "Contextualização" e "Fundamentação":** Para garantir que a base teórica estava sólida e alinhada com as metodologias de Engenharia de Software abordadas nas aulas, a equipa focou-se em pedir ao modelo que interligasse as nossas ideias. > "Temos de escrever a Contextualização e a Fundamentação. A nossa ideia é falar sobre como as arquiteturas distribuídas e os LLMs mudaram o desenvolvimento de *software*, e como vamos usar isso para a rede de lojas 'Taki'. Podes criar um rascunho inicial que ligue estes pontos de forma formal, fazendo referência ao livro do Sommerville?"
- **Para a "Apresentação do Caso de Estudo" e "Identidade do Projeto":** A equipa precisava de converter as breves anotações das reuniões num texto apresentável e criar elementos visuais limpos em `Typst`. > "Com base neste pequeno excerto do enunciado sobre a cadeia de lojas, expande o texto para a 'Apresentação do Caso de Estudo'. Depois, cria um *template* em Typst com uma tabela para a 'Ficha de Identidade do Projeto' (com Nome, Acrónimo, Cliente) e um espaço à direita para o logótipo."
- **Para o Refinamento de Estilo Geral:** Ao longo da escrita de secções como a Fundamentação e Objetivos, recorremos frequentemente ao assistente para avaliar o tom do relatório. > "Lê este parágrafo introdutório. O tom é demasiado coloquial para o contexto universitário? É normal usar palavras a negrito para destacar ideias no meio de um relatório deste género? Responde com as melhores práticas de documentação."

Através destas interações guiadas, o modelo *Gemini* 3.1 *Pro* permitiu à equipa poupar um tempo considerável em tarefas de formatação e estruturação frásica. Essa poupança canalizou o nosso esforço intelectual

para a discussão real do domínio do problema e das decisões técnicas do projeto. O assistente não substituiu o discernimento crítico dos alunos; pelo contrário, funcionou como uma ferramenta aceleradora de produtividade e de padronização documental, que alinhou a forma ao conteúdo idealizado.

2. Conceção e Engenharia de Requisitos Assistida por LLM

O presente capítulo documenta a fase de **Conceção e Engenharia de Requisitos**, etapa basilar que estabelece o **contrato técnico e funcional** para o Sistema de Gestão Integrada da 'Taki'. Com o intuito de traduzir a visão pragmática do negócio numa **arquitetura de *software* robusta**, a equipa adotou metodologias de levantamento rigorosas junto dos diversos intervenientes da empresa. Para otimizar este processo, integrámos **Modelos de Linguagem de Grande Escala (LLM)** em momentos-chave da recolha e análise. O recurso a estas ferramentas de Inteligência Artificial permitiu estruturar as informações obtidas, refinar a redação técnica e validar a consistência lógica dos artefactos finais, de forma a garantir a total **ausência de ambiguidades** antes da transição para o desenho arquitetural.

2.1. Levantamento de Requisitos

O sucesso do desenvolvimento do novo sistema de gestão integrada para a cadeia de lojas de conveniência 'Taki' depende de uma **compreensão clara, rigorosa e exaustiva** das necessidades reais do negócio. Este capítulo documenta o processo de Engenharia de Requisitos, com o intuito de estabelecer o **guião técnico e funcional** que alinha a visão estratégica dos *stakeholders* com a execução da equipa de desenvolvimento de *software*.

O objetivo principal desta fase é garantir que a solução final suporta eficazmente a **autonomia de cada loja**, e assegura simultaneamente o **processamento centralizado e fiável** dos dados de vendas, inventário e faturação.

Para fornecer uma **visão transparente e rastreável** sobre o sistema a construir, esta secção encontra-se estruturada nas seguintes subsecções:

- **Método de Levantamento e de Análise de Requisitos Adotado**: Detalha a abordagem metodológica utilizada para compreender o contexto operacional da 'Taki', com a inclusão da simulação de entrevistas com os principais *stakeholders* e a análise das restrições do domínio do negócio;
- **Recolha e Organização dos Requisitos**: Descreve como as necessidades e dores de negócio identificadas foram traduzidas em **artefactos técnicos estruturados**, de forma a criar uma **ponte clara** entre a linguagem de negócio e a arquitetura de *software*;
- **Requisitos Funcionais**: Apresenta, de forma detalhada e consistente, as **funcionalidades core** do sistema (o que o *software* tem de fazer), desde as políticas de preços até à geração de *dashboards* analíticos;
- **Requisitos Não Funcionais**: Define os **atributos de qualidade** e as **restrições arquiteturais** da plataforma, para estabelecer os padrões exigidos em termos de segurança de dados, escalabilidade, desempenho e usabilidade;
- **Validação dos Requisitos**: Expõe os mecanismos e critérios aplicados para garantir que os requisitos documentados são **completos, consistentes e tecnicamente exequíveis**, sem contradições lógicas, com vista a minimizar os riscos para a fase de desenvolvimento.

2.1.1. Método de Levantamento e de Análise de Requisitos Adotado

No âmbito da metodologia de trabalho da ***SegFault***, as reuniões com a administração da **Taki** constituíram a principal fonte de requisitos de alto nível. Este processo dividiu-se em dois momentos formais distintos:

- **Reunião de Recolha de Requisitos (05/03/2026)**: Sessão focada na identificação das necessidades operacionais. O foco incidiu sobre a gestão de inventário, os fluxos de faturação e a autonomia necessária para as filiais.

> **Sistema de Gestão da Taki - Reunião de Recolha de Requisitos**
> **Local:** Sede da Taki (Famalicão)
> **Duração:** 2h e 30min
> **Data:** 05 de março de 2026
> **Início:** 13h30 | **Término:** 16h00
>
> ---
>
> **Participantes:**
> - Sr. Manuel Pinto da Costa (Dono e Gerente, **Taki**)
> - Diogo Campos, João Delgado, Nelson Mendes, Simão Mendes, Tomás Machado (Equipa ***SegFault***)
>
> **Pauta da reunião:**
> - Identificação das necessidades operacionais das lojas autónomas;
> - Definição dos fluxos de faturação e gestão de inventário local;
> - Estruturação dos processos de comunicação com o servidor central.

Figura 6: Resumo da Ata de Reunião de Recolha de Requisitos. A versão integral e assinada do documento encontra-se documentada no **Anexo II**.

Questionários a Operadores e Recursos Humanos

Para complementar a visão estratégica da administração, a equipa considerou essencial auscultar quem interage diretamente com o cliente final nas frentes de caixa. Através da aplicação de um inquérito estruturado (via *Microsoft Forms*) aos **operadores de loja** e aos responsáveis de **Recursos Humanos**, foi possível identificar as dores reais do quotidiano operacional. A estrutura integral deste inquérito, bem como as respetivas métricas, encontram-se disponíveis no **Anexo IV**.

Os resultados evidenciaram uma necessidade premente de **rapidez e clareza**. Devido à elevada rotatividade de pessoal (*turnover*) típica do retalho, os inquiridos apontaram que o sistema deve apresentar uma **interface altamente intuitiva**, com uma curva de aprendizagem inferior a duas horas. Adicionalmente, estabeleceu-se o limite de **10 segundos para o processamento de vendas comuns**, com o intuito de evitar a formação de filas. Estas métricas refletem e honram a cultura de eficiência prática encapsulada no lema da marca: "Não inventes, **Taki**!". Os resultados completos encontram-se disponíveis no **Anexo V**.

Estudo de Mercado e Enquadramento Legal

A conceção de um sistema de gestão comercial e faturação obriga a um rigoroso escrutínio do contexto externo. Por essa razão, a equipa da ***SegFault*** conduziu um estudo de mercado focado em soluções concorrentes de Ponto de Venda (*POS*) no setor do retalho rápido, bem como nos padrões exigidos pela **legislação fiscal portuguesa**.

Desta análise resultou a incorporação de requisitos críticos de conformidade legal, como a obrigatoriedade de exportação de dados no formato **SAF-T (PT)** para a Autoridade Tributária. No campo da segurança de informação, o estudo determinou a imposição de **encriptação forte (protocolo TLS 1.3 ou superior)** nas comunicações entre as filiais e o servidor central. O relatório detalhado com a análise de sistemas semelhantes e as diretrizes fiscais encontra-se documentado no **Anexo VI**.

Método de Análise de Requisitos Adotado

Após a conclusão da fase de recolha, a equipa procedeu à **triagem e formalização** de toda a informação obtida. O processo de análise teve como objetivo principal detetar e resolver potenciais contradições

entre a visão da administração, as necessidades reais dos operadores de loja e as restrições impostas pela legislação fiscal e de segurança.

Para estruturar os dados brutos em artefactos técnicos acionáveis, a equipa adotou as metodologias de **Engenharia de *Software*** preconizadas na literatura da unidade curricular. As dores de negócio e as sugestões operacionais sofreram uma conversão rigorosa para cenários de utilização claros. Esta ação metódica permitiu a extração exata e documentada dos **Requisitos Funcionais** (os serviços transacionais que o sistema tem de executar) e dos **Requisitos Não Funcionais** (os critérios arquiteturais de qualidade, segurança e desempenho exigidos).

Esta sistematização garante que a arquitetura final responde de forma integral aos desafios da cadeia **Taki**, com vista a estabelecer uma base de concordância sólida que suporta as especificações detalhadas nas subsecções seguintes.

2.1.2. Elicitação e Organização dos Requisitos

A engenharia de requisitos constitui uma das fases mais críticas no ciclo de vida do desenvolvimento de **software**. O processo de elicitação e análise foca-se na compreensão profunda do domínio da aplicação, definindo os serviços que o sistema deve fornecer e as restrições operacionais a que estará sujeito.

Tendo por base as atividades de levantamento e a interação com os **stakeholders** previamente descritas neste documento, procedeu-se à organização e especificação formal da informação obtida para o Sistema de Gestão Integrada 'Taki'. Uma documentação rigorosa é essencial para estabelecer uma base de concordância sólida e inequívoca entre a equipa de desenvolvimento e o cliente. Deste modo, os requisitos do sistema foram categorizados em duas vertentes fundamentais, que serão detalhadas nas subsecções seguintes:

- **Requisitos Funcionais (RF):** Consistem em declarações concretas dos serviços que o sistema informático deve fornecer. Detalham de que forma o sistema deve reagir a determinadas entradas de dados e como se deve comportar em situações específicas do fluxo de negócio (como, por exemplo, o registo de transações de venda, a atualização de inventário ou a emissão de faturas).
- **Requisitos Não Funcionais (RNF):** Traduzem-se em restrições impostas aos serviços e funções oferecidos pelo sistema. Em vez de se focarem em operações transacionais diretas, definem atributos essenciais de qualidade e de arquitetura, tais como o desempenho temporal, a fiabilidade, a segurança dos dados e a conformidade legal.

Esta segregação garante que a arquitetura do sistema seja projetada não só para executar as tarefas de negócio com exatidão, mas também para operar com os níveis de robustez e segurança exigidos pela organização.

Requisitos Funcionais A definição dos Requisitos Funcionais é um dos pilares fundamentais para o sucesso de qualquer projeto de desenvolvimento. Eles descrevem, de forma clara e detalhada, as **funcionalidades específicas que o sistema deve executar**, ou seja, **"o que" o sistema fará para atender** às necessidades dos utilizadores e do negócio.

A formalização e estruturação destes requisitos fundamentar-se-á no **modelo paradigmático apresentado na obra de referência indicada** pela equipa docente, conforme detalhado na Secção 7. Deste modo, é assegurada a adoção das boas práticas e da arquitetura metodológica preconizadas para a garantia do rigor técnico do projeto.

O primeiro requisito funcional estabelece a base operacional de qualquer sistema de Ponto de Venda. Foca-se na capacidade de registar as transações diárias de forma exata e automatizada, garantindo que os cálculos de totais e impostos decorrem sem falhas para o operador.

Requisito de utilizador	O sistema deve permitir o registo de transações de venda, calculando o total da venda, incluindo a aplicação de taxas aplicáveis.
Fonte	Questionários aos Operadores de Loja
Área do sistema	Gestão de Vendas
Requisitos de sistema	1. Deve existir uma interface de registo de vendas (Ponto de Venda) acessível a utilizadores com permissões de vendedor. 2. O sistema local deve permitir a adição de produtos à transação atual através de pesquisa manual ou leitura de códigos de barras. 3. Para cada produto adicionado, o sistema local deve calcular o subtotal com base na quantidade e no preço unitário. 4. O sistema local deve calcular e aplicar automaticamente as taxas legais correspondentes (ex: IVA) a cada produto, exibindo o valor total final da transação. 5. Após a confirmação da transação e do pagamento, o sistema local regista a venda, deduz os produtos do inventário e disponibiliza a opção de emissão de fatura/recibo.
Relevância para a aplicação	O registo fiável das transações é o núcleo de qualquer operação comercial. A automatização do cálculo de totais e da aplicação de impostos minimiza o erro humano, garante a conformidade fiscal e agiliza o processo de checkout. Além disso, permite manter o sistema de inventário atualizado em tempo real face às saídas de stock.

Tabela 1: Requisito funcional 1.

Para garantir a flexibilidade no momento do pagamento e responder às diferentes preferências dos clientes, o sistema necessita de suportar múltiplos métodos de transação. O requisito seguinte define o processamento seguro e integrado de pagamentos em dinheiro ou por cartão bancário.

Requisito de utilizador	O sistema deve processar diferentes métodos de pagamento, como numerário e cartão.
Fonte	Reuniões com a Direção Geral
Área do sistema	Faturação e Pagamentos
Requisitos de sistema	1. A interface de finalização de compra (checkout) deve apresentar as opções de pagamento “Numerário” e “Cartão de Crédito/Débito”. 2. Se o método “Numerário” for selecionado, o sistema local deve permitir a introdução do valor entregue pelo cliente e calcular automaticamente o troco a devolver. 3. Se o método “Cartão” for selecionado, o sistema local deve comunicar com o terminal de pagamento automático (TPA) e aguardar a resposta (sucesso ou falha) da transação. 4. Em caso de falha no processamento eletrónico (ex: cartão rejeitado ou falha de rede), o sistema local deve notificar o operador e permitir a retoma do processo com o mesmo método ou um alternativo, sem perder os dados da venda.
Relevância para a aplicação	A diversificação dos métodos de pagamento é indispensável para acomodar as preferências dos consumidores. O processamento sistémico e preciso destas transações garante também que os fechos de caixa diários sejam exatos, facilitando a reconciliação bancária e a auditoria por parte da contabilidade.

Tabela 2: Requisito funcional 2.

Para a conclusão de uma transação comercial é exigido a entrega de um comprovativo legal e fiscal ao cliente. Este requisito assegura a geração e emissão de recibos detalhados, garantindo a transparência da compra e salvaguardando os direitos do consumidor para efeitos de garantia.

Requisito de utilizador	O sistema deve permitir a emissão de um recibo de venda para o cliente.
Fonte	Reuniões com a Administração e Legislação Fiscal
Área do sistema	Faturação e Pós-Venda
Requisitos de sistema	1. Após a conclusão bem-sucedida do pagamento, o sistema local deve gerar automaticamente um recibo detalhado associado à transação. 2. O recibo gerado deve conter, obrigatoriamente: identificação da loja (nome, NIF, morada), data e hora, lista de artigos (com quantidade e preço unitário), subtotal, impostos aplicados, valor total e método de pagamento. 3. O sistema local deve apresentar ao operador a opção de imprimir o recibo fisicamente (comunicando com a impressora de talões) ou enviá-lo em formato digital (ex: por e-mail, solicitando o endereço ao cliente). 4. Cada recibo emitido deve ser classificado com um número de identificação único e sequencial, garantindo a rastreabilidade da operação. 5. O sistema local deve permitir a consulta do histórico de vendas para que seja possível a reimpressão ou o reenvio de uma 2ª via do recibo em caso de perda pelo cliente.
Relevância para a aplicação	A emissão de um recibo é não só uma obrigação legal e fiscal que comprova a transação perante a Autoridade Tributária, mas também um direito do consumidor. É o documento fundamental que permite ao cliente acionar garantias, efetuar trocas ou solicitar devoluções. Para o negócio, garante a transparência e cria um rasto de auditoria fiável.

Tabela 3: Requisito funcional 3.

Lidar com devoluções é uma rotina incontornável do comércio e um ponto de contacto com o consumidor. Assim, este requisito foi desenhado para estruturar um fluxo ágil de devolução de artigos, com o duplo objetivo de garantir uma restituição dos fundos e sincronizar com entrada da mercadoria para o stock.

Requisito de utilizador	O sistema deve fornecer uma funcionalidade para o processamento de devoluções.
Fonte	Questionários aos Operadores de Loja
Área do sistema	Pós-Venda e Inventário
Requisitos de sistema	1. O sistema local deve permitir a identificação de uma venda anterior através da inserção do número da fatura. 2. Após a identificação da venda, o sistema local deve apresentar os artigos adquiridos e permitir a seleção dos produtos a devolver, bem como as respetivas quantidades. 3. O sistema local deve validar automaticamente se a devolução se encontra dentro do prazo estipulado pela política da loja (ex: 30 dias). 4. O sistema local deve calcular o montante a reembolsar e permitir que o operador registe o reembolso no método de pagamento original. 5. Concluído o processo, o sistema local deve emitir um comprovativo de devolução (nota de crédito) e atualizar automaticamente o stock dos artigos devolvidos no inventário.
Relevância para a aplicação	O processamento ágil de devoluções é crucial para um bom serviço de apoio ao cliente e para o cumprimento da legislação de consumo. Além disso, a automatização deste processo garante que o sistema de inventário é reposto corretamente e que os valores reembolsados ficam registados de forma exata para o fecho de caixa e a reconciliação contabilística.

Tabela 4: Requisito funcional 4.

A precisão do inventário depende de uma comunicação constante e sem falhas entre os terminais de faturação e o armazém. O requisito que se segue garante que cada transação de venda registe a saída do respetivo artigo no stock em tempo real, permitindo um controlo rigoroso das quantidades disponíveis e a emissão automática de alertas de reposição.

Requisito de utilizador	O sistema deve decrementar automaticamente a quantidade de um produto em stock após a sua venda.
Fonte	Reuniões com a Administração
Área do sistema	Gestão de Inventário
Requisitos de sistema	1. Após a conclusão bem-sucedida e o pagamento de uma transação de venda, o sistema local deve identificar todos os artigos transacionados e as respetivas quantidades. 2. O sistema local deve aceder à base de dados de inventário em tempo real e subtrair a quantidade vendida ao stock disponível de cada artigo correspondente. 3. O sistema local deve garantir a atomicidade da operação: em caso de falha no pagamento ou cancelamento da venda antes da sua conclusão, as quantidades em stock não devem ser alteradas. 4. Cada decremento de stock deve ser registado no histórico de movimentos de inventário local, ficando associado à data, hora e ao número identificador único da transação de venda.
Relevância para a aplicação	A atualização automática e em tempo real do inventário é o elo fundamental entre as operações de venda e a logística. Impede a venda de artigos que já não se encontram fisicamente disponíveis (evitando a insatisfação do cliente) e fornece dados fiáveis aos gestores de loja para a tomada de decisão sobre encomendas a fornecedores, minimizando ruturas de stock.

Tabela 5: Requisito funcional 5.

O abastecimento contínuo das lojas exige um mecanismo informático fiável para documentar a chegada de nova mercadoria. O requisito abaixo detalha como os responsáveis de logística devem proceder para registar e somar as quantidades de entregas ao sistema global.

Requisito de utilizador	O sistema deve permitir a atualização manual do stock para registar a entrada de entregas.
Fonte	Reuniões com a Administração
Área do sistema	Gestão de Inventário
Requisitos de sistema	1. O sistema local deve disponibilizar uma interface para a receção de mercadorias, acessível apenas a utilizadores com permissões de gestão de inventário. 2. O sistema local deve permitir a identificação do artigo recebido através da inserção da referência. 3. O operador deve introduzir a quantidade recebida, e o sistema local irá adicionar automaticamente ao valor atual do stock desse artigo. 4. O sistema local deve registar a entrada de stock guardando a data, a hora, o utilizador responsável e a fatura do fornecedor. 5. Após a confirmação, o sistema local deve atualizar imediatamente a quantidade disponível para venda e permitir a consulta desta operação no histórico de movimentos.
Relevância para a aplicação	O registo manual de entradas é fundamental para manter o inventário rigorosamente atualizado sempre que chegam novas encomendas dos fornecedores. Garantir que cada entrada fica associada à fatura do fornecedor e a um utilizador assegura a rastreabilidade da operação, facilitando futuras auditorias e evitando discrepâncias no stock.

Tabela 6: Requisito funcional 6.

Sabendo que as contagens de inventário físico raramente correspondem em absoluto aos dados do sistema informático, quer seja por quebras, furtos ou erros operacionais, tornou-se necessário criar um mecanismo corretivo. Este requisito assegura que esses ajustes sejam feitos de forma controlada.

Requisito de utilizador	O sistema deve permitir a realização de ajustes manuais ao inventário para corrigir discrepâncias.
Fonte	Reuniões com a Administração
Área do sistema	Gestão de Inventário
Requisitos de sistema	1. O sistema local deve disponibilizar uma funcionalidade de ajuste de *stock*, de acesso restrito a utilizadores com privilégios de gestão ou auditoria. 2. O sistema local deve permitir a seleção do artigo a ajustar através da pesquisa por referência. 3. O operador deve indicar a nova quantidade física real do artigo e selecionar obrigatoriamente um motivo para o ajuste (ex: quebra, furto, erro de contagem, validade expirada). 4. O sistema local deve calcular automaticamente a diferença entre o stock no sistema e a nova quantidade inserida, atualizando o valor no inventário em tempo real. 5. O sistema local deve criar um registo do ajuste, guardando a data, hora, o utilizador responsável, as quantidades anterior e nova, bem como o motivo declarado.
Relevância para a aplicação	A precisão do inventário é vital para a operação logística e contabilística da empresa. Como as discrepâncias físicas ocorrem inevitavelmente, o sistema tem de fornecer um mecanismo rigoroso e seguro para alinhar o stock informático com a realidade física. A obrigatoriedade de registar o motivo e o utilizador garante a rastreabilidade (auditoria) e desincentiva fraudes internas.

Tabela 7: Requisito funcional 7.

O presente requisito diz respeito à **criação de encomendas a fornecedores**, uma operação central para a reposição de inventário em cada loja. Descreve as funcionalidades necessárias para gerar encomendas localmente, de forma autónoma, garantindo o registo de todos os dados relevantes. Inclui ainda a consolidação noturna desta informação no sistema central para visibilidade global.

Requisito de utilizador	O sistema deve permitir a criação de encomendas a fornecedores.
Fonte	Stakeholders
Área do sistema	Gestão de Inventário
Requisitos de sistema	1. O sistema local de cada loja deve disponibilizar um formulário para registo de novas encomendas a fornecedores, solicitando a seleção do fornecedor, a data de entrega pretendida e a loja de destino. 2. O sistema local deve permitir a pesquisa e seleção de produtos a incluir na encomenda, registando para cada um a quantidade pretendida, o preço unitário acordado e as condições fiscais aplicáveis (ex: IVA). 3. O sistema local deve calcular automaticamente o valor total da encomenda (subtotal e valor global com impostos), com base nos itens inseridos. 4. O sistema local deve associar a encomenda a um estado específico (ex: “Rascunho”, “Pendente”, “Enviada”, “Concluída”, “Concluída”, “Em Trânsito”, “Entregue”, “Cancelado”), sendo que a criação inicial deve gerar uma encomenda no estado “Rascunho”. 5. O sistema local deve registar a data e hora da criação da encomenda e o utilizador que a realizou, para efeitos de auditoria e rastreabilidade. 6. O sistema local deve validar se o fornecedor selecionado está ativo no cadastro antes de permitir a criação da encomenda.
Relevância para a aplicação	A criação de encomendas a fornecedores é uma operação crucial para a aplicação, pois viabiliza a gestão de *stocks* ao garantir a reposição de inventário e prevenir ruturas. Adicionalmente, formaliza compromissos financeiros, permitindo o controlo orçamental, ao mesmo tempo que assegura a rastreabilidade do processo de aquisição para apoio à decisão estratégica.

Tabela 8: Requisito funcional 8.

Este requisito refere-se à **consulta em tempo real dos níveis de *stock* em cada loja**, fundamental para a tomada de decisão operacional diária. Especifica as funcionalidades que permitem a cada loja visualizar o seu inventário atualizado, bem como a disponibilização de uma visão consolidada no sistema central. Abrange ainda a filtragem, pesquisa e exportação dos dados para apoio à gestão.

Requisito de utilizador	O sistema deve fornecer uma visão em tempo real dos níveis de stock de cada produto por loja.
Fonte	Stakeholders
Área do sistema	Gestão de Inventário
Requisitos de sistema	1. O sistema deve disponibilizar a informação de *stock* em tempo real através de uma interface gráfica. 2. O sistema deve permitir a filtragem e pesquisa da informação por produto ou por intervalo de quantidades (ex: produtos com *stock* inferior a X unidades). 3. O sistema central deve disponibilizar uma *dashboard* consolidada, com os níveis de *stock* de todas as lojas, permitindo a filtragem por loja, produto ou categoria. 4. O sistema central deve permitir a exportação dos dados de *stock* consolidados para formatos como `CSV` ou `JSON`, para apoio à tomada de decisão.
Relevância para a aplicação	É crítico suportar a tomada de decisão operacional e comercial com informação atualizada e fiável. Esta visibilidade imediata permite otimizar a gestão de inventário ao identificar ruturas ou excessos, agilizar o atendimento ao cliente através da confirmação instantânea da disponibilidade de produtos e melhorar a eficiência logística ao possibilitar transferências entre lojas com base em necessidades reais e não em suposições.

Tabela 9: Requisito funcional 9.

O requisito aqui apresentado diz respeito à **geração automática de alertas para produtos com *stock* baixo**, permitindo uma gestão proativa do inventário. Descreve a monitorização autónoma por cada loja, a notificação dos responsáveis e a distinção entre diferentes níveis de criticidade. Inclui também o registo histórico dos alertas e a sua consolidação noturna no sistema central.

Requisito de utilizador	O sistema deve gerar alertas automáticos para produtos que atinjam um nível de stock baixo predefinido.
Fonte	Stakeholders
Área do sistema	Gestão de Inventário
Requisitos de sistema	1. O sistema de cada loja deve permitir a configuração, para cada produto, de um nível mínimo de *stock* (*stock* de segurança) que, quando atingido, deverá gerar um alerta. 2. Cada loja deve dispor de um módulo local de monitorização de *stocks*, autónomo e independente, responsável por comparar continuamente os seus níveis de inventário com os respetivos limites mínimos definidos, garantindo a geração de alertas. 3. O sistema deve gerar automaticamente um alerta sempre que a quantidade disponível de um produto for igual ou inferior ao seu nível mínimo predefinido. 4. O sistema deve distinguir diferentes tipos de alerta (ex: "*stock* crítico" para níveis muito baixos e "aviso de rutura" para quando o *stock* chega a zero). 5. O sistema deve notificar os utilizadores relevantes (ex: gestor de loja) através de mensagens no sistema. 6. Os alertas gerados localmente devem ser registados no sistema de cada loja. 7. O sistema deve permitir a consulta e gestão dos alertas ativos, possibilitando a sua marcação como "Resolvido" após a tomada de ação (ex: realização de uma encomenda).
Relevância para a aplicação	É essencial garantir a capacidade de transformação da gestão de inventário de reativa para proativa, pois, assim, prevenirá ruturas, que podem impactar negativamente as vendas ou uma operação. Ao automatizar a monitorização e a comunicação de situações críticas, o sistema liberta os gestores de loja da tarefa manual de verificação constante de *stocks*, garantindo uma resposta atempada na reposição de produtos e contribuindo decisivamente para a continuidade do negócio e a satisfação do cliente.

Tabela 10: Requisito funcional 10.

Este requisito refere-se à gestão autónoma do catálogo de produtos por cada loja, **permitindo adicionar, editar e descontinuar itens localmente**. Especifica a informação a registar para cada produto, bem como a sinalização visual dos itens descontinuados. Abrange ainda a sincronização noturna das alterações com o sistema central para uma visão global atualizada.

Requisito de utilizador	O sistema deve permitir adicionar, editar e descontinuar produtos no catálogo local.
Fonte	Stakeholders
Área do sistema	Gestão de Catálogo e Fornecedores
Requisitos de sistema	1. O sistema local de cada loja deve permitir a adição de novos produtos ao catálogo local, incluindo informação como: nome do produto, descrição, categoria, preço de venda sugerido, unidade de medida (ex: unidade, kg, litro), identificador único e estado (ex: "Ativo", "Descontinuado"). 2. O sistema local deve permitir a edição dos dados de qualquer produto existente no catálogo local, registando a data e o utilizador que realizou a alteração, para efeitos de auditoria. 3. O sistema local deve permitir a descontinuação de produtos, impedindo que estes passem a ser utilizados em novas operações (ex: vendas, encomendas), mas mantendo o seu histórico para consulta e relatórios. 4. Um produto descontinuado deve ser identificado visualmente nas interfaces de consulta (ex: marcado como "Descontinuado") e excluído dos processos ativos (como pesquisas rápidas para venda ou criação de encomendas). 5. O sistema local deve registar, para cada produto, a data de criação, a data da última alteração e a data de descontinuação (se aplicável), garantindo a rastreabilidade completa do ciclo de vida do produto no catálogo local.
Relevância para a aplicação	É necessário assegurar uma autonomia operacional conferida a cada loja para gerir o seu catálogo em tempo real, adaptando a oferta às necessidades específicas do seu público sem dependência do sistema central. Garante que apenas produtos ativos estão disponíveis para venda e encomenda, evitando erros e inconsistências. Adicionalmente, a consolidação noturna no sistema central assegura à administração uma visão global atualizada e fiável do sortido de toda a cadeia, fundamental para análise estratégica e planeamento.

Tabela 11: Requisito funcional 11.

O presente requisito diz respeito à **categorização dos produtos**, um elemento estruturante para a organização do catálogo. Descreve a criação de hierarquias de categorias, a associação de produtos a uma ou mais categorias e a pesquisa por categoria. Inclui ainda a consolidação noturna da estrutura de categorias no sistema central para análise global.

Requisito de utilizador	O sistema deve permitir a categorização de produtos.
Fonte	Stakeholders
Área do sistema	Gestão de Catálogo e Fornecedores
Requisitos de sistema	1. O sistema local de cada loja deve permitir a criação de categorias de produtos, incluindo informação como: nome da categoria, descrição, categoria "pai" (para suportar hierarquias ou subcategorias) e estado (ex: "Ativo", "Descontinuada"). 2. O sistema local deve permitir a edição dos dados de qualquer categoria existente, registando a data e o utilizador que realizou a alteração, para efeitos de auditoria. 3. O sistema local deve permitir a descontinuação de categorias, impedindo que estas passem a ser associadas a novos produtos, mas mantendo o seu histórico para consulta e relatórios (especialmente importante para não corromper dados históricos de vendas e relatórios por categoria). 4. O sistema local deve permitir associar cada produto a uma ou mais categorias (ex: um produto pode pertencer à categoria "Bebidas" e à subcategoria "Refrigerantes" ou "Águas"). 5. O sistema local deve disponibilizar funcionalidades de pesquisa e filtragem de produtos por categoria, facilitando a consulta e a análise local. 6. O sistema local deve registar, para cada categoria, a data de criação, a data da última alteração e a data de descontinuação (se aplicável), garantindo a rastreabilidade completa do ciclo de vida das categorias no catálogo local. 7. O sistema central deve disponibilizar uma visão consolidada das categorias utilizadas em todas as lojas, permitindo à administração analisar padrões de venda por categoria, comparar desempenho entre lojas e gerar relatórios globais com agregação por categoria.
Relevância para a aplicação	A categorização dos produtos constitui um elemento estruturante para a organização e análise da informação no sistema. Ao permitir agrupar produtos por "famílias", este requisito viabiliza uma gestão mais eficiente do catálogo e uma navegação mais intuitiva nas operações diárias de cada loja, como vendas ou consultas de *stock*. Do ponto de vista da gestão central, a categorização é indispensável para a geração de relatórios comparativos e análise de desempenho, possibilitando, por exemplo, identificar as categorias mais rentáveis. Desta forma, a categorização não só organiza o catálogo, como também fundamenta a tomada de decisão informada e a otimização contínua do negócio.

Tabela 12: Requisito funcional 12.

Este requisito refere-se à **gestão autónoma da base de dados de fornecedores por cada loja**, permitindo adicionar, editar e descontinuar registos localmente. Especifica a informação a manter para cada fornecedor, bem como a sinalização visual dos fornecedores descontinuados. Abrange ainda a sincronização noturna dos dados com o sistema central para visibilidade global.

Requisito de utilizador	O sistema deve permitir a gestão da informação da base de dados de fornecedores.
Fonte	Stakeholders
Área do sistema	Gestão de Catálogo e Fornecedores
Requisitos de sistema	1. O sistema local de cada loja deve permitir a adição de novos fornecedores à base de dados local, incluindo informação como: nome do fornecedor, NIF, morada, contacto telefónico, *email* e estado (ex: "Ativo", "Descontinuado"). 2. O sistema local deve permitir a edição dos dados de qualquer fornecedor existente, registando a data e o utilizador que realizou a alteração, para efeitos de auditoria e rastreabilidade. 3. O sistema local deve permitir a descontinuação (ou desativação) de fornecedores, impedindo que estes passem a ser selecionados em novas encomendas, mas mantendo o seu histórico para consulta em encomendas anteriores e relatórios. 4. Um fornecedor descontinuado deve ser identificado visualmente nas interfaces de consulta (ex: marcado como "Descontinuado") e excluído das pesquisas ativas para criação de encomendas. 5. O sistema local deve permitir a consulta e pesquisa de fornecedores por diversos critérios (ex: nome, NIF, localidade, produtos fornecidos), facilitando a gestão local das relações comerciais. 6. O sistema local deve registar, para cada fornecedor, a data de criação, a data da última alteração e a data de descontinuação (se aplicável), garantindo a rastreabilidade completa do ciclo de vida do fornecedor na base de dados local. 7. O sistema central deve disponibilizar uma visão consolidada dos fornecedores de todas as lojas, permitindo à administração identificar fornecedores comuns, negociar condições globais e analisar o desempenho por fornecedor ao nível de toda a cadeia.
Relevância para a aplicação	A gestão da base de dados de fornecedores constitui um pilar essencial para a operacionalidade e o rigor financeiro de qualquer cadeia de lojas. Este requisito garante que cada loja pode, de forma autónoma, registar e manter atualizada a informação dos seus fornecedores locais, assegurando que as encomendas são colocadas junto das entidades corretas e com as condições comerciais adequadas. A possibilidade de descontinuar fornecedores de forma controlada previne encomendas inadvertidas a entidades inativas, protegendo a integridade dos processos de compra. Adicionalmente, a consolidação noturna desta informação no sistema central permite à administração da cadeia obter uma visão global e estratégica das relações comerciais.

Tabela 13: Requisito funcional 13.

O requisito aqui apresentado diz respeito à **associação de produtos a um ou mais fornecedores**, estabelecendo as relações comerciais para aprovisionamento. Descreve o registo de condições específicas por fornecedor, a definição de fornecedor preferencial e a validação nas encomendas. Inclui ainda a consolidação noturna destas associações no sistema central para análise estratégica.

Requisito de utilizador	O sistema deve permitir associar produtos a um ou mais fornecedores.
Fonte	Stakeholders
Área do sistema	Gestão de Catálogo e Fornecedores
Requisitos de sistema	1. O sistema local de cada loja deve permitir associar um ou mais fornecedores a cada produto do catálogo local, estabelecendo quais os fornecedores que podem abastecer esse produto. 2. O sistema local deve permitir editar ou remover associações produto--fornecedor existentes, registando a data e o utilizador que realizou a alteração, para efeitos de auditoria. 3. O sistema local deve permitir a consulta das associações produto-fornecedor por produto ou por fornecedor, facilitando a análise local (ex: "que produtos é que este fornecedor nos abastece?" ou "quem são os fornecedores deste produto?"). 4. O sistema local deve garantir que, ao criar uma encomenda para um determinado produto, apenas sejam apresentados os fornecedores que têm esse produto associado, evitando seleções inválidas. 5. O sistema central deve disponibilizar uma visão consolidada das associações produto-fornecedor de todas as lojas, permitindo à administração identificar padrões de abastecimento, negociar condições globais e analisar a diversidade de fornecedores por produto em toda a cadeia.
Relevância para a aplicação	Ao registar quais os fornecedores de cada produto e as respetivas condições, o sistema agiliza a criação de encomendas e evita erros. Centralmente, a consolidação noturna permite à administração analisar o universo de fornecedores por produto, identificar oportunidades de negociação global e promover uma gestão estratégica do aprovisionamento em toda a cadeia.

Tabela 14: Requisito funcional 14.

O requisito funcional 15 diz respeito à gestão de preços e campanhas comerciais. Foi definido para assegurar que a administração consiga ajustar dinamicamente o valor dos artigos e criar promoções, de forma a atrair clientes, garantir a competitividade das lojas e otimizar as margens de lucro de forma centralizada.

Requisito de utilizador	O sistema deve permitir a definição de políticas de preços e promoções.
Fonte	Entrevista com a Administração da Taki / Gestor de Operações.
Área do sistema	Gestão de Catálogo e Fornecedores
Requisitos de sistema	1. O sistema local deve disponibilizar uma interface para criar, editar, ativar e desativar regras de promoção. 2. O sistema local deve permitir aplicar as promoções a produtos individuais ou categorias inteiras. 3. O sistema local deve atualizar o preço final do produto e propagar essa informação para os sistemas de venda. 4. A data de fim da promoção deve ser obrigatoriamente posterior à data de início. O sistema não deve permitir aplicar um desconto superior a 100% do preço base. 5. Se o gestor tentar criar uma promoção que se sobreponha nas datas a outra já existente para o mesmo produto, o sistema deve emitir um alerta de conflito e bloquear a ação.
Relevância para a aplicação	Essencial para o processamento centralizado de vendas e para apoiar os processos operacionais da cadeia de lojas.

Tabela 15: Requisito funcional 15.

Este requisito funcional 16 está focado na análise transacional e no acompanhamento de vendas. Foi concebido para garantir que os gestores consigam extrair dados precisos sobre o volume de negócios,

permitindo avaliar o desempenho comercial de cada loja ou categoria e suportar decisões financeiras fundamentadas no dia a dia.

Requisito de utilizador	O sistema deve ser capaz de gerar relatórios de vendas (por período, por loja, por categoria de produto).
Fonte	Reunião de Levantamento com o Departamento Financeiro.
Área do sistema	Gestão de Vendas
Requisitos de sistema	1. O sistema deve disponibilizar opções de filtragem obrigatórias ou opcionais para definir o período temporal (data de início e data de fim), a loja específica (ou todas as lojas da cadeia) e a categoria de produto (ou todas as categorias) . 2. O sistema deve extrair e calcular o volume total de vendas e a quantidade de artigos vendidos com base na informação que é consolidada diariamente no sistema central de dados. 3. O sistema deve impedir a geração do relatório e apresentar uma mensagem de erro caso a data de início selecionada seja cronologicamente posterior à data de fim. 4. O sistema deve apresentar uma notificação indicando a ausência de resultados caso não existam registos de vendas que correspondam aos filtros aplicados.
Relevância para a aplicação	A funcionalidade é estritamente necessária para apoiar adequadamente os processos de tomada de decisão da cadeia de lojas.

Tabela 16: Requisito funcional 16.

O requisito funcional 17 refere-se ao controlo físico e financeiro do inventário. Foi estabelecido para permitir a monitorização constante das quantidades disponíveis e do capital imobilizado nas lojas, com o intuito de prevenir ruturas de produtos críticos e planear de forma eficiente os processos de reposição.

Requisito de utilizador	O sistema deve ser capaz de gerar relatórios de inventário (e.g., valor do stock, produtos a esgotar).
Fonte	Entrevista com os Gerentes de Loja e Responsáveis de Logística.
Área do sistema	Gestão de Inventário
Requisitos de sistema	1. O sistema deve permitir a seleção de critérios de filtragem obrigatórios, nomeadamente a loja específica (ou visão global da cadeia) e o tipo de visão do relatório (valorização financeira do stock ou listagem de produtos com baixo *stock*). 2. O sistema deve calcular o valor financeiro do *stock* multiplicando a quantidade atual consolidada de cada artigo pelo seu respetivo preço de custo, com base nos dados do sistema central. 3. O sistema deve identificar os "produtos a esgotar" comparando a quantidade atual de cada artigo com um valor de "*stock* mínimo de segurança" que deve estar previamente definido na ficha de cada produto.
Relevância para a aplicação	É crucial para garantir a gestão eficiente da reposição de produtos nas lojas e evitar faixas de rutura de *stock* que prejudiquem as vendas.

Tabela 17: Requisito funcional 17.

O requisito funcional 18 aborda a monitorização global e a inteligência de negócio (*Business Intelligence*). Foi desenhado para agregar as métricas operacionais mais críticas num único painel visual, proporcionando à equipa de gestão uma visão imediata e consolidada sobre a saúde financeira e operacional de toda a cadeia `Taki`.

Requisito de utilizador	O sistema global deve fornecer um painel de controlo (dashboard) com os principais indicadores de desempenho (KPIs) da empresa.
Fonte	Entrevista com a Administração da Taki.
Área do sistema	Análise de Dados (Dashboard Central).
Requisitos de sistema	1. O sistema global deve apresentar visualmente (através de gráficos, tabelas ou cartões de resumo) métricas predefinidas, tais como o volume de vendas do dia, faturação total, e número de produtos em rutura de *stock*, baseando-se na informação consolidada no sistema central . 2. O sistema global deve permitir que os dados do *dashboard* sejam filtrados para mostrar a visão global de toda a cadeia de lojas ou a visão individualizada de uma loja específica . 3. Os indicadores devem ser calculados e atualizados automaticamente com base nos dados que são consolidados ao fim de cada dia, garantindo a consistência da informação. 4. O sistema global deve manter a estrutura visual do painel e exibir o valor zero (ou "N/A") nos indicadores aplicáveis caso não existam dados registados para a loja ou período visualizado, evitando ecrãs de erro ou falhas de carregamento da interface.
Relevância para a aplicação	Fundamental para proporcionar uma visão global e rápida do estado do negócio à equipa de gestão, apoiando diretamente o processo de tomada de decisão.

Tabela 18: Requisito funcional 18.

O requisito funcional 19 diz respeito à administração do sistema e à gestão do ciclo de vida das credenciais. Foi estipulado para assegurar que a gestão consegue manter um registo rigoroso e atualizado de todos os colaboradores ativos, garantindo a total rastreabilidade das operações no sistema e a revogação atempada de acessos quando necessário.

Requisito de utilizador	O sistema local deve permitir a criação, edição, bloqueio e remoção de contas de utilizador.
Fonte	Políticas de Segurança de Informação.
Área do sistema	Administração do Sistema (Segurança e Acessos).
Requisitos de sistema	1. O sistema local deve recolher e validar dados obrigatórios (nome, *email*, palavra-passe e loja associada) ao registar um novo utilizador ou ao atualizar os dados de uma conta existente. 2. O sistema local deve permitir a inativação temporária (bloqueio) ou a remoção lógica de uma conta, garantindo que o histórico de operações e vendas associadas a esse utilizador em cada loja autónoma se mantém intacto e rastreável. 3. O sistema local deve validar a unicidade do endereço de *email* introduzido, cancelando a gravação dos dados e exibindo uma mensagem de erro caso o *email* já pertença a outro utilizador. 4. O sistema local deve impedir que o utilizador com privilégios de administração remova ou bloqueie a sua própria conta durante a sessão ativa, exibindo um aviso de restrição de segurança.
Relevância para a aplicação	Essencial para garantir a segurança da plataforma e a responsabilização (rastreabilidade) de quem executa as operações nas lojas e no sistema central.

Tabela 19: Requisito funcional 19.

O requisito funcional 20 foca-se na segregação de funções e no controlo de privilégios dentro da plataforma. Foi definido para garantir que cada utilizador apenas visualiza e interage com as ferramentas estritamente necessárias para o exercício do seu cargo, protegendo informações sensíveis de gestão central e reduzindo o risco de acessos indevidos.

Requisito de utilizador	O sistema local deve permitir a atribuição de diferentes perfis de acesso a cada utilizador.
Fonte	Políticas de Segurança de Informação.
Área do sistema	Administração do Sistema (Segurança e Acessos).
Requisitos de sistema	1. O sistema local deve disponibilizar uma interface para configurar perfis (ex: Administrador Central, Gerente de Loja, Operador de Caixa), definindo especificamente as permissões de leitura, escrita ou eliminação para cada módulo. 2. O sistema local deve aplicar estas regras no momento do login do utilizador, restringindo a visibilidade de menus e ecrãs (ex: ocultar o painel de administração central a um operador de uma loja autónoma). 3. O sistema local deve exigir que cada conta ativa tenha, obrigatoriamente, pelo menos um perfil de acesso válido associado no momento da gravação dos dados. 4. O sistema local deve bloquear a navegação e apresentar um ecrã de "Acesso Negado" caso um utilizador tente forçar o acesso a um endereço (URL) ou serviço para o qual o seu perfil atual não tem autorização.
Relevância para a aplicação	É essencial que o sistema garanta que cada colaborador apenas executa as operações compatíveis com o seu cargo. A segregação de funções por perfil protege informação sensível de gestão central e reduz o risco de operações indevidas, como alterações de preçário ou acesso a dados financeiros consolidados por parte de operadores de caixa sem autorização para tal.

Tabela 20: Requisito funcional 20.

Requisitos Não Funcionais

Encontram-se, nesta secção, os requisitos não funcionais (RNF) do projeto do Sistema de Gestão Integrada 'Taki'.

O primeiro requisito não funcional diz respeito à segurança e ao controlo de acessos. Foi definido para assegurar que apenas pessoas autorizadas consigam interagir com áreas sensíveis do sistema, para evitar fugas de informação ou alterações indevidas.

Requisito Não Funcional	O sistema deve garantir que o utilizador autenticado apenas acede às funcionalidades permitidas pelo seu perfil.
Fonte	Reuniões com a Administração
Área do sistema	Segurança - Controlo de Acesso
Especificação e Restrições	1. O sistema deve implementar controlo de acesso baseado em papéis (*RBAC*). 2. Todas as rotas da aplicação, tanto a nível de *front-end* como *back-end* (*API*), devem validar o perfil do utilizador de forma estrita. 3. Uma tentativa de acesso não autorizado a qualquer recurso deve ser imediatamente bloqueada e registada num *log* do sistema para auditoria.
Relevância para a aplicação	Garantir a confidencialidade e a integridade da informação da cadeia de lojas, ao restringir operações de elevado impacto (como gestão de perfis, alteração de preçários) apenas às pessoas devidamente autorizadas. Cumpre com o Princípio do Menor Privilégio.

Tabela 21: Requisito não funcional 1.

O requisito que se segue foca-se na macroestrutura do sistema, para garantir a salvaguarda de toda a informação do negócio, em contexto global, e para minimizar possíveis ataques cibernéticos ou catástrofes naturais no servidor principal.

O requisito que se segue complementa as operações locais e garante a coesão da rede, para assegurar que toda a informação do negócio é consolidada no sistema central ao final do dia.

Requisito Não Funcional	O servidor local de cada loja deve exportar e transmitir os seus dados transacionais diários para a base de dados central, para fins de consolidação.
Fonte	Reuniões com a Administração
Área do sistema	Fiabilidade - Sincronização e Consolidação
Especificação e Restrições	1. A base de dados local de cada loja deve iniciar um processo de sincronização e envio dos dados transacionais (vendas, movimentos de inventário) para o servidor central a cada 24 horas, no final do dia de trabalho. 2. A transferência de dados deve garantir atomicidade, para que nenhuma informação seja parcialmente carregada em caso de quebra de ligação. 3. Caso ocorra uma falha na transferência, o sistema local deve emitir um alerta e reagendar automaticamente a sincronização.
Relevância para a aplicação	A consolidação diária é o mecanismo que alimenta a gestão central do negócio (*Business Intelligence*) e a geração de *dashboards* globais, para garantir que a Direção tem sempre acesso à performance real e atualizada da cadeia de lojas.

Tabela 22: Requisito não funcional 2.

Como o sistema operará como uma solução comercial e fiscal, o mesmo está sujeito às leis do Estado, o que torna imperativo que todas as transações, contabilidade e documentação fiscal acompanhem a respetiva legislação portuguesa.

Requisito Não Funcional	O sistema deve garantir a conformidade com as regulamentações legais e fiscais.
Fonte	Estudo de Mercado e Legislação do Retalho
Área do sistema	Conformidade Legal
Especificação e Restrições	1. O sistema deve ter a capacidade de exportar a faturação de forma regular e a pedido, através do formato *SAF-T* (PT) homologado pela Autoridade Tributária. 2. Deve contemplar mecanismos associados de assinatura digital de faturas que impossibilitem a sua adulteração após a emissão do documento.
Relevância para a aplicação	A obrigatoriedade de cumprir as obrigações fiscais dita a própria legalidade das operações nas lojas. Sistemas informáticos que não produzam relatórios de faturação de acordo com a legislação expõem gravemente a cadeia a coimas severas ou encerramento.

Tabela 23: Requisito não funcional 3.

Devido à arquitetura inerentemente distribuída do sistema (lojas e servidor central), todas as transferências de dados através da *Internet* estão sujeitas a riscos de interceção. Assim, foi estipulado o seguinte requisito para encriptar essa comunicação.

Requisito Não Funcional	O sistema deve encriptar toda a comunicação de dados entre as lojas e o servidor central.
Fonte	Estudo de Mercado de Cibersegurança
Área do sistema	Segurança - Comunicações
Especificação e Restrições	1. Todas as chamadas de sincronização de rede, de envio de vendas e validação de catálogo entre loja-servidor deverão ser feitas através do protocolo *TLS* 1.3 (ou superior). 2. A arquitetura de rede deve bloquear quaisquer tentativas de estabelecer ligação através de *HTTP* padrão (sem encriptação).
Relevância para a aplicação	Garantir que as informações de clientes, dados financeiros e credenciais de acesso não fiquem expostas ou passíveis de ser roubadas num ataque *man-in-the-middle*, para assegurar total confidencialidade da transmissão de dados sobre a rede.

Tabela 24: Requisito não funcional 4.

A *performance* de processamento na zona das caixas é vital no comércio de retalho para evitar constrangimentos com filas. Este requisito não funcional estipula um limite de tempo para transações convencionais.

Requisito Não Funcional	O sistema deve processar uma transação de venda comum em menos de 10 segundos.
Fonte	Questionários aos Operadores de Loja
Área do sistema	Eficiência - Desempenho
Especificação e Restrições	1. O sistema de *POS* deve ser desenvolvido por forma a garantir uma resposta rápida durante as operações de leitura de códigos de barras, com latência quase nula. 2. O tempo transcorrido a partir do momento em que um carrinho de 5 itens está registado até à sua comunicação para terminal e devolução do troco/emissão do recibo, deve ter uma média inferior a 10 segundos.
Relevância para a aplicação	Numa cadeia de lojas de conveniência, a fluidez de pagamentos é fulcral. Diminuir os tempos de transação reflete-se numa diminuição das filas de espera, na melhoria do serviço de atendimento ao cliente, e na maior capacidade de lucro.

Tabela 25: Requisito não funcional 5.

Dada a típica rotatividade de recursos humanos (*turnover*) no setor do retalho, a empresa quis mitigar custos com as formações, motivo pelo qual derivou o requisito abaixo, sobre os aspetos de facilidade de uso do novo sistema.

Requisito Não Funcional	O sistema deve ter uma interface suficientemente intuitiva para ser aprendida em menos de 2 horas por um novo operador.
Fonte	Questionários aos Recursos Humanos
Área do sistema	Usabilidade
Especificação e Restrições	1. As secções principais e botões de Ponto de Venda e Gestão de Inventário base devem estar visíveis com não mais de 2 ou 3 cliques a partir do ecrã inicial. 2. Toda a interface visual, botões e ações de erro e aviso devem ser desenvolvidas em linguagem clara (pt-PT) sem termos excessivamente técnicos associados ao *back-end*.
Relevância para a aplicação	O elevado nível de rotatividade (*turnover*) e as épocas altas necessitam da constante integração de novos funcionários na organização. Um *software* com excelente usabilidade reduzirá significativamente as despesas de tempo e formação inicial.

Tabela 26: Requisito não funcional 6.

A Direção requereu a disponibilidade constante de informação sobre as operações, por exigir um *SLA* elevado. Qualquer quebra dos servidores centrais impediria também as lojas de validarem clientes para certos fins e atrasaria as informações da gestão diária.

Requisito Não Funcional	O sistema central deve garantir uma disponibilidade de 99.5% durante o horário de funcionamento das lojas.
Fonte	Reuniões com a Direção Geral
Área do sistema	Fiabilidade - Disponibilidade do Sistema Central
Especificação e Restrições	1. A infraestrutura de *back-end* responsável por consolidar e dar acesso às bases de dados partilhadas deverá estar ativa e a receber *pings*/ requisições em 99.5% do tempo medido mensalmente. 2. A arquitetura distribuída garante que cada loja opera de forma completamente autónoma com base de dados local própria, pelo que uma eventual indisponibilidade do servidor central não interrompe as operações de venda nem a emissão de faturas. 3. O mecanismo de sincronização implementa um modelo de *failover* sequencial, percorrendo a lista de *endpoints* centrais configurados até obter uma resposta de sucesso (HTTP 200/201), garantindo resiliência nas comunicações entre loja e sede. 4. Excluem-se das métricas de inatividade os períodos de quebra provocados por falhas elétricas generalizadas ou fenómenos climatéricos extremos que afetem a infraestrutura.
Relevância para a aplicação	Para um processamento atempado e a disponibilidade imediata da informação à Direção e às entidades de auditoria. Dificultar atrasos entre requisições de sistema que exijam validação da central é elementar num ecossistema interligado.

Tabela 27: Requisito não funcional 7.

O último requisito não funcional foi estabelecido para salvaguardar a autenticação de cada utilizador e impedir perdas de acesso aos dados sistémicos, tal como foi sugerido pelos princípios de segurança *standard*.

Requisito Não Funcional	O sistema deve armazenar as palavras-passe dos utilizadores de forma segura.
Fonte	Estudo de Mercado e Práticas Standard
Área do sistema	Segurança - Armazenamento de Credenciais
Especificação e Restrições	1. Nenhum acesso do sistema deverá envolver a gravação, retenção ou transporte de palavras-passe nativas no banco de dados, em *plain text*. 2. A segurança da credencial deve ser feita através de algoritmos de codificação não reversíveis (*hashing* forte com *salt* dinâmico, ex: bcrypt ou Argon2).
Relevância para a aplicação	Impedir de todas as formas o acesso, pelo próprio Administrador de Base de Dados ou um invasor de sistema, aos códigos confidenciais e credenciais de uso pessoal, o que elimina o vetor de ataque mais grave em quebra de banco de dados.

Tabela 28: Requisito não funcional 8.

2.1.3. Validação dos Requisitos

A validação de requisitos consiste no processo de verificação sistemática da documentação produzida, com o intuito de garantir que os requisitos definem com exatidão o sistema que o cliente realmente necessita. De acordo com as metodologias de Engenharia de Software preconizadas, esta fase foca-se em assegurar a validade, a consistência, a completude, o realismo e a testabilidade das funcionalidades estipuladas.

Numa fase preliminar, a equipa da *SegFault* conduziu uma revisão interna minuciosa aos requisitos de utilizador e de sistema. O objetivo central desta etapa passou por detetar potenciais falhas de consistência (como requisitos contraditórios) e por avaliar a viabilidade técnica da sua implementação no prazo estipulado.

Após a resolução das anomalias internas, a equipa avançou para a validação formal com o cliente. Este passo materializou-se numa reunião presencial com a administração da *Taki*, cujo propósito assentou na revisão conjunta dos Requisitos Funcionais e Não Funcionais. A sessão permitiu confirmar o pleno alinhamento da especificação com a estratégia de negócio, com especial foco nos atributos de conformidade fiscal e de segurança. A aprovação da documentação nesta reunião estabeleceu o guião definitivo para a fase de desenho arquitetural. Apresenta-se, de seguida, o sumário da ata elaborada nesta sessão de validação:

Sistema de Gestão da Taki - Reunião de Validação
Local: Sede da Taki (Famalicão)
Duração: 1h e 30min
Data: 15 de março de 2026
Início: 10h00 | **Término:** 11h30

Participantes:
- Sr. Manuel Pinto da Costa (Dono e Gerente, **Taki**)
- Diogo Campos, João Delgado, Nelson Mendes, Simão Mendes, Tomás Machado (Equipa ***SegFault***)

Pauta da reunião:
- Leitura e revisão da lista de Requisitos Funcionais e Não Funcionais;
- Confirmação dos critérios de segurança e conformidade fiscal (**SAF-T**);
- Aprovação formal dos requisitos e autorização para o desenho arquitetural.

Figura 7: Resumo da Ata de Reunião de Validação de Requisitos. A versão integral e assinada do documento encontra-se documentada no **Anexo III**.

2.2. Análise de Requisitos

A Análise de Requisitos é uma fase fundamental no ciclo de desenvolvimento de qualquer sistema, pois atua como a ponte entre as necessidades dos *stakeholders* e a solução técnica a ser implementada. Ao garantir que os requisitos estão claros, completos e consistentes, esta prática aumenta significativamente as chances de entrega de um produto que realmente atenda às necessidades do negócio e dos usuários finais, minimizando riscos e falhas ao longo do projeto.

2.2.1. Modelo de Domínio

O Modelo de Domínio constitui uma representação estrutural e concetual das entidades que compõem o contexto de negócio do sistema. De acordo com as metodologias de Engenharia de *Software* (como as descritas por Ian Sommerville), este modelo não procura descrever pormenores de implementação técnica, métodos de programação ou a estrutura exata da base de dados. O seu propósito é, pelo contrário, capturar o vocabulário e os conceitos fundamentais partilhados entre os *stakeholders* e a equipa de desenvolvimento, para criar um entendimento comum do problema.

Metodologia de Construção

Para elaborar este modelo de forma exaustiva e rigorosa, a equipa aplicou um processo sistemático de análise textual aos artefactos previamente validados. O processo decorreu nos seguintes passos:

1. **Revisão Integral dos Requisitos:** Procedeu-se à releitura minuciosa de todos os Requisitos Funcionais e Não Funcionais documentados e aprovados para o projeto 'Taki'.
2. **Extração de Conceitos:** Durante esta análise, a equipa identificou e isolou os substantivos e os termos chave. A análise dos requisitos funcionais permitiu a extração das entidades de negócio e transacionais (como Venda, Produto, Inventário, Loja, Fornecedor e Encomenda). De forma complementar, a análise dos requisitos não funcionais revelou os conceitos ligados à infraestrutura, segurança e arquitetura física (como Servidor Central, Servidor Local e Perfil de Acesso).
3. **Mapeamento de Associações:** Por fim, a equipa observou as ações (verbos) descritas nos requisitos para traçar e justificar a forma como os vários conceitos se relacionam, o que ditou a aplicação das cardinalidades.

Diagrama do Modelo de Domínio

A figura seguinte apresenta o diagrama resultante da análise descrita anteriormente.

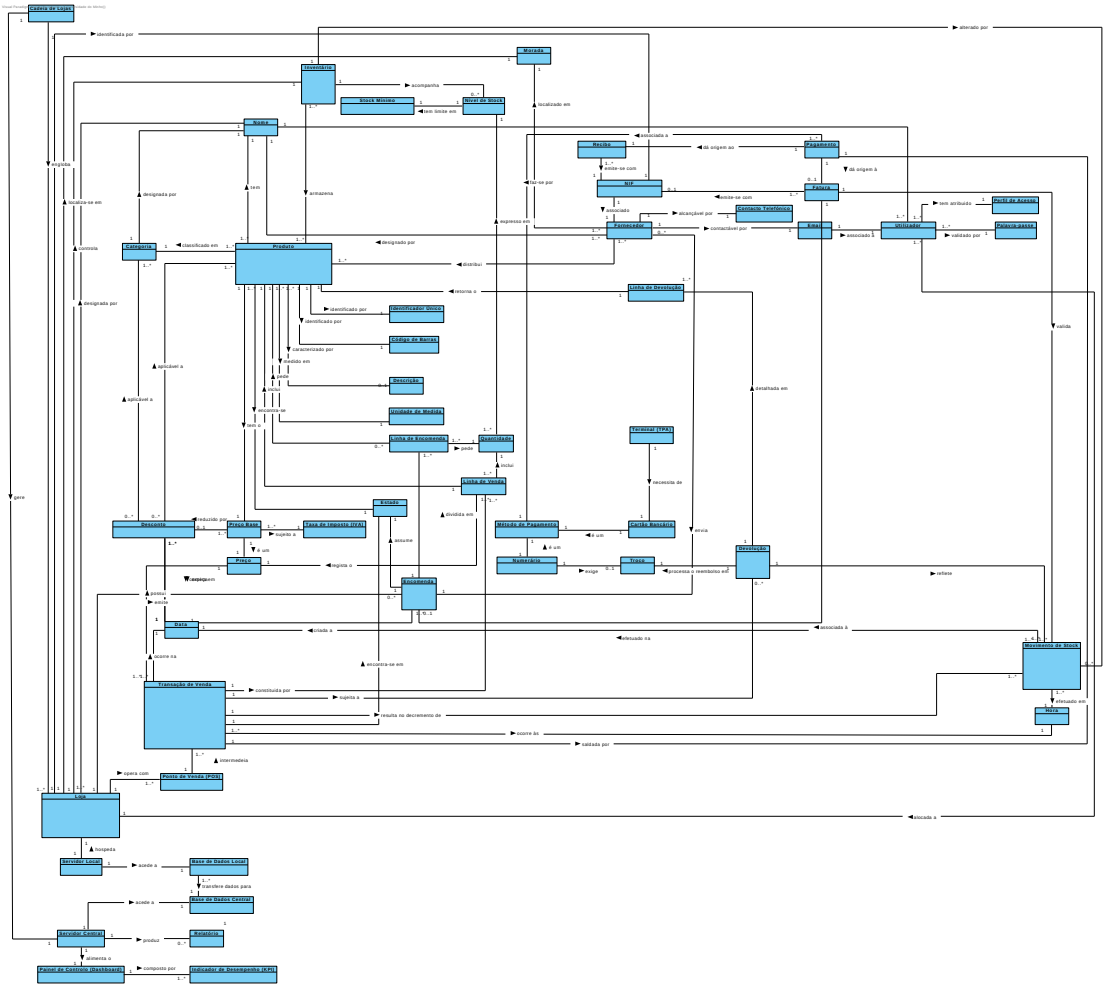


Figura 8: Modelo de Domínio

2.2.2. *User Stories*

No âmbito das **metodologias de desenvolvimento de Engenharia de *Software*** abordadas por *Ian Sommerville* [1], as ***User Stories*** (histórias de utilizador) consistem em **descrições narrativas de alto nível** que ilustram cenários práticos de utilização do sistema. Ao invés de uma especificação exaustiva, estas histórias capturam as **necessidades quotidianas** dos intervenientes e funcionam como um **contrato informal** entre a linguagem de negócio e a execução técnica.

Com base no contexto operacional da *Taki* e nas necessidades distintas de cada perfil de colaborador, a equipa estruturou os seguintes cenários para os dois papéis principais da plataforma: o Operador de Caixa (frente de loja) e o Gestor de Armazém (operações de retaguarda).

Perfil: Operador de Caixa (Frente de Loja)

- **História 1: Faturação Rápida e Atualização Automática de Inventário**

A Sofia encontra-se ao balcão durante um período de **forte afluência**. Um cliente aproxima-se com três produtos de conveniência. A Sofia utiliza a **interface minimalista do Ponto de Venda (POS)** para ler com o *scanner* os códigos de barras de cada artigo. O sistema local **calcula de imediato o subtotal** e aplica as taxas de IVA correspondentes, e exibe o valor total em poucos segundos. O cliente opta pelo pagamento através de cartão bancário. O sistema comunica ativamente com o **Terminal de Pagamento Automático (TPA)** e, ao confirmar a transação bem-sucedida, **deduz de forma autónoma os artigos vendidos** do inventário e **emite o respetivo recibo detalhado**, com a identificação sequencial e fiscal. Este fluxo rápido permite à Sofia atender prontamente o cliente seguinte.

- **História 2: Processamento de Devoluções e Reembolsos**

Um cliente dirige-se à frente de caixa para devolver um produto com defeito, e apresenta o recibo da transação efetuada na semana anterior. O operador da loja entra no **módulo de pós-venda** do sistema e introduz o número identificador do recibo. O sistema **recupera instantaneamente os detalhes originais** da venda e **valida, por via automática**, que a data da compra respeita a política de devoluções estipulada pela *Taki*. Após a seleção do produto, a plataforma **emite a correspondente nota de crédito**, calcula o **montante a reembolsar** pelo método de pagamento original e **incrementa de imediato o artigo devolvido ao stock local**. O inventário volta a apresentar a **contagem exata e atualizada**.

- **História 3: Aplicação de Descontos Promocionais em Tempo Real**

O Carlos atende um cliente que pretende adquirir diversos artigos, alguns dos quais estão abrangidos por campanhas promocionais ativas. À medida que o Carlos regista cada produto na **interface do POS**, o sistema consulta de forma automática a existência de promoções em vigor para esse item específico ou para a sua categoria. Ao detetar uma campanha válida, a plataforma **aplica o desconto mais favorável** em tempo real e apresenta o subtotal atualizado. A conclusão da venda debita os artigos e emite a fatura com o **detalhe da poupança acumulada**, o que garante total transparência sobre o benefício obtido pelo consumidor.

Perfil: Gestor de Armazém (Operações de Retaguarda)

- **História 4: Receção de Mercadoria e Registo de Entrada**

O Tiago, gestor da loja, acaba de receber uma palete de fornecimento de bebidas do seu distribuidor. Para garantir que o sistema **reflete com exatidão o *stock* real** do armazém, acede à funcionalidade de **gestão de inventário** no seu terminal. Introduz as referências dos artigos e insere as quantidades descritas na guia de remessa. O sistema **adiciona, de imediato, as novas unidades** ao valor do *stock*. De modo a **salvaguardar a rastreabilidade** da operação, a plataforma **regista esta entrada** associada à data, à hora, à referência da fatura do fornecedor e ao utilizador em funções, e **atualiza simultaneamente a disponibilidade** na frente de loja.

- **História 5: Consulta de Disponibilidade e Alerta de Rutura de *Stock***

Durante a ronda matinal, o Tiago acede ao **módulo de gestão de inventário** para verificar os níveis de produtos de alta rotação. O painel de controlo destaca, através de um **alerta visual preventivo**, que o *stock* de águas engarrafadas atingiu o limite mínimo configurado. Com base nesta informação gerada pelo sistema, o Tiago submete de imediato uma nota de encomenda ao fornecedor habitual, e **previne potenciais ruturas de *stock*** na loja.

- **História 6: Correção de Inventário por Quebra de Produto**

Ao arrumar o armazém, o Tiago depara-se com uma caixa de sumo danificada. Para manter a **integridade dos dados da plataforma**, acede ao sistema e seleciona a opção de **registo de quebra de *stock***. Insere a referência do produto, a quantidade afetada e o motivo da baixa ("Artigo danificado"). O sistema **deduz as unidades correspondentes** do inventário atual e gera um registo para efeitos de contabilidade. Esta ação assegura que a discrepância fica devidamente documentada e o sistema reflete a realidade física do espaço.

Perfil: Gestor (Administração e Gerência)

- **História 7: Exportação Mensal do Ficheiro SAF-T**

O Sr. Manuel, gerente da *Taki*, necessita de enviar os dados de faturação mensais para a contabilidade. Para o efeito, acede ao **módulo de administração e finanças** do sistema com as suas credenciais de acesso privilegiado. Na secção de conformidade fiscal, seleciona o mês transato e aciona a opção de exportação. O sistema compila todas as faturas e recibos emitidos nesse período e **gera automaticamente o ficheiro SAF-T (PT)** no formato XML regulamentar estipulado pela Autoridade Tributária. O Manuel descarrega o documento e partilha-o com o contabilista, e assegura, desta forma, o **cumprimento ágil das obrigações legais** da empresa.

- **História 8: Análise de Relatórios de Vendas e Desempenho**

No final de uma semana de campanhas promocionais, o Sr. Manuel pretende avaliar o impacto das mesmas nas receitas das lojas. Acede ao **painel de controlo (*dashboard*) analítico** e seleciona a vista de relatórios de desempenho. O sistema agrega os dados de vendas de todos os terminais em tempo real e apresenta métricas detalhadas com os produtos mais vendidos, os picos de afluência horária e a receita total. Com base nesta informação de negócio, o Manuel ajusta a estratégia de encomendas para a semana seguinte e **otimiza a rentabilidade** do espaço.

- **História 9: Gestão de Acessos e Perfis de Colaboradores**

A *Taki* acaba de contratar um novo funcionário para a frente de loja. O Sr. Manuel entra no **módulo de segurança e controlo de acessos** do sistema para criar a nova conta de utilizador. Introduz os dados do colaborador e atribui-lhe o perfil de "Operador de Caixa", o qual restringe o acesso a funcionalidades sensíveis (como a anulação de faturas sem supervisão ou a visualização de relatórios financeiros). O sistema **aplica as políticas de segurança com protocolos de cifra de topo** de imediato, e garante que o novo funcionário opera apenas com as permissões estritamente necessárias para a sua função.

Em suma, o conjunto de histórias de utilizador apresentado garante a **cobertura integral dos requisitos** validados na fase preliminar. Ao centrar a análise nas interações diárias dos três perfis principais, a equipa assegura que a futura arquitetura de software **responde de forma direta e inequívoca às expectativas do cliente**. Esta abordagem centrada no utilizador estabelece, assim, uma **base sólida para a transição** para a fase de desenho e modelação do sistema, com foco na eficiência operacional, na integridade dos dados e na estrita conformidade fiscal.

2.2.3. Casos de Uso

Esta subsecção detalha os Casos de Uso definidos para o Sistema de Gestão da `Taki`. Para cada operação vital do negócio, a equipa documentou o comportamento exato da plataforma através de tabelas estruturadas. Cada caso expõe os cenários aplicáveis, o fluxo normal de execução e os respetivos desvios (fluxos alternativos e de exceção). O objetivo desta modelação passa por estabelecer um guião técnico definitivo e claro, pronto para orientar a fase de implementação da solução.

Use Case	**Registar Venda**
Descrição	O operador regista os artigos a adquirir pelo cliente, processa o pagamento (numerário ou cartão) e emite o respetivo recibo, de modo a atualizar o inventário no final da transação.
Cenários	Um cliente dirige-se à caixa com os artigos físicos que pretende comprar e efetua o pagamento no local.
Pré-Condição	O sistema local encontra-se operacional e o utilizador tem a sessão iniciada com o perfil de vendedor/operador de caixa.
Pós-condição	A transação é registada com um identificador único, o pagamento é validado, o stock dos artigos é deduzido de forma atómica e o recibo (físico ou digital) é emitido.
Fluxo Normal	1. O operador inicia uma nova transação de venda. 2. O operador adiciona um produto através da leitura do respetivo código de barras. 3. O sistema calcula o subtotal e aplica automaticamente as taxas legais em vigor (ex.: IVA). 4. O operador verifica se existem mais produtos. 5. Não existem mais produtos a passar. 6. O operador indica o fim da introdução de artigos. 7. O sistema exibe o valor total final a pagar. 8. O cliente opta por pagar em numerário. 9. O operador insere o montante entregue pelo cliente. 10. O sistema calcula automaticamente o troco a devolver. 11. O sistema regista a venda e deduz, de forma atómica, as quantidades no stock local. 12. O sistema emite um recibo detalhado sequencial e solicita o formato de emissão. 13. O operador seleciona a impressão física. 14. O sistema imprime o documento na impressora de talões. 15. O operador entrega os artigos, o troco e o recibo ao cliente.
Fluxo Alternativo	**(1) [Erro na leitura do código de barras] (passo 2)** 2.1 O scanner não consegue ler a etiqueta do produto ou a mesma encontra-se danificada. 2.2 O operador seleciona a opção de introdução manual. 2.3 O operador insere o código numérico do artigo ou pesquisa pelo seu nome. 2.4 O sistema identifica o artigo e adiciona-o à transação. 2.5 O fluxo regressa ao passo 3. **(2) [Existem mais produtos] (passo 4)** 4.1 O operador passa o produto no scanner 4.2 Passo 4 **(3) [Cliente opta por pagamento eletrónico] (passo 8)** 8.1 O cliente informa que pretende pagar com cartão bancário ou dispositivo móvel. 8.2 O sistema comunica o valor a pagar ao Terminal de Pagamento Automático (TPA). 8.3 O cliente interage com o TPA para validar o pagamento.

	8.4 O TPA processa a transação com sucesso e informa o sistema local. 8.5 Passo 11. **(4) [Cliente solicita recibo em formato digital] (passo 13)** 13.1 O cliente informa que não deseja o papel e prefere receber o comprovativo por correio eletrónico. 13.2 O operador seleciona a opção correspondente e introduz o endereço de e-mail fornecido pelo cliente. 13.3 O sistema procede ao envio do documento digital. 13.4 Passo 15.
Fluxos de Exceção	**(1) [Cartão rejeitado pelo TPA ou falha de rede] (passo 8)** 8.1.1 O TPA devolve um erro de transação (ex.: PIN incorreto, falta de saldo, ou anomalia de comunicação). 8.1.2 O sistema notifica o operador de que o pagamento falhou, com a exibição do motivo retornado pelo TPA. **(2) [Cliente desiste da compra] (passo 8)** 8.2.1 O cliente decide abandonar a compra antes de finalizar o pagamento. 8.2.2 O operador seleciona a opção de cancelar transação no ecrã. 8.2.3 O sistema anula a operação e descarta os dados (quantidades em stock não sofrem alteração).

Tabela 29: Caso de Uso: Registar Venda

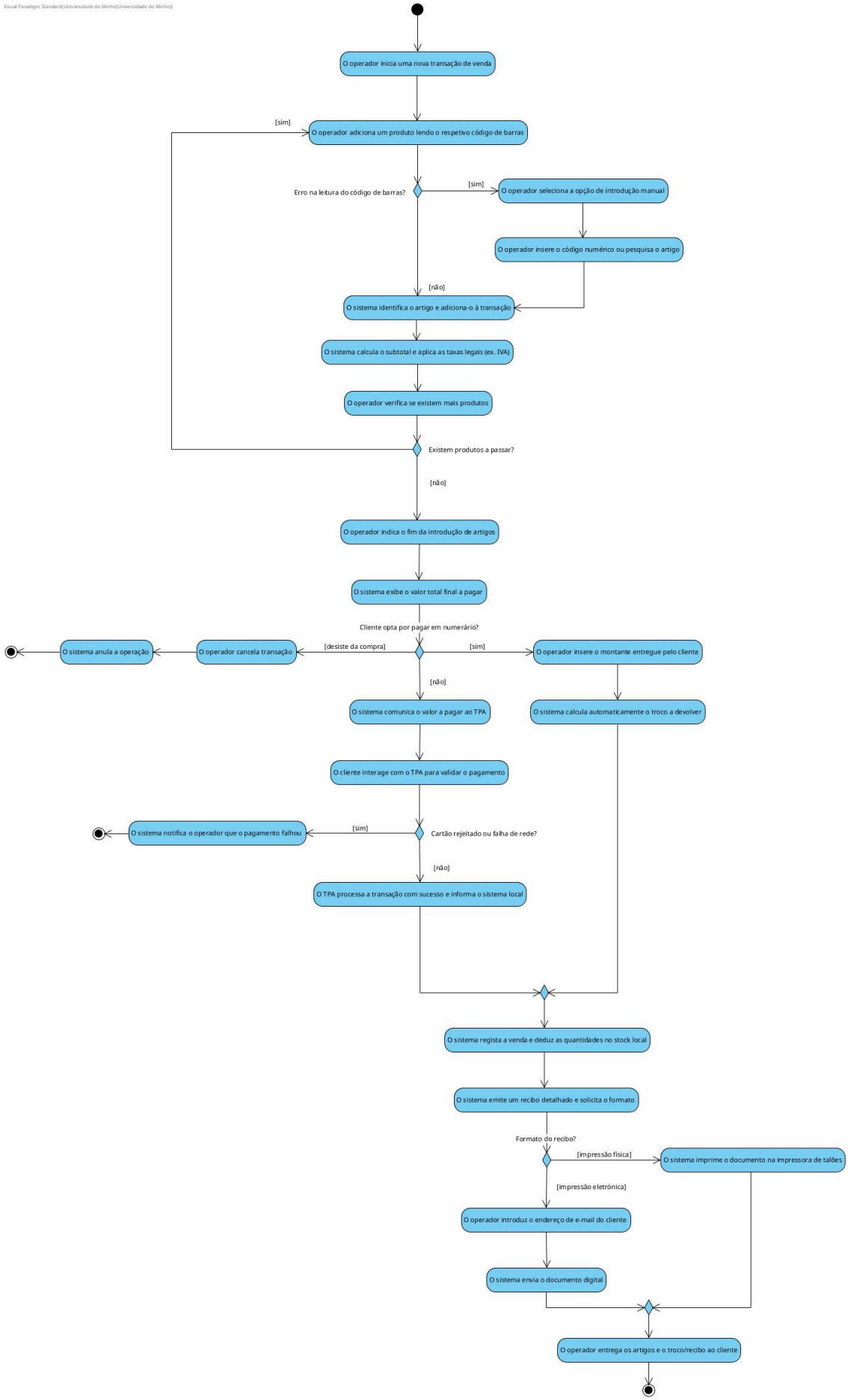


Figura 9: Diagrama de Atividades do Caso de Uso: Registar Venda

Use Case	Processar Devolução
Descrição	O operador regista a devolução de artigos, com a realização do reembolso ao cliente e a reposição do stock no inventário local.
Cenários	Um cliente dirige-se à loja para devolver produtos de uma compra anterior e apresenta o respetivo recibo/fatura.
Pré-Condição	O sistema local encontra-se operacional e o operador tem a sessão iniciada com permissões para processar devoluções.
Pós-condição	O comprovativo de devolução é emitido, o valor do reembolso é registado e o stock dos artigos devolvidos é incrementado.
Fluxo Normal	1. O operador introduz o número da fatura no sistema local. 2. O sistema identifica a transação e apresenta os artigos adquiridos. 3. O operador seleciona um produto a devolver e a respetiva quantidade. 4. O operador verifica se existem mais produtos a devolver. 5. Não existem mais produtos a devolver. 6. O sistema valida automaticamente se a devolução se encontra dentro do prazo estipulado pela política da loja. 7. O sistema calcula o montante total a reembolsar. 8. O operador regista o reembolso no método de pagamento original do cliente. 9. O sistema emite o comprovativo de devolução . 10. O sistema atualiza automaticamente o inventário, com o incremento do stock local dos artigos devolvidos.
Fluxo Alternativo	**(1) [Existem mais produtos a devolver] (passo 4)** 4.1 O operador seleciona outro produto a devolver e a respetiva quantidade. 4.2 Passo 4. **(2) [Método de pagamento original indisponível] (passo 8)** 8.1 O estorno no método original não é possível (ex.: cartão expirado ou falta de numerário na caixa). 8.2 O operador seleciona um método de reembolso alternativo no sistema, em acordo com o cliente. 8.3 Passo 9.
Fluxos de Exceção	**(1) [Fatura não encontrada] (passo 2)** 2.1 O sistema não encontra nenhum registo associado ao número introduzido. 2.2 O sistema exibe a mensagem de erro: “Fatura não encontrada”. **(2) [Prazo de devolução excedido] (passo 6)** 6.1 O sistema deteta que a compra ocorreu há mais tempo do que o permitido pela política da loja. 6.2 O sistema bloqueia a devolução e exibe uma mensagem de erro que indica o limite de prazo. 6.3 O operador informa o cliente de que a devolução não é autorizada. 6.4 O sistema anula a operação.

Tabela 30: Caso de Uso: Processar Devolução

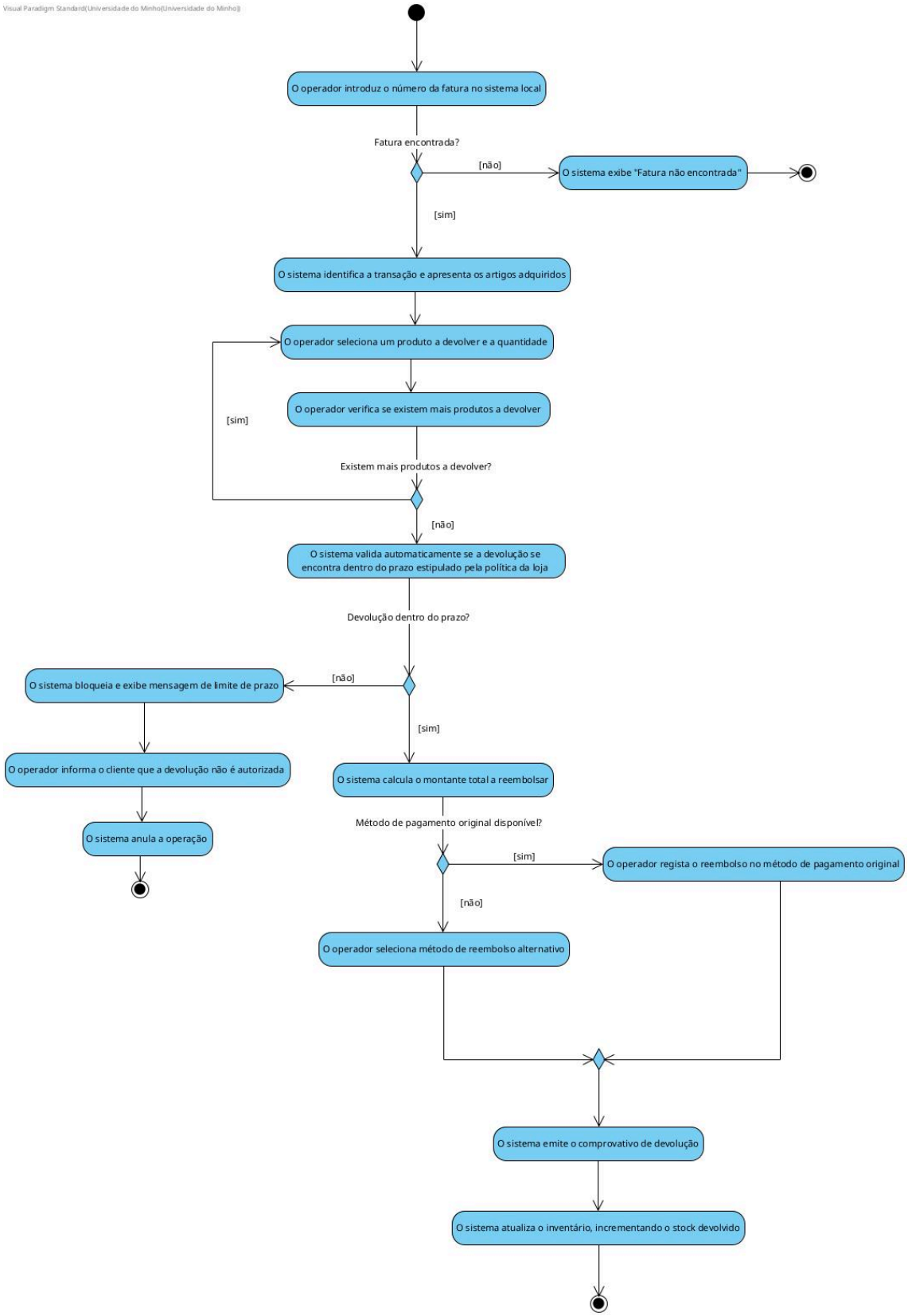


Figura 10: Diagrama de Atividades do Caso de Uso: Processar Devolução

Use Case	**Registar Entrada de Mercadoria**
Descrição	O responsável atualiza manualmente o stock da loja para registar a receção de novas entregas por parte dos fornecedores.
Cenários	Um fornecedor entrega mercadoria na loja e o utilizador responsável regista a entrada dos artigos no sistema.
Pré-Condição	O sistema local encontra-se operacional e o utilizador tem a sessão iniciada with permissões de gestão de inventário.
Pós-condição	A entrada de mercadoria é registada com os dados da fatura do fornecedor e as quantidades dos artigos são incrementadas no stock local.
Fluxo Normal	1. O utilizador acede à interface de receção de mercadorias. 2. O utilizador insere a referência do artigo recebido. 3. O sistema identifica o artigo no catálogo local. 4. O utilizador introduz a quantidade recebida. 5. O utilizador verifica se existem mais artigos a registar na entrega. 6. Não existem mais artigos a registar. 7. O utilizador insere os dados da fatura do fornecedor associada à entrega. 8. O utilizador confirma o registo da entrada de mercadoria. 9. O sistema adiciona automaticamente as novas quantidades ao stock disponível para venda. 10. O sistema guarda o registo da operação com a associação de data, hora, utilizador responsável e fatura.
Fluxo Alternativo	**(1) [Existem mais artigos a registar] (passo 5)** 5.1 O utilizador insere a referência de outro artigo recebido e a respetiva quantidade. 5.2 Passo 5.
Fluxos de Exceção	**(1) [Quantidade inválida] (passo 4)** 4.1 O sistema deteta que a quantidade introduzida é inválida (ex.: valor negativo ou igual a zero). 4.2 O sistema bloqueia a ação e exibe uma mensagem de erro que indica que o valor deve ser superior a zero. **(2) [Campos obrigatórios em falta] (passo 8)** 8.1 O sistema deteta que o campo correspondente à fatura do fornecedor se encontra vazio. 8.2 O sistema realça o campo em falta e apresenta a mensagem de erro para o preencher.

Tabela 31: Caso de Uso: Registar Entrada de Mercadoria

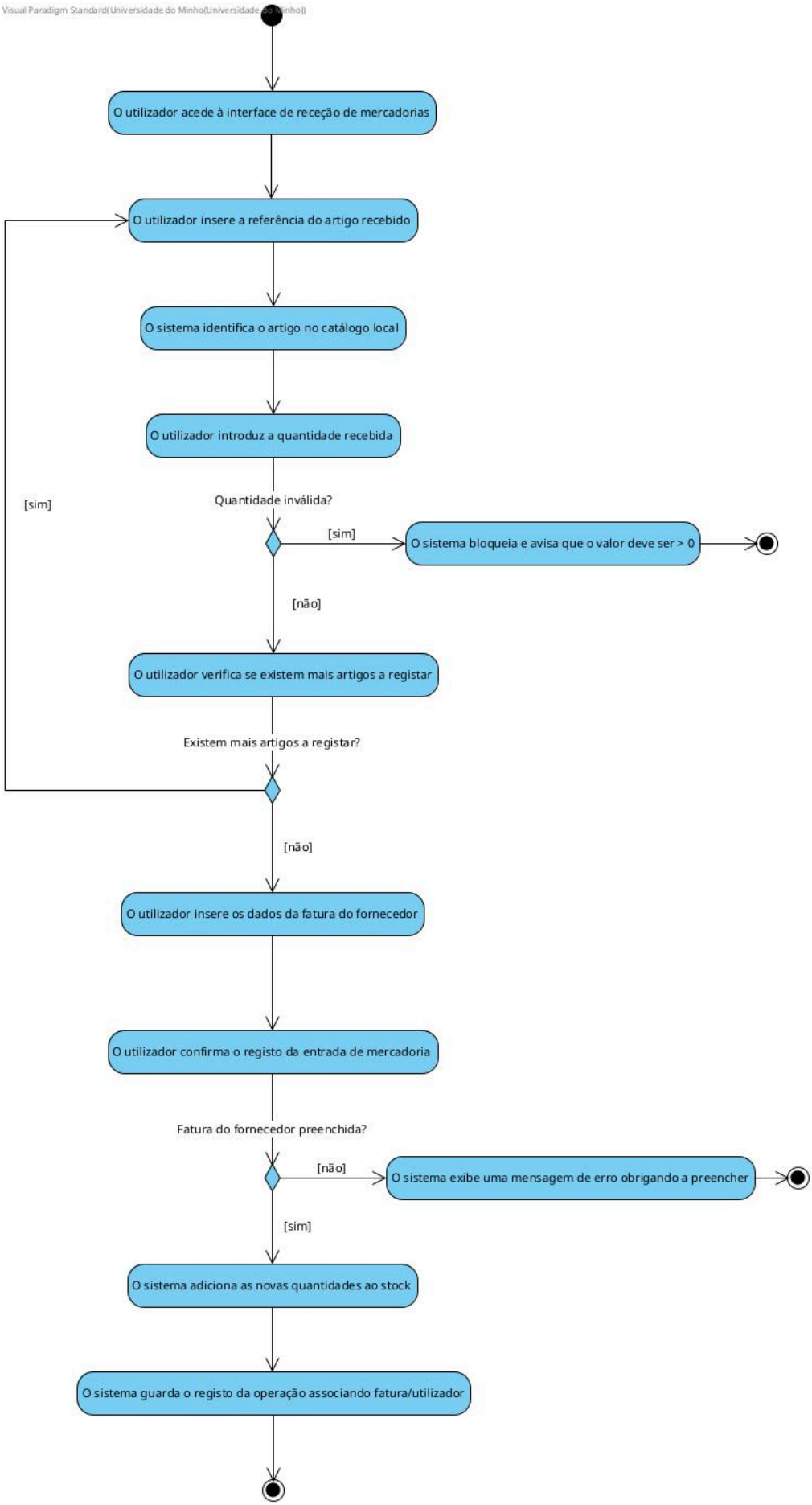


Figura 11: Diagrama de Atividades do Caso de Uso: Registar Entrada de Mercadoria

Use Case	**Realizar Ajuste de Inventário**
Descrição	O responsável realiza ajustes manuais ao inventário local para corrigir discrepâncias físicas (ex.: quebras, furtos, erros de contagem).
Cenários	O gestor de loja efetua uma contagem física e deteta que a quantidade na prateleira não corresponde à quantidade no sistema, para proceder à sua correção.
Pré-Condição	O sistema local encontra-se operacional e o utilizador tem a sessão iniciada com privilégios de gestão ou auditoria.
Pós-condição	A quantidade em stock é atualizada em tempo real com o valor físico e é criado um registo de auditoria com a justificação.
Fluxo Normal	1. O utilizador acede à funcionalidade de ajuste de stock. 2. O utilizador pesquisa o artigo a ajustar através da sua referência. 3. O sistema identifica o artigo e apresenta a quantidade atual em stock. 4. O utilizador introduz a nova quantidade física real do artigo. 5. O utilizador seleciona obrigatoriamente um motivo para o ajuste (ex.: quebra, furto, erro de contagem, validade expirada). 6. O utilizador verifica se existem mais artigos a ajustar. 7. Não existem mais artigos a ajustar. 8. O utilizador confirma a gravação dos ajustes. 9. O sistema calcula automaticamente a diferença entre o stock no sistema e a nova quantidade. 10. O sistema atualiza o valor no inventário em tempo real. 11. O sistema cria um registo do ajuste (com o registo de data, hora, utilizador, quantidades e motivo).
Fluxo Alternativo	**(1) [Existem mais artigos a ajustar] (passo 6)** 6.1 O utilizador pesquisa a referência de outro artigo a ajustar, introduz a nova quantidade e o respetivo motivo. 6.2 Passo 6.
Fluxos de Exceção	**(1) [Artigo não encontrado] (passo 2)** 2.1 O sistema deteta que a referência pesquisada não existe na base de dados do catálogo local. 2.2 O sistema exibe a mensagem de erro: “Artigo não encontrado”. **(2) [Quantidade física inválida] (passo 4)** 4.1 O sistema deteta que a quantidade introduzida possui um formato inválido ou negativo. 4.2 O sistema bloqueia a ação e exibe uma mensagem de erro indicando que o valor deve ser um número inteiro válido (zero ou superior). **(3) [Campos obrigatórios em falta] (passo 8)** 8.1 O sistema deteta que o motivo do ajuste não foi selecionado para um ou mais artigos. 8.2 O sistema realça o campo em falta e apresenta um aviso ao utilizador para o preencher.

Tabela 32: Caso de Uso: Realizar Ajuste de Inventário

Figura 12: Diagrama de Atividades do Caso de Uso: Realizar Ajuste de Inventário

Use Case	Consultar Stock
Descrição	O utilizador acede à interface de inventário para visualizar, pesquisar e filtrar os níveis de stock atualizados dos produtos em tempo real.
Cenários	Um operador ou gestor de loja necessita de verificar a disponibilidade de um artigo específico para um cliente ou listar produtos com stock reduzido.
Pré-Condição	O sistema encontra-se operacional e o utilizador tem a sessão iniciada com permissões para consulta de inventário.
Pós-condição	O sistema apresenta a informação de stock pretendida com base nos critérios aplicados pelo utilizador.
Fluxo Normal	1. O utilizador acede à funcionalidade de consulta de stock. 2. O sistema apresenta a listagem global de produtos com os respetivos níveis de stock em tempo real. 3. O utilizador verifica se pretende aplicar filtros de pesquisa. 4. Não pretende aplicar filtros de pesquisa. 5. O utilizador analisa a informação de inventário apresentada no ecrã. 6. O utilizador encerra a interface de consulta de stock.
Fluxo Alternativo	**(1) [Aplicar filtros ou pesquisa] (passo 3)** 3.1 O utilizador introduz critérios de pesquisa (ex.: nome do produto, referência) ou aplica filtros específicos (ex.: categoria, quantidade inferior a X unidades). 3.2 O sistema atualiza a listagem de produtos de forma a apresentar apenas os resultados que correspondem aos critérios definidos. 3.3 Passo 5. **(2) [Exportar dados de stock] (passo 5)** 5.1 O utilizador seleciona a opção de exportar a listagem atual de stock. 5.2 O sistema gera e inicia a transferência de um ficheiro com os dados estruturados (ex.: CSV ou JSON). 5.3 Passo 5.
Fluxos de Exceção	**(1) [Nenhum resultado encontrado] (passo 3)** 3.1 O sistema deteta que não existem produtos no catálogo que correspondam aos filtros de pesquisa aplicados (ex.: referência inexistente ou nenhuma quantidade abaixo do limite). 3.2 O sistema exibe uma listagem vazia acompanhada da mensagem de aviso: “Nenhum produto encontrado para os critérios selecionados”.

Tabela 33: Caso de Uso: Consultar Stock

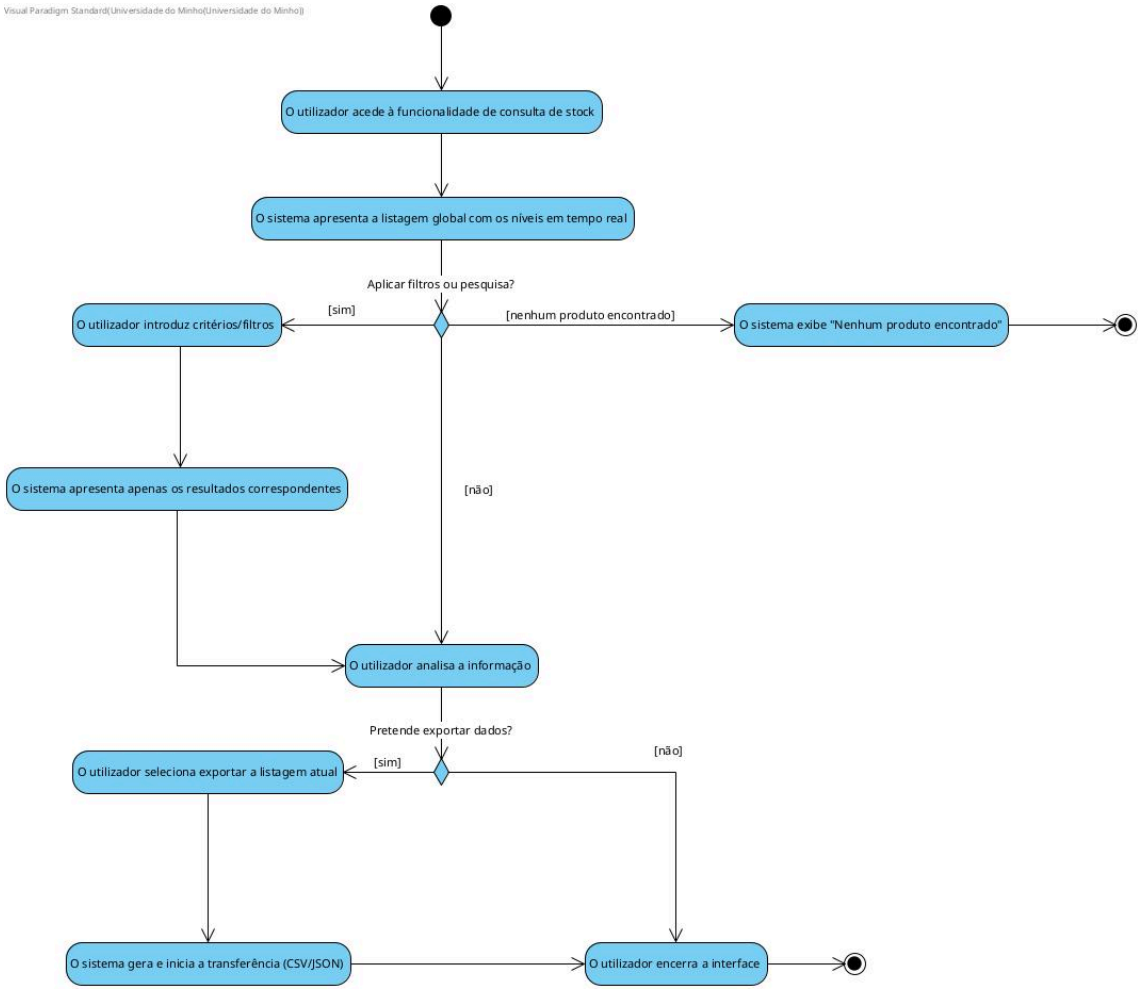


Figura 13: Diagrama de Atividades do Caso de Uso: Consultar Stock

Use Case	Criar Encomenda a Fornecedor
Descrição	O utilizador cria uma nova encomenda a um fornecedor, selecionando os artigos, quantidades e preços, e o sistema gere o estado da mesma (Rascunho, Pendente ou Enviada) consoante o preenchimento.
Cenários	O gestor de loja necessita de repor o inventário e regista o pedido de novos produtos a um fornecedor específico.
Pré-Condição	O sistema local encontra-se operacional e o utilizador tem a sessão iniciada com permissões de gestão de inventário.
Pós-condição	A encomenda é registada no sistema com o estado correspondente (Rascunho, Pendente ou Enviada), associando a data, hora e o utilizador responsável.
Fluxo Normal	1. O utilizador acede ao formulário de criação de encomendas a fornecedores. 2. O utilizador seleciona o fornecedor pretendido, a data de entrega e a loja de destino. 3. O sistema valida se o fornecedor selecionado se encontra ativo no cadastro. 4. O utilizador pesquisa e seleciona um produto a incluir na encomenda. 5. O utilizador introduz a quantidade pretendida, o preço unitário e as taxas aplicáveis. 6. O utilizador verifica se existem mais produtos a incluir. 7. Não existem mais produtos a incluir. 8. O sistema calcula automaticamente o valor total da encomenda. 9. O utilizador seleciona a opção de concluir e submeter a encomenda. 10. O sistema verifica que todos os dados necessários se encontram preenchidos. 11. O sistema guarda a encomenda no estado “Enviada” e regista a operação.
Fluxo Alternativo	**(1) [Existem mais produtos a incluir] (passo 6)** 6.1 O utilizador pesquisa e seleciona outro produto, indicando a quantidade, preço e impostos. 6.2 Passo 6. **(2) [Utilizador não conclui a encomenda] (passo 9)** 9.1 O utilizador decide não concluir a encomenda no momento (ex.: seleciona guardar progresso ou tenta sair da página). 9.2 O sistema guarda os dados introduzidos com o estado “Rascunho”. 9.3 O caso de uso termina. **(3) [Informação em falta na submissão] (passo 10)** 10.1 O sistema deteta que a encomenda submetida tem algo em falta (ex.: aguarda validação de preço ou dados adicionais). 10.2 O sistema guarda a encomenda no estado “Pendente”. 10.3 O caso de uso termina.
Fluxos de Exceção	**(1) [Fornecedor inativo] (passo 3)** 3.1 O sistema deteta que o fornecedor selecionado se encontra descontinuado ou inativo. 3.2 O sistema bloqueia a seleção e exibe a mensagem de erro: “Fornecedor inativo. Selecione um fornecedor válido”. **(2) [Quantidade ou preço inválidos] (passo 5)** 5.1 O sistema deteta que a quantidade ou o preço unitário introduzidos são inválidos (ex.: zero ou valores negativos). 5.2 O sistema exibe um aviso indicando que os valores devem ser superiores a zero.

Tabela 34: Caso de Uso: Criar Encomenda a Fornecedor

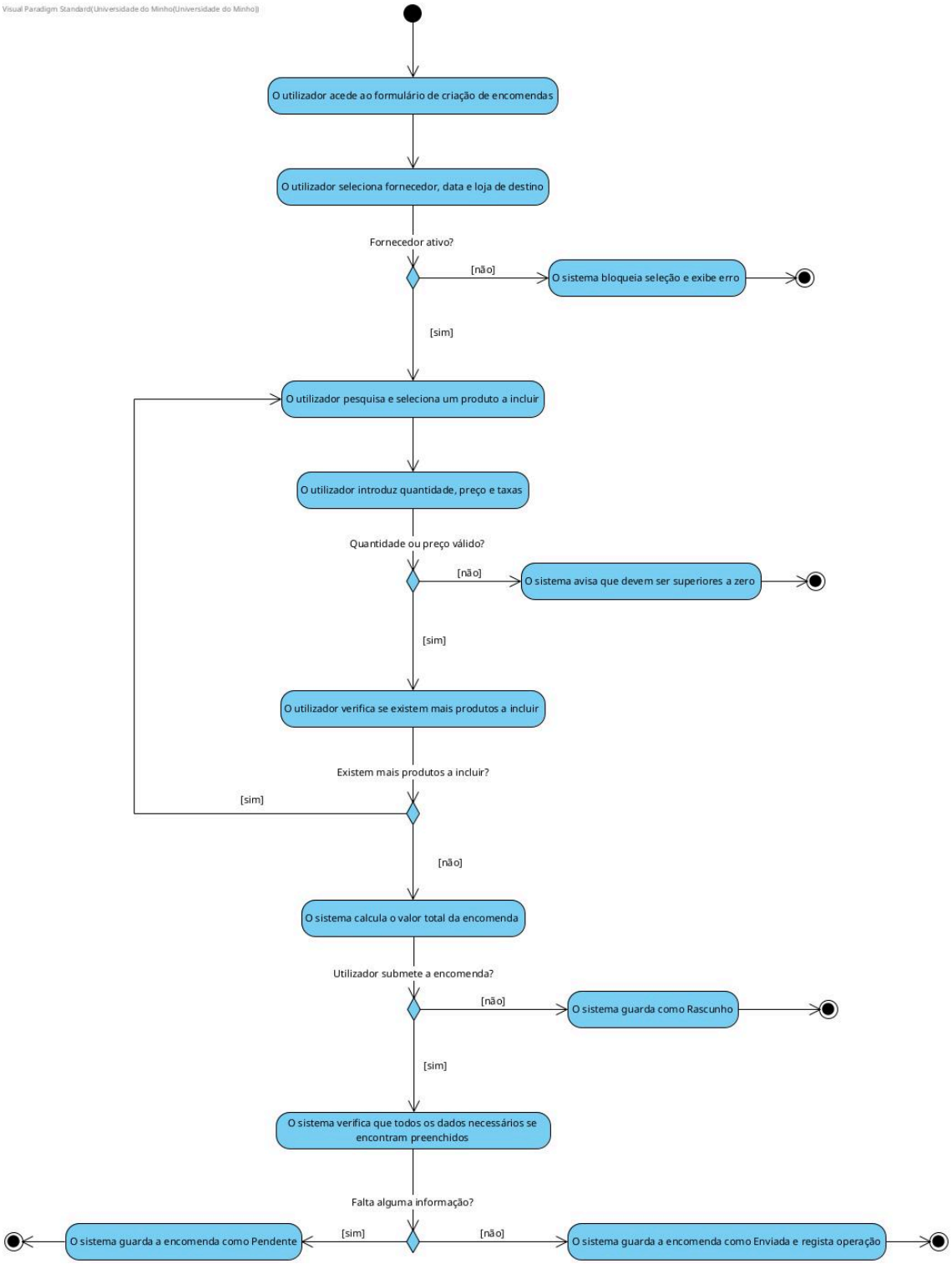


Figura 14: Diagrama de Atividades do Caso de Uso: Criar Encomenda a Fornecedor

Use Case	**Configurar Alerta de Stock**
Descrição	O utilizador define a quantidade mínima de stock para um produto, estabelecendo o alerta que será ativado quando o inventário atingir esse valor.
Cenários	O gestor de loja pretende ser notificado quando um produto estiver quase a esgotar e introduz a quantidade mínima que aciona o alerta.
Pré-Condição	O sistema local encontra-se operacional e o utilizador tem a sessão iniciada com permissões de gestão de inventário.
Pós-condição	A quantidade mínima do produto é guardada, com funcionamento automático como o alerta de stock baixo no sistema.
Fluxo Normal	1. O utilizador acede à interface de alertas de stock. 2. O utilizador pesquisa e seleciona o produto pretendido. 3. O sistema apresenta a informação do produto e o valor atual do alerta (nível mínimo de stock). 4. O utilizador introduz a nova quantidade mínima pretendida. 5. O utilizador verifica se existem mais produtos a configurar. 6. Não existem mais produtos a configurar. 7. O utilizador confirma a gravação. 8. O sistema guarda a nova quantidade mínima, que passa a servir de alerta automático para esse produto.
Fluxo Alternativo	**(1) [Existem mais produtos a configurar] (passo 5)** 5.1 O utilizador pesquisa e seleciona outro produto, com a introdução da respetiva quantidade mínima para o alerta. 5.2 Passo 5. **(2) [Remoção do alerta] (passo 4)** 4.1 O utilizador decide que o produto já não necessita de alerta de stock. 4.2 O utilizador apaga o valor existente ou introduz o valor zero. 4.3 Passo 5.
Fluxos de Exceção	**(1) [Produto não encontrado] (passo 2)** 2.1 O sistema deteta que a referência pesquisada não corresponde a nenhum produto no catálogo. 2.2 O sistema exibe a mensagem de erro: “Produto não encontrado”. **(2) [Quantidade inválida] (passo 7)** 7.1 O sistema deteta que o valor introduzido possui um formato inválido (ex.: número negativo). 7.2 O sistema bloqueia a gravação e apresenta um aviso para corrigir o valor assinalado.

Tabela 35: Caso de Uso: Configurar Alerta de Stock

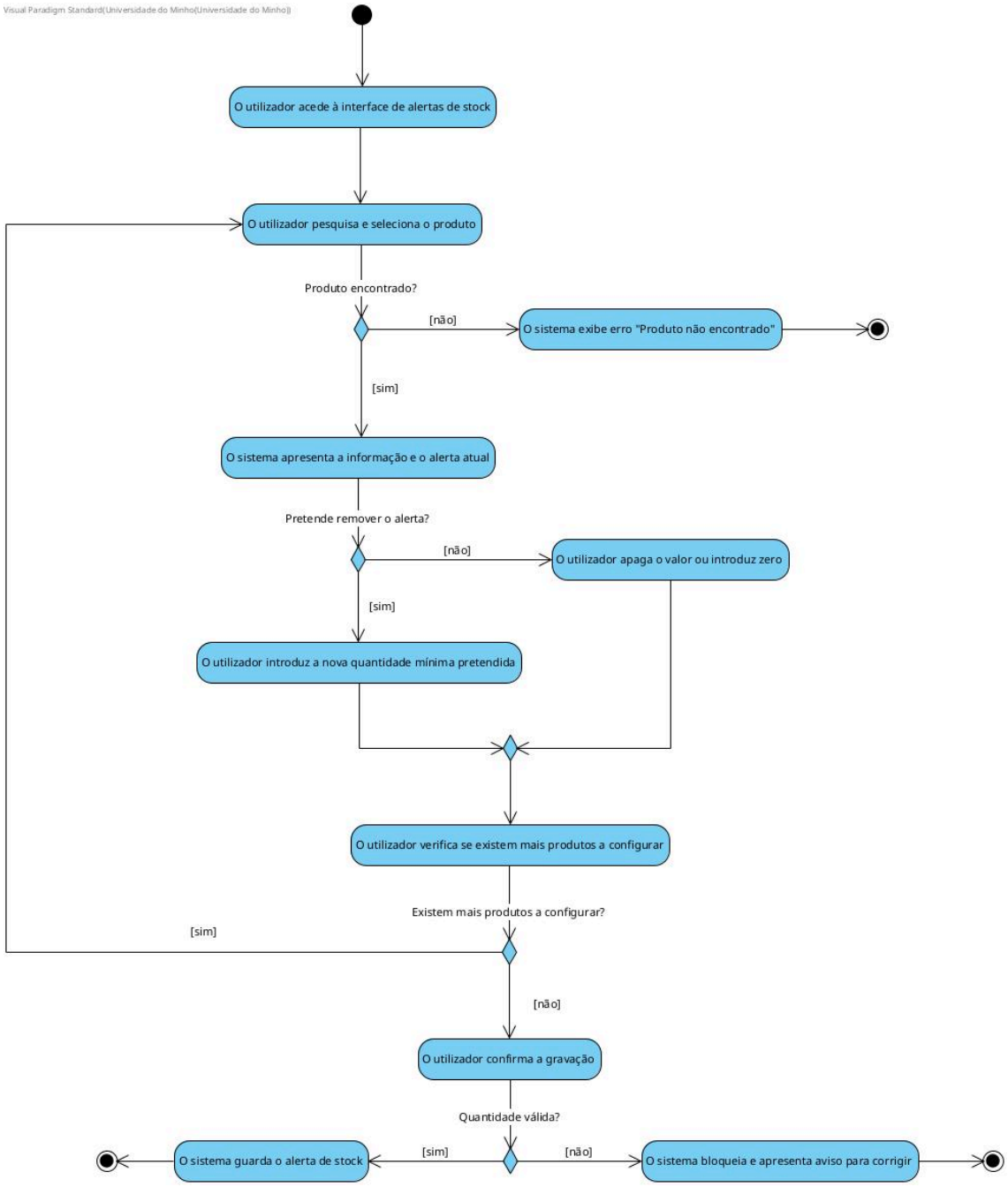


Figura 15: Diagrama de Atividades do Caso de Uso: Configurar Alerta de Stock

Use Case	**Consultar Alertas de Stock**
Descrição	O utilizador acede ao sistema para visualizar a listagem de avisos automáticos gerados para produtos com stock reduzido.
Cenários	O gestor de loja verifica diariamente as notificações do sistema para identificar quais os produtos que necessitam de reposição imediata.
Pré-Condição	O sistema local encontra-se operacional e o utilizador tem a sessão iniciada com permissões de gestão de inventário.
Pós-condição	A listagem atualizada de alertas ativos é apresentada ao utilizador.
Fluxo Normal	1. O utilizador acede à interface de alertas de stock. 2. O sistema verifica os níveis de inventário através da comparação dos mesmos com os limites configurados. 3. O sistema apresenta a listagem de alertas ativos (produtos com stock igual ou inferior ao nível mínimo). 4. O utilizador analisa a informação apresentada. 5. O utilizador encerra a interface de alertas de stock.
Fluxo Alternativo	**(1) [Filtrar alertas ativos] (passo 4)** 4.1 O utilizador aplica um filtro à listagem (ex.: filtrar por categoria). 4.2 O sistema atualiza a vista para exibir apenas os alertas correspondentes ao filtro. 4.3 Passo 4.
Fluxos de Exceção	**(1) [Nenhum alerta ativo no sistema] (passo 3)** 3.1.1 O sistema deteta que nenhum produto se encontra com stock igual ou abaixo do respetivo nível de segurança. 3.1.2 O sistema exibe uma listagem vazia acompanhada da mensagem de aviso: “Não existem alertas de stock ativos”.

Tabela 36: Caso de Uso: Consultar Alertas de Stock

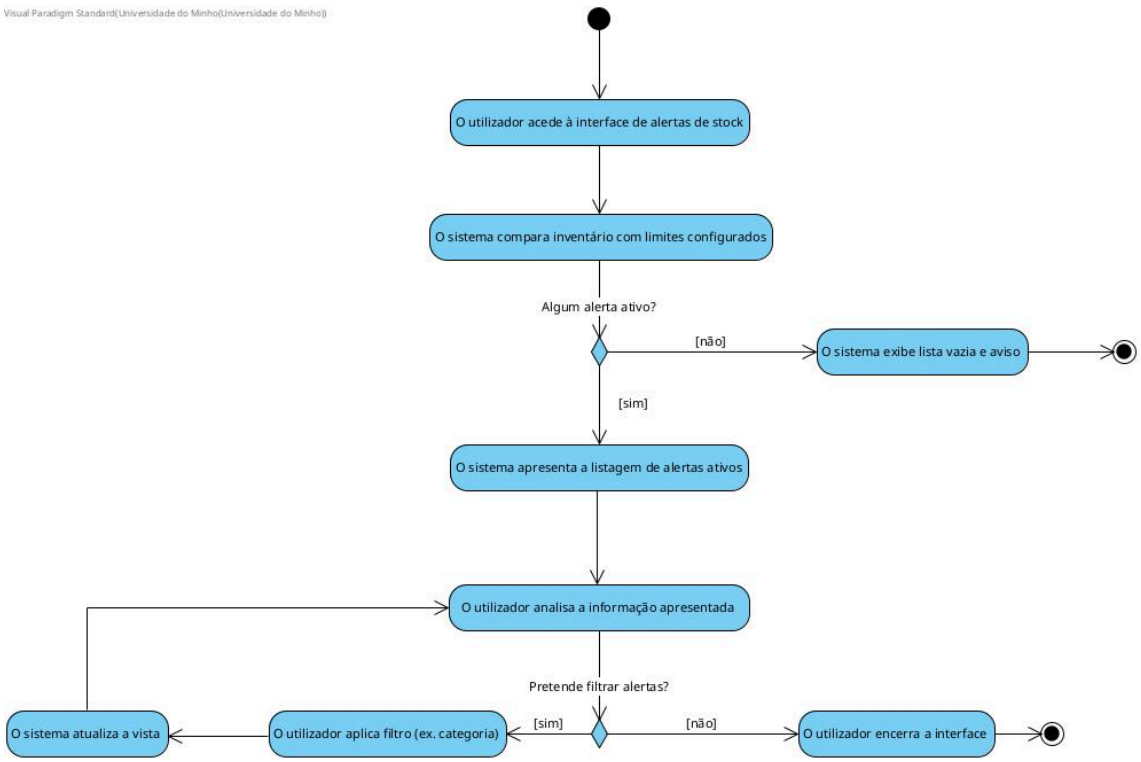


Figura 16: Diagrama de Atividades do Caso de Uso: Consultar Alertas de Stock

Use Case	Adicionar Produto
Descrição	O utilizador cria um novo registo de produto no catálogo local da loja, introduzindo toda a informação necessária para a sua comercialização.
Cenários	A loja passa a comercializar um novo artigo e o gestor necessita de o adicionar ao sistema para que possa ser lido na caixa e encomendado a fornecedores.
Pré-Condição	O sistema local encontra-se operacional e o utilizador tem a sessão iniciada com permissões de gestão de catálogo.
Pós-condição	O novo produto é guardado no catálogo local com o estado inicial 'Ativo', com o registo da data de criação e do utilizador responsável.
Fluxo Normal	1. O utilizador acede à funcionalidade de gestão de catálogo de produtos. 2. O utilizador seleciona a opção para adicionar um novo produto. 3. O sistema disponibiliza o formulário de criação em branco. 4. O utilizador introduz os dados obrigatórios do produto(nome, descrição, categoria, preço de venda sugerido e unidade de medida). 5. O utilizador confirma a gravação do novo registo. 6. O sistema valida os dados inseridos. 7. O sistema guarda o produto na base de dados local com o estado “Ativo”. 8. O sistema exibe uma mensagem de sucesso e atualiza a listagem do catálogo.
Fluxo Alternativo	**(1) [Cancelar criação de produto] (passo 4)** 4.1 O utilizador decide cancelar a operação antes de gravar. 4.2 O sistema descarta todas as informações introduzidas no formulário. 4.3 Passo 1.
Fluxos de Exceção	**(2) [Campos obrigatórios em falta] (passo 6)** 6.1.1 O sistema deteta que um ou mais campos obrigatórios se encontram vazios. 6.1.2 O sistema realça os campos em falta e apresenta um aviso ao utilizador para os preencher. **(3) [Formato de dados inválido] (passo 6)** 6.2.1 O sistema deteta que o formato do preço inserido é inválido. 6.2.2 O sistema bloqueia a gravação e apresenta uma mensagem de erro.

Tabela 37: Caso de Uso: Adicionar Produto

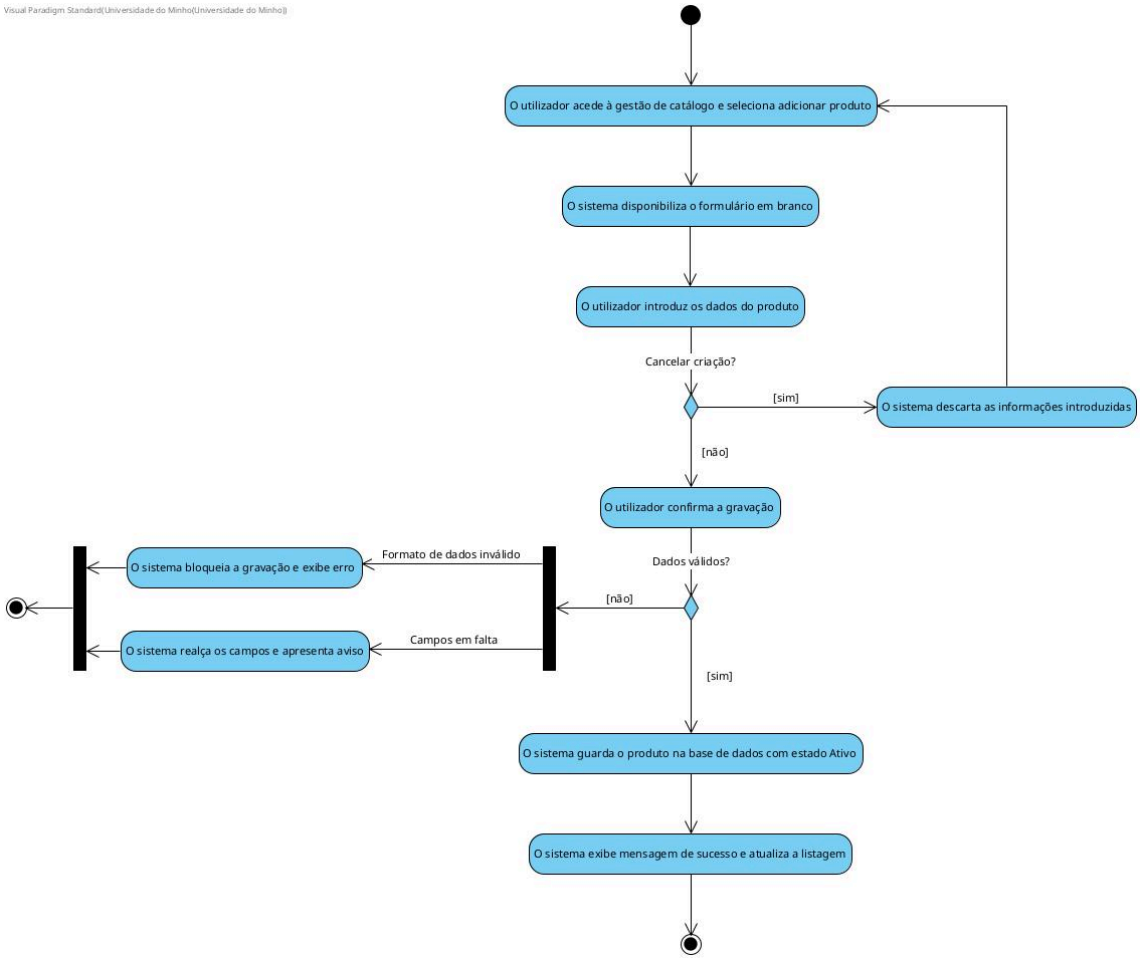


Figura 17: Diagrama de Atividades do Caso de Uso: Adicionar Produto

Use Case	Editar Produto
Descrição	O utilizador altera os dados de um produto existente no catálogo local, com o registo da operação no sistema para efeitos de auditoria.
Cenários	O gestor de loja necessita de atualizar o preço de venda de um artigo devido a uma alteração de preço e modifica o respetivo registo no sistema.
Pré-Condição	O sistema local encontra-se operacional, o utilizador tem a sessão iniciada com permissões de gestão de catálogo e o produto já existe na base de dados.
Pós-condição	Os dados do produto são atualizados no catálogo local e o sistema guarda a data da alteração e a identificação do utilizador responsável.
Fluxo Normal	1. O utilizador acede à funcionalidade de gestão de catálogo de produtos. 2. O utilizador pesquisa e seleciona o produto que pretende editar. 3. O sistema disponibiliza o formulário preenchido com os dados atuais do produto. 4. O utilizador altera os campos pretendidos (ex.: preço, descrição, categoria). 5. O utilizador confirma a gravação das alterações. 6. O sistema valida os novos dados inseridos. 7. O sistema atualiza o produto na base de dados local. 8. O sistema regista a data da alteração e o utilizador responsável pela mesma (auditoria). 9. O sistema exibe uma mensagem de sucesso e atualiza a listagem do catálogo.
Fluxo Alternativo	
Fluxos de Exceção	**(1) [Produto não encontrado] (passo 2)** 2.1.1 O sistema deteta que a referência ou o nome pesquisado não corresponde a nenhum produto no catálogo. 2.2.1 O sistema exibe a mensagem de erro: “Produto não encontrado”. **(2) [Campos obrigatórios em falta] (passo 6)** 6.1.1 O sistema deteta que o utilizador apagou o conteúdo de um ou mais campos obrigatórios (ex.: nome ou preço). 6.1.2 O sistema realça os campos em falta e apresenta um aviso ao utilizador para os preencher. **(3) [Formato de dados inválido] (passo 6)** 6.2.1 O sistema deteta que um novo valor inserido possui um formato inválido (ex.: introdução de letras no campo do preço ou valor negativo). 6.2.2 O sistema bloqueia a gravação e apresenta uma mensagem de erro que indica o formato correto.

Tabela 38: Caso de Uso: Editar Produto

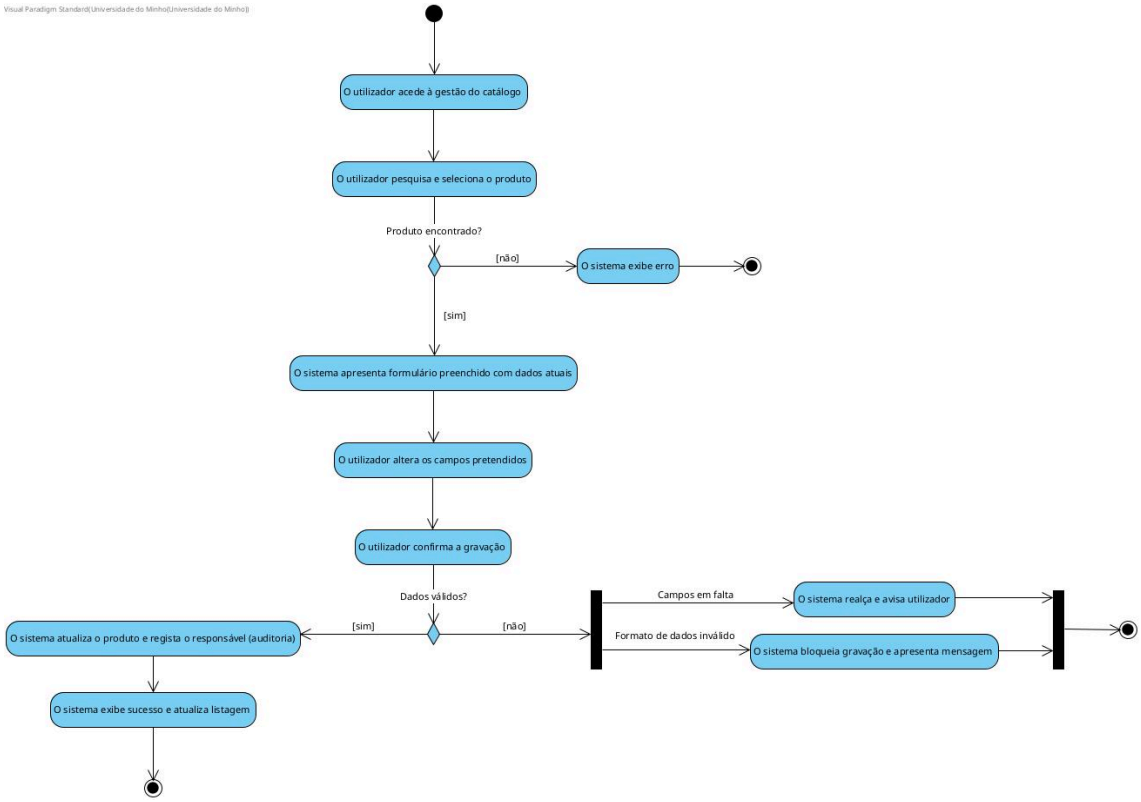


Figura 18: Diagrama de Atividades do Caso de Uso: Editar Produto

Use Case	Descontinuar Produto
Descrição	O utilizador inativa um produto no catálogo local, o que impede a sua utilização em novas operações, mas com a preservação do seu histórico para efeitos de relatórios e consultas.
Cenários	Um determinado produto deixou de ser comercializado pela loja e o gestor inativa-o no sistema para que este deixe de estar disponível na caixa e nas encomendas.
Pré-Condição	O sistema local encontra-se operacional, o utilizador tem a sessão iniciada com permissões de gestão de catálogo e o produto encontra-se registado com o estado 'Ativo'.
Pós-condição	O estado do produto é alterado para 'Descontinuado', a data da operação é registada e o produto passa a ser ocultado das pesquisas ativas (vendas e encomendas).
Fluxo Normal	1. O utilizador acede à funcionalidade de gestão de catálogo de produtos. 2. O utilizador pesquisa e seleciona o produto que pretende descontinuar. 3. O sistema apresenta os detalhes atuais do produto. 4. O utilizador seleciona a opção para descontinuar o produto. 5. O sistema exibe uma mensagem de aviso para solicitar a confirmação da operação. 6. O utilizador confirma a descontinuação. 7. O sistema altera o estado do produto para “Descontinuado” na base de dados local. 8. O sistema regista a data de descontinuação e o utilizador responsável pela ação. 9. O sistema atualiza a listagem do catálogo, aplicando uma marcação visual (ex.: etiqueta “Descontinuado”) ao produto afetado.
Fluxo Alternativo	**(1) [Cancelar descontinuação] (passo 6)** 6.1 O utilizador seleciona a opção de cancelar na mensagem de confirmação. 6.2 O sistema interrompe a operação e o estado do produto mantém-se inalterado. 6.3 Passo 3.
Fluxos de Exceção	**(1) [Produto não encontrado] (passo 2)** 2.1.1 O sistema deteta que a referência ou o nome pesquisado não corresponde a nenhum produto no catálogo. 2.2.1 O sistema exibe a mensagem de erro: “Produto não encontrado”.

Tabela 39: Caso de Uso: Descontinuar Produto

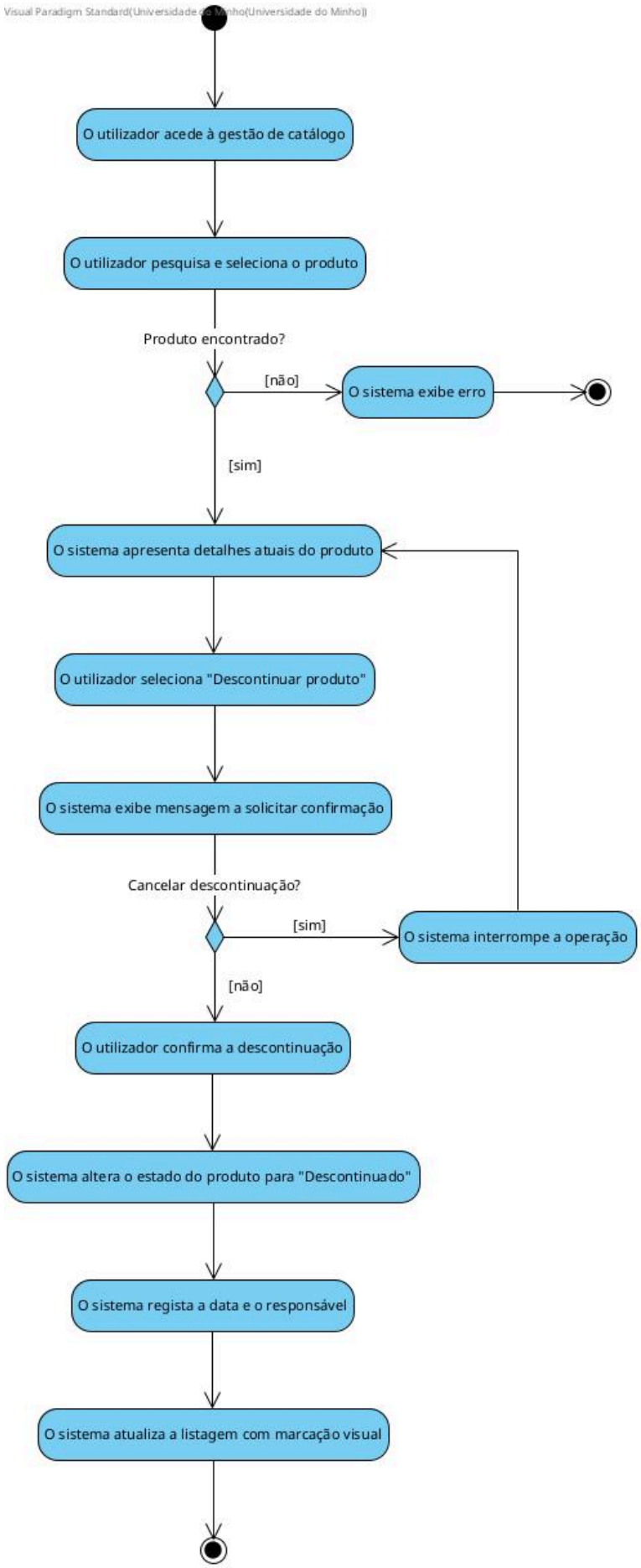


Figura 19: Diagrama de Atividades do Caso de Uso: Descontinuar Produto

Use Case	Consultar Produto
Descrição	O utilizador acede ao catálogo local para pesquisar e visualizar de forma detalhada as informações de um produto específico.
Cenários	O operador de caixa ou gestor de loja necessita de verificar o preço, a categoria, o estado ou outras características de um artigo a pedido de um cliente.
Pré-Condição	O sistema local encontra-se operacional e o utilizador tem a sessão iniciada com permissões de consulta ao catálogo.
Pós-condição	O sistema apresenta no ecrã os detalhes completos do produto pretendido sem alteração dos seus dados.
Fluxo Normal	1. O utilizador acede à funcionalidade de consulta do catálogo de produtos. 2. O utilizador introduz o termo de pesquisa (ex.: código de barras, referência ou nome). 3. O sistema pesquisa a informação na base de dados local. 4. O sistema apresenta a listagem de resultados correspondentes. 5. O utilizador seleciona o produto pretendido na listagem. 6. O sistema exibe a ficha detalhada do produto (nome, descrição, preço, categoria, estado e unidade de medida). 7. O utilizador encerra a vista de detalhes.
Fluxo Alternativo	**(1) [Pesquisa exata por código de barras] (passo 4)** 4.1 O sistema deteta que a pesquisa resultou numa correspondência única e exata através da leitura do código de barras. 4.2 O sistema ignora a apresentação da listagem de resultados e avança diretamente para a ficha do produto. 4.3 Passo 6.
Fluxos de Exceção	**(1) [Termo de pesquisa inválido] (passo 3)** 3.1 O sistema deteta que o termo introduzido é demasiado curto ou contém caracteres não permitidos. 3.2 O sistema bloqueia a pesquisa e apresenta um aviso para introduzir um termo válido. **(2) [Nenhum produto encontrado] (passo 4)** 4.1 O sistema deteta que o termo de pesquisa não corresponde a nenhum registo no catálogo local. 4.2 O sistema exibe uma listagem vazia acompanhada da mensagem de aviso: “Nenhum produto encontrado”.

Tabela 40: Caso de Uso: Consultar Produto

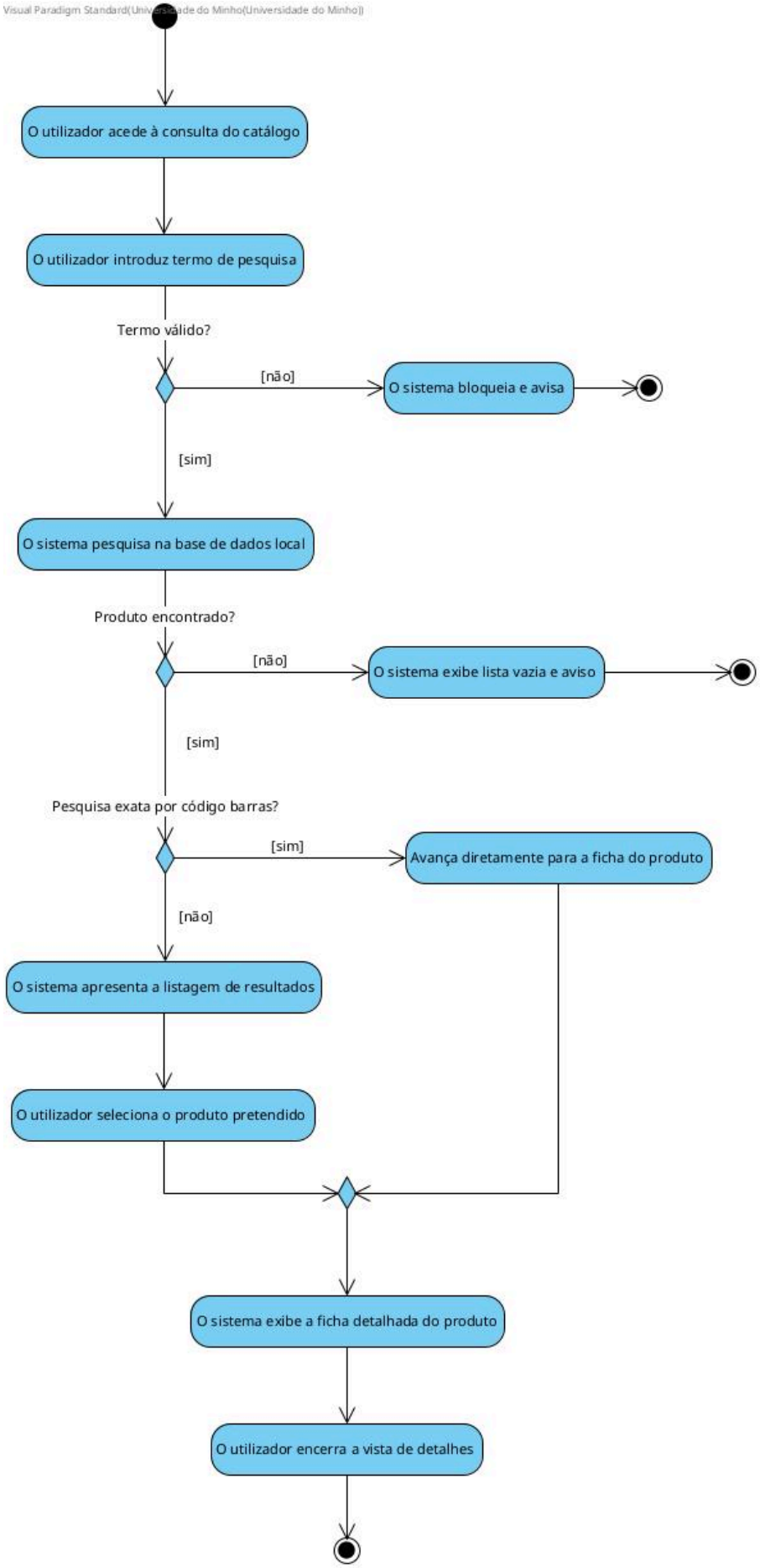


Figura 20: Diagrama de Atividades do Caso de Uso: Consultar Produto

Use Case	Criar Categoria
Descrição	O utilizador cria uma nova categoria no sistema para organizar e classificar estruturalmente os produtos do catálogo local.
Cenários	A loja passa a comercializar uma nova linha de artigos (ex.: 'Laticínios') e o gestor cria a categoria correspondente no sistema antes de adicionar os respetivos produtos.
Pré-Condição	O sistema local encontra-se operacional e o utilizador tem a sessão iniciada com permissões de gestão de catálogo.
Pós-condição	A nova categoria é registada na base de dados local, ficando disponível para ser associada a novos ou atuais produtos.
Fluxo Normal	1. O utilizador acede à funcionalidade de gestão de categorias. 2. O utilizador seleciona a opção para adicionar uma nova categoria. 3. O sistema disponibiliza o formulário de criação em branco. 4. O utilizador introduz o nome da nova categoria e, opcionalmente, uma descrição. 5. O utilizador confirma a gravação do novo registo. 6. O sistema valida os dados inseridos e verifica a unicidade do nome da categoria. 7. O sistema guarda a categoria na base de dados local. 8. O sistema exibe uma mensagem de sucesso e atualiza a listagem de categorias.
Fluxo Alternativo	**(1) [Cancelar criação de categoria] (passo 5)** 5.1 O utilizador decide cancelar a operação antes de gravar (ex.: seleciona a opção de retroceder). 5.2 O sistema descarta todas as informações introduzidas no formulário. 5.3 Passo 1.
Fluxos de Exceção	**(1) [Categoria já existente] (passo 6)** 6.1.1 O sistema deteta que o nome introduzido já pertence a outra categoria registada no catálogo. 6.2.1 O sistema bloqueia a gravação e exibe a mensagem de erro: “Já existe uma categoria registada com este nome”. **(2) [Campos obrigatórios em falta] (passo 6)** 6.1.2 O sistema deteta que o campo correspondente ao nome da categoria se encontra vazio. 6.2.2 O sistema realça o campo em falta e apresenta um aviso ao utilizador para o preencher.

Tabela 41: Caso de Uso: Criar Categoria

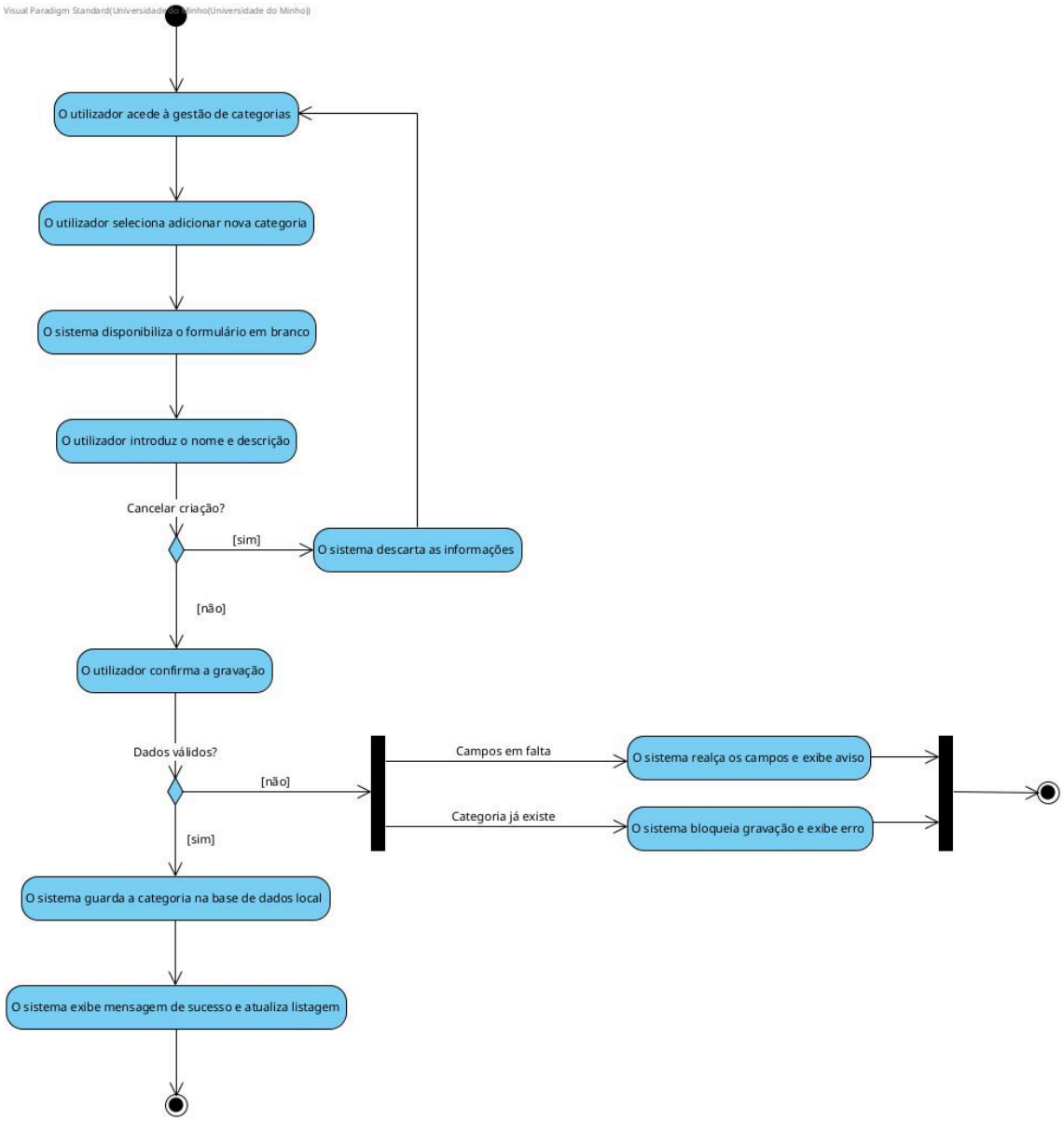


Figura 21: Diagrama de Atividades do Caso de Uso: Criar Categoria

Use Case	Editar Categoria
Descrição	O utilizador altera os dados de uma categoria existente no sistema para atualizar a organização e classificação do catálogo de produtos.
Cenários	O gestor de loja decide alterar o nome de uma categoria de 'Bebidas' para 'Bebidas e Sumos' para ser mais descritivo na organização dos produtos.
Pré-Condição	O sistema local encontra-se operacional, o utilizador tem a sessão iniciada com permissões de gestão de catálogo e a categoria já existe na base de dados.
Pós-condição	Os dados da categoria são atualizados no catálogo local, refletindo-se imediatamente em todos os produtos a ela associados.
Fluxo Normal	1. O utilizador acede à funcionalidade de gestão de categorias. 2. O utilizador pesquisa e seleciona a categoria que pretende editar. 3. O sistema disponibiliza o formulário preenchido com os dados atuais da categoria. 4. O utilizador altera os campos pretendidos (ex.: nome ou descrição). 5. O utilizador confirma a gravação das alterações. 6. O sistema valida os novos dados inseridos e verifica a unicidade do novo nome (se o mesmo tiver sido alterado). 7. O sistema atualiza a categoria na base de dados local. 8. O sistema exibe uma mensagem de sucesso e atualiza a listagem de categorias.
Fluxo Alternativo	**(1) [Cancelar edição de categoria] (passo 5)** 5.1 O utilizador decide cancelar a operação antes de gravar (ex.: seleciona a opção de retroceder). 5.2 O sistema descarta todas as alterações introduzidas no formulário. 5.3 Passo 1.
Fluxos de Exceção	**(1) [Categoria não encontrada] (passo 2)** 2.1.1 O sistema deteta que o nome pesquisado não corresponde a nenhuma categoria no catálogo. 2.2.1 O sistema exibe a mensagem de erro: “Categoria não encontrada”. **(2) [Nome de categoria já existente] (passo 6)** 6.1 O sistema deteta que o novo nome introduzido já se encontra em uso por outra categoria registada. 6.2 O sistema bloqueia a gravação e exibe a mensagem de erro: “Já existe uma categoria registada com este nome”.

Tabela 42: Caso de Uso: Editar Categoria

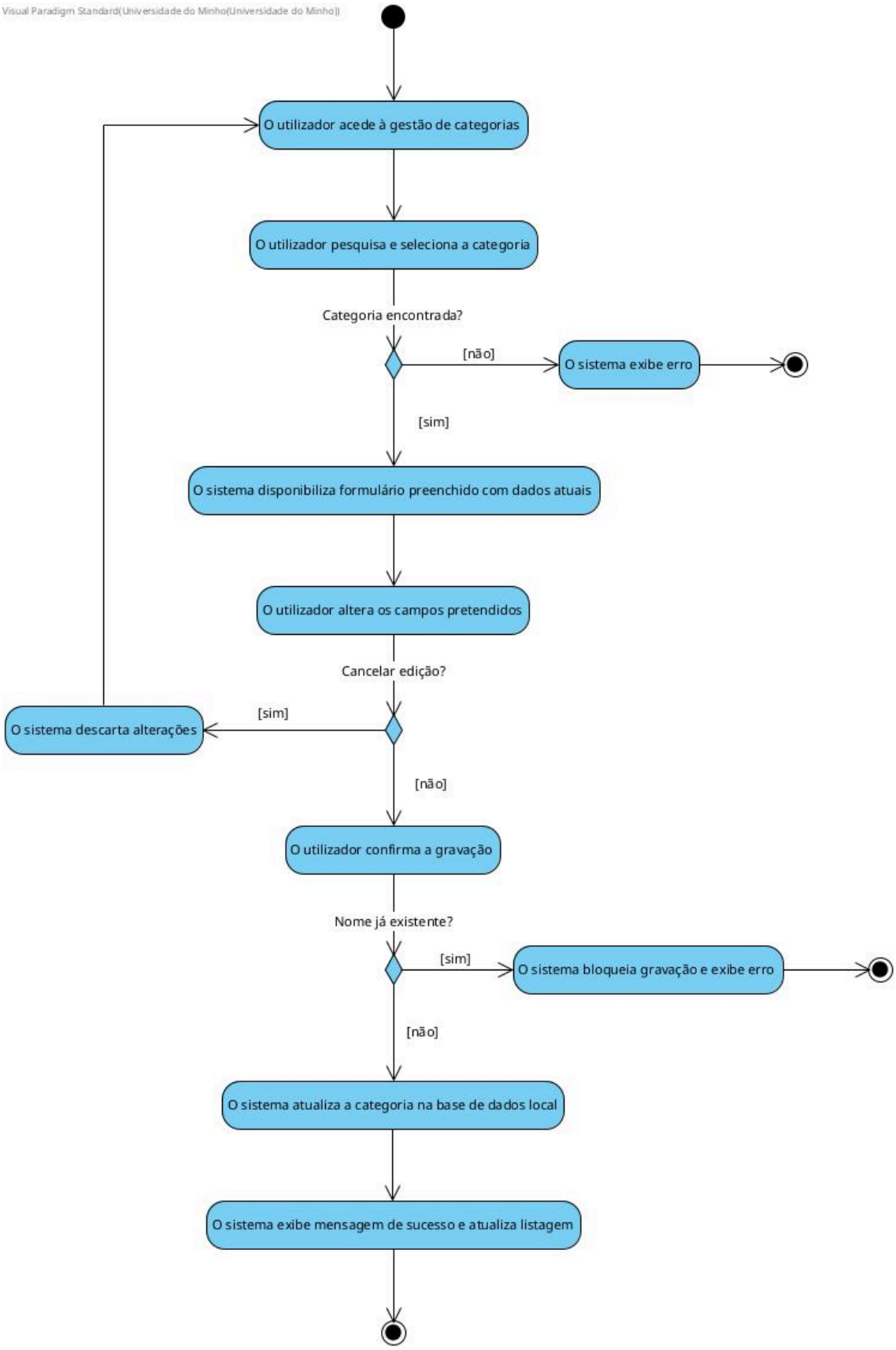


Figura 22: Diagrama de Atividades do Caso de Uso: Editar Categoria

Use Case	Descontinuar Categoria
Descrição	O utilizador inativa uma categoria no sistema, impedindo que novos produtos lhe sejam associados, mas preservando o seu histórico para efeitos de relatórios e organização dos produtos já existentes.
Cenários	A loja deixa de comercializar a linha de 'Laticínios' de forma definitiva. O gestor inativa a categoria para limpar as opções de criação de novos artigos, sem perder o histórico de vendas associado.
Pré-Condição	O sistema local encontra-se operacional, o utilizador tem a sessão iniciada com permissões de gestão de catálogo e a categoria encontra-se com o estado 'Ativa'.
Pós-condição	O estado da categoria é alterado para 'Descontinuada' e a mesma é ocultada dos menus de seleção para novos produtos, mantendo a sua integridade no histórico de dados.
Fluxo Normal	1. O utilizador acede à funcionalidade de gestão de categorias. 2. O utilizador pesquisa e seleciona a categoria que pretende descontinuar. 3. O sistema apresenta os detalhes atuais da categoria e indica a quantidade de produtos que lhe estão associados. 4. O utilizador seleciona a opção para descontinuar a categoria. 5. O sistema exibe uma mensagem de aviso para solicitar a confirmação da operação. 6. O utilizador confirma a descontinuação. 7. O sistema altera o estado da categoria para “Descontinuada” na base de dados local. 8. O sistema atualiza a listagem de categorias, aplicando uma marcação visual (ex.: etiqueta “Descontinuada”) ao registo afetado.
Fluxo Alternativo	**(1) [Cancelar descontinuação] (passo 6)** 6.1 O utilizador seleciona a opção de cancelar na mensagem de confirmação. 6.2 O sistema interrompe a operação e o estado da categoria mantém-se inalterado. 6.3 Passo 3.
Fluxos de Exceção	**(1) [Categoria não encontrada] (passo 2)** 2.1 O sistema deteta que o nome pesquisado não corresponde a nenhuma categoria ativa no catálogo. 2.2 O sistema exibe a mensagem de erro: “Categoria não encontrada”.

Tabela 43: Caso de Uso: Descontinuar Categoria

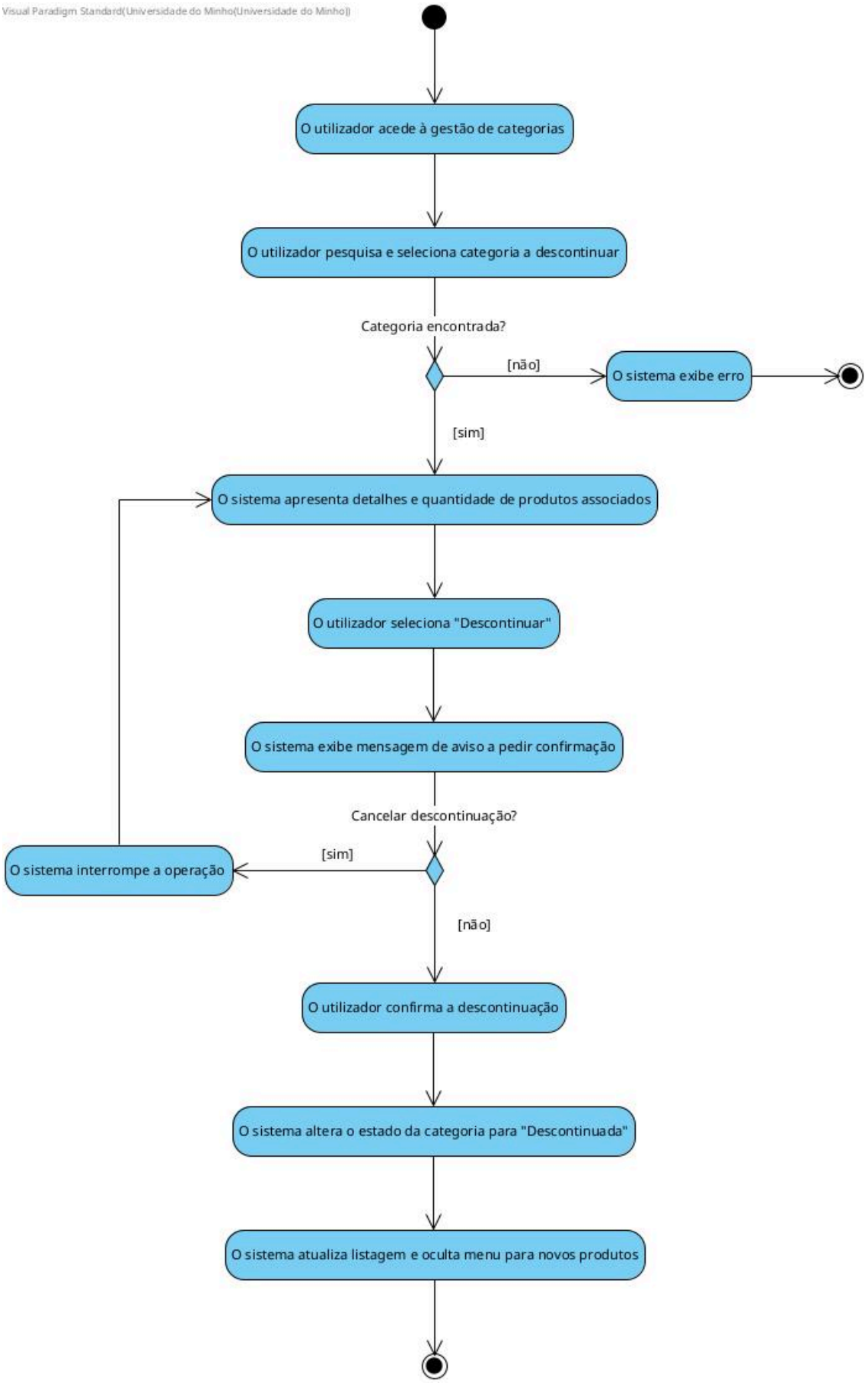


Figura 23: Diagrama de Atividades do Caso de Uso: Descontinuar Categoria

Use Case	Registar Fornecedor
Descrição	O utilizador autorizado adiciona os dados de um novo parceiro comercial à base de dados local para permitir futuras operações de aprovisionamento na loja.
Cenários	Início de uma nova parceria comercial que requer o registo da entidade no sistema para permitir futuras interações e aprovisionamentos.
Pré-Condição	O utilizador encontra-se autenticado no sistema informático e possui um perfil de acesso com permissões ativas de escrita no módulo de Gestão de Fornecedores.
Pós-condição	Os dados do novo fornecedor encontram-se guardados e persistidos na base de dados local com o estado "Ativo", o que o torna imediatamente disponível.
Fluxo Normal	1. O utilizador acede ao módulo “Fornecedores” no sistema. 2. O utilizador seleciona a opção “Registar Novo Fornecedor”. 3. O sistema apresenta um formulário de registo em branco. 4. O utilizador introduz os dados do fornecedor (Nome, NIF, Morada, Contacto e Email). 5. O utilizador seleciona a opção “Guardar”. 6. O sistema valida os dados. 7. O sistema regista o fornecedor e apresenta uma mensagem de sucesso. 8. O sistema redireciona o utilizador para a lista atualizada de fornecedores.
Fluxo Alternativo	**(1) [Campos Obrigatórios em Falta] (passo 6)** 6.1 O sistema deteta que um ou mais campos obrigatórios estão vazios. 6.2 O sistema realça os campos em falta (ex: a vermelho) e apresenta um aviso ao utilizador para os preencher. 6.3 A gravação é interrompida e o sistema aguarda que o utilizador complete a informação. **(2) [NIF já existente] (passo 6)** 6.1.1 O sistema deteta que o NIF introduzido já pertence a outro fornecedor registado na base de dados. 6.2.1 O sistema bloqueia a gravação e apresenta a mensagem de erro “Já existe um fornecedor registado com este NIF”. 6.3.1 O sistema mantém os dados no formulário para que o utilizador possa corrigir a informação.
Fluxos de Exceção	**(3) [Cancelamento da Operação] (passo 4)** 4.1 O utilizador seleciona a opção “Cancelar” em vez de submeter os dados. 4.2 O sistema descarta a informação introduzida e redireciona o utilizador para a listagem sem realizar alterações.

Tabela 44: Caso de Uso: Registar Fornecedor

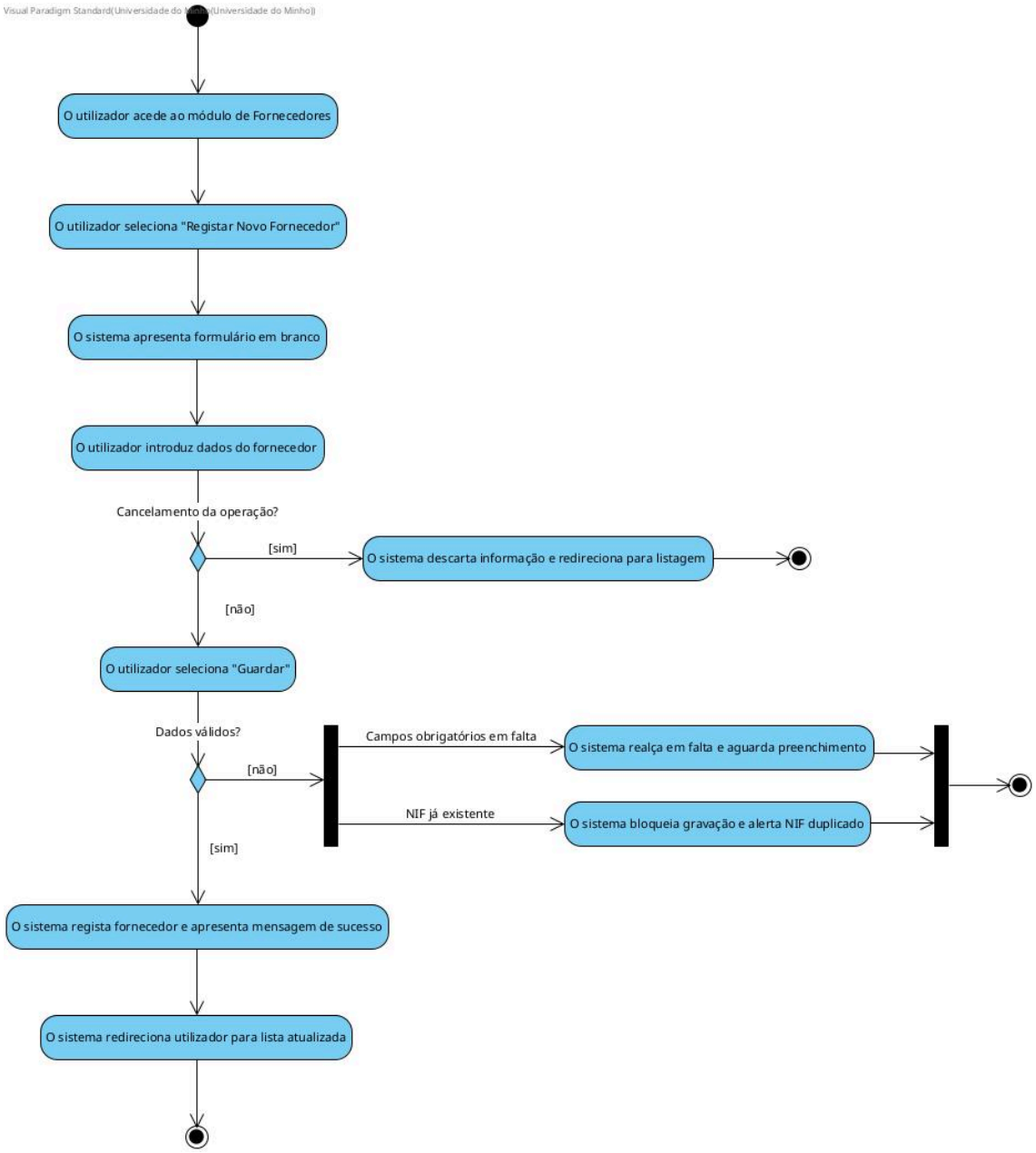


Figura 24: Diagrama de Atividades do Caso de Uso: Registar Fornecedor

Use Case	Editar Dados do Fornecedor
Descrição	O utilizador atualiza as informações de um parceiro comercial já existente na base de dados.
Cenários	Necessidade de atualizar os dados de um parceiro comercial devido a alterações nas suas informações institucionais ou de contacto.
Pré-Condição	O utilizador está autenticado, tem permissões de escrita em fornecedores e o fornecedor existe no sistema local.
Pós-condição	Os novos dados do fornecedor estão atualizados na base de dados e prontos para a sincronização noturna.
Fluxo Normal	1. O utilizador acede à lista de Fornecedores e pesquisa pelo fornecedor pretendido. 2. O utilizador seleciona a opção “Editar”. 3. O sistema apresenta o formulário pré-preenchido com os dados atuais do fornecedor. 4. O utilizador altera os dados pretendidos nos campos correspondentes. 5. O utilizador clica em “Guardar”. 6. O sistema valida o formato dos dados. 7. O sistema atualiza o registo do fornecedor na base de dados. 8. O sistema exibe a mensagem “Dados atualizados com sucesso” e regressa à listagem de fornecedores.
Fluxo Alternativo	**(1) [NIF Duplicado] (passo 6)** 6.1 O sistema deteta que o NIF alterado já pertence a outro fornecedor registado. 6.2 O sistema bloqueia a gravação e apresenta uma mensagem de erro. 6.3 O sistema aguarda que o utilizador corrija a informação. 6.4 Passo 5
Fluxos de Exceção	**(2) [Cancelamento da Edição] (passo 5)** 5.1 O utilizador seleciona a opção “Cancelar” em vez de submeter os dados. 5.2 O sistema descarta as alterações efetuadas, caso existam, e redireciona o utilizador para a listagem de fornecedores.

Tabela 45: Caso de Uso: Editar Dados do Fornecedor

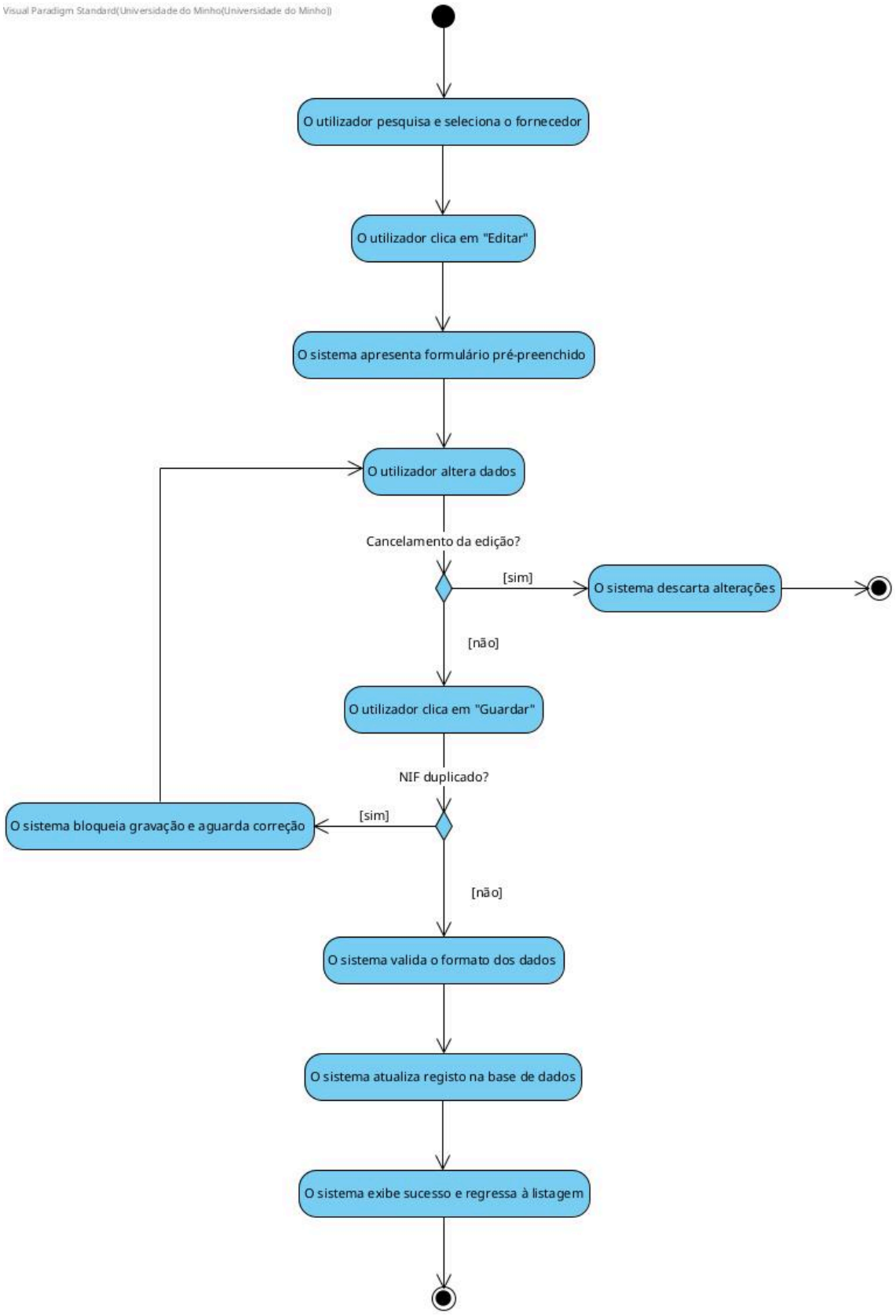


Figura 25: Diagrama de Atividades do Caso de Uso: Editar Dados do Fornecedor

Use Case	**Remover Fornecedor**
Descrição	O utilizador inativa um fornecedor com o qual a Taki deixou de trabalhar.
Cenários	Término de uma parceria comercial ou inatividade prolongada que justifica a desativação da entidade no sistema.
Pré-Condição	O utilizador está autenticado e tem permissões para gerir fornecedores.
Pós-condição	O fornecedor passa ao estado "Inativo" (soft delete), com a manutenção do seu histórico de faturas intacto.
Fluxo Normal	1. O utilizador pesquisa o fornecedor pretendido na listagem de fornecedores. 2. O utilizador seleciona a opção “Remover/Desativar”. 3. O sistema apresenta um ecrã a pedir a confirmação da ação. 4. O utilizador confirma a inativação. 5. O sistema verifica se existem dependências ativas. 6. O sistema altera o estado do fornecedor para “Inativo” na base de dados. 7. O sistema exibe uma mensagem de sucesso e oculta o fornecedor das listagens ativas.
Fluxo Alternativo	
Fluxos de Exceção	**(1) [Cancelamento da Remoção] (passo 4)** 4.1 O utilizador seleciona a opção “Cancelar” ou “Não” na janela de confirmação. 4.2 O sistema aborta a operação e mantém o fornecedor ativo. **(2) [Encomendas Pendentes] (passo 5)** 5.1 O sistema deteta que existem notas de encomenda em curso para este fornecedor. 5.2 O sistema bloqueia a inativação e exibe o erro “Não é possível inativar o fornecedor: existem encomendas pendentes”. 5.3 A operação é abortada.

Tabela 46: Caso de Uso: Remover Fornecedor

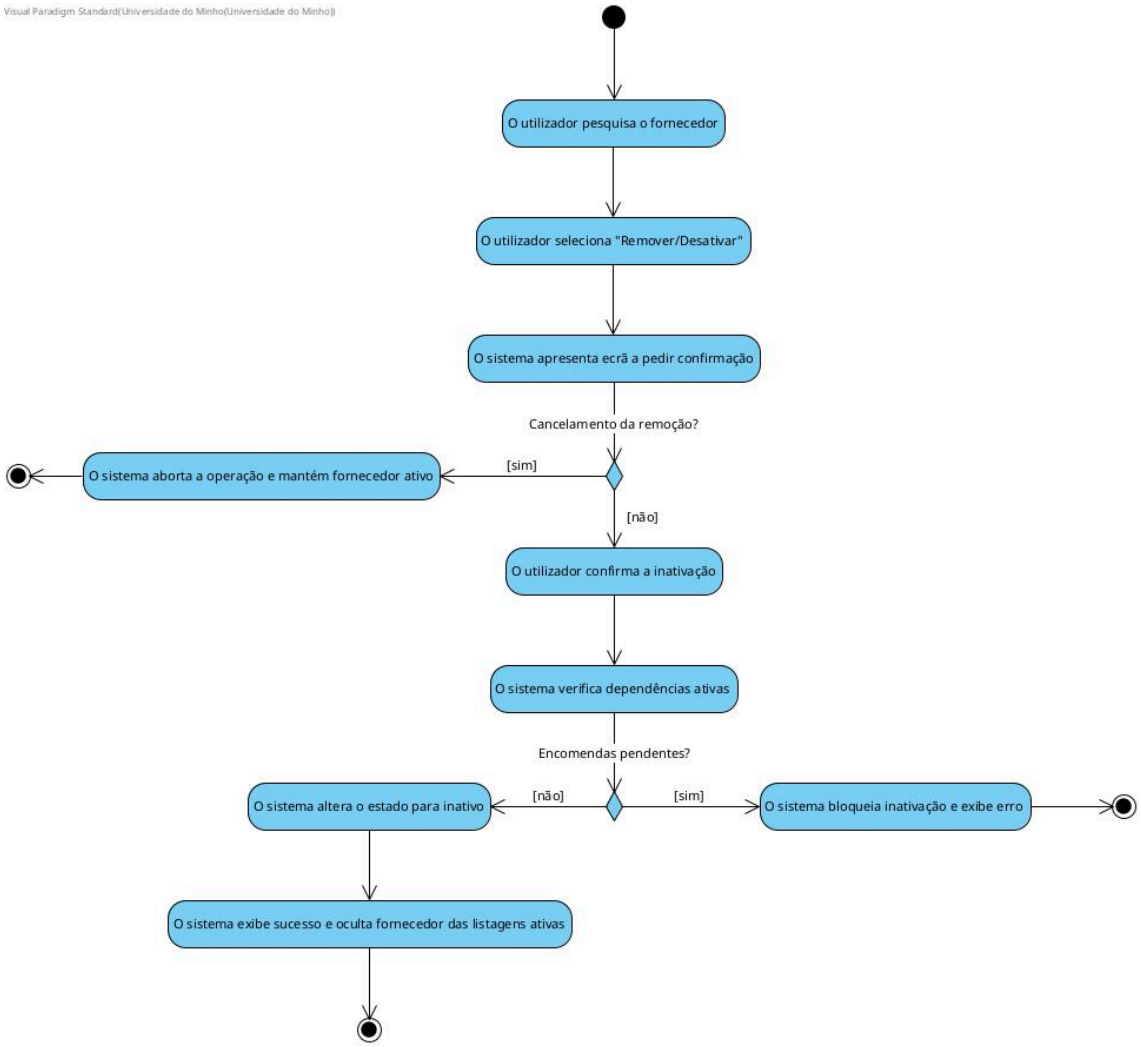


Figura 26: Diagrama de Atividades do Caso de Uso: Remover Fornecedor

Use Case	**Associar Produto a Fornecedor**
Descrição	O utilizador vincula um produto do catálogo a um fornecedor específico.
Cenários	Definição ou atualização da origem de fornecimento de um determinado artigo do catálogo para viabilizar futuras encomendas.
Pré-Condição	O produto e o fornecedor já existem na base de dados e encontram-se ativos.
Pós-condição	O produto passa a estar associado ao fornecedor com um respetivo preço.
Fluxo Normal	1. O utilizador acede à ficha do Produto e escolhe a opção “Associar a Fornecedor”. 2. O sistema apresenta a lista de fornecedores ativos. 3. O utilizador pesquisa e seleciona o fornecedor pretendido. 4. O utilizador insere o preço de custo acordado. 5. O utilizador clica em “Guardar”. 6. O sistema valida os dados inseridos. 7. O sistema regista a associação na base de dados. 8. O sistema exibe uma mensagem de sucesso.
Fluxo Alternativo	**(1) [Associação Existente] (passo 6)** 6.1 O sistema verifica que o produto já se encontra associado ao fornecedor selecionado. 6.2 O sistema bloqueia a gravação e avisa “O produto já se encontra associado a este fornecedor”. 6.3 O sistema aguarda correção por parte do utilizador. 6.4 Passo 1.
Fluxos de Exceção	**(2) [Cancelamento da Operação] (passo 5)** 5.1 O utilizador seleciona “Cancelar”. 5.2 O sistema aborta a associação e mantém a ficha do Produto.

Tabela 47: Caso de Uso: Associar Produto a Fornecedor

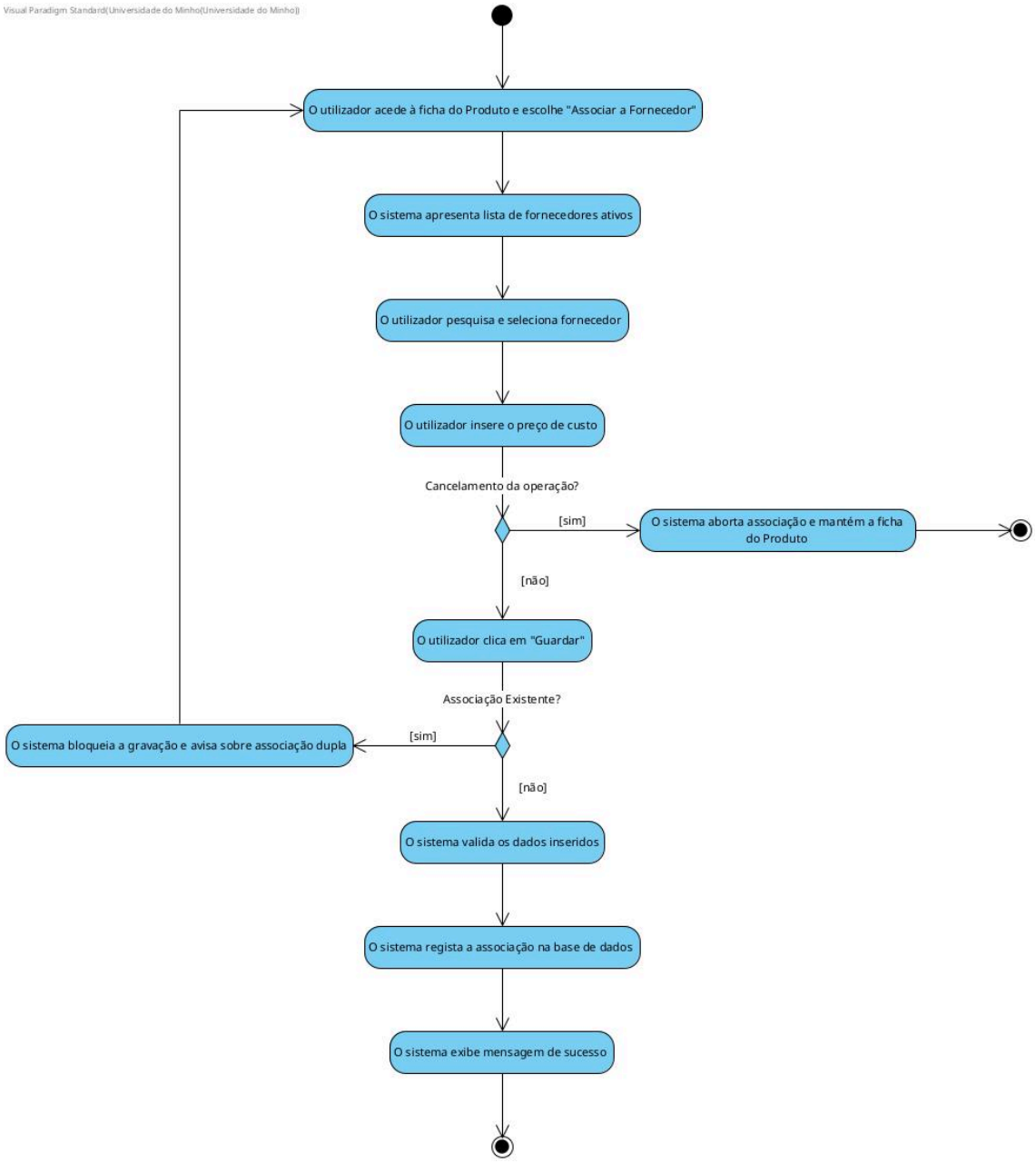


Figura 27: Diagrama de Atividades do Caso de Uso: Associar Produto a Fornecedor

Use Case	**Criar política de promoção**
Descrição	O utilizador define um desconto temporário para um artigo ou categoria.
Cenários	Aplicação de uma estratégia de preços dinâmica para incentivar a venda de produtos ou categorias num determinado período.
Pré-Condição	O utilizador tem permissões de gestão de catálogo e os produtos encontram-se ativos no sistema.
Pós-condição	A regra de promoção é guardada e o sistema de POS aplicará o desconto automaticamente nas datas definidas.
Fluxo Normal	1. O utilizador acede ao módulo de Promoções e seleciona “Nova Promoção”. 2. O sistema apresenta o formulário de configuração. 3. O utilizador insere a Data/Hora de Início, Data/Hora de Fim e o valor do desconto (em percentagem). 4. O utilizador seleciona os produtos ou categorias abrangidas. 5. O utilizador clica em “Guardar”. 6. O sistema valida as datas de vigência e as regras de sobreposição. 7. O sistema ativa a promoção na base de dados. 8. O sistema apresenta uma mensagem de sucesso.
Fluxo Alternativo	**(1) [Promoção Permanente] (passo 3)** 3.1 O utilizador não preenche o campo “Data/Hora de Fim”. 3.2 No passo 7, o sistema regista a promoção como vitalícia até ser alvo de cancelamento manual.
Fluxos de Exceção	**(2) [Datas Inválidas] (passo 6)** 6.1 O sistema deteta que a Data de Fim é anterior à Data de Início. 6.2 O sistema exibe o erro “O período da promoção é inválido” e bloqueia a gravação. **(3) [Conflito de Promoções] (passo 6)** 6.1 O sistema deteta que os produtos selecionados já possuem uma promoção ativa no mesmo intervalo de tempo. 6.2 O sistema bloqueia a gravação e apresenta um aviso de sobreposição temporal.

Tabela 48: Caso de Uso: Criar política de promoção

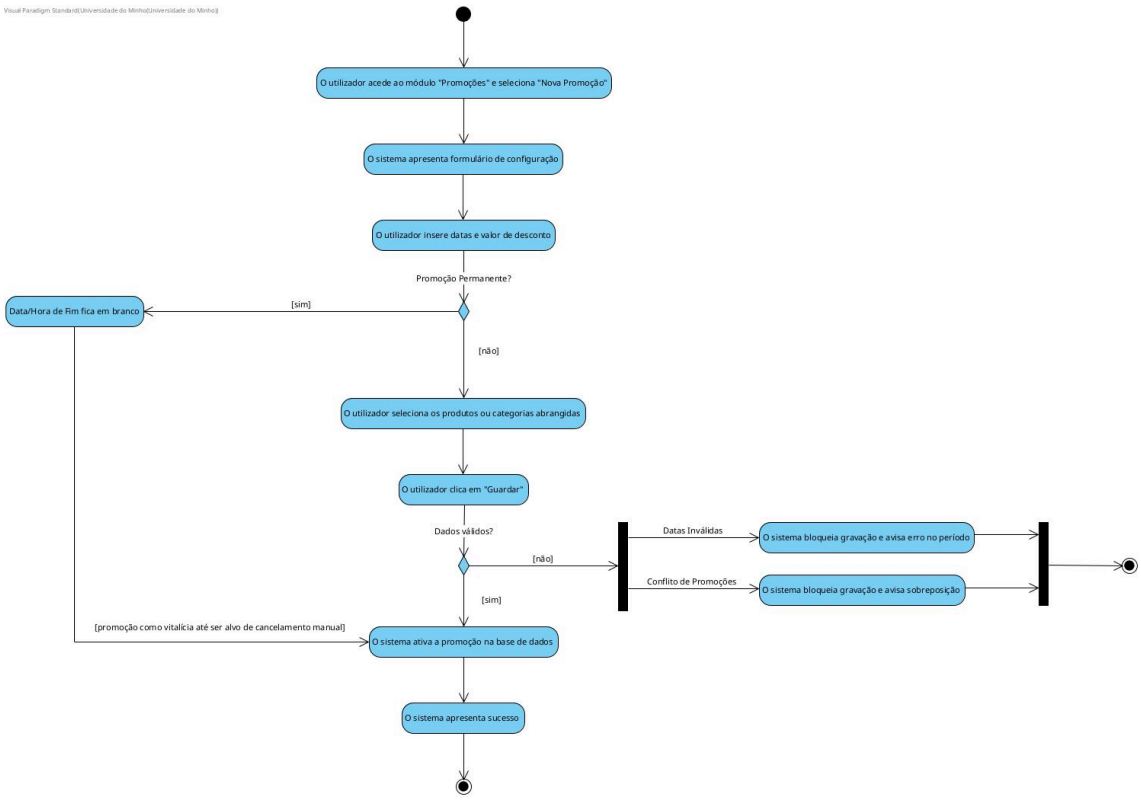


Figura 28: Diagrama de Atividades do Caso de Uso: Criar política de promoção

Use Case	**Cancelar política de promoção**
Descrição	O utilizador força o término de uma campanha antes da sua data final prevista.
Cenários	Necessidade de interromper antecipadamente uma campanha de descontos em vigor devido a alterações na estratégia comercial ou de stock.
Pré-Condição	Existe pelo menos uma promoção registada no sistema com o estado "Ativa".
Pós-condição	A promoção é alterada para o estado "Cancelada" e o seu motivo registado.
Fluxo Normal	1. O utilizador acede à listagem de Promoções Ativas. 2. O utilizador seleciona a promoção pretendida e escolhe a opção “Cancelar Promoção”. 3. O sistema apresenta um ecrã de confirmação, solicitando a introdução de um motivo para o cancelamento. 4. O utilizador introduz o motivo e confirma a operação. 5. O sistema altera o estado da política de promoção para “Cancelada”. 6. O sistema exibe uma mensagem de sucesso.
Fluxo Alternativo	**(1) [Desistir do Cancelamento] (passo 4)** 4.1 O utilizador seleciona a opção “Voltar” ou “Cancelar”. 4.2 O sistema mantém a promoção ativa e descarta a operação.
Fluxos de Exceção	**(1) [Motivo de Cancelamento em Falta] (passo 4)** 4.1 O utilizador tenta confirmar a operação sem introduzir um motivo. 4.2 O sistema exibe uma mensagem de erro que informa que o motivo é obrigatório. 4.3 O caso de uso regressa ao passo 3 do fluxo normal.

Tabela 49: Caso de Uso: Cancelar política de promoção

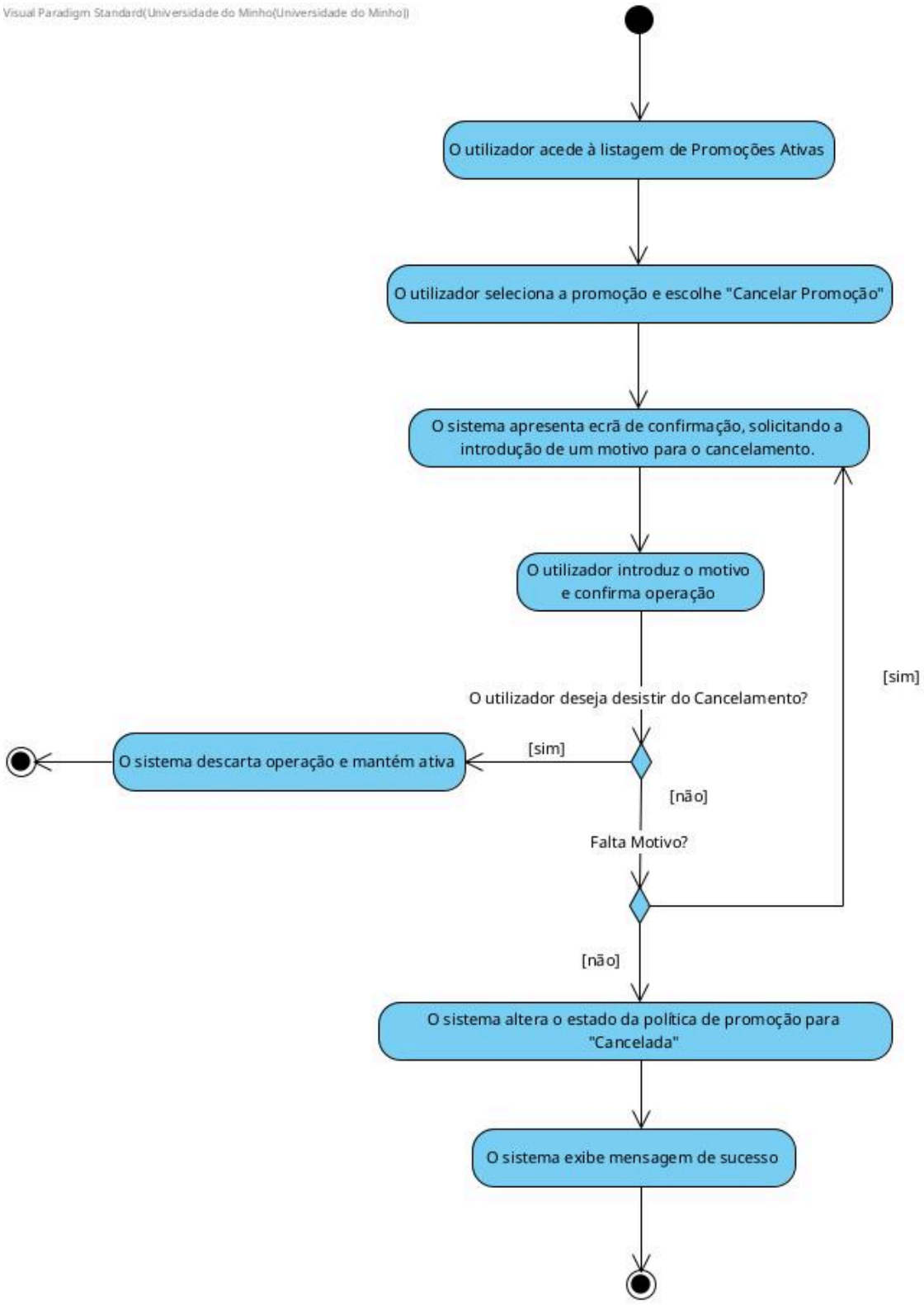


Figura 29: Diagrama de Atividades do Caso de Uso: Cancelar política de promoção

Use Case	**Gerar Relatório de Vendas**
Descrição	O utilizador extrai os dados de faturação baseados em filtros temporais e categóricos.
Cenários	Extração periódica ou pontual de métricas de faturação para análise do desempenho financeiro do negócio.
Pré-Condição	O utilizador possui permissões de visualização de dados financeiros.
Pós-condição	O relatório é gerado, o que permite a visualização no ecrã e a respetiva exportação.
Fluxo Normal	1. O utilizador acede ao módulo de Relatórios e escolhe “Relatório de Vendas”. 2. O utilizador define os filtros obrigatórios (Data de Início e Data de Fim). 3. O utilizador define os filtros opcionais (Loja ou Categoria). 4. O utilizador clica em “Gerar”. 5. O sistema compila os dados transacionais e efetua os cálculos de totais. 6. O sistema exibe os resultados num formato tabular ou gráfico. 7. O sistema disponibiliza o botão para “Exportar (PDF/Excel)”.
Fluxo Alternativo	
Fluxos de Exceção	**(1) [Sem Resultados] (passo 5)** 5.1 O sistema constata que não existem registos de faturação para os filtros selecionados. 5.2 O sistema interrompe a geração do documento e exibe o aviso “Não existem dados para os filtros selecionados”. **(2) [Datas Inválidas] (passo 5)** 5.1 O sistema deteta inconsistência temporal (Data Início > Data Fim). 5.2 O sistema bloqueia a operação e avisa o utilizador.

Tabela 50: Caso de Uso: Gerar Relatório de Vendas

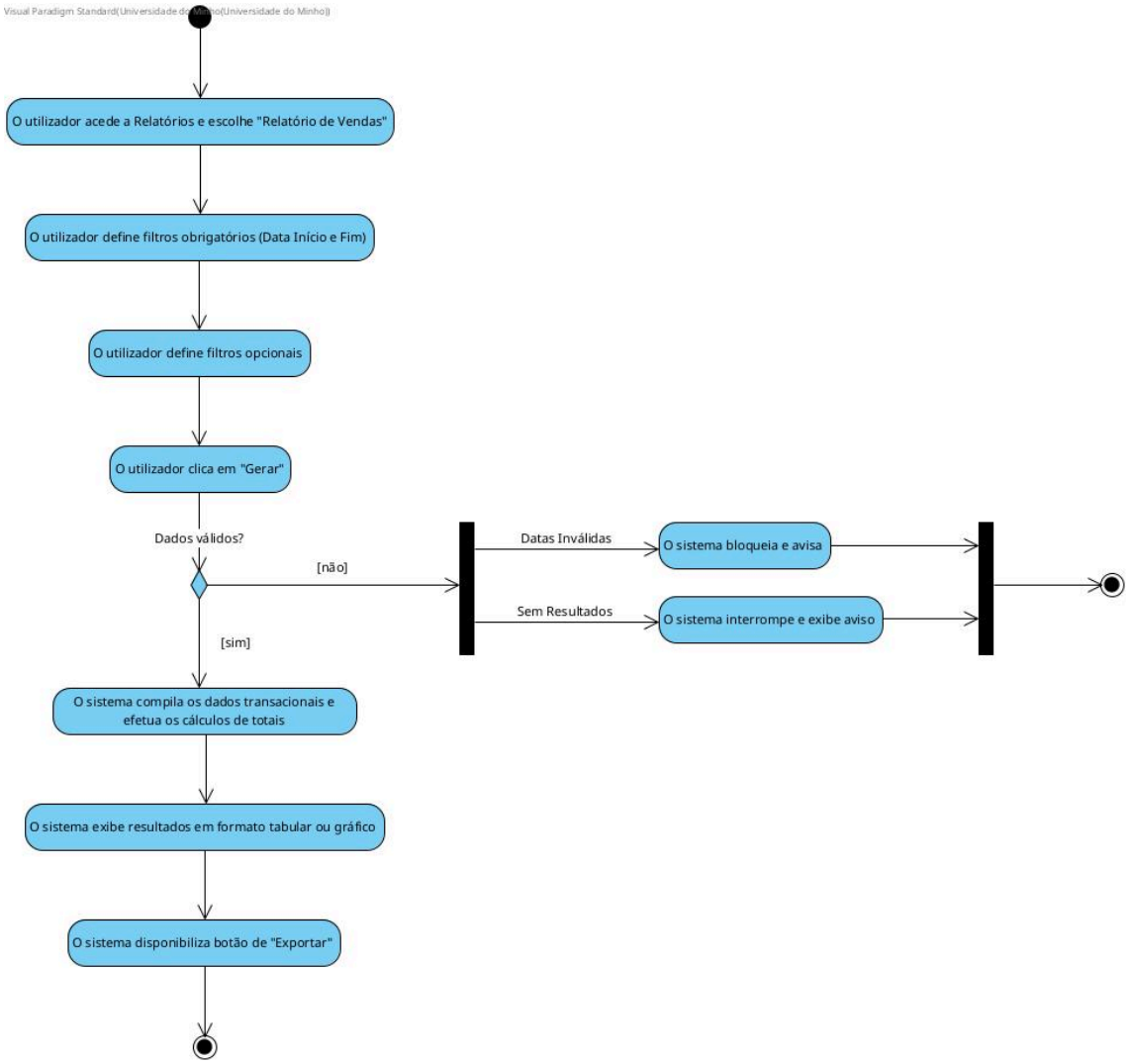


Figura 30: Diagrama de Atividades do Caso de Uso: Gerar Relatório de Vendas

Use Case	Gerar Relatório de Inventário
Descrição	O utilizador consulta a valorização do stock e/ou a lista de produtos em risco de rutura.
Cenários	Consulta do estado atual dos stocks para apuramento do valor imobilizado e identificação de necessidades de reposição.
Pré-Condição	O utilizador encontra-se autenticado e possui permissões de acesso ao inventário.
Pós-condição	A listagem de inventário atualizada é gerada com sucesso e os alertas visuais são aplicados aos produtos em rutura.
Fluxo Normal	1. O utilizador acede ao módulo de Relatórios e escolhe “Relatório de Inventário”. 2. O sistema solicita o tipo de visão (Ex: “Produtos a esgotar” ou “Valorização Financeira”). 3. O utilizador seleciona o tipo pretendido e clica em “Gerar”. 4. O sistema recolhe as quantidades físicas atuais e cruza-as com os stocks de segurança. 5. O sistema exibe a listagem no ecrã, com o destaque a vermelho dos produtos em alerta.
Fluxo Alternativo	
Fluxos de Exceção	**(1) [Inventário Vazio] (passo 4)** 4.1 O sistema verifica que não existem produtos registados no catálogo da loja. 4.2 O sistema emite a notificação “Não existem produtos para contabilizar no inventário”.

Tabela 51: Caso de Uso: Gerar Relatório de Inventário

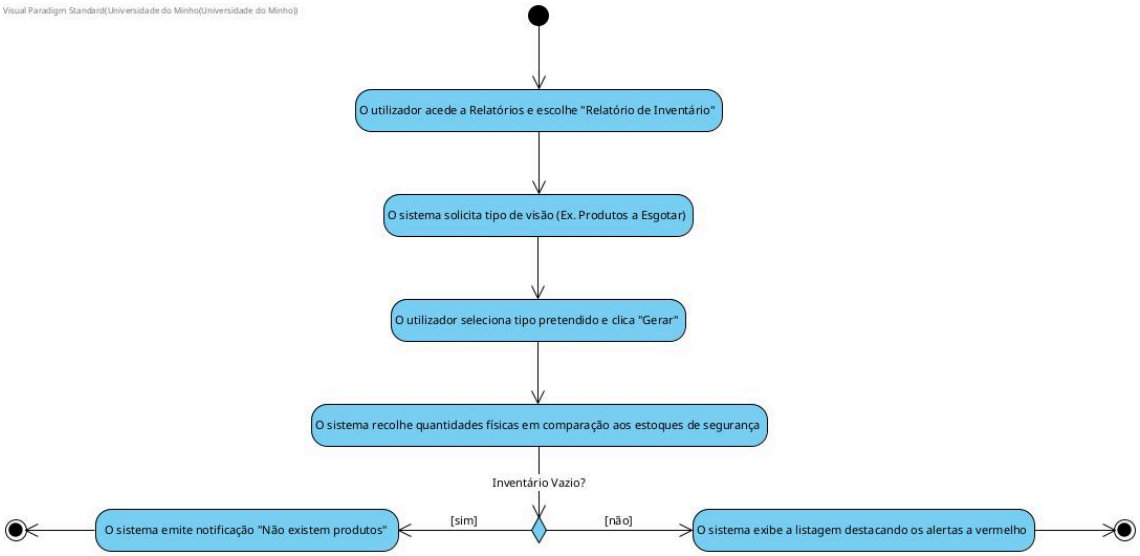


Figura 31: Diagrama de Atividades do Caso de Uso: Gerar Relatório de Inventário

Use Case	Efetuar Autenticação
Descrição	O sistema valida a identidade do utilizador para lhe conceder o acesso devido à plataforma.
Cenários	Acesso à plataforma por parte de um colaborador autorizado para iniciar as suas funções operacionais.
Pré-Condição	O utilizador possui credenciais ativas e o sistema encontra-se no ecrã de login.
Pós-condição	A sessão é iniciada com sucesso, um token de autenticação é gerado e o utilizador acede às funcionalidades estipuladas pelo seu perfil.
Fluxo Normal	1. O utilizador insere os seus dados nos campos de login. 2. O utilizador clica em "Entrar". 3. O sistema verifica a palavra-passe e compara com a base de dados. 4. O sistema confirma a correspondência das credenciais. 5. O sistema verifica que o estado da conta é "Ativo" e carrega as permissões do perfil. 6. O sistema redireciona o utilizador para a página inicial (Dashboard).
Fluxo Alternativo	
Fluxos de Exceção	**(1) [Credenciais Inválidas] (passo 4)** 4.1 O sistema deteta que a palavra-passe ou o e-mail não correspondem aos registos. 4.2 O sistema recusa a autenticação e exibe o erro genérico "E-mail ou palavra--passe incorretos". 4.3 O formulário é limpo para nova tentativa. **(2) [Conta Bloqueada] (passo 5)** 5.1 O sistema deteta que a conta tem o estado "Bloqueada". 5.2 O sistema recusa o acesso e informa "A sua conta encontra-se inativa. Contacte a administração".

Tabela 52: Caso de Uso: Efetuar Autenticação

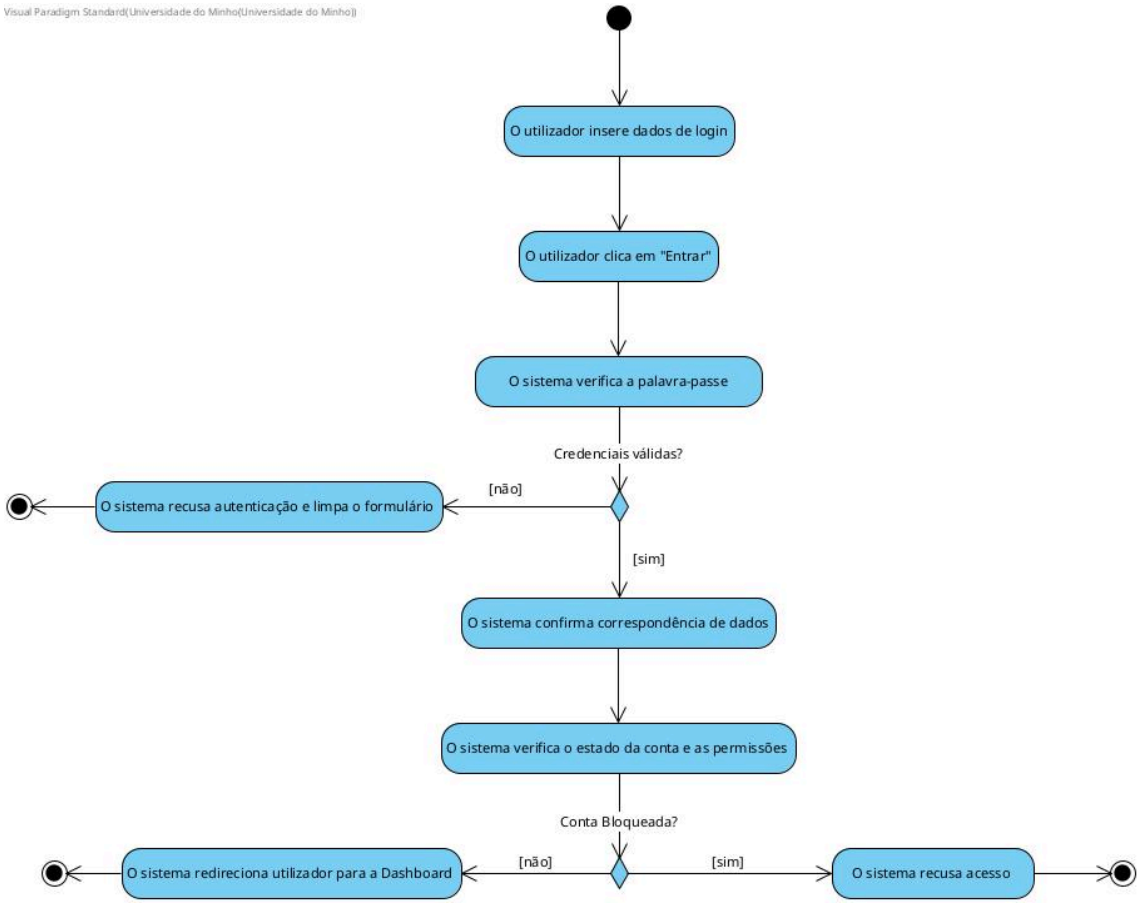


Figura 32: Diagrama de Atividades do Caso de Uso: Efetuar Autenticação

Use Case	**Registar conta de utilizador**
Descrição	O administrador cria credenciais de acesso para um novo funcionário da Taki.
Cenários	Integração de um novo colaborador na organização, requerendo a criação de credenciais de acesso ao sistema.
Pré-Condição	O administrador está autenticado com privilégios de gestão global de utilizadores.
Pós-condição	O novo utilizador fica registado na base de dados, com a respetiva palavra-passe encriptada, o que o torna apto a iniciar sessão.
Fluxo Normal	1. O administrador acede a Gestão de Utilizadores e seleciona “Novo Utilizador”. 2. O sistema apresenta um formulário em branco. 3. O administrador introduz o Nome, E-mail, Palavra-passe e seleciona a Loja de afetação. 4. O administrador clica em “Guardar”. 5. O sistema verifica se o e-mail introduzido é único na base de dados. 6. O sistema encripta a palavra-passe e guarda o registo do utilizador. 7. O sistema apresenta a mensagem de sucesso.
Fluxo Alternativo	**(1) [E-mail já registado] (passo 5)** 5.1 O sistema deteta que o e-mail inserido já pertence a outro utilizador. 5.2 O sistema bloqueia a gravação e apresenta o erro “Este endereço de e-mail já se encontra em uso”. 5.3 Passo 4
Fluxos de Exceção	**(2) [Cancelamento da Operação] (passo 4)** 4.1 O administrador clica em “Cancelar”. 4.2 O sistema aborta a criação da conta.

Tabela 53: Caso de Uso: Registar conta de utilizador

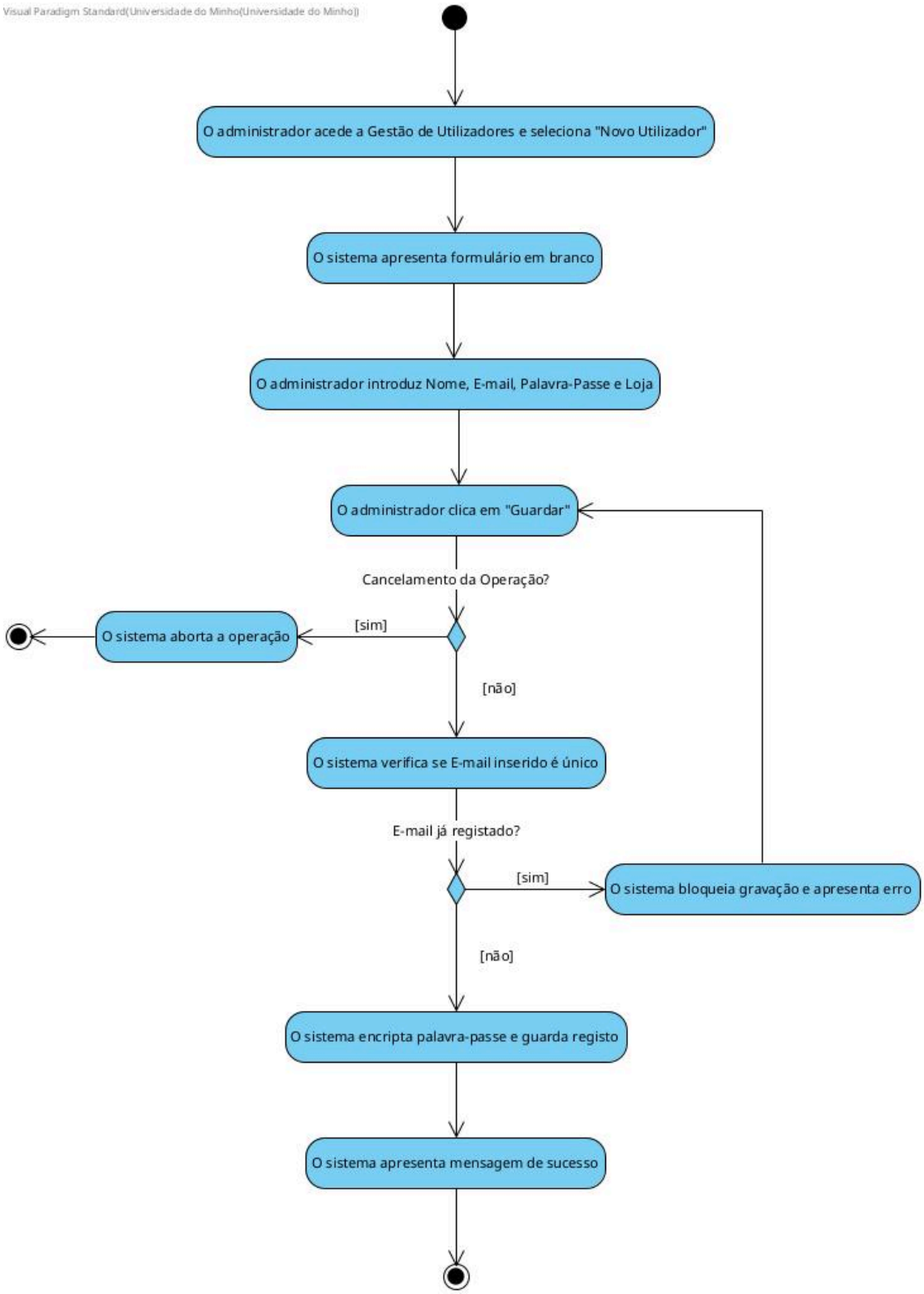


Figura 33: Diagrama de Atividades do Caso de Uso: Registar conta de utilizador

Use Case	**Editar conta de utilizador**
Descrição	O administrador atualiza os dados pessoais ou força a alteração da palavra-passe de um colaborador.
Cenários	Manutenção do ciclo de vida de uma conta de utilizador, como a atualização de dados pessoais ou a renovação de credenciais.
Pré-Condição	O administrador encontra-se autenticado com privilégios e a conta alvo existe no sistema.
Pós-condição	Os dados da conta são atualizados. Se a password tiver sido alterada, o utilizador terá de a usar no próximo login.
Fluxo Normal	1. O administrador localiza o utilizador na lista e clica na opção “Editar”. 2. O sistema pede a password de administrador. 3. Utilizador preenche a password de administrador. 4. Sistema verifica que todos os dados estão corretos. 5. O sistema preenche o formulário com a informação atual. 6. O administrador altera os dados pretendidos (ex: escreve uma nova password). 7. O administrador clica em “Guardar”. 8. O sistema valida a conformidade dos dados. 9. O sistema atualiza a informação na base de dados. 10. O sistema apresenta a mensagem de sucesso e regressa à lista.
Fluxo Alternativo	
Fluxos de Exceção	**(1) [Cancelamento da Edição] (passo 4)** 4.1 O administrador clica em “Cancelar”. 4.2 O sistema não efetua qualquer gravação. **(2) [E-mail Duplicado] (passo 5)** 5.1 O sistema verifica que o novo e-mail introduzido colide com uma conta existente. 5.2 O sistema bloqueia a alteração e reporta o erro visualmente.

Tabela 54: Caso de Uso: Editar conta de utilizador

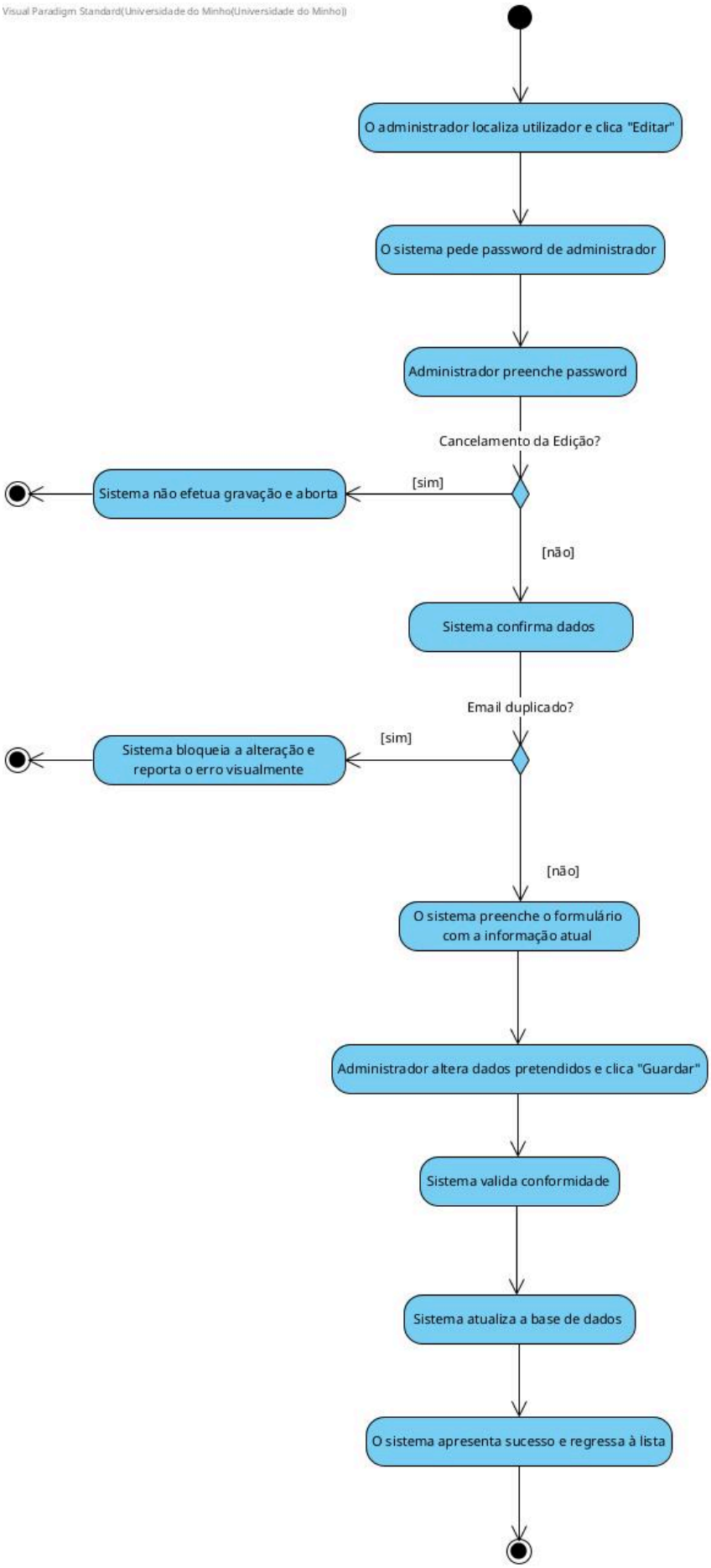


Figura 34: Diagrama de Atividades do Caso de Uso: Editar conta de utilizador

Use Case	**Bloquear conta de utilizador**
Descrição	O administrador inativa temporária ou permanentemente um acesso, preservando o histórico de operações.
Cenários	Suspensão ou desvinculação de um colaborador, exigindo a revogação imediata do seu acesso ao sistema de forma segura.
Pré-Condição	O administrador está autenticado e a conta alvo encontra-se com o estado "Ativo".
Pós-condição	O estado da conta passa a "Bloqueado", o que impede novas autenticações e revoga a sessão atual do utilizador.
Fluxo Normal	1. O administrador localiza a conta na listagem de utilizadores. 2. O administrador seleciona a option “Bloquear”. 3. O sistema pede a password de administração. 4. Utilizador preenche o campo da password. 5. Sistema confirma que a password está correta. 6. O sistema apresenta uma janela a pedir a confirmação da operação. 7. O administrador confirma. 8. O sistema valida as regras de segurança do utilizador. 9. O sistema altera o estado da conta para “Bloqueado” e revoga os tokens de sessão ativos. 10. O sistema apresenta uma mensagem de sucesso.
Fluxo Alternativo	**(1) [Desbloquear Conta] (passo 2)** 2.1 Se a conta já estiver bloqueada, o administrador escolhe “Desbloquear”. 2.2 O sistema pede a password de administração. 2.3 Utilizador preenche o campo da password. 2.4 Sistema confirma que a password está correta. 2.5 O sistema altera o estado novamente para “Ativo”, restaurando o acesso normal. 2.6 Passo 10.
Fluxos de Exceção	**(2) [Bloqueio Próprio] (passo 5)** 5.1 O sistema verifica que a conta alvo é a mesma que o administrador está a usar na sessão atual. 5.2 O sistema bloqueia a ação e informa: “Restrição de segurança: Não é possível bloquear a sua própria conta”.

Tabela 55: Caso de Uso: Bloquear conta de utilizador

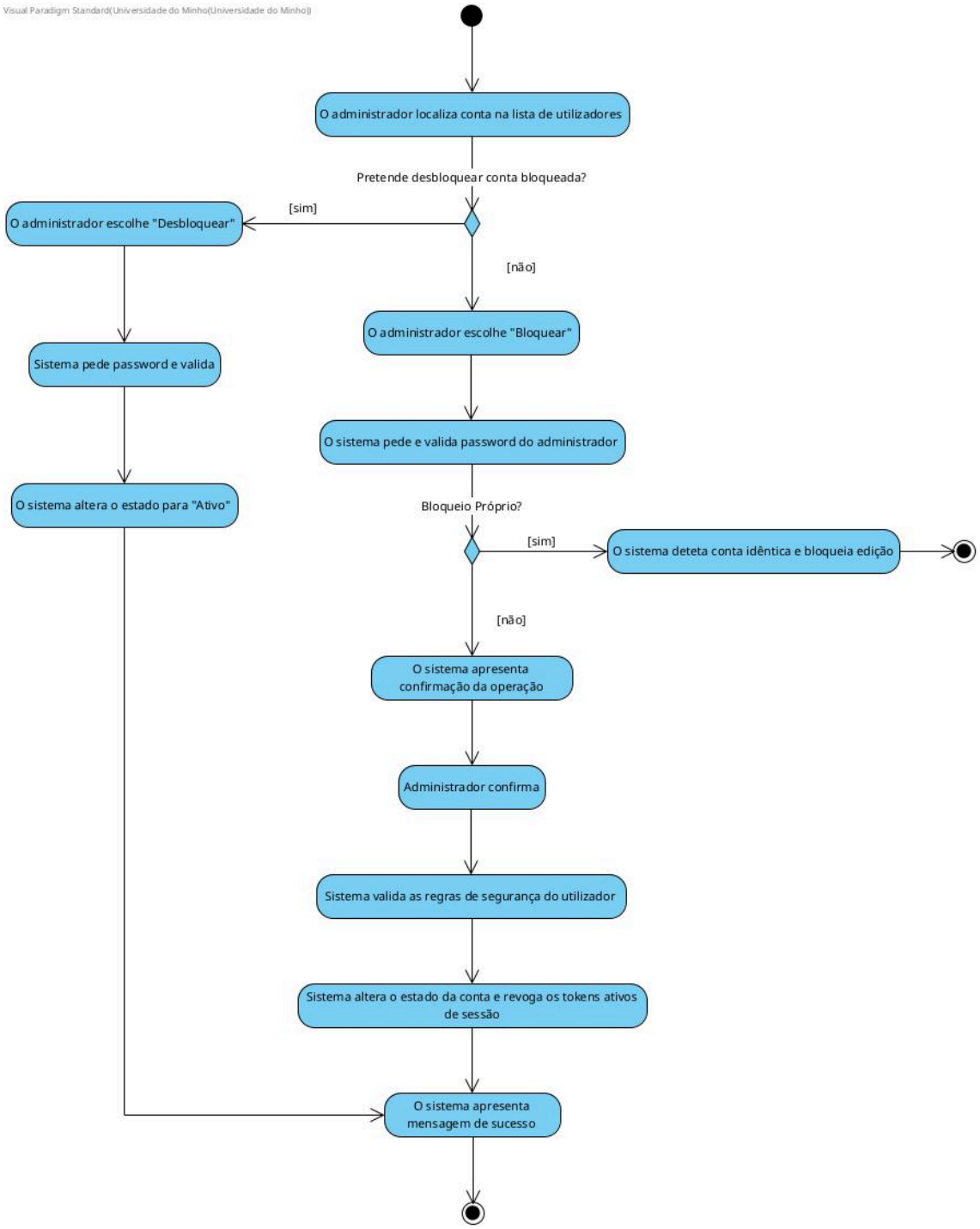


Figura 35: Diagrama de Atividades do Caso de Uso: Bloquear conta de utilizador

Use Case	**Criar perfil de acesso**
Descrição	O sistema permite a configuração de "cargos" (roles) lógicos, agregando níveis de permissão específicos.
Cenários	Estruturação de um novo nível de autorização no sistema para acomodar um novo cargo ou função organizacional.
Pré-Condição	O utilizador possui privilégios de Administração de Topo do sistema central.
Pós-condição	Um novo perfil com as permissões exatas é criado e fica disponível para ser atribuído a contas de utilizador.
Fluxo Normal	1. O administrador acede ao menu de “Perfis de Acesso” e clica em “Criar Perfil”. 2. Sistema pede por password de administrador. 3. Sistema verifica que a password está correta. 4. O sistema apresenta um ecrã para introdução do Nome e uma listagem de todas as permissões possíveis (ex: checkboxes). 5. O administrador introduz o Nome (ex: Auditor Interno) e marca as permissões desejadas. 6. O administrador clica em “Guardar”. 7. O sistema verifica a unicidade do Nome do perfil. 8. O sistema grava as configurações do perfil. 9. O sistema exibe mensagem de sucesso.
Fluxo Alternativo	**(1) [Nome Duplicado] (passo 5)** 5.1 O sistema constata que já existe um perfil com o nome introduzido. 5.2 O sistema previne a gravação e lança o alerta “Já existe um perfil com este nome”. 5.3 Sistema aguarda por alteração do nome. 5.4 Passo 6
Fluxos de Exceção	**(2) [Cancelamento da Criação] (passo 4)** 4.1 O administrador clica em “Cancelar”. 4.2 O sistema aborta o processo.

Tabela 56: Caso de Uso: Criar perfil de acesso

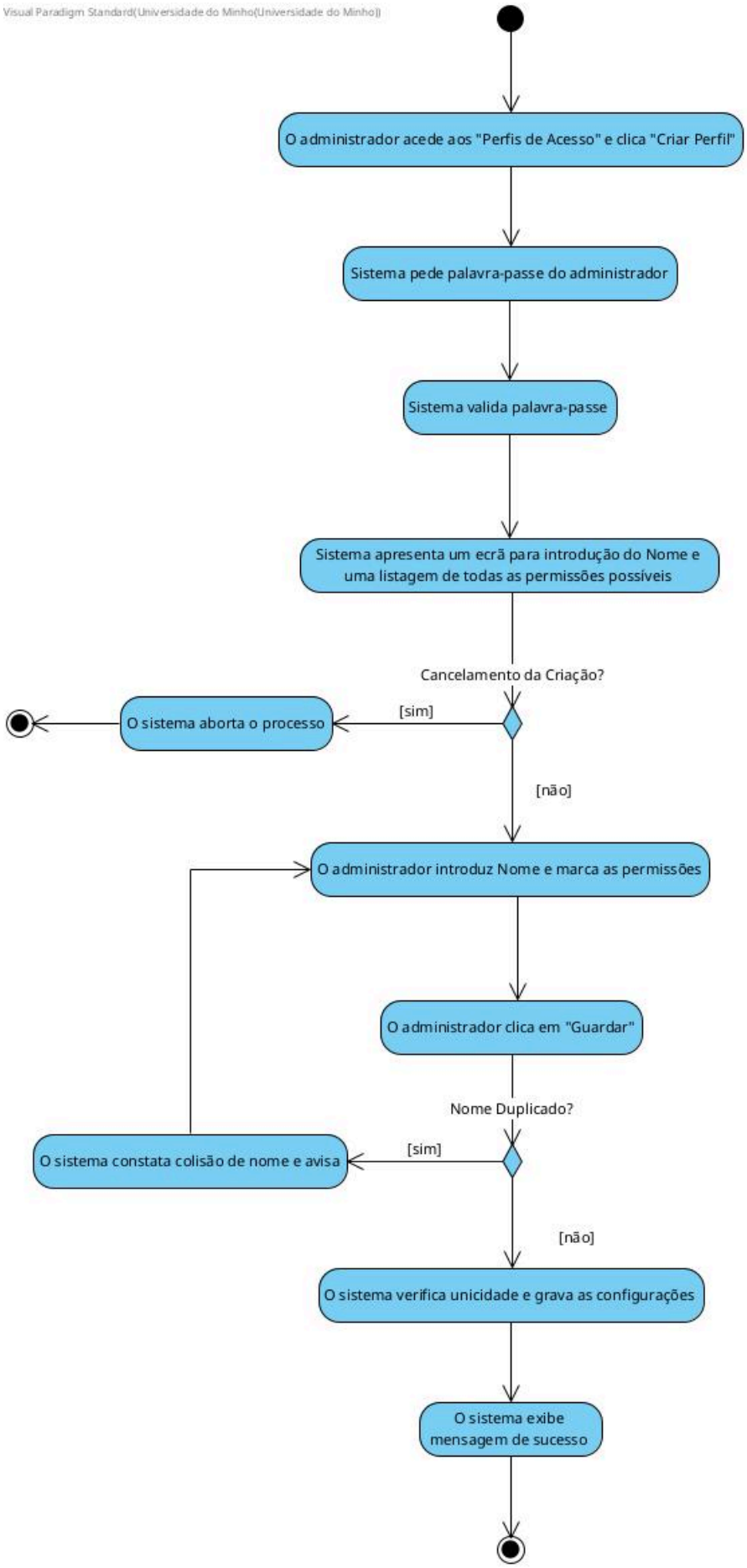


Figura 36: Diagrama de Atividades do Caso de Uso: Criar perfil de acesso

Use Case	**Editar perfil de acesso**
Descrição	O administrador altera as permissões (adiciona ou remove acessos) a um perfil já existente.
Cenários	Reajuste das permissões de um cargo existente para adaptação a novas políticas de segurança interna.
Pré-Condição	O perfil de acesso existe previamente na plataforma e o utilizador possui permissões de gestão de segurança.
Pós-condição	As alterações entram em vigor e aplicam-se a todos os utilizadores que partilham esse perfil, no seu próximo login.
Fluxo Normal	1. O administrador escolhe o perfil desejado na listagem e clica em “Editar”. 2. Sistema pede por password de administrador. 3. Sistema verifica que a password está correta. 4. O sistema apresenta as permissões atualmente ativas para o perfil. 5. O administrador seleciona ou remove as caixas de seleção das permissões. 6. O administrador clica em “Guardar”. 7. O sistema atualiza as regras do perfil na base de dados. 8. O sistema avisa do sucesso da operação.
Fluxo Alternativo	
Fluxos de Exceção	

Tabela 57: Caso de Uso: Editar perfil de acesso

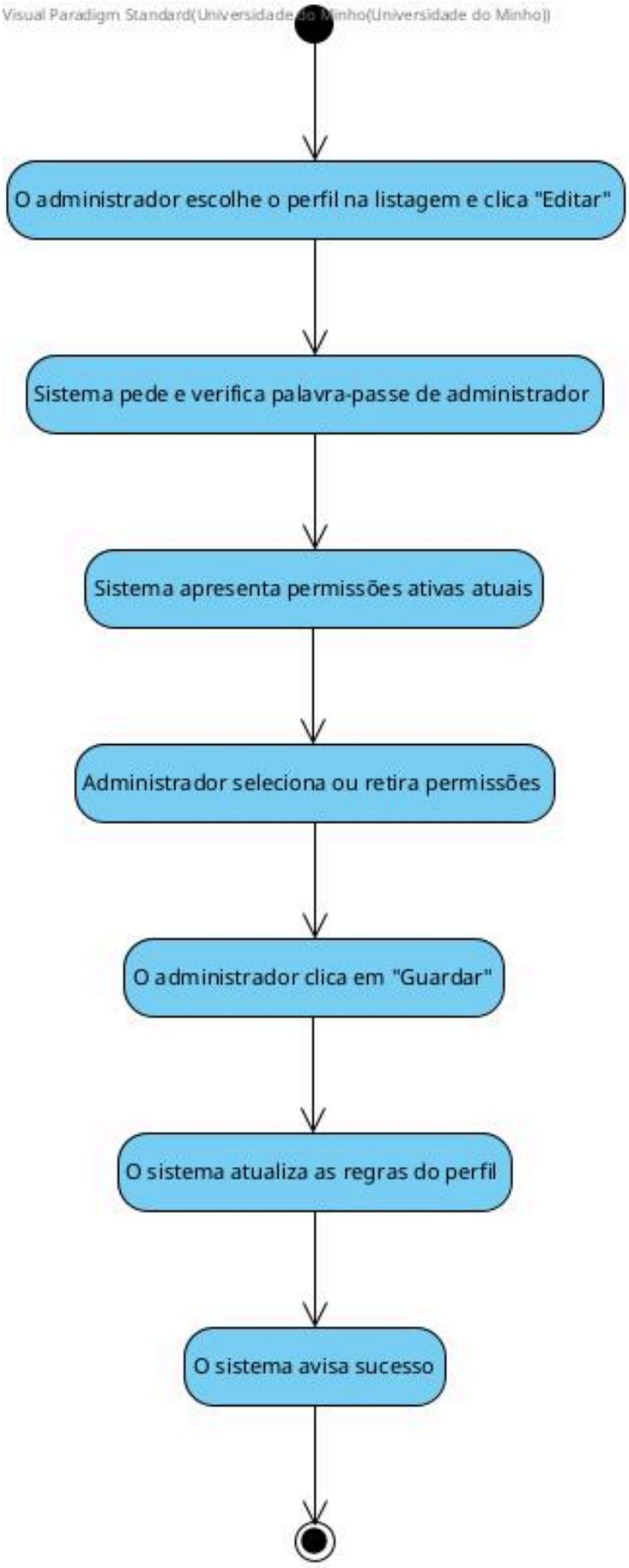


Figura 37: Diagrama de Atividades do Caso de Uso: Editar perfil de acesso

Use Case	**Atribuir perfil a utilizador**
Descrição	O administrador vincula um colaborador específico aos perfis de acesso que definem o seu alcance no software.
Cenários	Alteração das funções de um colaborador na empresa, exigindo a readequação dos seus privilégios no sistema.
Pré-Condição	O utilizador e o(s) perfil(is) de acesso já existem registados e válidos no sistema.
Pós-condição	O utilizador adquire (ou perde) os privilégios estipulados, com impacto imediato na sua interface de navegação no login seguinte.
Fluxo Normal	1. O administrador acede à edição de uma conta de utilizador e transita para o separador “Perfis”. 2. O sistema apresenta a lista de perfis disponíveis na plataforma. 3. O administrador seleciona os perfis desejados para aquele colaborador. 4. Sistema pede por password de administrador. 5. Sistema verifica que a password está correta. 6. O administrador clica em “Guardar”. 7. O sistema valida que pelo menos um perfil foi selecionado. 8. O sistema efetua a vinculação na base de dados. 9. O sistema apresenta a notificação de êxito da operação.
Fluxo Alternativo	
Fluxos de Exceção	**(1) [Sem Perfil Selecionado] (passo 5)** 5.1 O sistema deteta que todas as opções de perfil foram desmarcadas. 5.2 O sistema não processa a gravação e exige o cumprimento da regra de negócio: “O utilizador tem de estar associado a, pelo menos, um perfil de acesso válido”.

Tabela 58: Caso de Uso: Atribuir perfil a utilizador

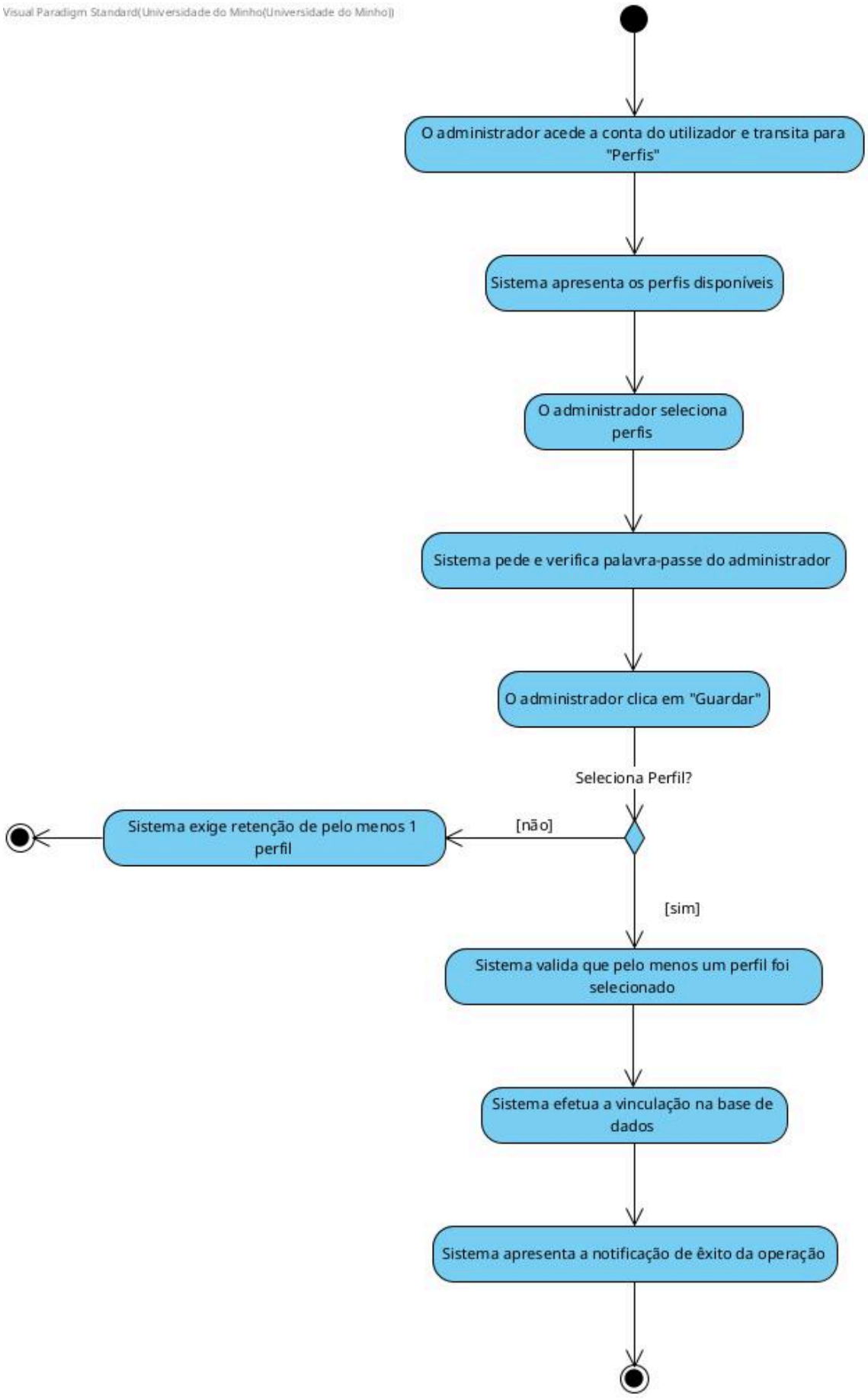


Figura 38: Diagrama de Atividades do Caso de Uso: Atribuir perfil a utilizador

Diagrama dos Casos de Uso

A presente subsecção expõe os Diagramas de Casos de Uso do Sistema de Gestão da Taki, modelados de acordo com as normas da UML. O principal objetivo destas representações visuais é ilustrar as fronteiras do sistema e as interações diretas entre os utilizadores e as funcionalidades da plataforma.

Para garantir uma leitura estruturada e evitar diagramas excessivamente complexos, optou-se por modelar os casos de uso em três vistas separadas, correspondentes aos três principais perfis de acesso da organização: o Operador de Caixa, o Gestor de Loja e o Gestor Central.

Importa clarificar que, embora modelados em diagramas distintos para efeitos de clareza visual, existe uma hierarquia cumulativa de permissões na plataforma. Ou seja, os perfis de hierarquia superior (Gestor de Loja e Gestor Central) têm a capacidade inerente de executar as operações base do Operador de Caixa, estando os diagramas focados em ilustrar as responsabilidades e ferramentas exclusivas de cada nível.

As três vistas do sistema dividem-se da seguinte forma:

- **Operações do Operador de Caixa:** Ilustra as funcionalidades base e de operação diária na frente de loja;
- **Operações do Gestor de Loja:** Focado na gestão da filial, controlo de inventário local e notas de encomenda;
- **Operações do Gestor Central:** Focado na gestão global da cadeia, administração da plataforma, relatórios consolidados e definição de catálogos.

Este diagrama foca-se nas interações do Operador de Caixa com o sistema. Representa o nível base de acesso à plataforma, ilustrando os casos de uso estritamente necessários para a operação diária da frente de loja, sem comprometer as áreas sensíveis de gestão ou administração.

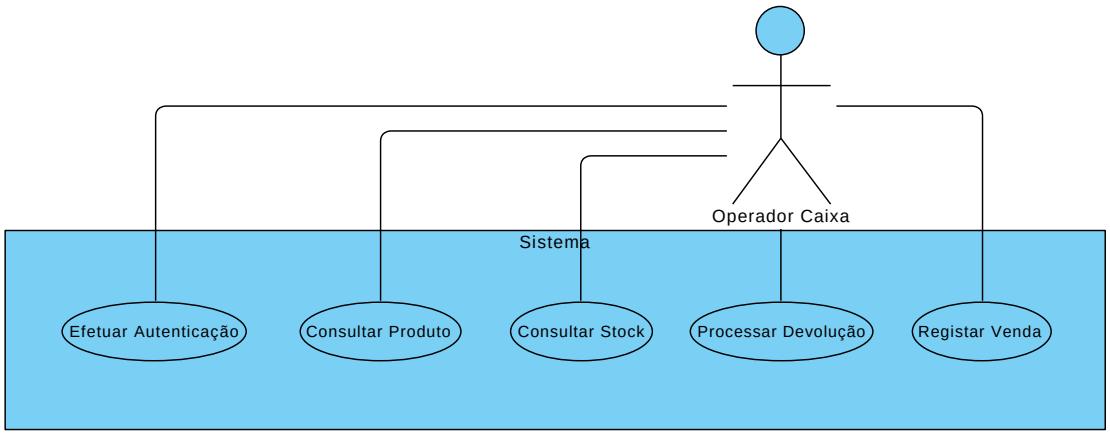


Figura 39: Diagrama Casos de Uso 1

Este modelo ilustra as permissões exclusivas do Gestor de Loja. Destaca-se o foco nas operações de retaguarda da filial local ligadas ao controlo de alertas de inventário, ajustes de stock e emissão de requisições a fornecedores.

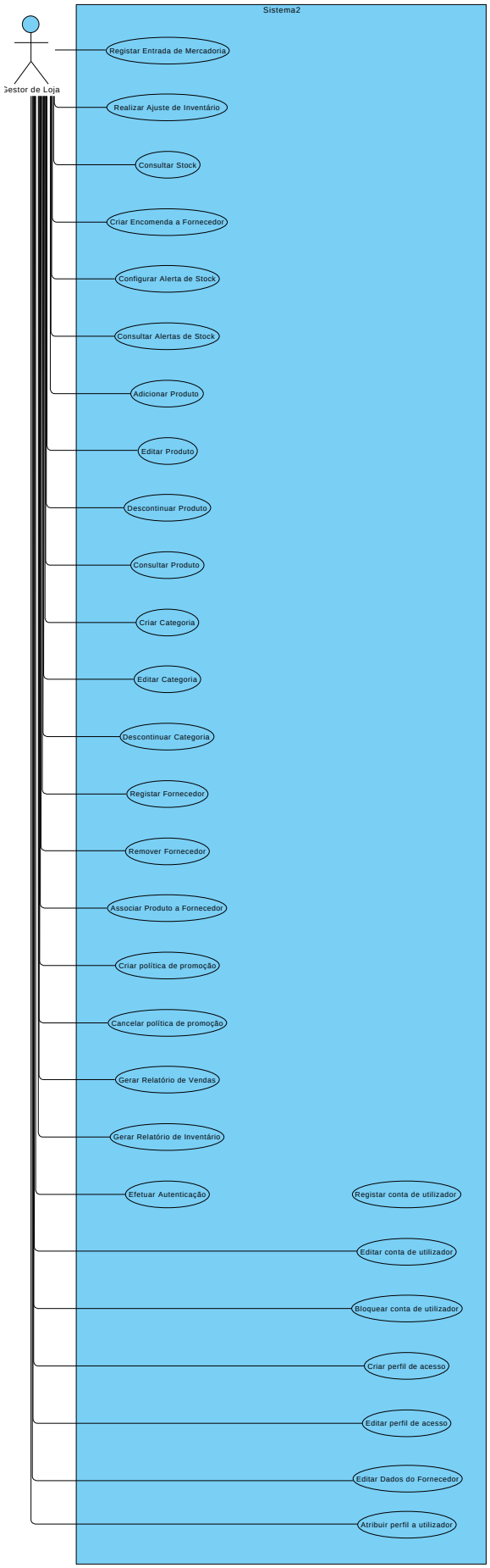


Figura 40: Diagrama Casos de Uso 2

O terceiro diagrama mapeia as responsabilidades do Gestor Central. Este ator interage com casos de uso de caráter macro e estratégico, tais como a gestão do catálogo global de produtos, definição de promoções para toda a rede, gestão de contas de utilizador e a extração de relatórios consolidados para apoio à decisão da direção.

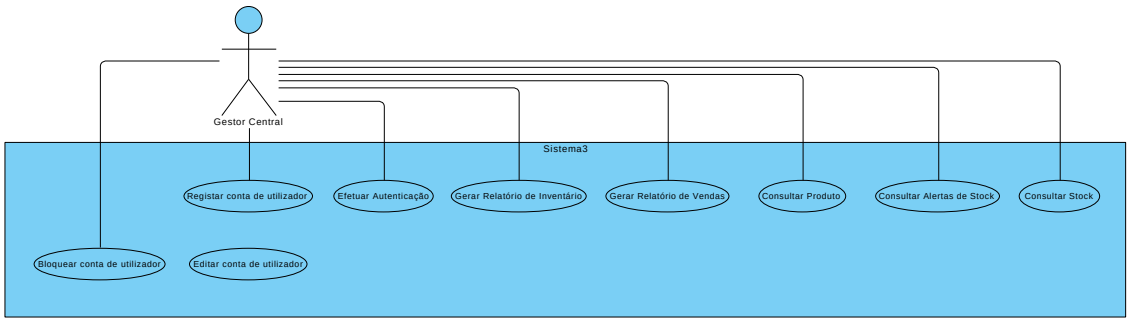


Figura 41: Diagrama Casos de Uso 3

2.3. Utilização de Inteligência Artificial

De acordo com os objetivos metodológicos delineados para a unidade curricular de Laboratórios de Informática IV, que fomenta o uso intensivo de assistentes baseados em Inteligência Artificial Generativa (IAG), a presente secção documenta a forma como os Modelos de Linguagem de Grande Escala (LLM) foram empregues durante a fase de **Conceção e Engenharia de Requisitos**. A aplicação destas ferramentas visou a otimização de tarefas analíticas, a estruturação de artefactos de engenharia de *software* e a aceleração da modelação visual.

2.3.1. Estratégia de Interação e Diretrizes

Durante a etapa de levantamento e análise de requisitos, o assistente de IA assumiu o papel de co-piloto analítico. A estratégia de interação baseou-se na aplicação de técnicas de ***Prompt Engineering***, nomeadamente ***Role-Prompting*** (atribuição de um papel de especialista à IA) e ***Few-Shot Prompting*** (fornecimento de exemplos prévios para calibrar o formato de saída). O processo iniciou-se sempre com o fornecimento de um contexto exaustivo, incluindo as transcrições das reuniões com os ***stakeholders*** e os resultados dos questionários operacionais, seguido de instruções rigorosas quanto ao formato técnico pretendido.

Foi estipulada uma diretriz inegociável de ***human-in-the-loop*** (validação humana obrigatória). Todos os artefactos gerados, desde grelhas de requisitos a código para diagramas, foram iterativamente analisados, corrigidos e refinados pela equipa de desenvolvimento (*SegFault*). Esta etapa de curadoria garantiu que os resultados gerados pela IA mantinham o rigor técnico exigido e o alinhamento com a filosofia operacional e minimalista da *Taki* ("**Não inventes, Taki!**").

2.3.2. Interações e *Prompts* Chave na Engenharia de Requisitos

A integração da IA incidiu de forma transversal sobre as várias subfases da Análise de Requisitos, desde a elicitação formal até à modelação funcional e comportamental. Destacam-se as seguintes interações e os respetivos ***prompts*** representativos, utilizados para orientar a geração de cada artefacto:

1. Elicitação e Estruturação de Requisitos

Após a recolha dos dados empíricos e a análise do estudo de mercado, a IA foi incumbida de interpretar os dados em bruto e transpor as necessidades de negócio para tabelas rigorosas de Requisitos Funcionais e Não Funcionais.

- **Exemplo de *Prompt:*** *"Atua como um Engenheiro de Requisitos especializado. Com base nas atas de reunião com os stakeholders e no relatório do inquérito de avaliação operacional em anexo, identifica e extrai os requisitos funcionais e não funcionais para o Sistema de Gestão da Taki. Estrutura os resultados num formato tabular rigoroso, contendo as seguintes colunas: ID, Requisito de Utilizador, Fonte, Área do Sistema, Especificação detalhada e Relevância para o negócio."*

2. Definição do Modelo de Domínio

Para assegurar que a arquitetura de dados suportava eficazmente as operações do sistema, recorreu-se ao assistente para realizar a análise semântica inicial, extraindo as principais entidades do domínio e as suas inter-relações a partir do texto dos requisitos.

- **Exemplo de** Prompt: *"Analisa os requisitos funcionais e não funcionais documentados e identifica as principais entidades conceptuais para o Modelo de Domínio do sistema da Taki. Para cada entidade, define os seus atributos chave e especifica as cardinalidades das relações com as restantes entidades (ex: 1 para N, N para M). Apresenta a resposta num formato estruturado que facilite a posterior modelação visual em UML."*

3. Redação de *User Stories*

Com o domínio e os requisitos consolidados, a IA foi instruída a traduzir as funcionalidades técnicas em épicos e histórias de utilizador orientadas ao valor de negócio, segmentadas pelos vários atores do sistema.

- **Exemplo de *Prompt:*** *"Assume o papel de um Product Owner. Considerando a lista de requisitos funcionais previamente consolidada, redige as User Stories correspondentes, segmentando-as rigorosamente pelos perfis de acesso do sistema: 'Operador de Caixa (Frente de Loja)', 'Gestor de Armazém (Operações de Retaguarda)' e 'Gestor (Administração)'. Aplica a sintaxe ágil ('Como [perfil], quero [ação] para que [valor/motivo]') e inclui uma lista concisa de critérios de aceitação para cada história."*

4. Casos de Uso e Geração de Diagramas em *PlantUML*

Um dos marcos mais evidentes da eficiência promovida pela IA revelou-se na transição das especificações dos Casos de Uso para a engenharia modelar. O assistente interpretou os fluxos normais e alternativos para conceber rapidamente as lógicas de atividade precisas, gerando diretamente a sintaxe de código necessária para a sua renderização gráfica.

- **Exemplo de *Prompt:*** *"Com base no documento 'relatorio_etapa2.pdf' em anexo, analisa as descrições textuais, os fluxos normais e as exceções dos Casos de Uso detalhados. A partir desta análise, gera o código correspondente em sintaxe estrita de PlantUML para os respetivos diagramas de atividade. Garante que os fluxos de decisão, as bifurcações (fork/join) e os estados finais e iniciais estão sintaticamente corretos e em total conformidade com a notação UML 2.0."*

2.3.3. Impacto Operacional e Validação

A adoção desta metodologia assistida por LLM reduziu drasticamente o tempo despendido em formatação processual e redação de ***boilerplate***. Esta delegação burocrática libertou a equipa para atividades de maior valor, canalizando o foco para a validação da arquitetura lógica, resolução de ambiguidades de negócio e escrutínio do Modelo de Domínio. Em suma, a IA não substituiu o pensamento crítico dos engenheiros, atuando antes como um catalisador de produtividade que acelerou a transição entre o levantamento de necessidades e a modelação formal do sistema.

3. Arquitetura e *Design* do *Software* utilizando LLM

Nesta secção, dissecamos a arquitetura e o *design* do *software* sob a ótica da integração com *Large Language Models* (LLMs), **estabelecendo uma estrutura robusta que orquestra a complexidade inerente a estes sistemas**. A análise será dividida em camadas fundamentais que garantem tanto a escalabilidade quanto a manutenibilidade da solução. Iniciamos com o ***Data Layer***, que define a gestão e o fluxo da informação crítica; seguimos para a **Lógica de Negócio**, onde as regras centrais da aplicação são encapsuladas; e avançamos para o **UI**, o ponto de interação direta com o usuário. Complementando esta visão estrutural, abordaremos a **Validação da Arquitetura e *Design***, assegurando que as decisões tomadas atendam aos requisitos não funcionais de performance e segurança, culminando na camada de **Utilização de Inteligência Artificial**, que detalha como os LLMs foram orquestrados para potencializar a inteligência da aplicação de forma controlada e eficiente.

3.1. *Data Layer*

Esta secção descreve a arquitetura da camada de dados do sistema Taki, responsável pela persistência, integridade e segurança da informação. A abordagem adotada transitou da abstração do modelo conceptual para o desenho lógico rigoroso, desenhado especificamente para um Sistema de Gestão de Bases de Dados Relacionais (*PostgreSQL*). Para além das regras de normalização e das estruturas relacionais, este capítulo documenta as decisões críticas de *design* arquitetural, tais como a escolha dos tipos de dados para suporte à sincronização distribuída (UUIDs), as políticas de retenção e as estratégias de encriptação (cifragem) aplicadas a dados sensíveis, garantindo a total conformidade com os requisitos não funcionais.

3.1.1. Modelo Concetual do *Data Layer*

Neste sub-capítulo, aborda-se o desenho concetual da base de dados, partindo dos requisitos previamente validados para a construção de um modelo sólido e sem anomalias lógicas. O objetivo é clarificar a arquitetura de informação que suportará a rede de lojas Taki, documentando de forma estruturada as entidades cruciais ao negócio e as regras que ditam a sua interação.

Apresentação da Abordagem de Modelação Realizada

Após o levantamento e classificação dos requisitos, foi iniciada a fase na qual se criou o modelo conceptual necessário para exprimir as entidades com os respetivos atributos, bem como os relacionamentos entre entidades do Sistema de Gestão Integrada Taki.

Para criar o esquema foi utilizada a ferramenta *open source brModelo*, na qual a representação das entidades dá-se por retângulos e a dos atributos por círculos ligados aos retângulos. Os relacionamentos são representados por losangos entre as entidades.

Finalmente, antes da realização da modelação, foram relidos sistematicamente os requisitos, de modo a não perder qualquer atributo, entidade ou relacionamento. Toda a análise respeitou as regras do negócio descritas nas atas de reunião e questionários, de forma a promover uma ponte direta entre a linguagem natural dos *stakeholders* e a abstração tecnológica.

Identificação e Caracterização das Entidades

A primeira entidade identificada foi a **Loja**, descrita em vários requisitos funcionais operacionais. Esta entidade representa os espaços físicos da rede, fundamentais para contextualizar onde ocorrem as operações. A presença de sinónimos como "Filial" ou "Ponto de Venda" confirma a variedade de nomenclatura usada para esta entidade.

Seguidamente, identificou-se a entidade **Funcionario**, que agrupa a informação dos trabalhadores que operam o sistema. A utilização do termo "Utilizador" ou "Operador" como sinónimo indica que estes funcionários interagem diretamente com o sistema, com permissões baseadas no seu cargo.

As entidades **Produto** e **Categoria** são relativas ao catálogo de vendas. O Produto armazena os dados mestre dos artigos transacionáveis, enquanto a Categoria atua como forma hierárquica de organização, referenciada nos requisitos de geração de relatórios de *stock* e vendas.

Posteriormente, identificaram-se as entidades relativas à cadeia de abastecimento: **Fornecedor**, **Encomenda** e **Linha_Encomenda**. A entidade Fornecedor regista os parceiros comerciais, a Encomenda documenta o pedido global e a Linha de Encomenda especifica os produtos e respetivas quantidades solicitadas.

A gestão da componente física logística gerou a entidade **Inventario**, que controla as quantidades disponíveis por loja, e a entidade **Movimento_Inventario**, que assegura um histórico inalterável e rastreável de todas as modificações nas existências. Em complemento, a entidade **Alerta_Stock** representa os alertas automáticos de nível de *stock* (baixo, crítico e rutura) gerados a partir das alterações do inventário.

No coração da vertente comercial do sistema identificou-se a entidade **Venda**, que agrupa os dados totais de uma transação, e a sua dependente **Linha_Venda**, que reflete a fotografia do preço e quantidade do produto no momento do registo. Associado à conclusão da transação temos o **Pagamento**, que regista a via de liquidação, e a **Fatura**, que atua como o documento fiscal exigido legalmente.

Finalmente, para dar resposta a campanhas comerciais e insatisfações de clientes, surgem as entidades **Promocao** (para regulação de descontos temporários) e a dupla **Devolucao** com **Linha_Devolucao**, que revertem valores e inventário de vendas passadas.

A tabela abaixo resume a identificação e caracterização de todas as entidades:

Designação	**Descrição**	**Sinónimos**	**Ocorrência**	**Requisito**
Loja	Representa os locais físicos de operação da empresa Taki.	Filial, Ponto de Venda	Cada loja tem um ID único atribuído na sua abertura.	RF15, RNF2
Funcionario	Dados dos colaboradores que operam o sistema.	Utilizador, Operador	Cada funcionário recebe um identificador único.	RF19, RF20, RNF1
Categoria	Organização hierárquica do catálogo de artigos.	Família, Secção	Cada categoria tem um ID único e sequencial.	RF12, RF16
Produto	Dados sobre os artigos na rede de lojas.	Artigo, Mercadoria	Cada produto recebe um identificador único aquando do seu registo no catálogo.	RF11, RF14
Promocao	Condições comerciais para aplicar descontos.	Campanha, Desconto	Cada promoção tem um identificador único de registo.	RF15

Fornecedor	Entidades que abastecem as lojas com produtos.	Parceiro, Distribuidor	Cada fornecedor possui um identificador interno único.	RF13, RF14
Encomenda	Documento interno de pedido a um fornecedor.	Pedido, Ordem de Compra	Cada encomenda tem um ID único atribuído após criação.	RF08
Linha_Encomenda	Especifica a quantidade e preço de um artigo encomendado.	Artigo Encomendado	Identificada pela linha inserida no contexto de uma encomenda.	RF08
Inventario	Registo de existências de um produto numa loja.	Stock, Existências	Cada inventário cruza a loja com um produto específico.	RF09, RF10
Movimento_Inventario	Histórico de entrada ou saída física de artigos.	Ajuste, Registo de Stock	Cada movimento recebe um ID único gerado no momento.	RF05, RF06, RF07
Alerta_Stock	Alerta operacional de nível de stock por loja/produto.	Alerta de Rutura, Alerta Crítico	Cada alerta tem um identificador único e estado de resolução.	RF10, RF17
Venda	Registo global de uma transação efetuada.	Transação, Compra	Cada venda possui um ID único do talão/caixa.	RF01, RF16
Linha_Venda	Registo individual de artigo cobrado numa venda.	Artigo Vendido	Gera um identificador para a linha dentro da Venda.	RF01
Pagamento	Via de liquidação financeira da Venda.	Transação Financeira	Cada pagamento tem o seu identificador próprio ligado à venda.	RF02
Fatura	Documento fiscal comprobatório.	Recibo, Talão Fiscal	Cada fatura tem um número fiscal da loja.	RF03, RNF4
Devolucao	Registo de reversão de uma venda a pedido do cliente.	Nota de Crédito	Cada devolução adquire um ID numérico da sua execução.	RF04
Linha_Devolucao	Detalha qual a linha da venda original a reverter.	Artigo Devolvido	Cada linha é numerada em função da devolução mestra.	RF04

Tabela 59: Identificação e Caracterização das Entidades

Identificação e Caracterização dos Relacionamentos

A identificação dos relacionamentos entre entidades foi realizada com base na análise detalhada dos requisitos funcionais, nomeadamente na forma como diferentes entidades interagem entre si no contexto de uma loja de conveniência.

A tabela seguinte resume de forma exaustiva os relacionamentos identificados para a correta modelação concetual, com as respetivas restrições de cardinalidade.

Entidade 1	**Relacionamento**	**Cardinalidade**	**Participação**	**Entidade 2**	**Requisito**
Loja	emprega	1:N	P:T	Funcionario	RF19
Categoria	contem	1:N	P:P	Categoria	RF12
Produto	categorizado	N:M	T:P	Categoria	RF12
Fornecedor	fornece	N:M	P:P	Produto	RF14
Loja	cria	1:N	P:T	Promocao	RF15
Promocao	aplica	N:M	P:P	Produto	RF15
Promocao	abrange	N:M	P:P	Categoria	RF15
Loja	emite	1:N	P:T	Encomenda	RF08
Encomenda	requisitada	N:1	T:T	Fornecedor	RF08
Linha_Encomenda	integra	N:1	T:T	Encomenda	RF08
Linha_Encomenda	pertence	N:1	T:P	Produto	RF08
Loja	guarda	1:N	T:T	Inventario	RF09
Inventario	tem	1:N	T:P	Produto	RF09
Inventario	sofre	1:N	P:T	Movimento_Inventario	RF05
Loja	monitoriza	1:N	P:T	Alerta_Stock	RF10
Produto	origina	1:N	P:T	Alerta_Stock	RF10
Funcionario	regista	1:N	P:T	Movimento_Inventario	RF06
Loja	realiza	1:N	T:T	Venda	RF01
Funcionario	processa	1:N	P:T	Venda	RF01
Venda	contem	1:N	T:T	Linha_Venda	RF01
Linha_Venda	vende	N:1	T:P	Produto	RF01
Venda	liquida	1:1	T:T	Pagamento	RF02
Venda	gera	1:1	P:T	Fatura	RF03
Venda	origina	1:N	P:T	Devolucao	RF04
Funcionario	processa	1:N	P:T	Devolucao	RF04
Devolucao	contem	1:N	T:T	Linha_Devolucao	RF04
Linha_Devolucao	devolve	1:1	T:P	Linha_Venda	RF04

Tabela 60: Identificação e Caracterização dos Relacionamentos

Identificação e Caracterização dos Atributos das Entidades e dos Relacionamentos

A caracterização dos atributos das entidades foi realizada com base nos requisitos recolhidos como complemento à identificação prévia. O processo consistiu na análise da documentação, com o fim de associar a cada entidade os respetivos campos de dados. Para cada atributo definiu-se um nome normalizado (sem espaços ou caracteres especiais) para a sua transição até à base de dados relacional. Apresentam-se de seguida as tabelas individuais para cada entidade.

Entidade Loja: A entidade Loja requer atributos que identifiquem a filial a nível operacional e geográfico. O atributo `morada` é de natureza composta e dele derivam `rua`, `cidade` e `distrito`.

Atributo	Descrição	Domínio/Tamanho	Nulo	Exemplo
`id_loja`	Identificador único	INT	N	1
`nome`	Nome comercial	VARCHAR(50)	N	Taki Braga
`telefone`	Telefone principal	VARCHAR(15)	N	253123456
`email`	Correio eletrónico	VARCHAR(100)	N	braga@taki.pt
`nif`	Identificação fiscal	VARCHAR(15)	N	501234567
`rua`	Morada (Rua)	VARCHAR(50)	N	Rua S. Vicente
`cidade`	Morada (Cidade)	VARCHAR(50)	N	Braga
`distrito`	Morada (Distrito)	VARCHAR(50)	N	Braga

Tabela 61: Atributos - Loja

Entidade Funcionario: A entidade Funcionario representa os colaboradores e os seus acessos de sistema através da cifragem no atributo `password_hash`.

Atributo	Descrição	Domínio/Tamanho	Nulo	Exemplo
`id_funcionario`	ID interno colaborador (UUID)	VARCHAR(36)	N	45
`nome`	Nome do funcionário	VARCHAR(80)	N	Diogo Costa
`email`	Email institucional	VARCHAR(100)	N	diogo@taki.pt
`cargo`	Cargo / Acesso RBAC	VARCHAR(30)	N	Gestor
`password_hash`	Palavra-passe cifrada	VARCHAR(255)	N	$2b$12eX...
`estado`	Condição atual	VARCHAR(15)	N	Ativo

Tabela 62: Atributos - Funcionario

Entidade Categoria: A entidade Categoria organiza os produtos do catálogo para pesquisa e obtenção de relatórios.

Atributo	Descrição	Domínio/Tamanho	Nulo	Exemplo
`id_categoria`	Chave da categoria (UUID)	VARCHAR(36)	N	5
`designacao`	Família de produtos	VARCHAR(50)	N	Bebidas
`estado`	Estado de disponibilidade	VARCHAR(20)	N	Ativa

Tabela 63: Atributos - Categoria

Entidade Produto: A entidade Produto dita os metadados do artigo, os preços e os impostos de referência na hora da venda.

Atributo	Descrição	Domínio/Tamanho	Nulo	Exemplo
`id_produto`	Identificador interno (UUID)	VARCHAR(36)	N	1024
`codigo_barras`	EAN comercial	VARCHAR(20)	N	560123...

nome	Designação produto	VARCHAR(80)	N	Água 1L
descricao	Detalhes adicionais	TEXT	S	Garrafa PET
preco_custo	Custo fornecedor	DECIMAL(10,2)	N	0.40
preco_venda	Preço venda público	DECIMAL(10,2)	N	1.20
unidade_medida	Unidade (Un, Kg, L)	VARCHAR(10)	N	Unidade
taxa_iva	IVA da venda	DECIMAL(4,2)	N	0.06
estado	Disponibilidade	VARCHAR(15)	N	Ativo

Tabela 64: Atributos - Produto

Entidade Promocao: A entidade Promocao possibilita o decréscimo de preços dentro de uma janela temporal.

Atributo	Descrição	Domínio/Tamanho	Nulo	Exemplo
id_promocao	Chave da campanha (UUID)	VARCHAR(36)	N	14
designacao	Nome da campanha	VARCHAR(50)	N	Promo Verão
desconto	Percentagem aplicada	DECIMAL(5,2)	N	0.15
data_inicio	Arranque desconto	DATETIME	N	2026-06-01
data_fim	Término desconto	DATETIME	N	2026-09-01
estado	Validade no sistema	VARCHAR(15)	N	Ativa

Tabela 65: Atributos - Promocao

Entidade Fornecedor: A entidade Fornecedor detém as entidades comerciais que providenciam a mercadoria.

Atributo	Descrição	Domínio/Tamanho	Nulo	Exemplo
id_fornecedor	Chave do parceiro (UUID)	VARCHAR(36)	N	12
nome	Firma / Designação	VARCHAR(100)	N	Bebidas SA
nif	Número fiscal	VARCHAR(15)	N	509876543
telefone	Linha de apoio	VARCHAR(15)	N	253987654
email	E-mail oficial	VARCHAR(100)	N	apoio@empresa
estado	Condição comercial	VARCHAR(15)	N	Ativo

Tabela 66: Atributos - Fornecedor

Entidade Encomenda: A entidade Encomenda representa o documento macro (cabeçalho) de um pedido de *stock*.

Atributo	Descrição	Domínio/Tamanho	Nulo	Exemplo
id_encomenda	Número da ordem (UUID)	VARCHAR(36)	N	568
data_criacao	Instante do pedido	DATETIME	N	2026-05-12
data_entrega	Data de chegada	DATE	N	2026-05-15
estado	Fase do pedido	VARCHAR(20)	N	Pendente

Tabela 67: Atributos - Encomenda

Entidade Linha_Encomenda: A entidade Linha_Encomenda garante o detalhe atómico dos produtos a fornecer.

Atributo	Descrição	Domínio/Tamanho	Nulo	Exemplo
id_linha_encomenda	ID da linha do pedido (UUID)	VARCHAR(36)	N	1
quantidade	Volume pretendido	DECIMAL(10,3)	N	200.000
preco	Preço de custo faturado	DECIMAL(10,2)	N	0.40

Tabela 68: Atributos - Linha_Encomenda

Entidade Inventario: A entidade Inventario cruza o Produto com a Loja para fornecer os dados reais e em tempo real do armazém.

Atributo	Descrição	Domínio/Tamanho	Nulo	Exemplo
id_inventario	Chave da fila de stock (UUID)	VARCHAR(36)	N	890
quantidade	Unidades disponíveis	DECIMAL(10,3)	N	45.000
quantidade_minima	Ponto de alerta	DECIMAL(10,3)	N	10.000

Tabela 69: Atributos - Inventario

Entidade Movimento_Inventario: A entidade Movimento_Inventario documenta historicamente qualquer adição ou subtração de artigo na loja.

Atributo	Descrição	Domínio/Tamanho	Nulo	Exemplo
id_movimento	Registo de evento (UUID)	VARCHAR(36)	N	14502
tipo	Entrada/Saída stock	VARCHAR(10)	N	Saída
quantidade	Unidades movidas	DECIMAL(10,3)	N	2.000
data_registo	Momento da alteração	DATETIME	N	2026-05-20
motivo	Justificação do ajuste	VARCHAR(100)	S	Venda 102

Tabela 70: Atributos - Movimento_Inventario

Entidade Alerta_Stock: A entidade Alerta_Stock regista automaticamente estados de risco de inventário por loja e produto.

Atributo	Descrição	Domínio/Tamanho	Nulo	Exemplo
id_alerta	Identificador do alerta (UUID)	VARCHAR(36)	N	uuid-alerta
tipo_alerta	Classificação do risco	VARCHAR(20)	N	STOCK_CRITICO
quantidade_atual	Quantidade observada	DECIMAL(10,3)	N	2.000
limite_minimo	Limite de segurança	DECIMAL(10,3)	N	10.000
estado	Estado do alerta	VARCHAR(15)	N	Ativo
data_criacao	Data de abertura	DATETIME	N	2026-05-21
data_resolucao	Data de resolução	DATETIME	S	2026-05-22

Tabela 71: Atributos - Alerta_Stock

Entidade Venda: A entidade Venda formaliza o documento de compra global executado pelo cliente no terminal POS.

Atributo	Descrição	Domínio/Tamanho	Nulo	Exemplo
id_venda	Número de recibo (UUID)	VARCHAR(36)	N	102
data_hora	Fecho da transação	DATETIME	N	2026-05-20
subtotal	Valor bruto artigos	DECIMAL(10,2)	N	5.00

imposto	Soma do IVA retido	DECIMAL(10,2)	N	0.30
total	Montante final cobrado	DECIMAL(10,2)	N	5.30
estado	Pendente ou Concluída	VARCHAR(15)	N	Concluída

Tabela 72: Atributos - Venda

Entidade Linha_Venda: A entidade Linha_Venda fotografa as condições precisas de uma unidade transacionada durante uma Venda.

Atributo	Descrição	Domínio/Tamanho	Nulo	Exemplo
id_linha_venda	ID interno de item (UUID)	VARCHAR(36)	N	1
quantidade	Unidades transacionadas	DECIMAL(10,3)	N	2.000
preco	Custo comercial da hora	DECIMAL(10,2)	N	1.20
imposto	Taxa IVA do produto	DECIMAL(4,2)	N	0.23
subtotal	Resultado desta linha	DECIMAL(10,2)	N	2.40

Tabela 73: Atributos - Linha_Venda

Entidade Pagamento A entidade Pagamento regista a prova financeira de concretização da Venda.

Atributo	Descrição	Domínio/Tamanho	Nulo	Exemplo
id_pagamento	Registo interno (UUID)	VARCHAR(36)	N	102
metodo	Via de pagamento	VARCHAR(20)	N	Numerário
valor	Montante liquidado	DECIMAL(10,2)	N	5.30
troco	Devolução de moeda	DECIMAL(10,2)	N	4.70

Tabela 74: Atributos - Pagamento

Entidade Fatura: A entidade Fatura lida com o aspeto estrito de conformidade fiscal perante a Autoridade Tributária.

Atributo	Descrição	Domínio/Tamanho	Nulo	Exemplo
id_fatura	Documento legal (UUID)	VARCHAR(36)	N	1
num_fatura	Número legal SAF-T	VARCHAR(50)	N	FT 2026/102
data_emissao	Criação do documento	DATETIME	N	2026-05-20
nif_cliente	NIF do cliente (opcional)	VARCHAR(15)	S	212345678
hash	Assinatura criptográfica do documento	TEXT	S	base64…
hash_control	Versão/controlo da assinatura	VARCHAR(10)	N	1

Tabela 75: Atributos - Fatura

Entidade Devolucao: A entidade Devolucao possibilita o retorno financeiro em oposição direta a uma Venda efetuada.

Atributo	Descrição	Domínio/Tamanho	Nulo	Exemplo
id_devolucao	Recibo negativo (UUID)	VARCHAR(36)	N	1
data_hora	Momento do retorno	DATETIME	N	2026-05-21
valor	Balanço devolvido	DECIMAL(10,2)	N	2.40
metodo_reembolso	Método de devolução do valor	VARCHAR(20)	N	Original

num_nota_credito	Número da nota de crédito	VARCHAR(50)	S	NC 2026/15

Tabela 76: Atributos - Devolucao

Entidade Linha_Devolucao: A entidade Linha_Devolucao detalha perfeitamente a retoma física do artigo para o voltar a injetar no inventário.

Atributo	Descrição	Domínio/Tamanho	Nulo	Exemplo
id_linha_devolucao	Item reavido (UUID)	VARCHAR(36)	N	1
quantidade	Unidades devolvidas	DECIMAL(10,3)	N	1.500
valor	Preço deduzido	DECIMAL(10,2)	N	1.20

Tabela 77: Atributos - Linha_Devolucao

Apresentação e Explicação do Diagrama ER Produzido

Depois de identificadas as entidades, os seus relacionamentos e os atributos que as caracterizam, foi desenvolvido o diagrama ER. Este diagrama funciona como uma representação gráfica rigorosa da organização dos dados, idealizada para agilizar a compreensão da arquitetura do ecossistema informático da rede Taki.

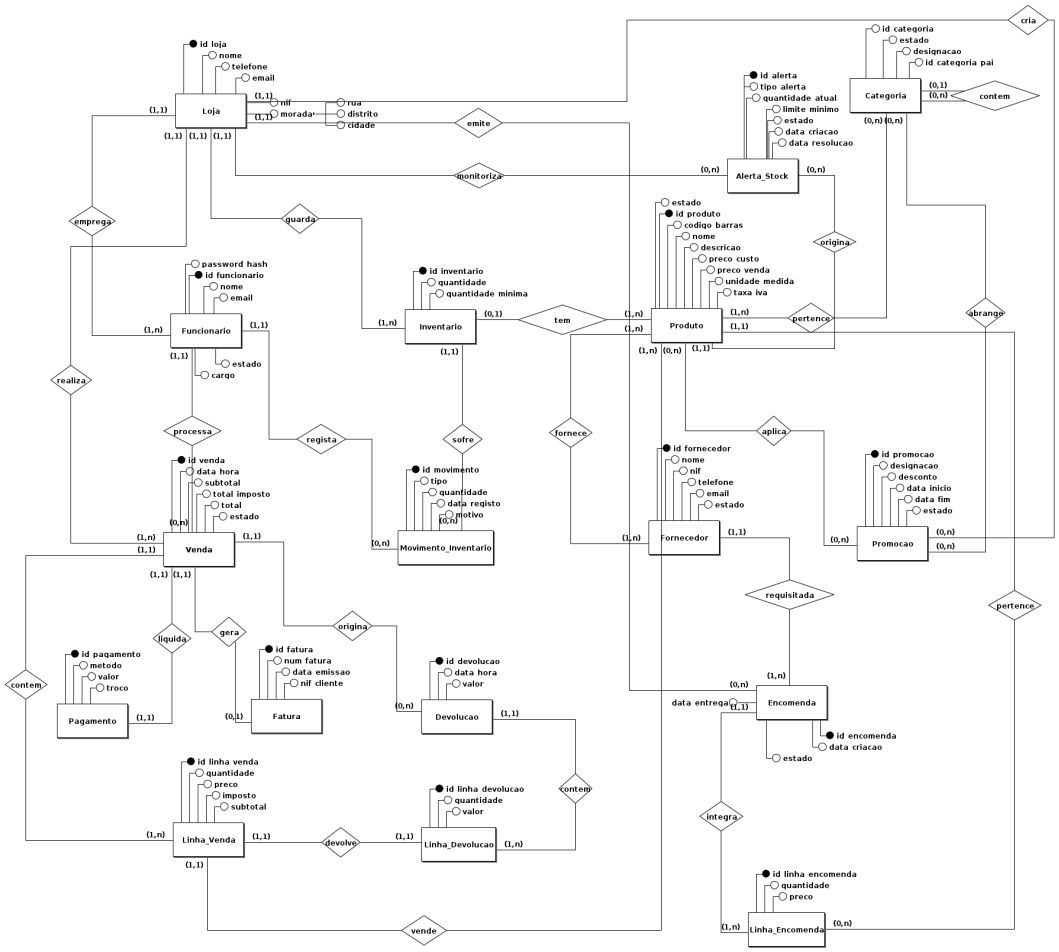


Figura 42: Modelo Conceptual de Dados (Diagrama ER) do Sistema

O diagrama ER foi desenhado na aplicação *brModelo* e construído com base nas tabelas de identificação definidas nas secções transatas deste relatório. O esquema separa intrinsecamente os processos opera-

cionais visíveis aos utentes na frente de caixa, e a vertente de bastidores onde se processa a logística e aquisições.

A base teórica de `Produto` assume uma postura forte no diagrama através das ligações de retaguarda à sua `Categoria` de origem e à `Promocao` aplicável no decorrer do tempo. Para assegurar que o artigo transita de facto da base de dados para as prateleiras, a `Encomenda` desdobra-se em `Linha_Encomenda`, de forma a comunicar inequivocamente ao `Fornecedor` o pretendido pela `Loja`.

Assim que a mercadoria atinge a `Loja`, ela entra no domínio do `Inventario`. A estrutura modular adotada obriga a que qualquer manipulação dessa tabela seja acompanhada por um `Movimento_Inventario` rastreável, assinado por um `Funcionario`. O diagrama traduz esta dependência de forma sólida e explícita para cumprimento de requisitos de controlo de acessos.

Do outro espetro encontra-se a rede comercial de balcão. O modelo isola inteligentemente as camadas do fluxo de faturação. O diagrama denota a criação da `Venda` mestre gerada pelo `Funcionario`, que acumula os preços transitórios num vetor designado por `Linha_Venda`. Posteriormente a esta estrutura ramificam-se os fechos contabilísticos (`Pagamento`) e as submissões legais tributárias (`Fatura`).

A integração isolada de uma ramificação para `Devolucao` e `Linha_Devolucao`, capaz de interligar cardinalmente apenas e unicamente com a linha original de venda, protege o modelo contra anomalias fiscais ou a injeção artificial de reembolsos incorretos. Trata-se assim de uma abstração que cobre em completude a exigência técnica de um ponto de retalho.

3.1.2. Decisões Arquiteturais e de Implementação

Antes da transição do paradigma conceptual para a modelação lógica do sistema, estabeleceram-se diversas premissas técnicas e restrições estruturais. Tais decisões asseguram que a tradução das entidades e relacionamentos para o Sistema de Gestão de Bases de Dados (`SGBD`) cumpre não apenas as regras de normalização, mas também os requisitos de negócio e de *performance* exigidos para o ecossistema distribuído da rede `Taki`.

As diretrizes adotadas enquadram-se nas seguintes vertentes críticas de desenvolvimento:

Proteção e Cifragem de Credenciais

Por forma a cumprir estritamente com os normativos de segurança definidos (como ilustrado no requisito **RNF-09**), assumiu-se que as palavras-passe dos funcionários nunca são gravadas ou manipuladas no seu estado legível nativo (*plain text*). Para este efeito, a modelação lógica reflete esta proteção através da designação do atributo `password_hash` no seio da entidade `Funcionario`. O sistema compromete-se a utilizar um algoritmo de *hashing* forte e irreversível (**BCrypt**), acrescido de um *salt* aleatório. Esta barreira neutraliza o vetor de ataque primário em caso de comprometimento da base de dados, garantindo a segurança integral das credenciais dos colaboradores.

Resolução de Identificadores num Sistema Distribuído

Devido à arquitetura autónoma inerente ao modelo de franquias da rede `Taki` (ver **RNF-03**), as lojas necessitam de manter uma operacionalidade de caixa totalmente local e, apenas num momento posterior de processamento, sincronizar a sua faturação e registos para a Sede Central.

Perante isto, inviabilizou-se a utilização comum do formato inteiro auto-incremental (`INT/SERIAL`) como Chave Primária nas tabelas de geração local. Se duas lojas abrissem de manhã e efetuassem a sua primeira transação, ambas gerariam localmente uma "Venda 1". A tentativa de agregar estas vendas no Servidor Central culminaria obrigatoriamente numa falha do SGBD por colisão de chave primária idêntica.

Assim, decidiu-se recorrer ao formato **UUID** (*Universally Unique Identifier*), traduzido à escala lógica num `VARCHAR(36)`, para a identificação de todas as tabelas operacionais e transacionais (`Venda`, `Fatura`, `Movimento_Inventario`, `Encomenda`, `Promocao` e `Produto`). A adoção desta chave alfanumérica complexa de 128-*bits* extingue a probabilidade matemática de colisões aquando do agrupamento dos dados no catálogo global da empresa. Por seu turno, a gestão estrutural exclusiva da Sede, como a designação

dos pólos no campo `Loja`, detém autorização única de inserção e, consequentemente, preserva o padrão clássico sequencial (`INT`).

Integridade Financeira e Tipos de Dados Exatos

No contexto da estipulação do formato lógico de campos que comportam capital ou grandezas de inventário a retalho, anulou-se explicitamente o uso de numéricos de vírgula flutuante (como os formatos nativos `FLOAT` ou `REAL`). Os dados neste tipo de formato sofrem sistematicamente de perdas por arredondamento binário da arquitetura processual.

Para mitigar a inaceitabilidade legal, fiscal e financeira destes desvios de micro-cêntimo nos balanços e *subtotais* do Ponto de Venda da Taki, fixou-se a regra imutável de utilização exclusiva do formato exato de armazenamento monetário. A base de dados comportará estes valores sempre com alocação numérica fixa: `DECIMAL(10,2)` (ou `NUMERIC`) para preservação rigorosa da moeda (Euros e cêntimos) e o `DECIMAL(10,3)` para o balanço de massa no Inventário (de forma a registar o movimento logístico a peso, como 0.500 kg).

Retenção de Histórico através de Exclusão Lógica (*Soft Deletes*)

Na eventualidade de um produto do catálogo transitar para inativo, ou um funcionário ser dispensado da rede, a arquitetura restringe terminantemente a eliminação física da linha de registo (instrução SQL `DELETE`) nas referidas tabelas parentais.

Caso a informação original desaparecesse do registo mestre, todas as `Faturas`, `Vendas` ou `Movimentos de Inventário` ocorridos nos anos precedentes ficariam órfãos pela restrição da Chave Estrangeira (*Foreign Key*), originando erros ou resultando numa perigosa cascata de perda de informação contabilística exigida pelos normativos `SAF-T` em Portugal.

A decisão recaiu antes no conceito estrutural denominado por Exclusões Lógicas (*Soft Deletes*). Por esse motivo, incorporou-se globalmente o atributo `estado` (`Ativo`/`Descontinuado`) nas entidades vitais (`Produto`, `Funcionario`, `Fornecedor`). O *software* e a camada de dados limitam-se a atualizar este campo booleano em caso de remoção, garantindo a permanência histórica do registo vital para preservação fidedigna dos dados da organização.

Integridade de Domínio (CHECK Constraints vs ENUMs)

Uma das pedras basilares na transição do domínio conceptual para o **PostgreSQL** prende-se com a blindagem dos atributos baseados em estado (tais como “Ativo”, “Inativo”, ou vias de liquidação como “Multibanco”, “Numerário” ou “MBWay”).

Na modelação relacional moderna, existe a tentação de se recorrer à criação de tipos fechados estáticos (`ENUM`). Contudo, face aos desafios inerentes à replicação de bases de dados distribuídas e assíncronas no contexto das diferentes franquias da Taki, a equipa preteriu terminantemente este caminho. A alteração de um `ENUM` obriga a **scripts** extra de migração (`ALTER TYPE ... ADD VALUE`) que tendem a complicar a sincronização entre a Sede e as Lojas.

Em substituição, a arquitetura abraça incondicionalmente a metodologia flexível de `CHECK Constraints` (`CHECK (coluna IN ('Valor1', 'Valor2'))`). Estas restrições fornecem um nível de inviolabilidade de dados 100% nativo no motor de Base de Dados (impossibilitando inserções com erros, como “Ativo”, a partir das lojas), possuindo a grande vantagem operadora de serem mais ágeis e menos destrutivas de gerir durante expansões de negócio futuras.

Racional de Simplificação de Escopo

Na revisão final do *Data Layer*, foram removidas estruturas adicionais que não estavam explicitamente requeridas (por exemplo, tabelas dedicadas a monitorização e auditoria transversal). Esta decisão garantiu aderência estrita aos `RF/RNF` e evitou complexidade técnica sem valor direto para os casos de uso definidos.

3.1.3. Modelo Lógico

A transição do modelo conceptual para a arquitetura relacional exige um elevado grau de precisão tecnológica que ferramentas de conversão automática (como o *brModelo*) frequentemente não conseguem assegurar sem introduzir redundâncias de nomenclatura ou anomalias nas chaves primárias. Por este motivo, a equipa optou por desenhar o Esquema Lógico diretamente com as regras e dialetos do motor que suportará a aplicação: o **PostgreSQL**.

Esta abordagem manual garante o controlo total sobre a integridade referencial, a otimização dos tipos de dados e a aplicação estrita das decisões arquiteturais documentadas na secção anterior.

Mapeamento de Tipos de Dados para PostgreSQL

As abstrações estipuladas no Modelo Conceptual foram mapeadas para os tipos de dados nativos do PostgreSQL, maximizando a *performance* e a fiabilidade do SGBD:

Conceptual	**PostgreSQL**	**Justificação Técnica**
`INT (Chave)`	`SERIAL` / `INTEGER`	Otimizado para tabelas de gestão centralizada com auto-incremento (ex: `Loja`).
`VARCHAR(36)`	`UUID`	Garante a singularidade universal sem concorrência para a sincronização das Lojas.
`DATETIME`	`TIMESTAMP`	Preserva a data e hora absolutas das transações financeiras e logísticas.
`DECIMAL(10,x)`	`NUMERIC(10,x)`	Garante a eliminação de erros de vírgula flutuante nos cálculos de dinheiro e peso.
`VARCHAR(x)`	`VARCHAR(x)`	Armazenamento padrão de texto com limite de caracteres para alocação eficiente.
`TEXT`	`TEXT`	Utilizado exclusivamente para descrições longas sem limite estrito (ex: `Produto`).

Tabela 78: Mapeamento de Domínios para PostgreSQL

Esquema Relacional (Notação Textual)

Abaixo apresenta-se o esquema lógico final, onde as entidades se transformaram em **Relações** (tabelas) e as restrições foram resolvidas através de **Chaves Primárias** (sublinhadas e com a *tag* PK) e **Chaves Estrangeiras** (*tag* FK). As associações de cardinalidade $N : M$ deram origem a tabelas de interseção independentes.

Loja (id_loja *(PK)*, nome, telefone, email, nif, rua, cidade, distrito)
Funcionario (id_funcionario *(PK)*, nome, email, cargo, password_hash, estado, id_loja *(FK)*)
Categoria (id_categoria *(PK)*, designacao, estado, id_categoria_pai *(FK)*)
Produto (id_produto *(PK)*, codigo_barras, nome, descricao, preco_custo, preco_venda, unidade_medida, taxa_iva, estado)
Fornecedor (id_fornecedor *(PK)*, nome, nif, telefone, email, estado)
Produto_Fornecedor (id_produto *(PK, FK)*, id_fornecedor *(PK, FK)*, preco_custo, preferencial)
Produto_Categoria (id_produto *(PK, FK)*, id_categoria *(PK, FK)*)
Promocao (id_promocao *(PK)*, designacao, desconto, data_inicio, data_fim, estado, id_loja *(FK)*)
Promocao_Produto (id_promocao *(PK, FK)*, id_produto *(PK, FK)*)
Promocao_Categoria (id_promocao *(PK, FK)*, id_categoria *(PK, FK)*)
Encomenda (id_encomenda *(PK)*, data_criacao, data_entrega, estado, id_loja *(FK)*, id_fornecedor *(FK)*)
Linha_Encomenda (id_linha_encomenda *(PK)*, quantidade, preco, id_encomenda *(FK)*, id_produto *(FK)*)
Inventario (id_inventario *(PK)*, quantidade, quantidade_minima, id_loja *(FK)*, id_produto *(FK)*)
Movimento_Inventario (id_movimento *(PK)*, tipo, quantidade, data_registo, motivo, id_inventario *(FK)*, id_funcionario *(FK)*)
Venda (id_venda *(PK)*, data_hora, subtotal, imposto, total, estado, id_loja *(FK)*, id_funcionario *(FK)*)
Linha_Venda (id_linha_venda *(PK)*, quantidade, preco, imposto, subtotal, id_venda *(FK)*, id_produto *(FK)*)
Pagamento (id_pagamento *(PK)*, metodo, valor, troco, id_venda *(FK)*)
Fatura (id_fatura *(PK)*, num_fatura, data_emissao, nif_cliente, hash, hash_control, id_venda *(FK)*)
Devolucao (id_devolucao *(PK)*, data_hora, valor, metodo_reembolso, num_nota_credito, id_venda *(FK)*, id_funcionario *(FK)*)
Linha_Devolucao (id_linha_devolucao *(PK)*, quantidade, valor, id_devolucao *(FK)*, id_linha_venda *(FK)*)
Alerta_Stock (id_alerta *(PK)*, id_loja *(FK)*, id_produto *(FK)*, tipo_alerta, quantidade_atual, limite_minimo, estado, data_criacao, data_resolucao)

Restrições de Integridade Adicionais (*Constraints*)

Para além da translação de chaves primárias e estrangeiras, a equipa definiu restrições de unicidade (`UNIQUE`) para garantir a consistência absoluta da lógica de negócio ao nível do motor relacional:

- O atributo `codigo_barras` da tabela **Produto** foi marcado como único para impedir que a mesma loja registe artigos repetidos em catálogos autónomos;
- Os atributos `email` (**Funcionario**) e `num_fatura` (**Fatura**) receberam restrição única para evitar sobreposição de credenciais ou falsificação fiscal;
- Na resolução do relacionamento de cardinalidade $1:1$ entre a **Venda** e a **Fatura**, optou-se por posicionar a Chave Estrangeira na tabela `Fatura` (sendo também restrita como única), dado que o documento fiscal é opcional e obrigatoriamente subsequente ao talão da venda;
- No cumprimento fiscal, a tabela `Fatura` inclui ainda `hash`, `hash_control` e validações de formato para `num_fatura` (`FT YYYY/N`) e `nif_cliente` (9 dígitos, opcional);
- A tabela `Devolucao` mantém ligação referencial explícita ao `Funcionario` que processou a operação.

Apresentação do Modelo Lógico Visual

O esquema textual acima exposto foi posteriormente modelado numa ferramenta de *design* oficial de bases de dados (o *DBeaver*), resultando na representação visual final que orientará o código SQL.

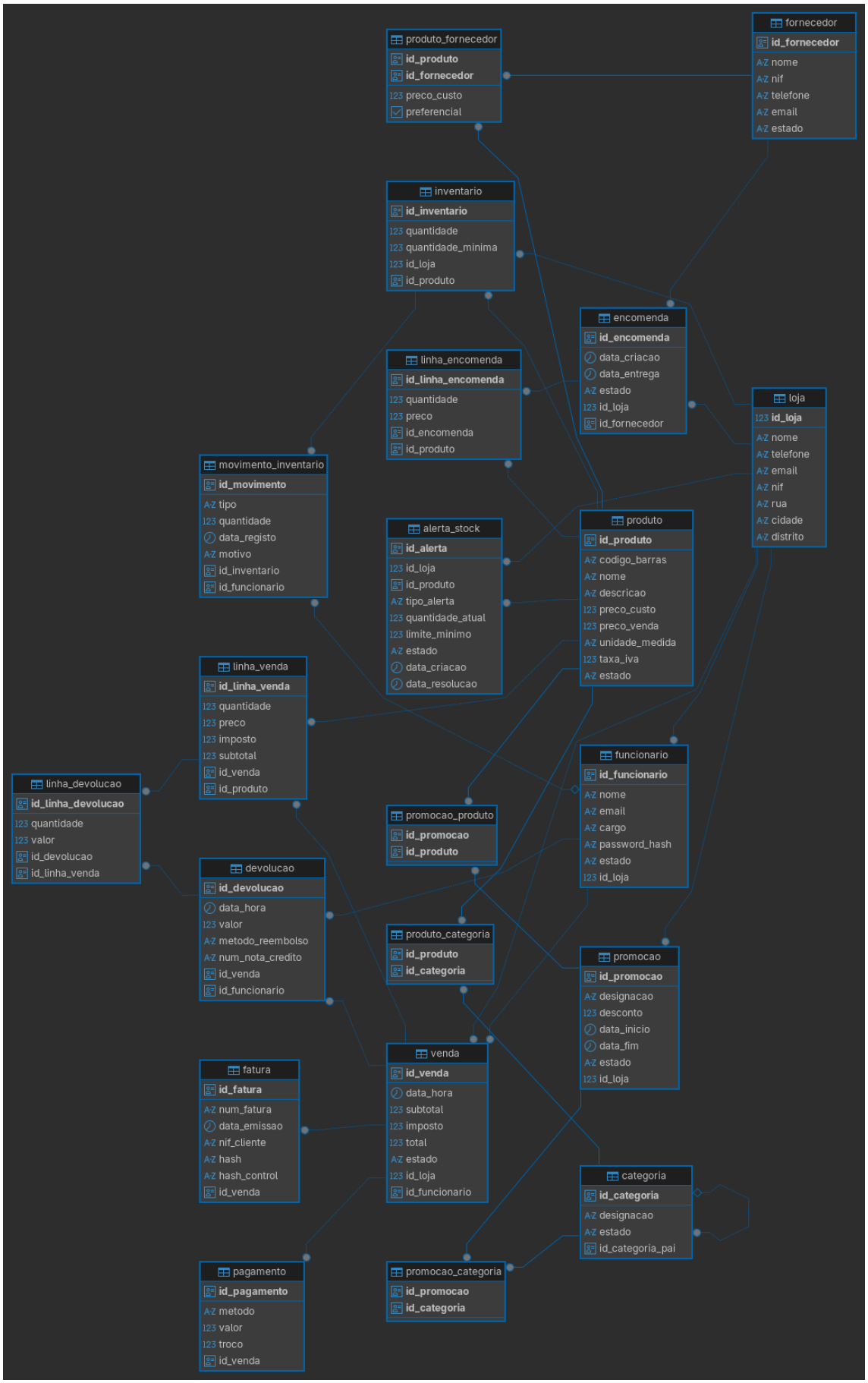


Figura 43: Modelo Lógico de Base de Dados (PostgreSQL)

3.1.4. Normalização da BD

A normalização de uma base de dados é um processo analítico formal que visa mitigar redundâncias e prevenir anomalias de inserção, atualização e eliminação (vulgarmente conhecidas por anomalias DML). O modelo relacional do ecossistema Taki, projetado no subcapítulo anterior, foi concebido estruturalmente para cumprir os requisitos das três principais Formas Normais (1NF, 2NF e 3NF).

Nas subsecções seguintes, prova-se a validade matemática do esquema face às restrições de cada uma destas formas normais.

Primeira Forma Normal (1NF)

A regra fundamental da 1NF estipula que todos os atributos de uma relação (tabela) devem ser atómicos. Isto significa que não podem existir atributos multivalorados, domínios compostos ou grupos repetitivos de dados dentro de uma única célula da tabela.

O modelo da Taki respeita a 1NF de forma exaustiva:

- **Resolução de Atributos Compostos:** O domínio "Morada" da entidade `Loja` não foi inserido como um bloco de texto inteiro ou *array*. O modelo "achatou" o conceito nas colunas perfeitamente atómicas `rua`, `cidade` e `distrito`;
- **Eliminação de Grupos Repetitivos:** O "carrinho de compras" de um cliente (múltiplos produtos numa só fatura) nunca é guardado num *array* dentro da tabela `Venda`. O modelo normalizou este comportamento ao externalizar os itens faturados para a tabela `Linha_Venda`, onde cada produto ocupa uma linha unívoca própria;
- Adicionalmente, todas as tabelas detêm uma Chave Primária (PK) definida, garantindo a unicidade de cada registo (tuplo).

Segunda Forma Normal (2NF)

Para uma relação alcançar a 2NF, necessita de cumprir os ditames da 1NF e assegurar a inexistência de **Dependências Funcionais Parciais**. Uma dependência parcial ocorre quando um atributo não-chave depende de apenas uma parte de uma Chave Primária Composta.

No caso do esquema lógico produzido, a vasta maioria das tabelas possui Chaves Primárias simples e atómicas (ex: `id_loja`, `id_produto` através de UUIDs), logo, a 2NF é atingida de forma intrínseca.

Nas entidades associativas onde existem Chaves Primárias compostas, a regra é igualmente respeitada:

- Nas tabelas `Promocao_Produto` e `Promocao_Categoria`, a Chave Primária é a união de duas Chaves Estrangeiras e não existem atributos não-chave, pelo que não há dependências parciais;
- Na tabela `Produto_Fornecedor`, apesar de existirem atributos não-chave (`preco_custo`, `preferencial`), estes dependem da combinação completa (`id_produto`, `id_fornecedor`), mantendo a conformidade com a 2NF.

O modelo cumpre, assim, a Segunda Forma Normal a 100%.

Terceira Forma Normal (3NF)

O último grande limiar de otimização estrutural, a 3NF, exige que a tabela se encontre em 2NF e que não comporte quaisquer **Dependências Funcionais Transitivas**. Isto traduz-se na regra de ouro de Codd: "Cada atributo não-chave tem de depender da chave, de toda a chave, e de nada mais do que a chave". Nenhum atributo comum pode derivar o valor de outro atributo comum.

O modelo Lógico desenhado oblitera a possibilidade destas anomalias:

- **Proteção do Catálogo:** Na tabela `Produto`, o sistema regista a Chave Estrangeira `id_categoria`, mas abstém-se deliberadamente de guardar o texto com o `nome_da_categoria`. Se a gestão central decidir alterar o nome de uma categoria de "Laticínios" para "Laticínios Naturais", a alteração ocorre unica-

mente na tabela `Categoria`. Nenhum produto fica com dados desatualizados, prevenindo anomalias de atualização (*Update Anomaly*);

- **Proteção de Vendas e Inventário:** A entidade `Venda` regista o `id_loja` e o `id_funcionario`, mas não armazena a morada da loja nem o cargo do operador de caixa. Toda a informação periférica depende inteiramente da chave primária do seu próprio domínio.

Em suma, a estrita aplicação destas regras de normalização garante um *Data Layer* robusto, limpo e imune a dados "órfãos" ou duplicados, preparando a infraestrutura para escalar ao ritmo da expansão da rede de lojas da franquia.

3.2. Lógica de Negócio

A definição da arquitetura da Lógica de Negócio constitui um dos pilares centrais do desenvolvimento do sistema, pois é nela que se materializa o fluxos de trabalho que dá sentido à aplicação. Nesta secção, são apresentadas as decisões arquiteturais adotadas para organizar e estruturar a camada responsável pela execução das operações essenciais do negócio, garantindo clareza, manutenibilidade e alinhamento com os requisitos funcionais previamente identificados.

3.2.1. Diagrama de Componentes

O **Diagrama de Componentes** ilustra a macroarquitetura lógica do sistema de gestão integrada da `Taki`. O principal propósito desta representação visual é mapear a organização estrutural do *software*, evidenciando as dependências e as interfaces de comunicação estabelecidas entre os diversos módulos do sistema.

A segmentação da lógica de negócio em blocos funcionais autónomos constitui um pilar estratégico para garantir a **escalabilidade**, a **resiliência** e a facilidade de manutenção da solução técnica. A estruturação destes subsistemas reflete uma clara separação de responsabilidades, promovendo uma arquitetura global pautada pela **alta coesão** e pelo **baixo acoplamento**. Estes princípios assumem um carácter essencial para assegurar a estabilidade da operação diária e facilitar a futura expansão do ecossistema tecnológico.

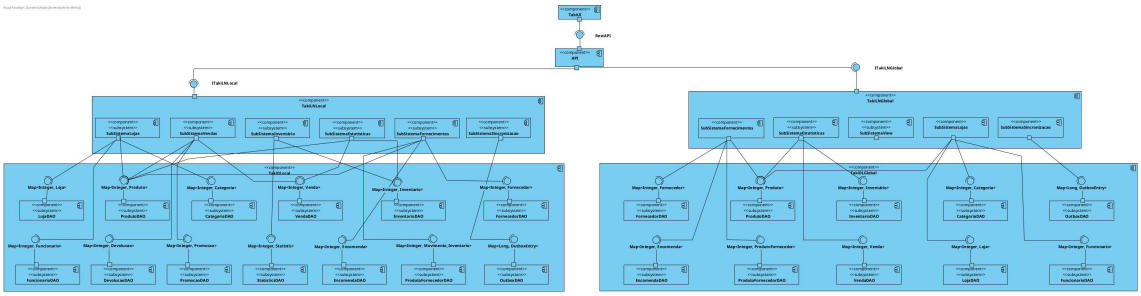


Figura 44: Diagrama de Componentes — Macroarquitetura do Sistema Taki

3.2.2. *Data Access Objects* (`DAO`)

A escolha do padrão ***Data Access Object*** **(`DAO`)** no sistema `Taki` fundamenta-se na necessidade de isolar completamente a lógica de negócio das particularidades da persistência física dos dados. Esta abordagem garante que o motor de regras da aplicação não seja afetado por mudanças na estrutura da base de dados ou no motor SQL utilizado.

Diferente de padrões como o ***ActiveRecord***, onde os modelos de domínio carregam a responsabilidade de se auto-persistirem, o DAO atua como uma camada de tradução independente. Esta separação preserva a pureza dos objetos de negócio, permitindo que estes se foquem estritamente nas regras operacionais. Comparativamente à execução direta de comandos SQL dentro das classes de serviço, a utilização de DAOs centraliza o conhecimento sobre o esquema relacional, evitando a dispersão de lógica de infraestrutura e facilitando a manutenção e a escalabilidade do sistema.

Cobertura de DAOs e Entidades: Para cobrir integralmente o modelo relacional e suportar a operação distribuída, o sistema define um conjunto de DAOs que agregam as entidades do domínio:

- **LojaDAO**: Gere os dados das filiais da rede.
- **FuncionarioDAO**: Gere as contas e permissões dos utilizadores.
- **CategoriaDAO**: Gere a estrutura hierárquica do catálogo.
- **ProdutoDAO**: Gere os dados mestre dos artigos.
- **FornecedorDAO**: Gere a persistência da entidade fornecedora.
- **ProdutoFornecedorDAO**: Gere as relações de custo entre produtos e fornecedores.
- **EncomendaDAO**: Gere o agregado de encomendas e as respetivas linhas de detalhe.
- **PromocaoDAO**: Gere as campanhas, descontos e as tabelas de associação a produtos ou categorias.
- **VendaDAO**: Gere o fluxo transacional completo de venda, incluindo pagamentos.
- **FaturaDAO**: Gera a faturação em formato SAF-T.
- **DevolucaoDAO**: Gere os processos de retorno e estorno de artigos.
- **InventarioDAO**: Gere os níveis de stock e o registo histórico de movimentos.
- **OutboxDAO**: Gere a fila de eventos necessária para a sincronização distribuída.
- **StatisticsDAO**: Especializado na extração de indicadores através de vistas e funções de agregação.

Arquitetura Modular (DAO Genérico): A arquitetura do sistema adota um fundamento modular através de um **DAO Genérico**. Este conceito estabelece uma base comum para todos os componentes de acesso a dados, padronizando operações elementares como a criação, consulta e remoção de registos. Ao definir este contrato de comportamento uniforme, o sistema garante que todos os DAOs seguem os mesmos princípios de funcionamento, facilitando a substituição da implementação real por simulações em memória durante as fases de desenvolvimento e validação.

Interação com Lógica Ativa: Para além da persistência convencional, a arquitetura prevê a interação com a lógica ativa residente no motor de base de dados. Através de componentes especializados, o sistema consome **Vistas** para apresentação de dados agregados e invoca **Funções** para a execução de cálculos complexos no servidor. Esta estratégia otimiza a performance ao delegar o processamento de grandes volumes de dados para a base de dados, devolvendo à aplicação apenas os resultados necessários para a tomada de decisão.

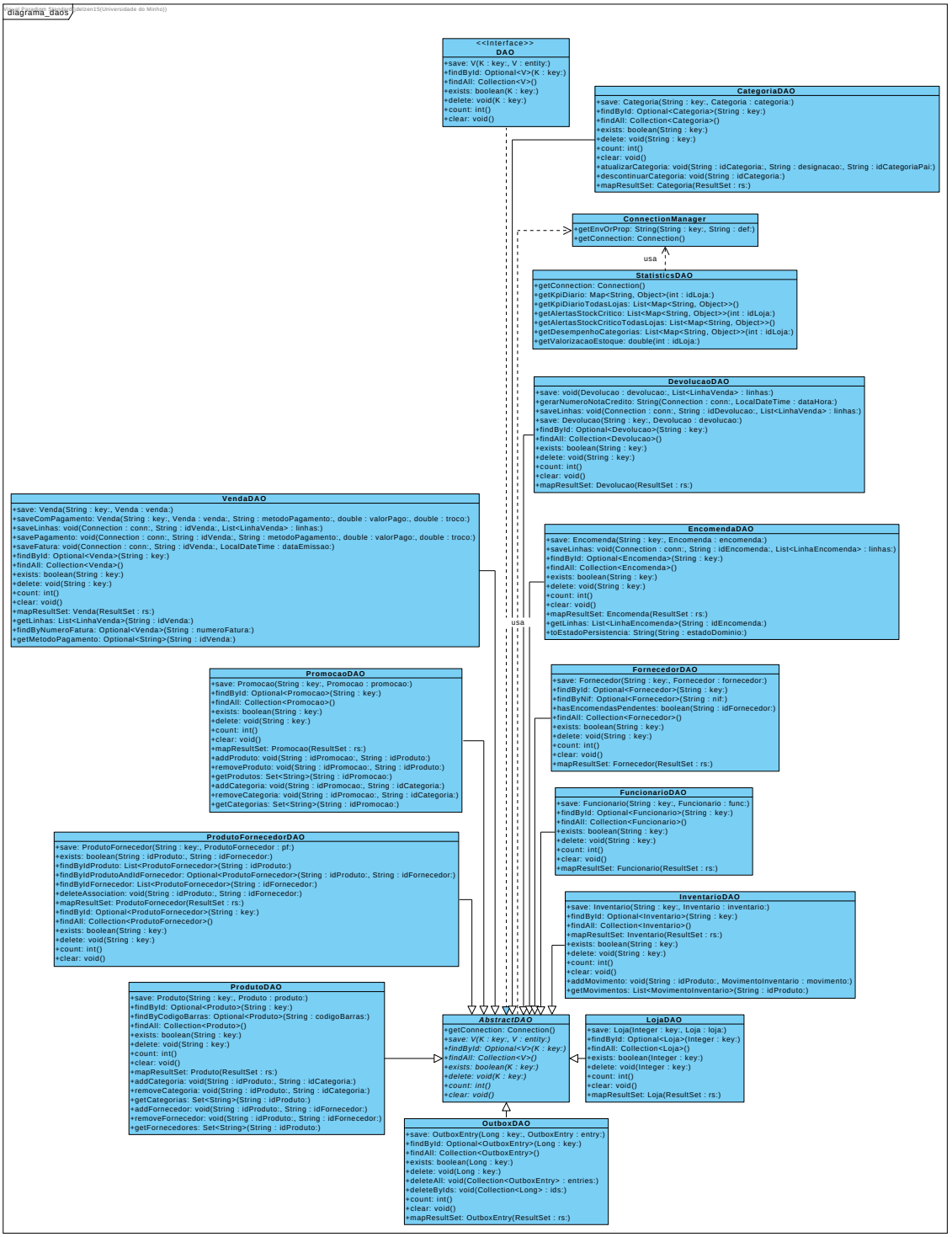


Figura 45: Diagrama UML ilustrando a Generalização dos DAOs

3.2.3. Gestão da Sincronização

A gestão de sincronização do sistema `Taki` é o componente crítico que assegura a consolidação da informação operacional das lojas no servidor central. Esta arquitetura distribuída baseia-se num modelo de reporte onde as lojas possuem autonomia total para realizar transações, delegando a harmonização dos dados para um processo assíncrono de envio à Sede.

Estratégia de Consolidação Unidirecional: A decisão fundamental nesta camada foi a implementação de um fluxo de sincronização unidirecional (Loja para Sede), focado na integridade do histórico transacional.

Em vez de uma comunicação bidirecional, o sistema garante que todos os dados transacionais gerados localmente, como registos de venda, faturas e movimentos de inventário, são replicados de forma fiável para o servidor central em Famalicão. Esta abordagem preserva a autonomia das filiais e simplifica a gestão da consistência, uma vez que a Sede atua como o repositório final para auditoria e análise global de toda a franquia. A esta lógica unidirecional acrescenta-se um canal complementar de importação periódica (*pull*), pelo qual as filiais sincronizam as regras de negócio transversais emitidas pela Sede.

Periodicidade e Agendamento: O fluxo de consolidação de dados rege-se por um modelo de reporte periódico, ocorrendo obrigatoriamente a cada 24 horas. Para garantir a estabilidade do sistema e minimizar o impacto nas operações de venda, a sincronização é agendada para ocorrer diariamente às $\mathbf{00:00}$ **(meia-noite)**. Esta cadência assegura que a Sede Central recebe a totalidade das transações diárias de forma consolidada, permitindo o processamento de indicadores de desempenho e o cumprimento de obrigações fiscais com base em ciclos de fecho diários completos.

Mecanismo de Captura de Eventos: Para garantir que nenhuma transação é perdida, o sistema utiliza gatilhos ao nível do motor de base de dados que monitorizam as mutações de estado em tempo real. Sempre que ocorre uma venda ou um acerto de *stock*, o evento é registado numa fila de saída persistente local. Esta estratégia garante a atomicidade: a operação de negócio local só é validada se a intenção de sincronização for igualmente gravada, protegendo o sistema contra falhas de conectividade ou erros de aplicação no momento da transação.

Identificação Global e Agregação: Para suportar a fusão de dados provenientes de múltiplas filiais sem conflitos, foi adotada a utilização de identificadores universais únicos. Esta decisão permite que uma venda realizada em qualquer ponto da rede possua um identificador irrepetível, facilitando o processo de agregação na Sede. O sistema opera sob um modelo de reporte periódico, onde os dados locais são empacotados e remetidos para o catálogo global, permitindo à administração uma visão consolidada da rentabilidade e do estado do inventário de toda a cadeia de lojas.

Resiliência e Confirmação de Receção: O protocolo de sincronização rege-se por um princípio de confirmação estrita. Os eventos capturados permanecem na fila local até que exista uma validação de que os dados foram integrados com sucesso pelo servidor central. Este processo de limpeza da fila apenas após a confirmação garante que, mesmo perante instabilidades de rede prolongadas, a informação é preservada localmente e reenviada de forma ordenada até que a consolidação na Sede seja concluída com sucesso.

Desenho Estrutural

A arquitetura deste subdomínio baseia-se na separação entre o orquestrador (`SubSistemaSincronizacao`) e a lógica de comunicação e rede (`SincronizacaoService`). Destaca-se o uso rigoroso do padrão arquitetural *Outbox*, gerido pelo componente `OutboxDAO`, que assegura a persistência e resiliência no armazenamento local dos eventos que aguardam envio.

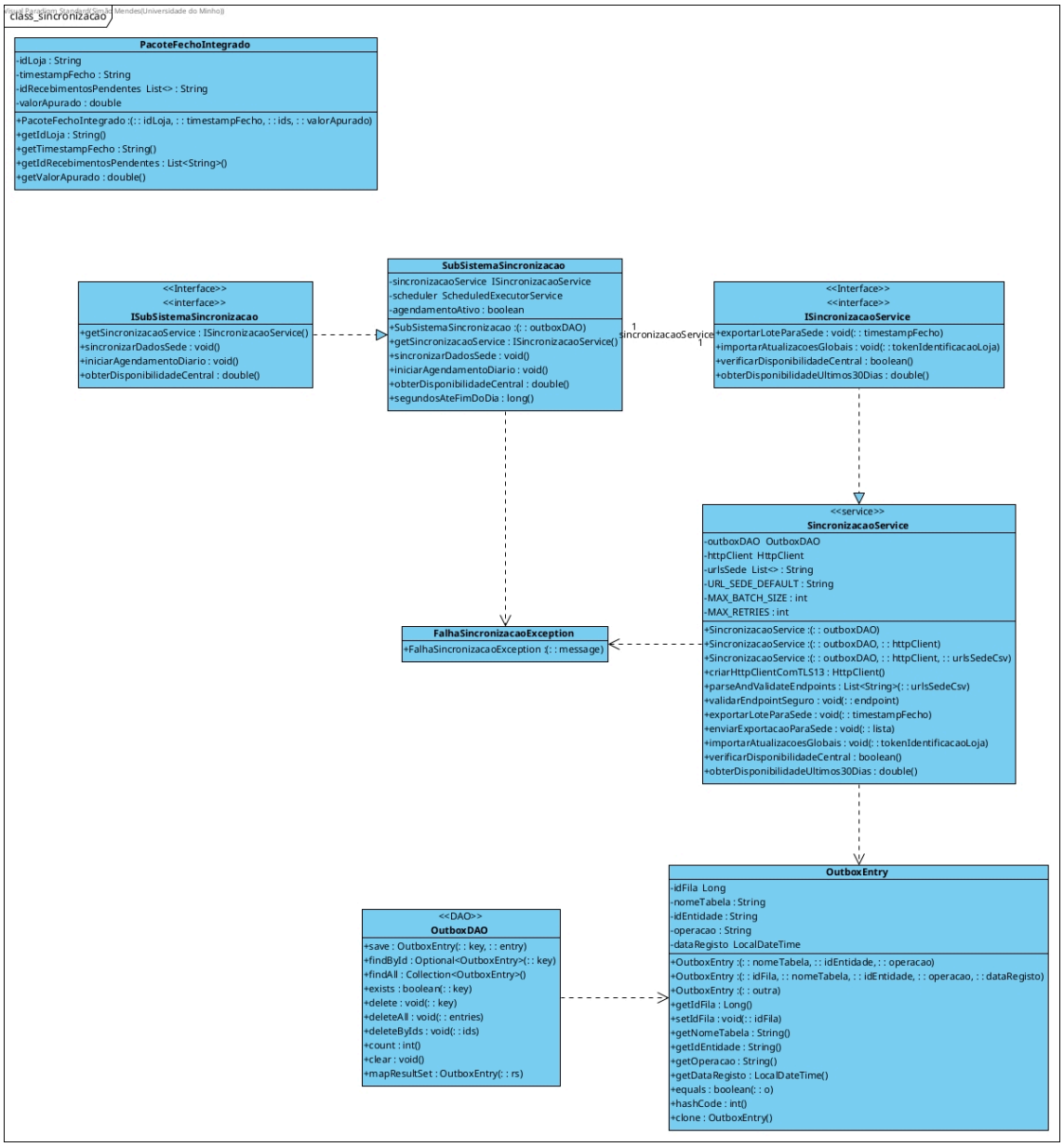


Figura 46: Diagrama de Classes: Subdomínio de Sincronização

Desenho Comportamental: Captura e Agendamento

A base da sincronização é a captura contínua de eventos acoplada à execução de rotinas diárias e manuais.

O fluxo seguinte ilustra como o sistema reage de forma transparente a alterações críticas (como o registo de novas vendas), assegurando a criação de uma entrada na tabela *Outbox* na mesma transação atómica que o evento de negócio.

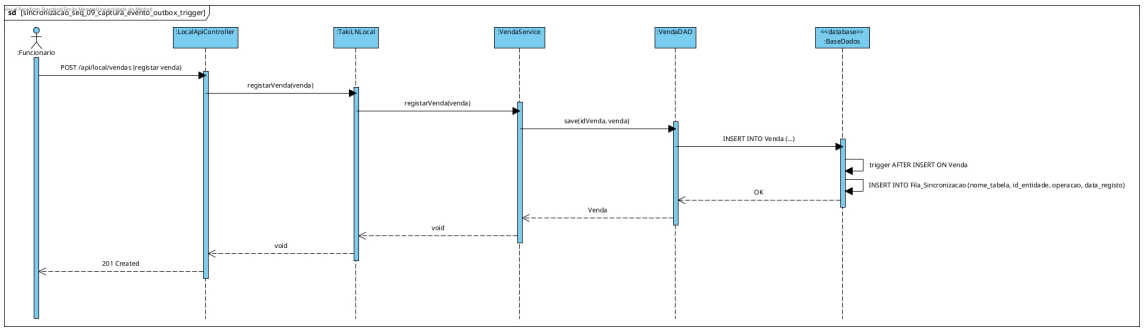


Figura 47: Diagrama de Sequência: Captura de Evento (*Outbox Trigger*)

Para garantir a operação autónoma, o sistema inicializa automaticamente uma rotina em plano de fundo aquando do arranque da aplicação na loja.

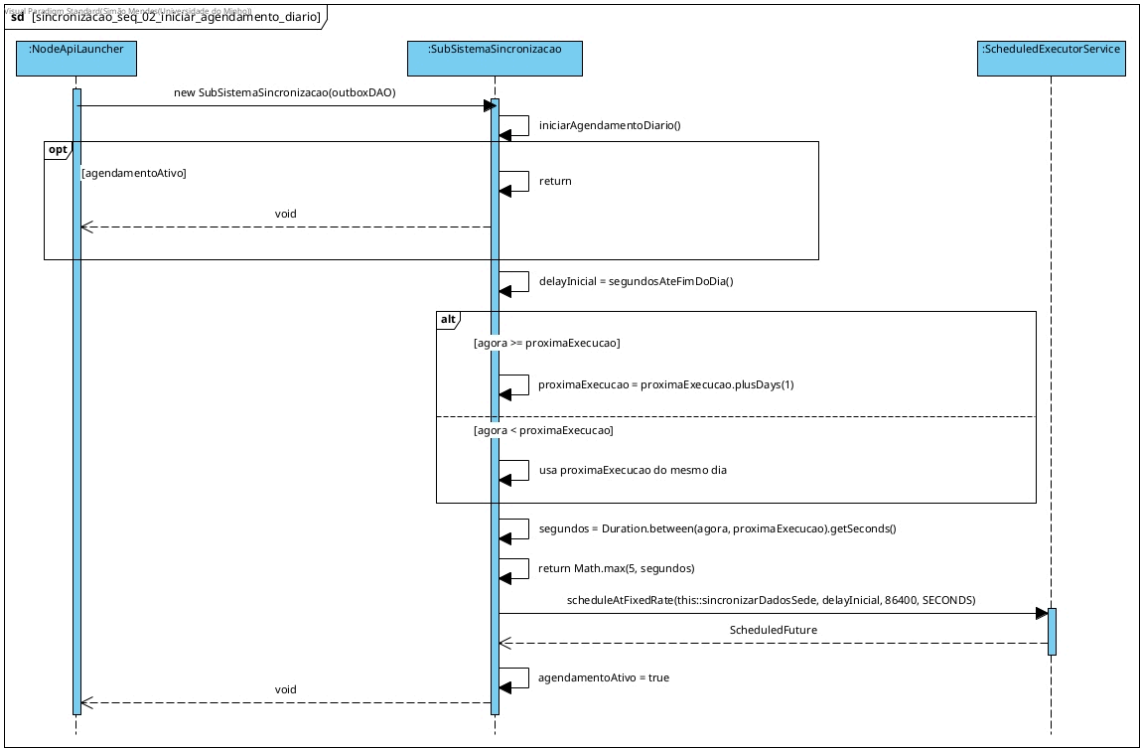


Figura 48: Diagrama de Sequência: Iniciar Agendamento Diário

Sempre que se atinge o horário estabelecido ($00:00$), a rotina agendada é ativada, despoletando o mecanismo de compilação do fecho de caixa e forçando o contacto com a Sede.

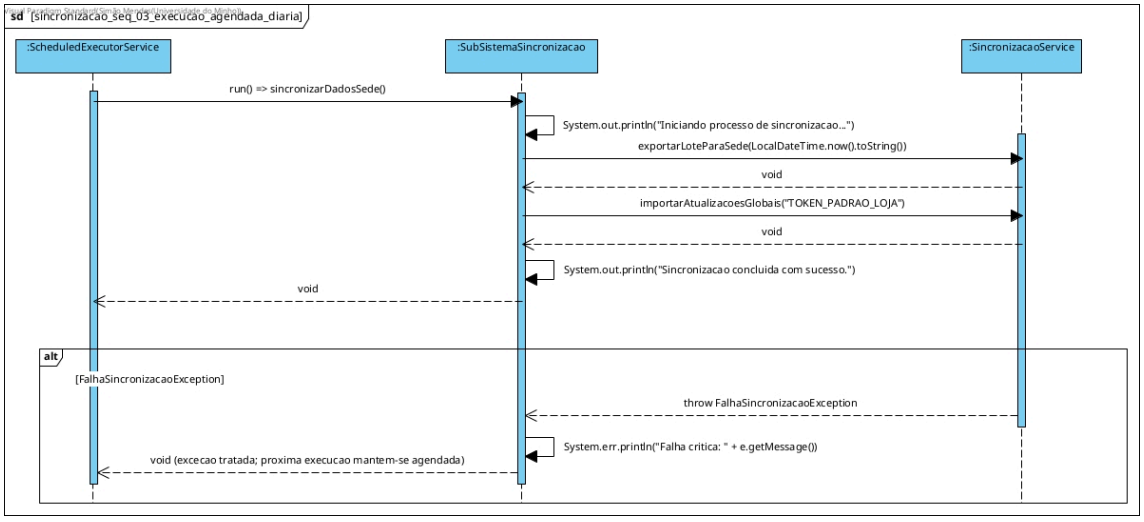


Figura 49: Diagrama de Sequência: Execução Agendada Diária

Apesar da previsibilidade do agendamento diário, os operadores do sistema possuem a capacidade de forçar o envio imediato da informação sempre que motivos operacionais assim o exijam.

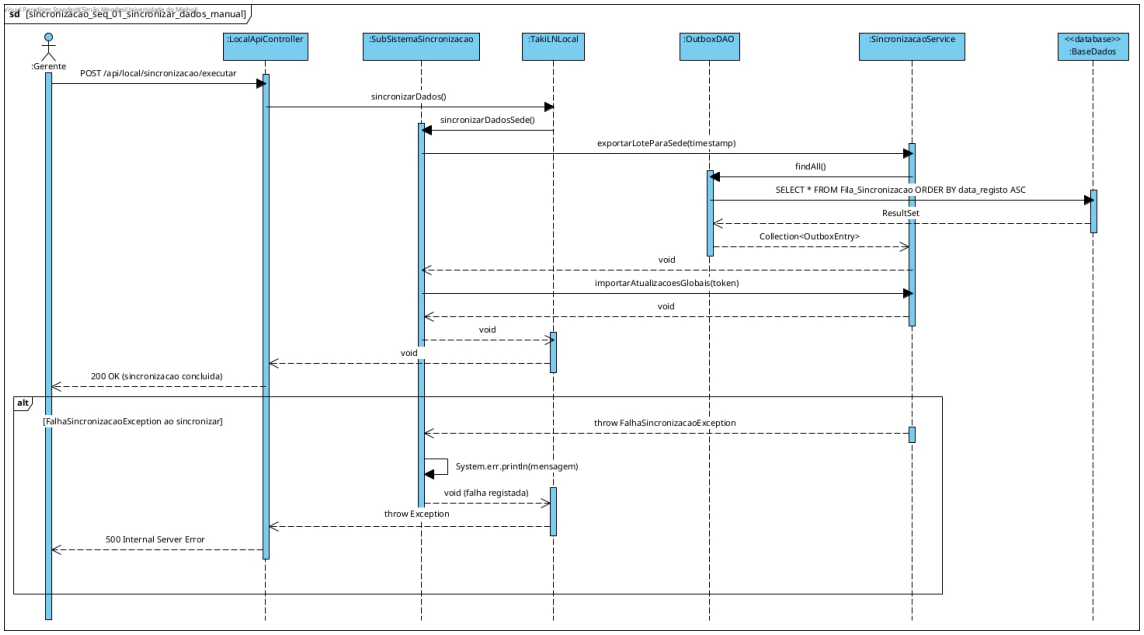


Figura 50: Diagrama de Sequência: Sincronização Manual de Dados

Desenho Comportamental: Exportação e Envio Seguros

A segunda fase do ciclo é a agregação dos dados e a sua transmissão segura para os servidores em Famalicão.

Para rentabilizar os recursos de rede, a aplicação agrega múltiplos eventos acumulados na *Outbox* num único *payload* (lote), reduzindo a latência da comunicação e a sobrecarga nos servidores recetores. Este fluxo assegura ainda que os registos processados só são removidos da fila local após a obtenção de um código HTTP de sucesso, materializando o princípio de confirmação estrita que sustenta a resiliência do mecanismo.

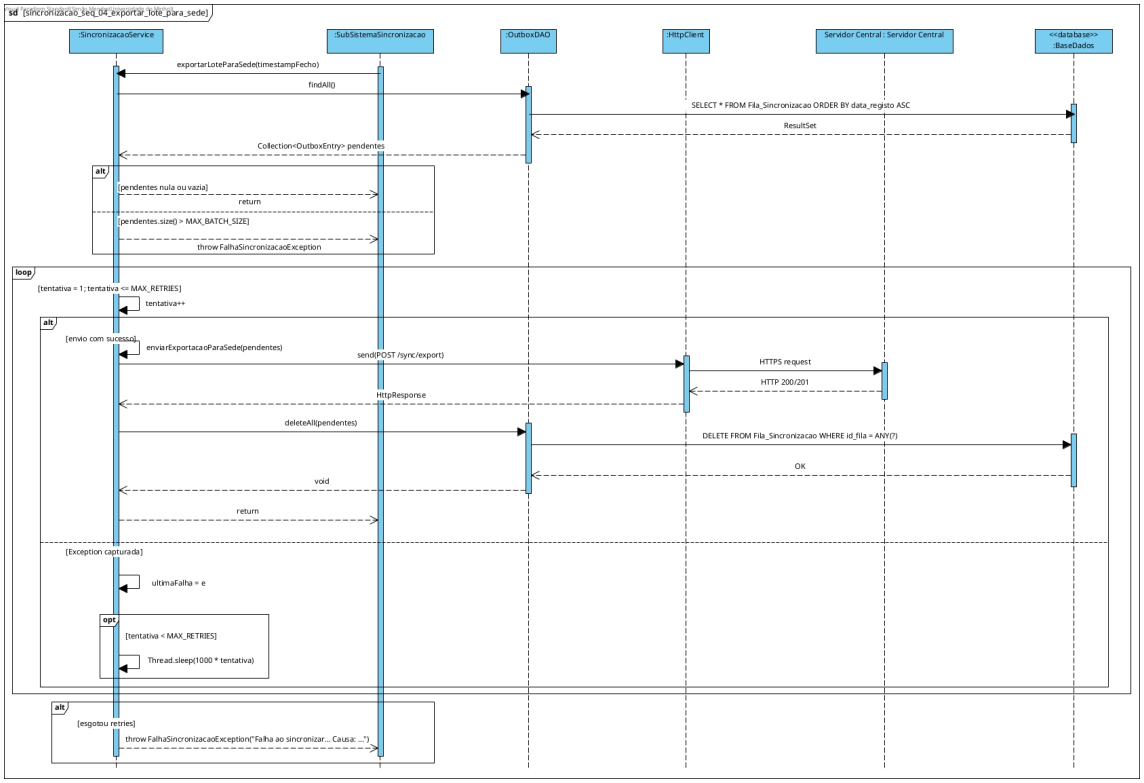


Figura 51: Diagrama de Sequência: Exportar Lote para a Sede

Após o loteamento, o sistema centraliza o processo de submissão do *payload* à API da Sede através de um mecanismo de tolerância a falhas por *failover*: a aplicação percorre sequencialmente a lista de *endpoints* centrais configurados, tentando cada um até obter uma resposta de sucesso (HTTP $200/201$). Em caso de erro HTTP inesperado ou exceção de rede, a falha é registada e o sistema avança para o próximo *endpoint*, garantindo a continuidade do serviço perante a indisponibilidade pontual de uma das instâncias da Sede.

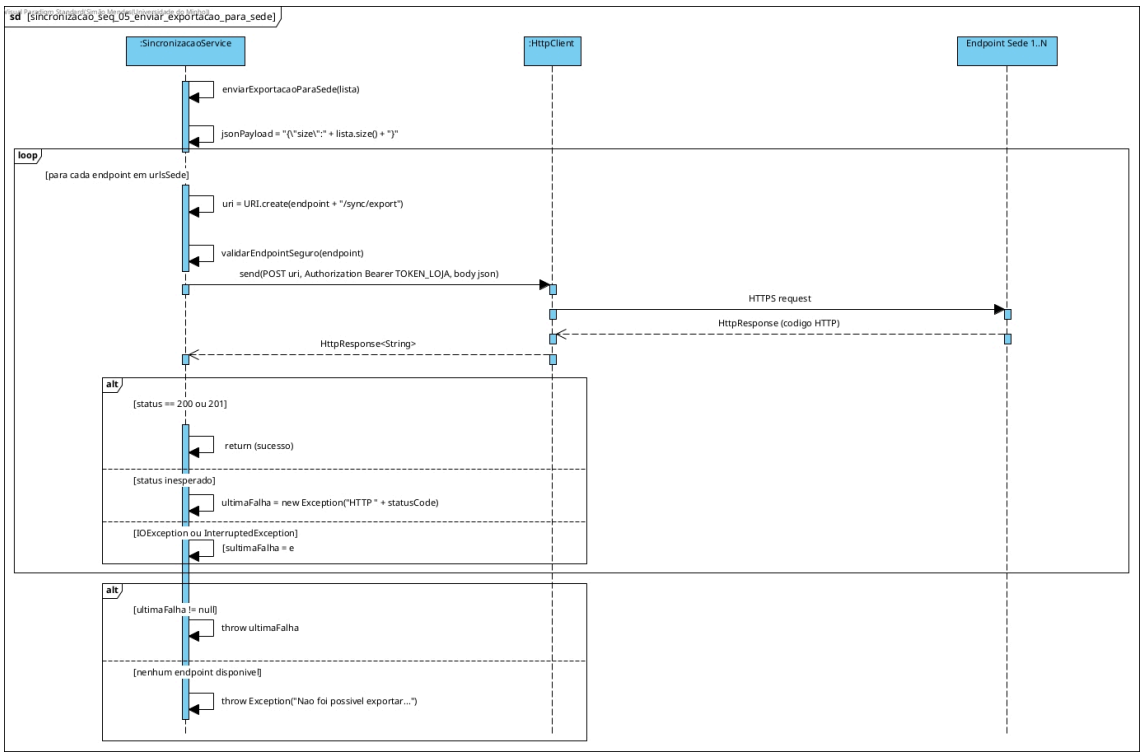


Figura 52: Diagrama de Sequência: Enviar Exportação para a Sede

O ponto de vista do servidor central é detalhado neste fluxo, evidenciando como a Sede recebe o pacote enviado pela filial e devolve a confirmação de receção que permite à loja proceder à limpeza da *Outbox* local. A integração transacional plena no repositório global — desserialização do *payload*, validação semântica e inserção dos registos — encontra-se prevista como evolução futura do *endpoint*, mantendo-se neste momento o contrato HTTP estável para suportar o ciclo de confirmação.

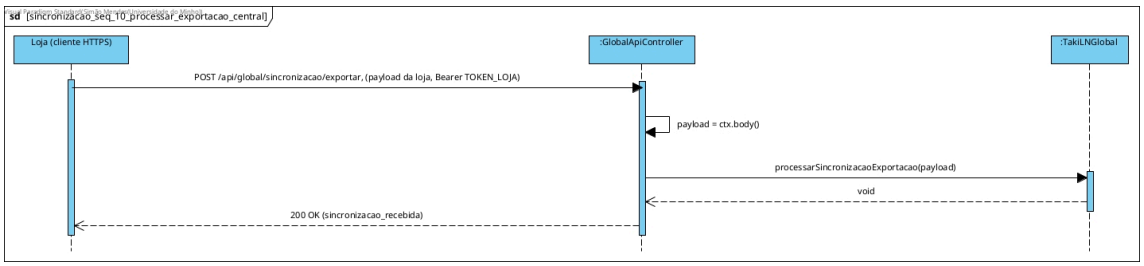


Figura 53: Diagrama de Sequência: Processar Exportação na Central

Dado o cariz confidencial da faturação e da logística, a aplicação obriga à negociação de *endpoints* seguros e à definição de canais cifrados em TLS 1.3 para qualquer comunicação *extranet*, cumprindo o requisito não-funcional RNF04 de segurança no transporte.

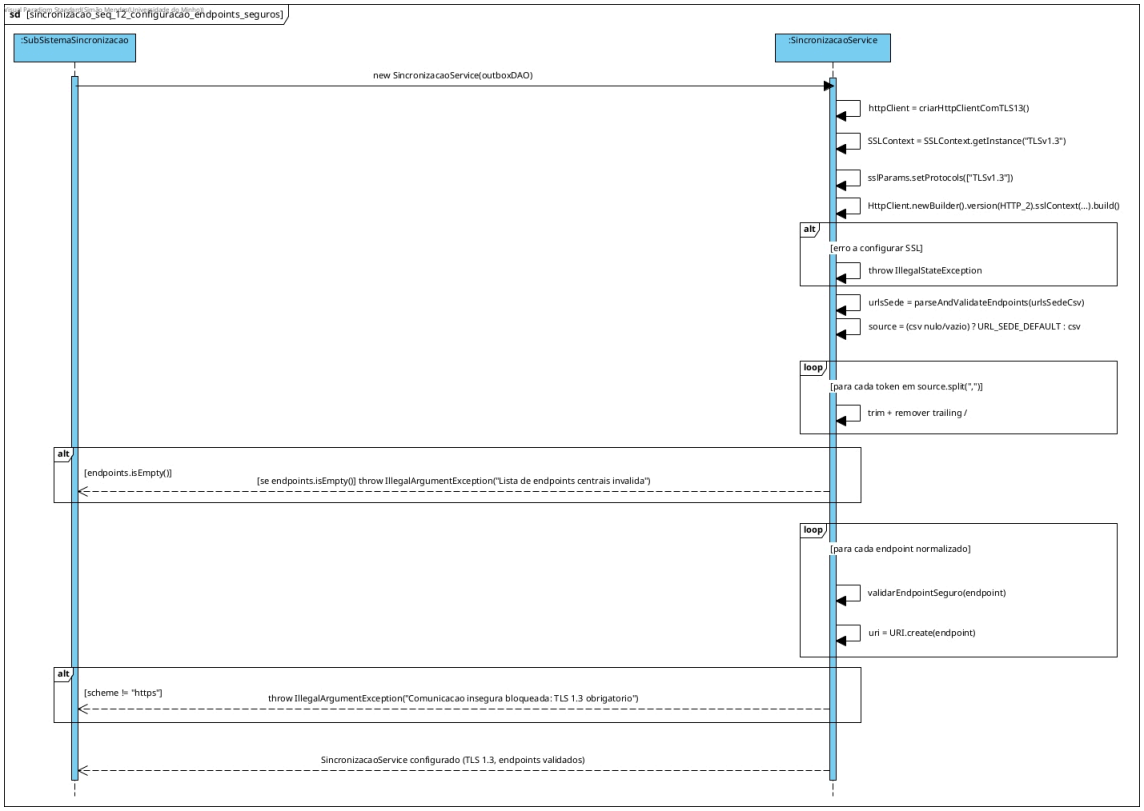


Figura 54: Diagrama de Sequência: Configuração de *Endpoints* Seguros

Desenho Comportamental: Importação e Monitorização de Saúde

Além da emissão de dados, as lojas importam periodicamente regras de negócio transversais provenientes da Sede (por exemplo, atualizações de taxas fiscais ou de catálogo).

Este fluxo assegura que eventuais diretrizes centrais sejam ativamente puxadas pelas filiais, prevenindo a desatualização local face a ordens emitidas pela Sede. Tal como na exportação, o processo é resiliente a falhas individuais de *endpoints*, percorrendo sequencialmente a lista de servidores centrais configurados até obter resposta.

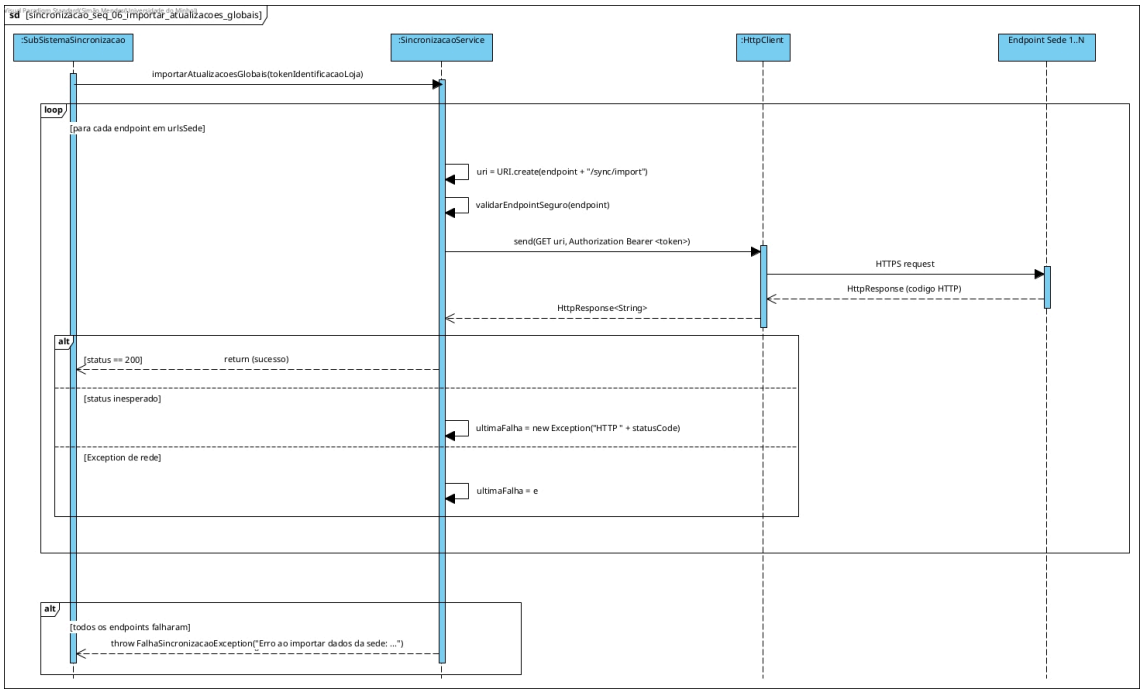


Figura 55: Diagrama de Sequência: Importar Atualizações Globais

Em contrapartida, este diagrama evidencia o processamento no servidor central durante um pedido de sincronização *"pull"* por parte da filial, em que o nó central responde com o pacote de atualizações globais a aplicar. À semelhança do *endpoint* de exportação, a composição completa do *payload* de regras transversais (catálogo, fiscalidade) está prevista como evolução futura, mantendo-se atualmente o contrato HTTP que assegura a periodicidade do pedido.

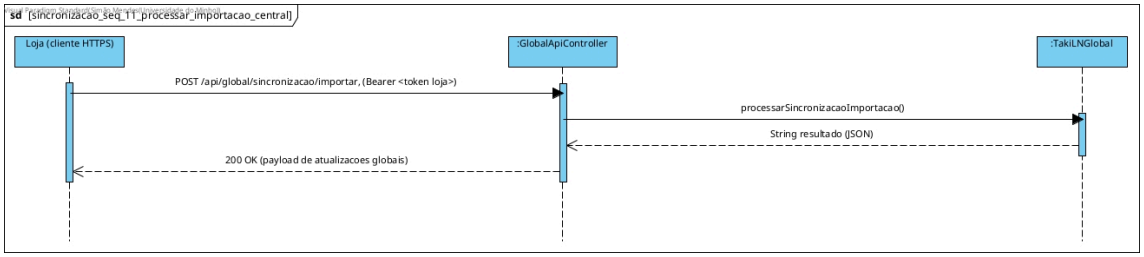


Figura 56: Diagrama de Sequência: Processar Importação na Central

Para prevenir interrupções operacionais surpresa, um mecanismo de sinalização (*heartbeat* / *ping*) interroga periodicamente o servidor central através do *endpoint* `/health`, avaliando a disponibilidade do sistema principal. A lógica de *failover* é igualmente aplicada nesta verificação, considerando-se a Sede acessível se pelo menos um dos *endpoints* configurados responder com sucesso.

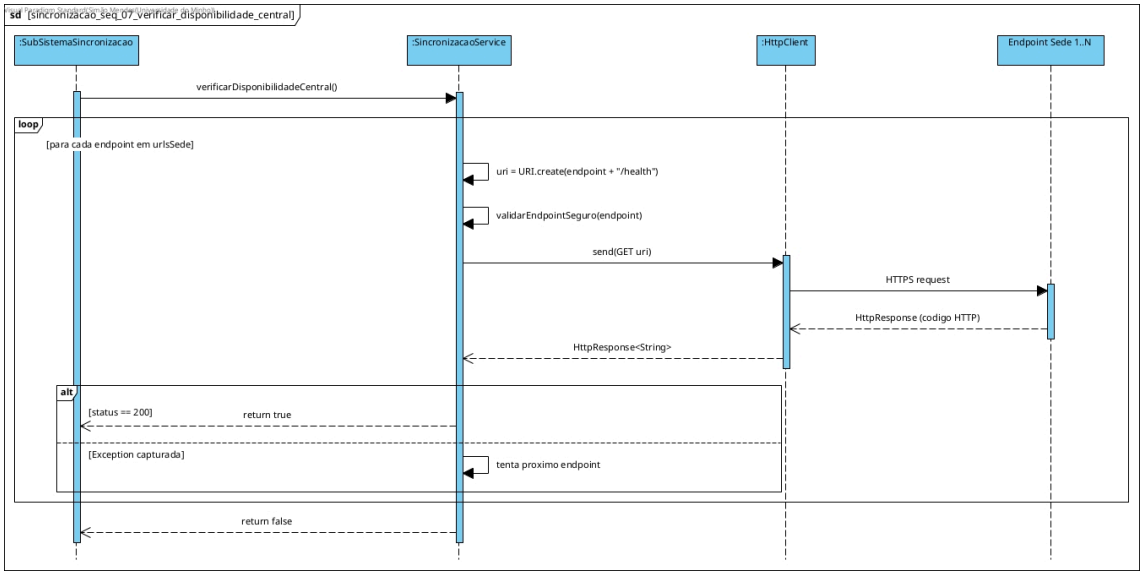


Figura 57: Diagrama de Sequência: Verificar Disponibilidade Central

Os administradores têm ao dispor ferramentas de telemetria baseadas neste fluxo, permitindo consultar em tempo real o estado da ligação à Sede (disponível / indisponível) bem como a percentagem de disponibilidade observada nos últimos 30 dias — um indicador de qualidade de serviço usado para sustentar o objetivo de fiabilidade da plataforma (RNF07).

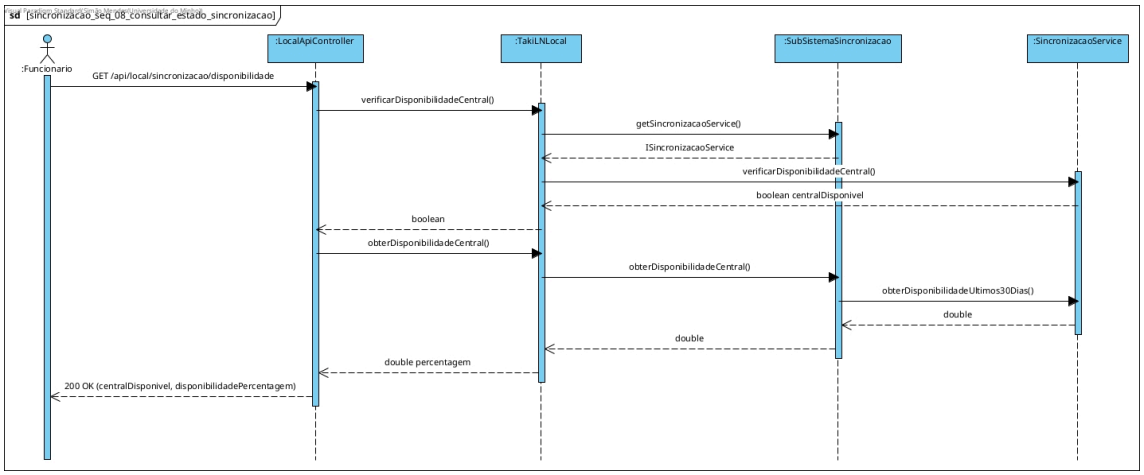


Figura 58: Diagrama de Sequência: Consultar Estado de Sincronização

3.2.4. Gestão de Catálogo (Lojas)

Desenho da Arquitetura: Catálogo de Produtos e Categorias

A gestão do catálogo é a base da plataforma `Taki`. Este módulo define como os artigos são organizados e estabelece as regras para a proteção dos dados. A arquitetura foi pensada para dar autonomia às lojas e, ao mesmo tempo, manter um controlo rigoroso da integridade dos dados a longo prazo.

Inativação Lógica (Padrão *Soft Delete*) Para não quebrar a integridade de vendas ou registos antigos, a plataforma opta por marcar os registos como "inativos" em vez de os apagar fisicamente. Esta abordagem tem duas vantagens principais:

- **Preservação do Histórico**: Os artigos que saem de linha são mantidos na base de dados. Isto é essencial para auditorias e para gerar relatórios de vendas de anos anteriores que sejam fiáveis;
- **Filtragem nas Operações**: O sistema filtra automaticamente os artigos inativos no dia a dia. Assim, os operadores de caixa ou de armazém só veem e interagem com os produtos que estão realmente à venda. Ao mesmo tempo, faturas e registos de inventário antigos continuam a apontar para os artigos originais, o que é uma exigência para a conformidade fiscal.

Estrutura Hierárquica de Categorias A organização do catálogo segue um modelo em árvore, permitindo que os artigos sejam classificados em múltiplos níveis. Esta decisão de design oferece os seguintes benefícios:

- **Navegação e Agrupamento**: Esta estrutura facilita a pesquisa de produtos no Ponto de Venda (POS) e ajuda a organizar de forma mais eficiente a gestão de inventários com muitos artigos;
- **Mecanismo de Herança**: A hierarquia permite que regras definidas num nível superior (como uma promoção de verão para "Vestuário") sejam automaticamente aplicadas a todas as subcategorias (como "Calças" e "Camisas"). Este mecanismo simplifica muito a gestão central do catálogo.

Desenho Estrutural

O diagrama de classes seguinte apresenta a estrutura estática do módulo. A interface `ISubSistemaLojas` atua como a *Facade* principal, distribuindo as responsabilidades pelos serviços especializados `ProdutoService` e `CategoriaService`. É visível a associação entre `Produto` e a enumeração `TaxaIva`, a relação muitos-para-muitos entre `Produto` e `Categoria` (materializada na tabela `Produto_Categoria`), bem como a autorreferenciação na classe `Categoria`, que permite a construção da árvore hierárquica mencionada anteriormente.

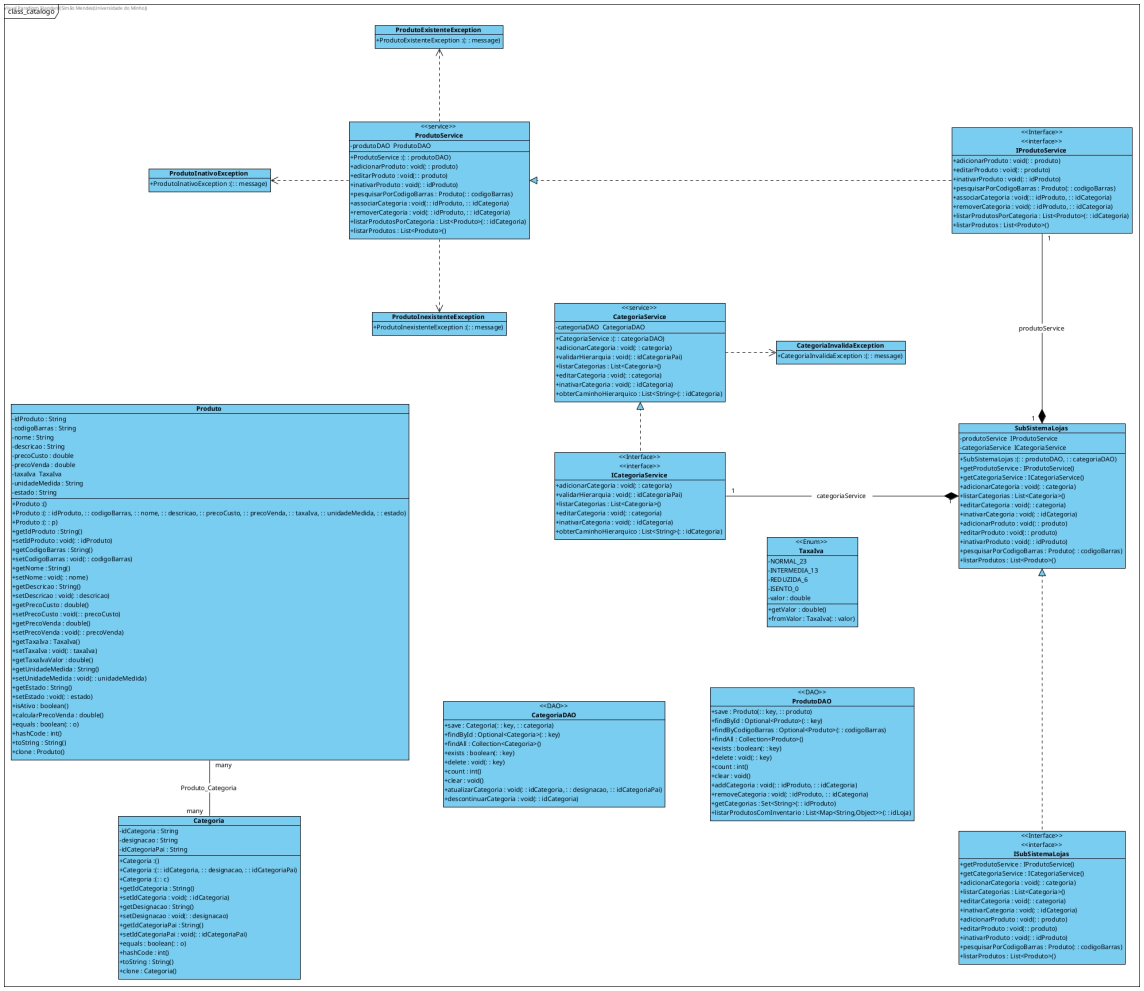


Figura 59: Diagrama de Classes: Subdomínio de Catálogo

Desenho Comportamental: Gestão e Ciclo de Vida de Artigos

As operações de gestão do catálogo seguem validações rigorosas das regras de negócio antes de qualquer alteração ser guardada na base de dados.

O processo de adição de um novo produto exige que o `ProdutoService` valide a unicidade do código de barras via `ProdutoDA0`, garantindo a integridade referencial antes da persistência.

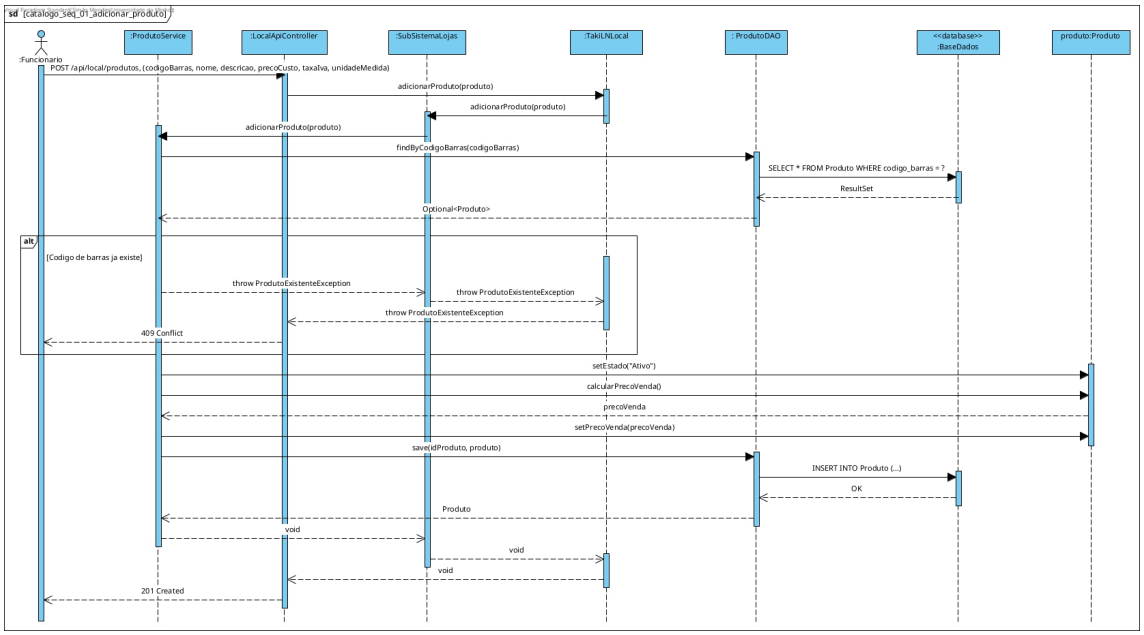


Figura 60: Diagrama de Sequência: Adicionar Produto

A edição de um produto permite atualizar atributos mantendo o vínculo com o histórico. O fluxo evidencia a recuperação da entidade e a aplicação de alterações validadas.

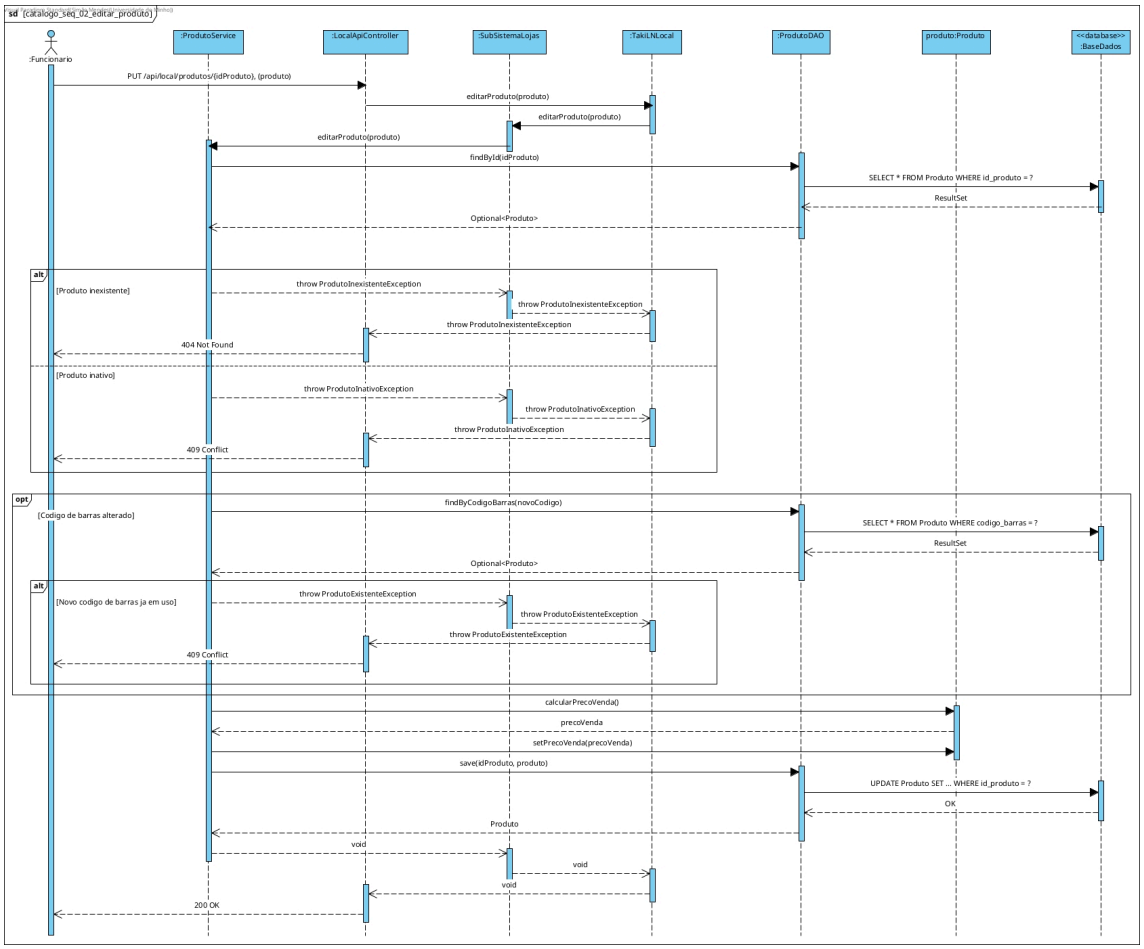


Figura 61: Diagrama de Sequência: Editar Produto

Em conformidade com o padrão *Soft Delete*, o diagrama abaixo demonstra que a inativação não remove o registo, mas altera o seu estado lógico para impedir operações comerciais futuras.

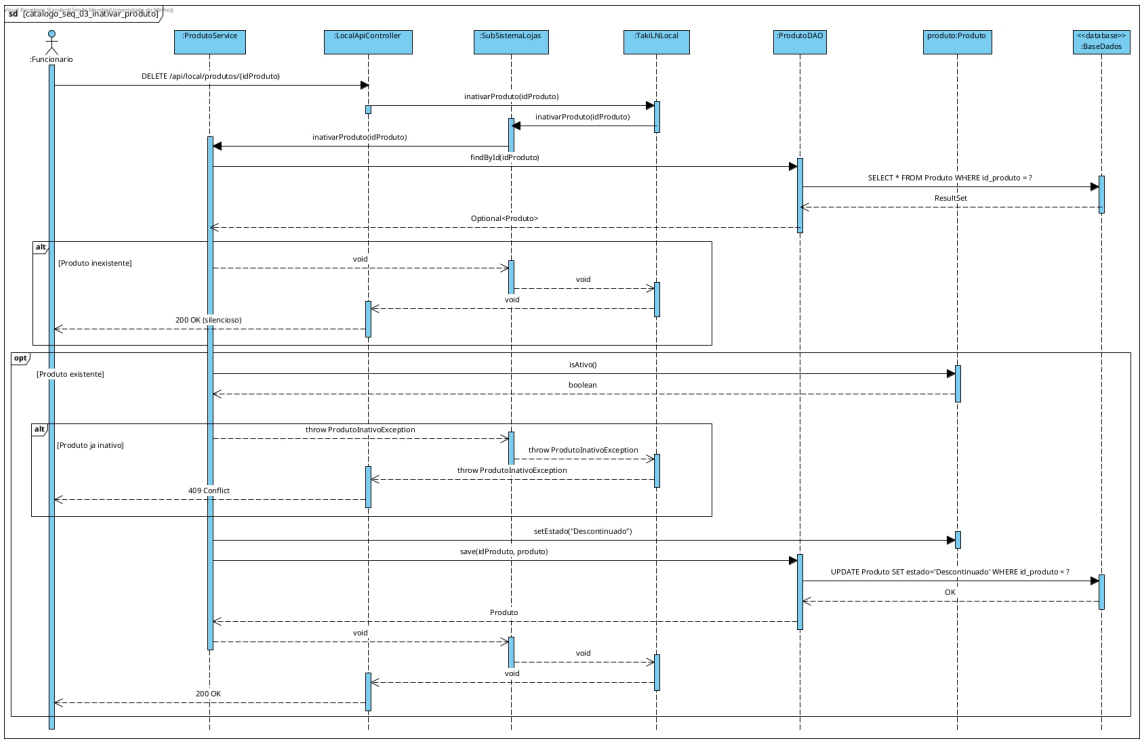


Figura 62: Diagrama de Sequência: Inativar Produto

A pesquisa por código de barras é exposta como *endpoint* global do catálogo, permitindo que qualquer nó da franquia recupere a ficha do artigo (designação, preço de venda, taxa de IVA aplicável) a partir do código lido. O fluxo detalha a resolução direta via `ProdutoDAO.findByCodigoBarras`, devolvendo 404 caso o código não corresponda a nenhum produto.

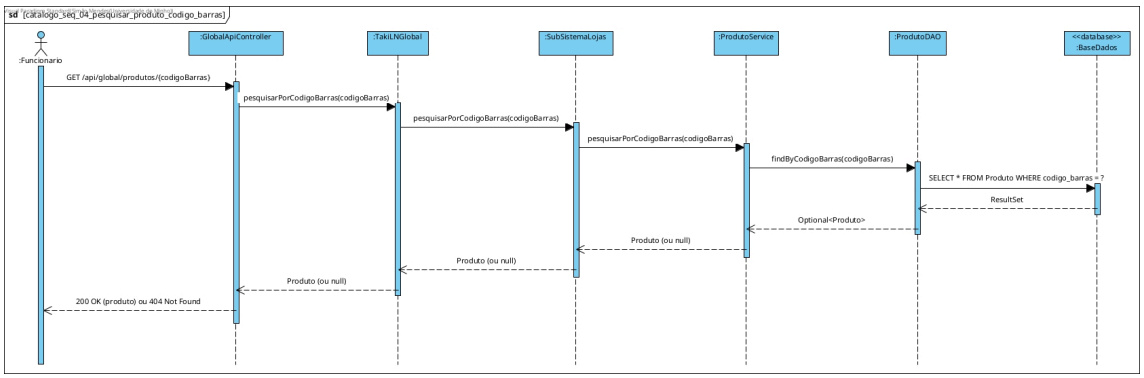


Figura 63: Diagrama de Sequência: Pesquisar Produto por Código de Barras

Para a gestão de inventário, o sistema disponibiliza a listagem completa de produtos, filtrando os registos marcados como inativos para que apenas o catálogo ativo na loja seja devolvido.

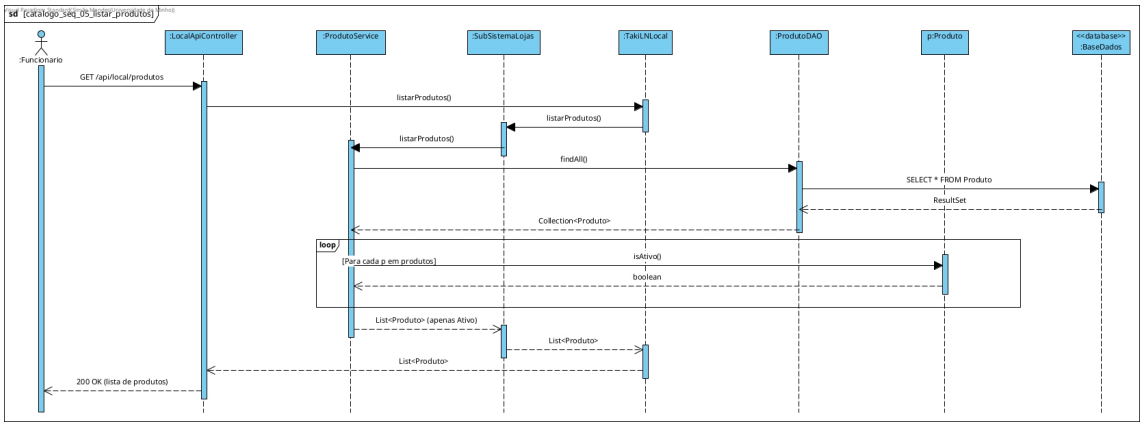


Figura 64: Diagrama de Sequência: Listar Produtos

Desenho Comportamental: Classificação e Taxonomia

A flexibilidade do catálogo assenta na associação dinâmica entre produtos e a estrutura de categorias.

O diagrama seguinte ilustra a criação de um vínculo entre um artigo e uma categoria, permitindo que o produto herde as propriedades organizacionais da mesma.

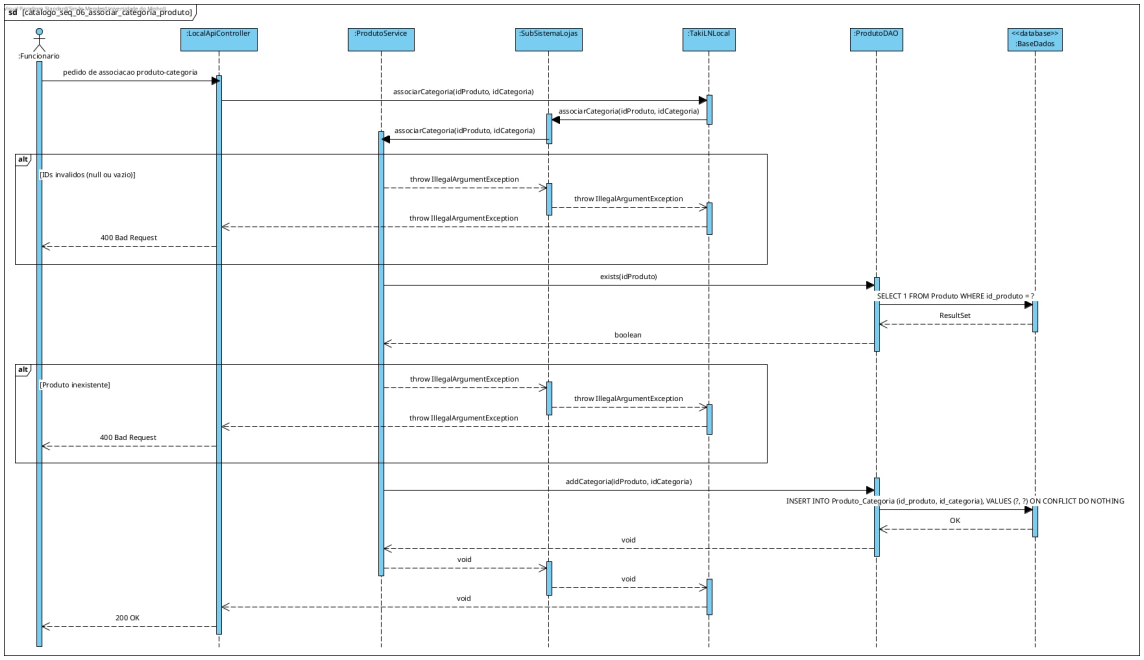


Figura 65: Diagrama de Sequência: Associar Categoria a Produto

Inversamente, a remoção da associação permite reclassificar artigos sem afetar a sua existência na base de dados principal.

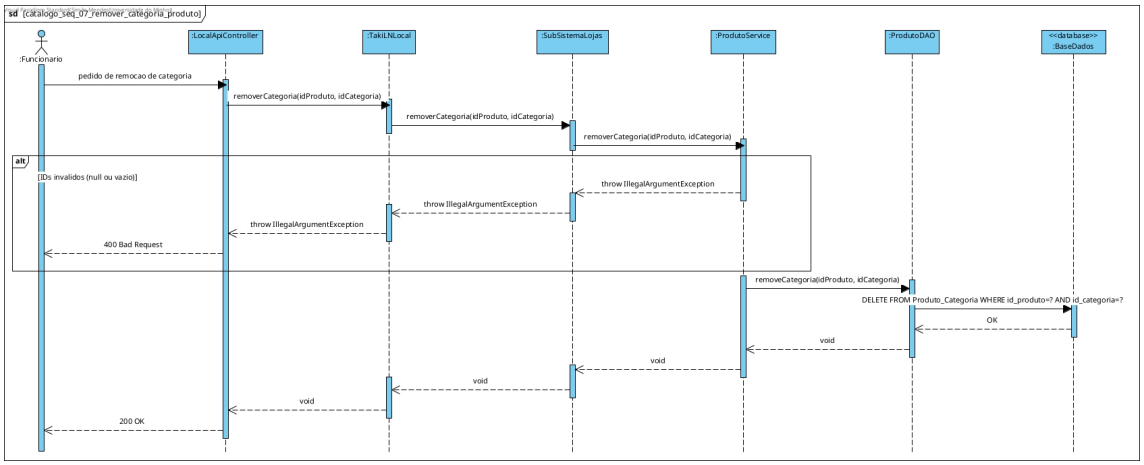


Figura 66: Diagrama de Sequência: Remover Categoria de Produto

Este fluxo demonstra a capacidade do sistema em filtrar o catálogo por ramos da árvore hierárquica, facilitando a navegação por menus no interface do utilizador.

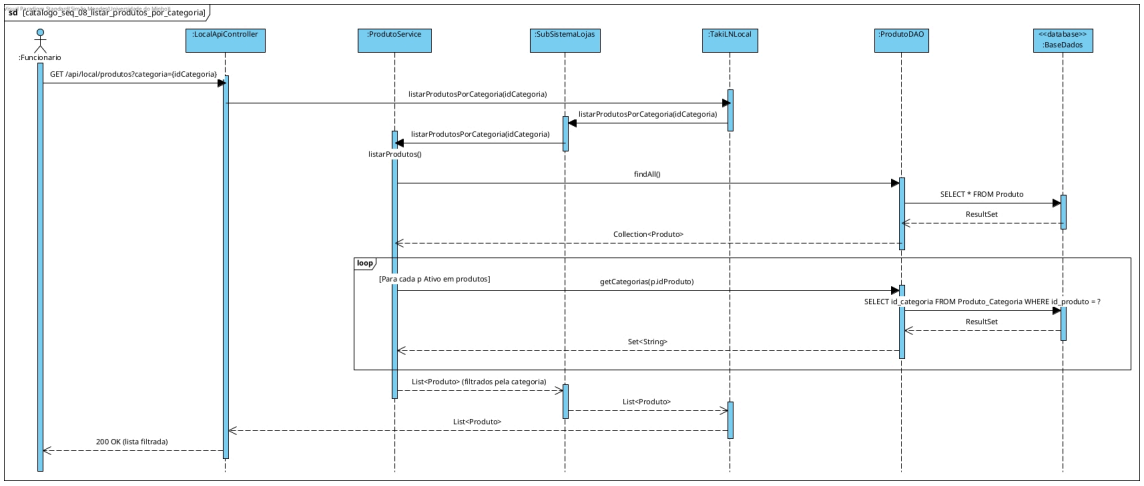


Figura 67: Diagrama de Sequência: Listar Produtos por Categoria

Desenho Comportamental: Hierarquia de Categorias

A manutenção da árvore de categorias requer cuidados especiais para garantir que a herança de regras e a navegação permanecem consistentes.

A criação de novas categorias inclui a definição opcional de uma categoria-pai, expandindo a árvore taxonómica da franquia.

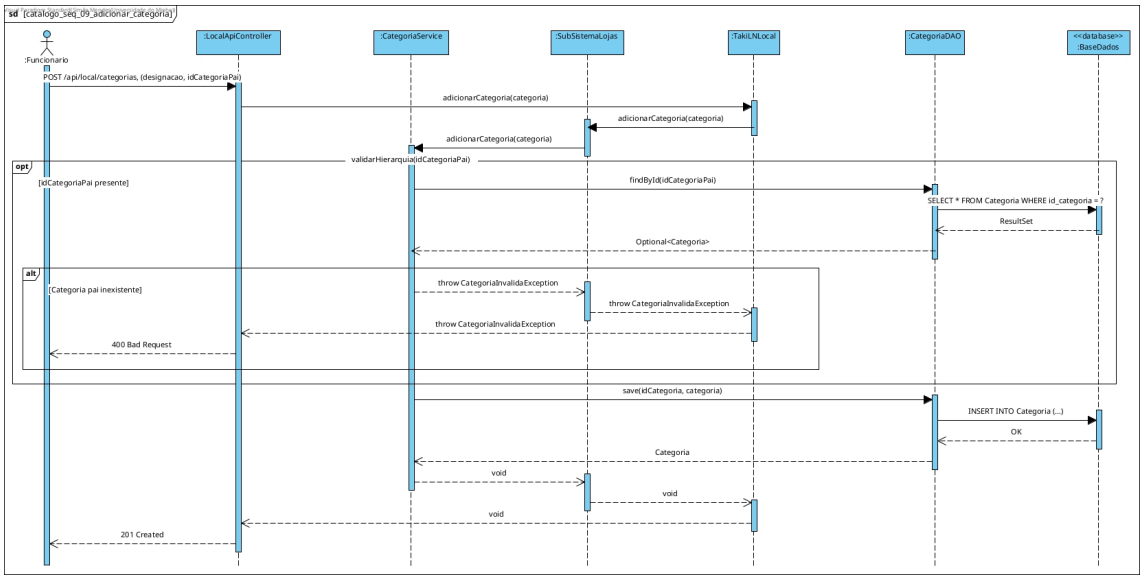


Figura 68: Diagrama de Sequência: Adicionar Categoria

A edição de categorias permite a reorganização estrutural, o que pode implicar a alteração do posicionamento de um ramo inteiro na hierarquia.

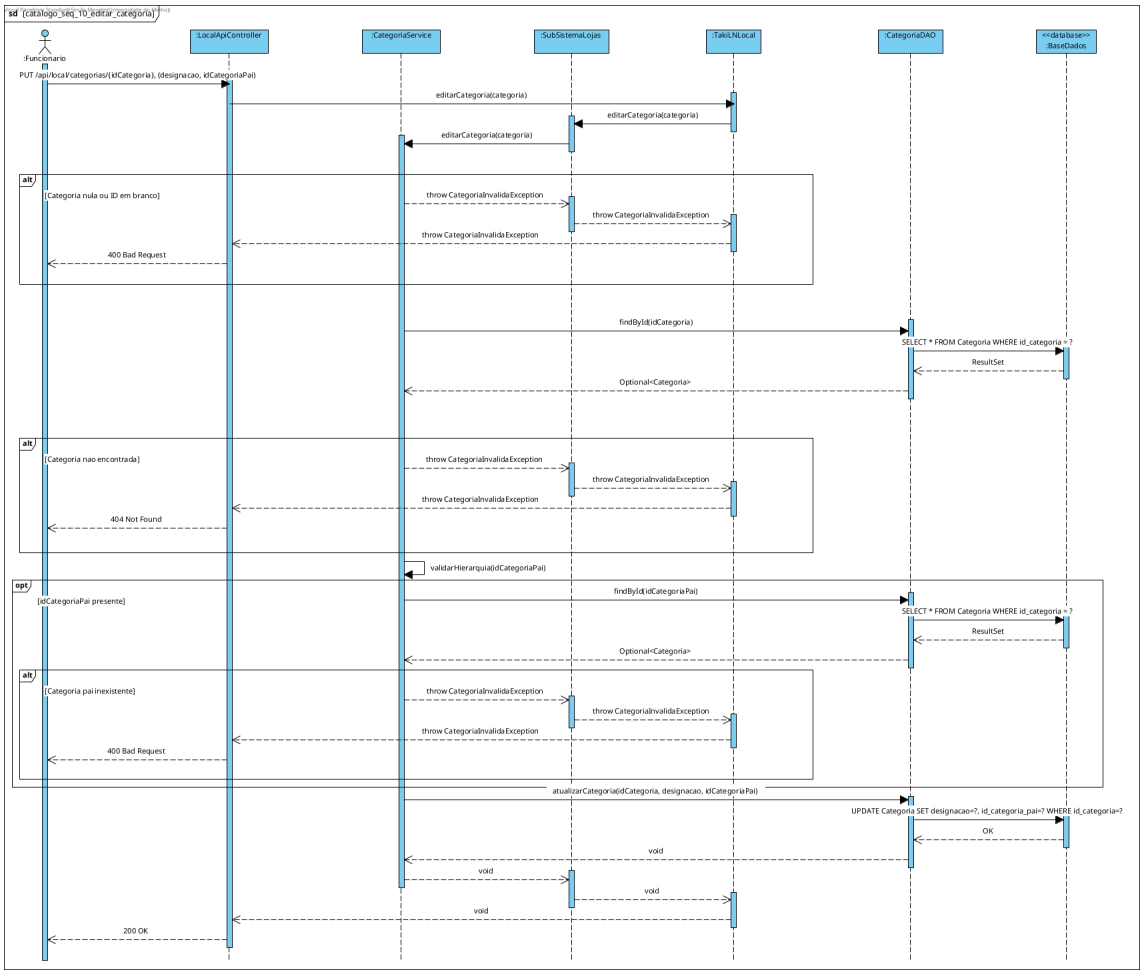


Figura 69: Diagrama de Sequência: Editar Categoria

Tal como nos produtos, a inativação de categorias segue uma lógica de preservação, garantindo que categorias antigas associadas a vendas históricas não desapareçam dos relatórios de auditoria.

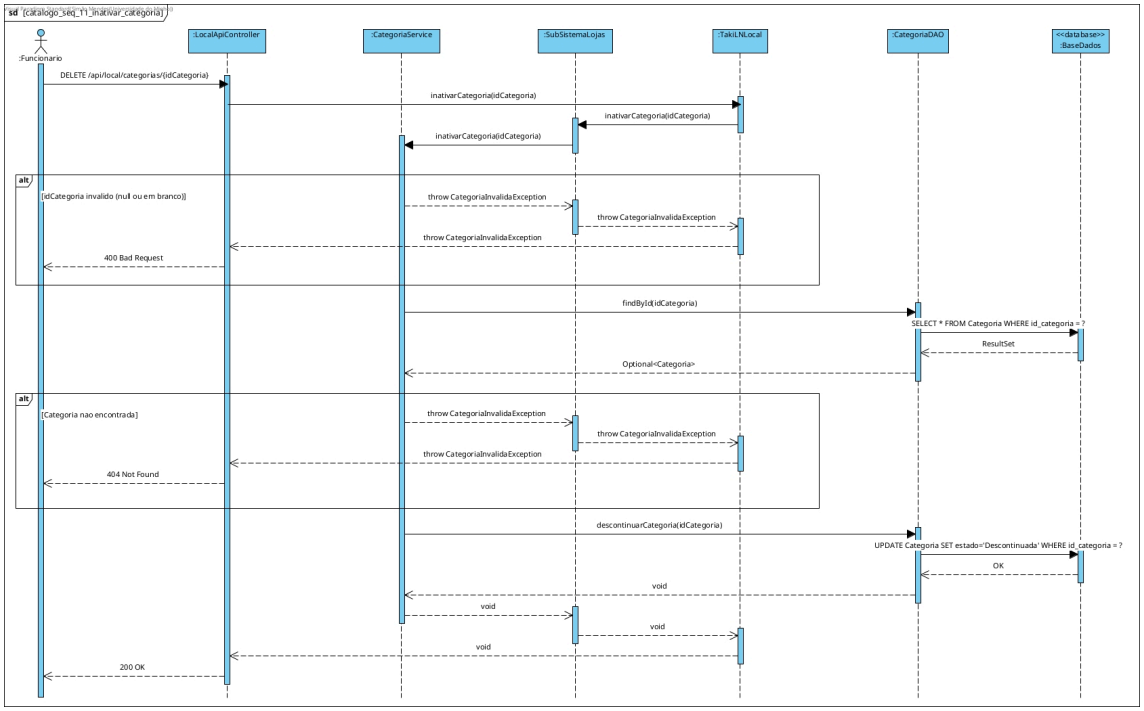


Figura 70: Diagrama de Sequência: Inativar Categoria

Finalmente, a resolução do caminho hierárquico permite gerar *breadcrumbs* na aplicação e materializar a navegação ascendente da categoria-folha até à raiz, percorrendo iterativamente o atributo `idCategoriaPai` até este ficar nulo.

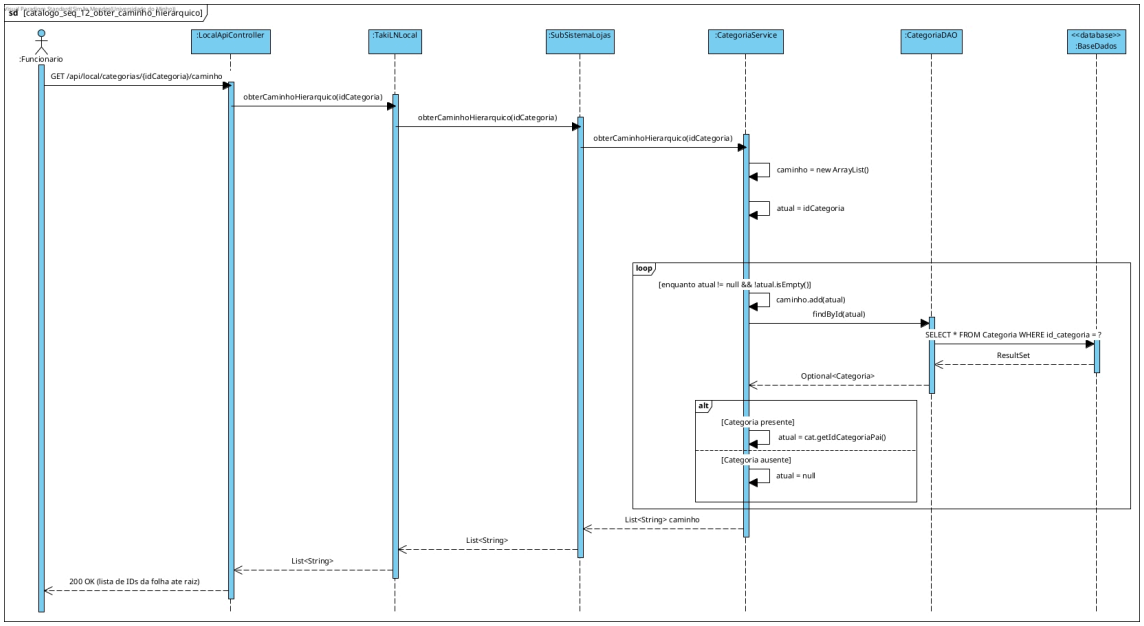


Figura 71: Diagrama de Sequência: Obter Caminho Hierárquico da Categoria

3.2.5. Gestão de Inventário

Desenho da Arquitetura: Gestão de Inventário e Lógica Ativa (PostgreSQL)

O subsistema de Inventário constitui o núcleo logístico da loja local na plataforma Taki. O controlo de mercadorias adota uma arquitetura centrada em dados (*Database-Centric*), delegando responsabilidades críticas no motor relacional PostgreSQL através de Lógica Ativa. Esta decisão assegura que as garantias de integridade e auditoria são independentes da camada aplicacional Java.

Integridade Transacional e Prevenção de Inconsistências

- **Bloqueio de Stock Negativo**: Um gatilho `BEFORE UPDATE` sobre a tabela `Inventario` interceta qualquer UPDATE na tabela Inventario e aborta a transação se a nova quantidade resultante for inferior a zero, abortando a transação antes de qualquer escrita ser confirmada. Esta barreira é definitiva e não contornável pela camada aplicacional;
- **Isolamento Aplicacional**: A adoção desta barreira ao nível do motor liberta o código Java da obrigatoriedade de verificar saldos de stock antes de cada operação, confiando a atomicidade absoluta ao próprio PostgreSQL.

Auditoria Logística Automatizada

- **Movimentos por Evento de Venda e Devolução**: A inserção de uma linha de venda aciona automaticamente um gatilho `AFTER INSERT` que abate a quantidade no inventário e regista um movimento de saída. De forma simétrica, a inserção de uma linha de devolução repõe a quantidade e regista um movimento de entrada. Em ambos os casos, a geração dos registos de auditoria é uma reação autónoma da base de dados, sem intervenção da camada Java;
- **Movimentos por Receção de Encomenda**: A transição de estado de uma encomenda para Entregue ou Concluída aciona um gatilho `AFTER UPDATE` que percorre todas as linhas da encomenda, repõe o stock de cada produto na loja e regista o respetivo movimento de entrada de forma automática;
- **Movimentos por Ajuste Manual**: Correções de inventário iniciadas por um operador são registadas pela camada Java, que persiste o movimento de forma explícita, associando-o ao funcionário responsável pela operação.

Motor Reativo de Alertas

- **Avaliação por Mutação**: Qualquer inserção ou alteração à quantidade ou quantidade mínima na tabela `Inventario` aciona um gatilho `AFTER INSERT OR UPDATE` que avalia matematicamente o saldo remanescente em proporção ao limite mínimo estipulado para o artigo;
- **Classificação Autónoma**: O gatilho classifica o estado do alerta em três níveis: rutura, *stock* crítico ou *stock* baixo e persiste o resultado em `Alerta_Stock`, criando um novo alerta ou atualizando o existente. A recuperação do nível acima do mínimo resolve automaticamente o alerta ativo, registando a data de resolução sem intervenção humana.

Desenho Estrutural

O diagrama abaixo mostra as principais entidades do subsistema de inventário e a forma como se relacionam entre si e com a lógica ativa da base de dados.

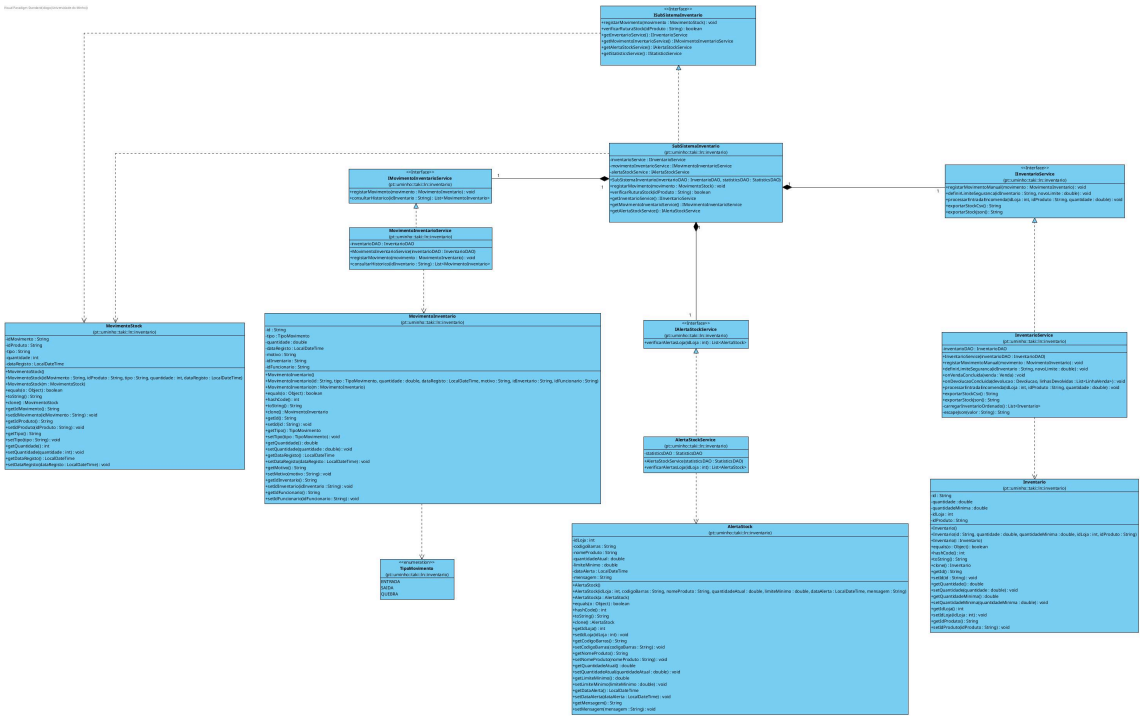


Figura 72: Diagrama de Classes: Subdomínio de Inventário

Desenho Comportamental

As operações de gestão de inventário envolvem a coordenação entre a camada Java e os mecanismos reativos da base de dados. Os diagramas seguintes detalham o comportamento de cada operação principal.

O **diagrama de sequência de gestão autónoma de alertas de *stock*** ilustra o mecanismo reativo implementado em PostgreSQL através da *trigger* `trg_inventario_alerta_stock`, que monitoriza automaticamente alterações nas colunas `quantidade` e `quantidade_minima` da tabela `Inventario`. Sempre que ocorre um `INSERT` ou `UPDATE` nestes campos, a função de *trigger* `trg_func_alerta_stock(NEW)` é executada, avaliando o novo registo com base em quatro limiares distintos. Para situações de rutura de *stock* (`NEW.quantidade <= 0`), a função verifica se já existe um alerta ativo para a chave composta `(id_loja, id_produto)`: caso exista, atualiza o tipo de alerta para `'RUTURA'` e os campos `quantidade_atual` e `limite_minimo`; caso contrário, insere um novo registo na tabela `AlertaStock` com `id_alerta = gen_random_uuid()`, `tipo_alerta = 'RUTURA'`, `estado = 'Ativo'` e `data_criacao = CURRENT_TIMESTAMP`. O mesmo padrão aplica-se às condições de *stock* crítico (`NEW.quantidade <= NEW.quantidade_minima * 0.5`, tipo `'STOCK_CRITICO'`) e *stock* baixo (`NEW.quantidade <= NEW.quantidade_minima`, tipo `'STOCK_BAIXO'`). Quando o *stock* é reposto e ultrapassa o limite mínimo (`NEW.quantidade > NEW.quantidade_minima`), a *trigger* resolve automaticamente o alerta ativo, atualizando o estado para `'Resolvido'` e registando `data_resolucao = CURRENT_TIMESTAMP`. Este mecanismo garante que a deteção e resolução de alertas ocorre de forma autónoma na camada de base de dados, sem intervenção da aplicação Java, reduzindo a latência e assegurando consistência transacional entre inventário e alertas através da chave composta por loja e produto.

Figura 73: Diagrama de Sequência: Gestão Autónoma de Alertas de *Stock*

O **diagrama de sequência de ajuste manual de inventário** ilustra o fluxo iniciado pelo operador ao submeter um pedido `POST /api/local/inventario/movimentos` com os dados do movimento, incluindo o identificador de inventário, a quantidade e o tipo de movimento. O `LocalApiController` valida o DTO recebido, resolve o tipo de operação e mapeia-o para um objeto de domínio `MovimentoInventario`, delegando depois no método `registarMovimentoManual` exposto pela fachada local. Este serviço recupera o registo de inventário através do `InventarioDAO`, aplicando regras de negócio como a deteção de artigos inexistentes e a prevenção de *stock* negativo em movimentos de saída ou quebra; só após estas validações atualiza a quantidade com `save(...)` e regista o histórico com `addMovimento(...)`, devolvendo ao controlador o movimento completo para que a resposta HTTP `201 Created` seja enviada ao operador.

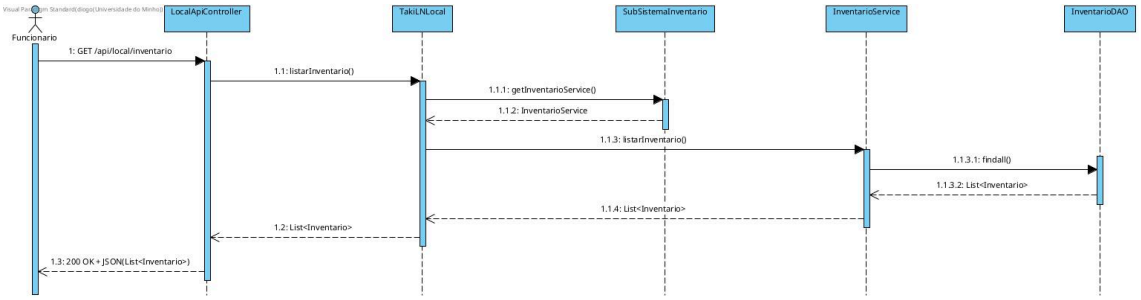


Figura 74: Diagrama de Sequência: Ajuste Manual de Inventário

O **diagrama de sequência de listagem de inventário** detalha o fluxo de consulta iniciado pelo operador através do endpoint `GET /api/local/inventario`. O processo começa no `LocalApiController`, que invoca o método `listarInventario()` da fachada principal `TakiLNLocal` através da sua interface `ITakiLNLocal`. Para manter a modularidade do sistema, a fachada acede ao `SubSistemaInventario` para obter o `InventarioService`, que atua como motor de lógica de domínio. Este serviço coordena a persistência ao chamar o `InventarioDAO`, que executa o método `findAll()` para recuperar todos os registos de inventário da loja, incluindo atributos críticos como `quantidade`, `quantidade_minima` e a associação ao `Produto`. A coleção resultante de objetos de domínio percorre o caminho inverso através da pilha de chamadas até chegar ao controlador. No `LocalApiController`, a lista é serializada para o formato JSON através do método `ctx.json(inventarios)`, culminando no envio de uma resposta HTTP `200 OK`. Este fluxo garante que o operador visualize o estado atualizado do *stock* em tempo real, respeitando rigorosamente a separação entre a interface da API e a lógica de acesso aos dados.

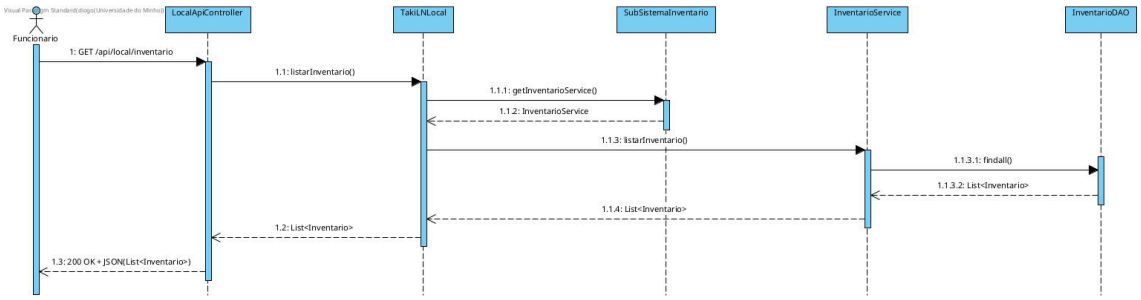


Figura 75: Diagrama de Sequência:Listar Inventário

O **diagrama de sequência de consulta de alertas de stock** ilustra o fluxo de obtenção de indicadores gerais através do endpoint `GET /api/local/estatisticas/kpi`. O `LocalApiController` invoca o método `obterKpiGerais(idLoja)` exposto pela interface `ITakiLNLocal`, cuja implementação, por sua vez, acede ao `SubSistemaInventario` para obter o `StatisticsService` — garantindo que a lógica de cálculo de métricas e alertas permaneça encapsulada. O serviço consulta a view `View_Alertas_StockCritico` através do `StatisticsDAO.getAlertasStockCritico(idLoja)`, recuperando uma lista de mapas com os produtos cujo stock se encontra abaixo do limite mínimo; estes dados são agregados com outras métricas num mapa de KPIs, serializado em JSON e devolvido ao operador, permitindo detetar situações críticas de inventário sem cálculos manuais na camada aplicacional.

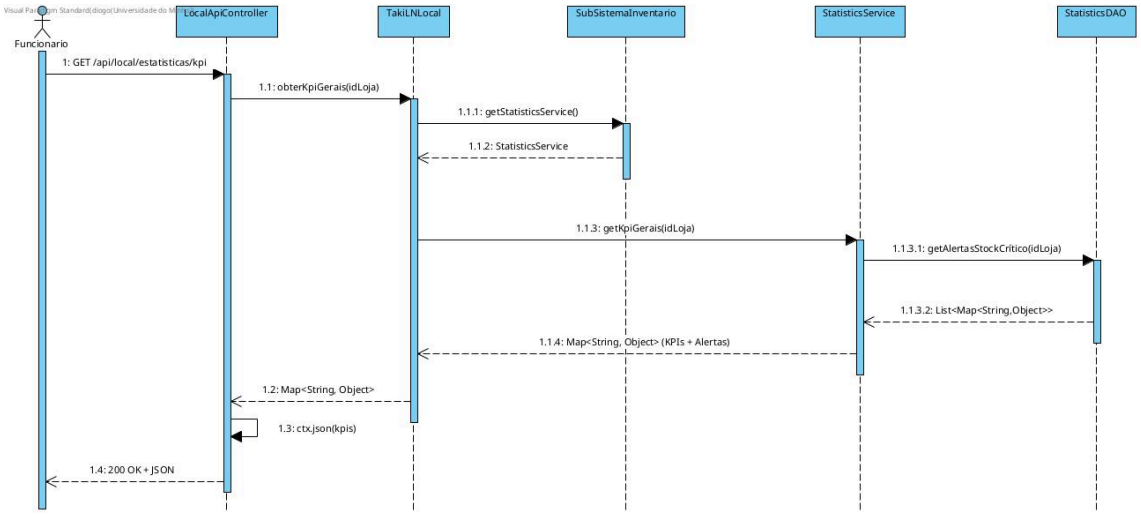


Figura 76: Diagrama de Sequência:Consultar Alertas

3.2.6. Gestão de Fornecimentos e Encomendas

Desenho da Arquitetura: Fornecimentos e Encomendas

O módulo de Fornecimentos gere a parte logística do abastecimento das lojas. A sua arquitetura define um fluxo de entrada de mercadoria estritamente controlado, rastreável e sujeito a validação constante com base nos acordos comerciais definidos com os fornecedores.

Ciclo de Vida da Encomenda (Padrão *State*) A arquitetura gere o processamento de encomendas através de uma máquina de estados, materializada na hierarquia da interface `IEstadoEncomenda`. O objetivo desta decisão é impor um processo rigoroso e garantir que os dados não podem ser alterados em fases críticas. O ciclo de vida divide-se nos seguintes estágios:

- **Estágio de Elaboração (`Rascunho`)**: Nesta fase inicial, o operador pode editar livremente os itens e as quantidades da encomenda, preparando o rascunho da transação. O método `podeModificarLinhas` devolve `true` apenas neste estado;
- **Estágio de Submissão (`Pendente`)**: A encomenda é dada como pronta a remeter ao fornecedor. As linhas ficam imutáveis e a transação pode, a partir daqui, avançar para `Enviada` ou ser revertida via `Cancelada`;

- **Estágio de Bloqueio (`Enviada`)**: Depois de a encomenda ser formalmente enviada ao fornecedor, os seus dados são "selados" e o estado fica bloqueado. Este bloqueio garante que o que foi pedido corresponde exatamente ao que será faturado, mantendo a conformidade legal;
- **Estágio de Conclusão (`Concluída`)**: A transição final é desencadeada pela receção da mercadoria. O acto regista a data de entrega e propaga os eventos necessários para dar entrada dos artigos no inventário da loja (`IInventarioService.processarEntradaEncomenda`).

Além destes estágios sequenciais, o estado terminal `Cancelada` permite abortar uma encomenda que ainda se encontre em `Pendente`, garantindo reversibilidade controlada antes de qualquer compromisso logístico.

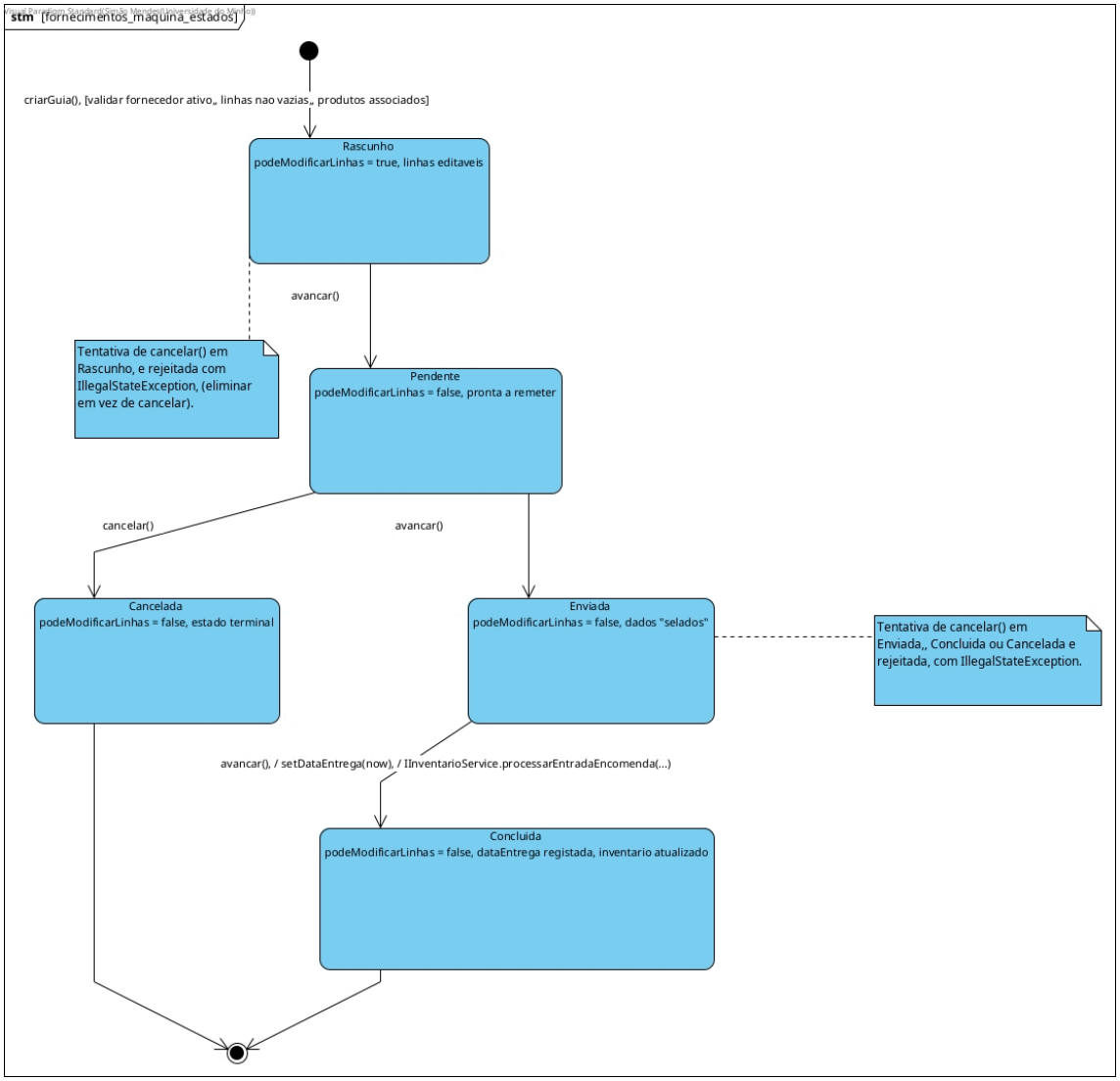


Figura 77: Diagrama de Máquina de Estados: Ciclo de Vida da Encomenda

Validação de Vínculos Comerciais O sistema impõe restrições rigorosas na relação entre fornecedores e os produtos do catálogo, de modo a evitar erros operacionais e financeiros. Para isso, foram implementadas as seguintes regras:

- **Conformidade de Preços**: A arquitetura garante que o preço de custo registado em cada linha da encomenda é o preço negociado e armazenado na associação `ProdutoFornecedor`. No momento da criação da guia, o serviço sobrescreve o valor recebido com o `precoCusto` vigente, eliminando divergências entre o pedido do operador e o acordo comercial;
- **Bloqueio Preventivo**: O sistema não permite a criação de encomendas irregulares. No acto de `criarGuia`, cada linha é validada contra a tabela `Produto_Fornecedor`: qualquer produto que não

esteja contratualizado com aquele fornecedor faz abortar a operação, validando assim as permissões de fornecimento ainda antes da persistência da encomenda.

Desenho Estrutural

O esquema abaixo clarifica as entidades que compõem o domínio de fornecimentos, incluindo as abstrações de estado que controlam o ciclo de vida da encomenda. A interface `IEstadoEncomenda` é o núcleo da implementação do padrão *State*, sendo realizada pelas classes concretas `EstadoRascunho`, `EstadoPendente`, `EstadoEnviada`, `EstadoConcluida` e `EstadoCancelada`. A associação muitos-para-muitos entre produtos e fornecedores é materializada pela classe `ProdutoFornecedor`, que carrega o preço de custo e a marca `preferencial` da associação.

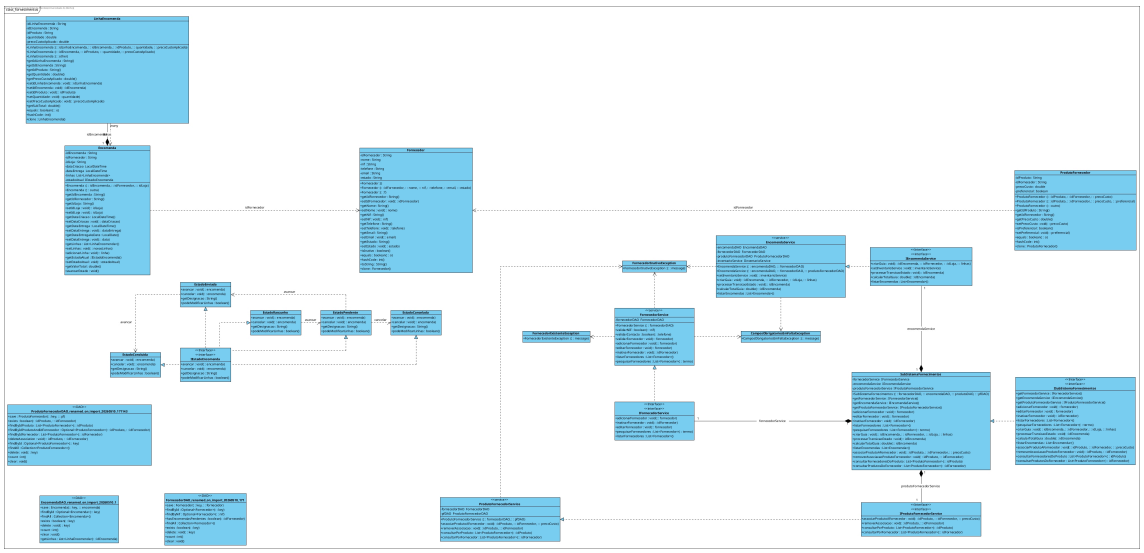


Figura 78: Diagrama de Classes: Subdomínio de Fornecimentos

Desenho Comportamental: Gestão de Fornecedores

A base de um abastecimento seguro começa com a manutenção rigorosa das entidades fornecedoras.

O registo de novos fornecedores adiciona novas entidades ao sistema. A validação cobre os campos obrigatórios, o formato do NIF e do contacto, e ainda a unicidade do NIF face aos fornecedores ativos já existentes.

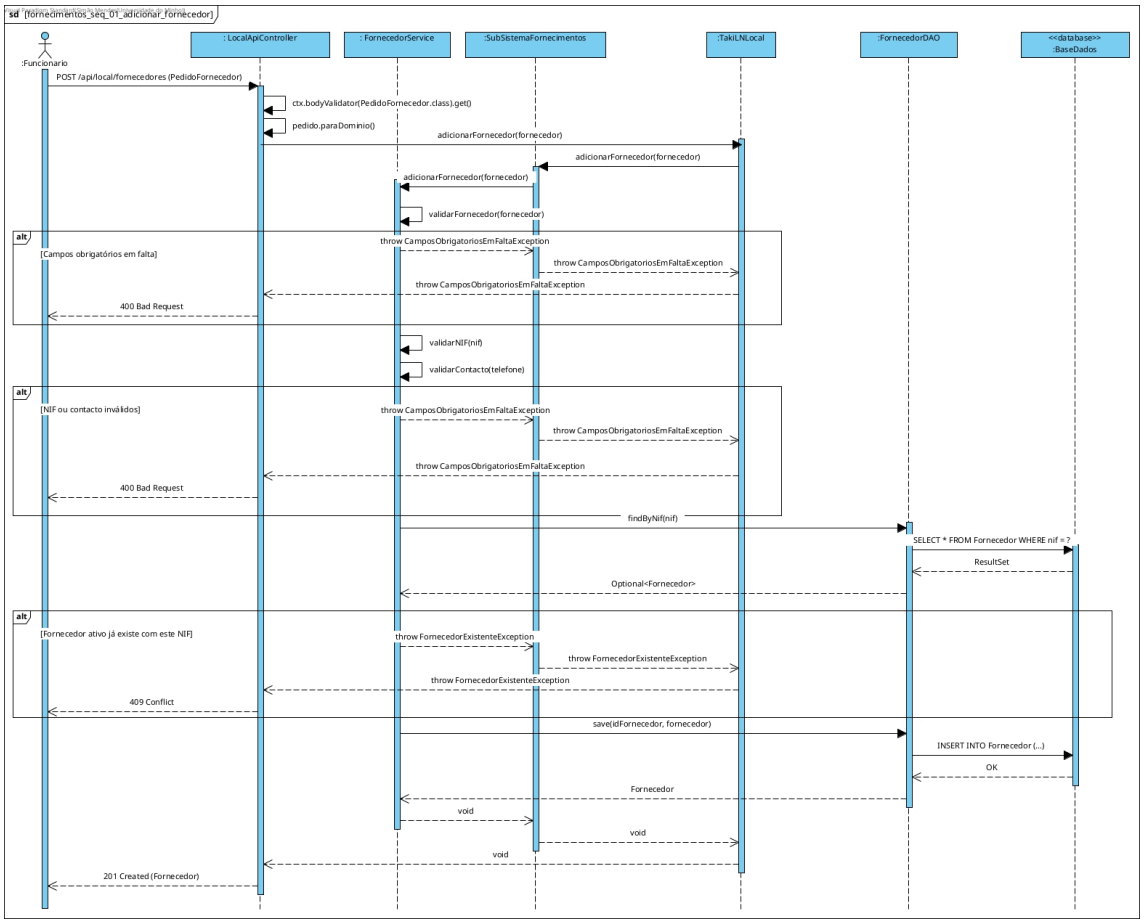


Figura 79: Diagrama de Sequência: Registar Fornecedor

A alteração dos dados de um fornecedor reaplica as mesmas regras de validação do registo inicial e impede que o NIF colida com o de outro fornecedor ativo, preservando a unicidade fiscal exigida pelo domínio.

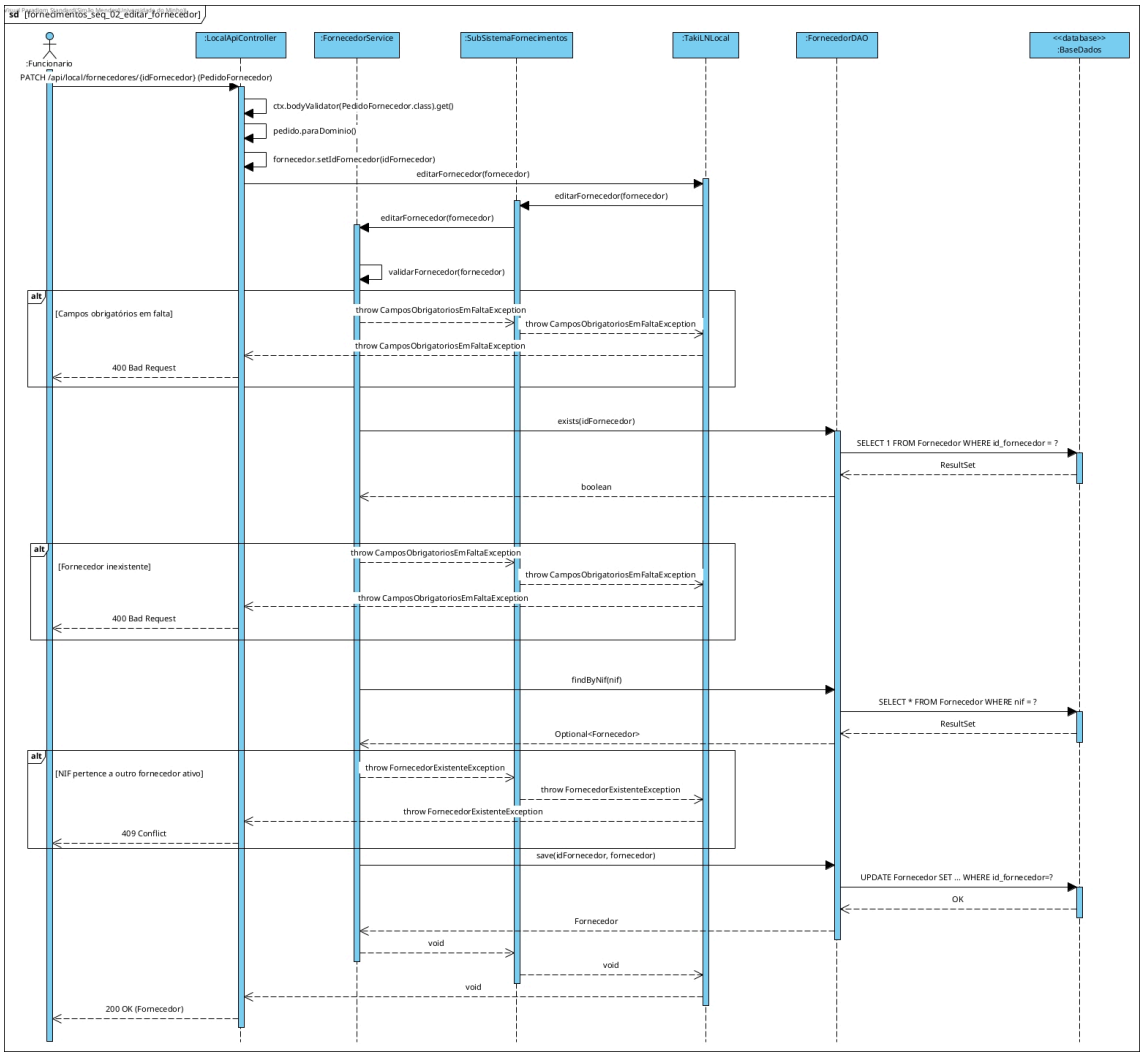


Figura 80: Diagrama de Sequência: Editar Dados do Fornecedor

A inativação de um fornecedor (remoção lógica via `estado = "Inativo"`) impede a criação de novas encomendas para esse parceiro, mas preserva todo o histórico de transações passadas. Como salvaguarda, o fluxo recusa a inativação enquanto existirem encomendas pendentes (`FornecedorDAO.hasEncomendasPendentes`), evitando deixar guias órfãs de um fornecedor ativo.

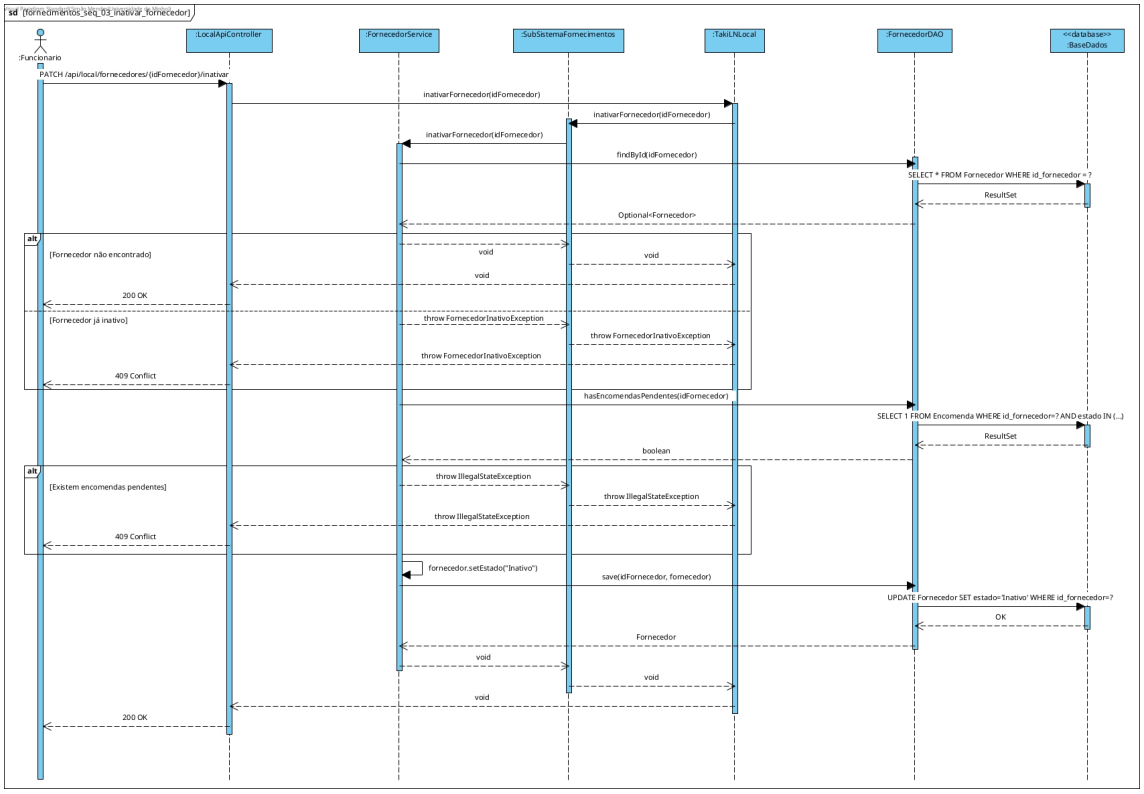


Figura 81: Diagrama de Sequência: Inativar Fornecedor

Para suportar os ecrãs de seleção do utilizador e auditorias, o sistema disponibiliza um fluxo de listagem que devolve a totalidade dos fornecedores registados, complementado por uma variante de pesquisa por termo que filtra o conjunto sobre o lado do serviço.

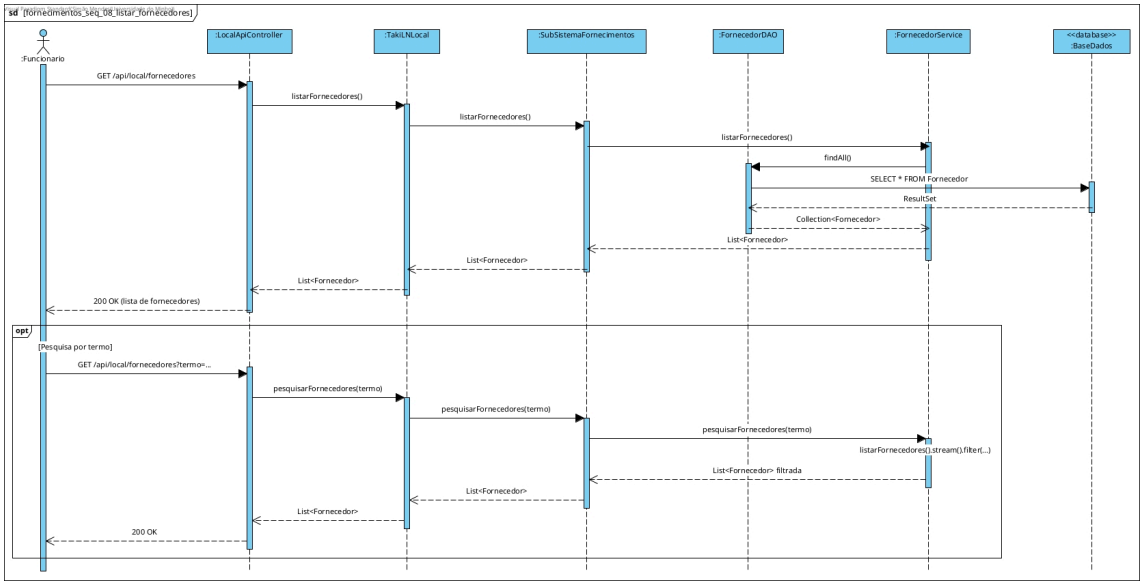


Figura 82: Diagrama de Sequência: Listar Fornecedores

Desenho Comportamental: Ciclo de Vida da Encomenda

O fluxo de uma transação de compra é a espinha dorsal deste módulo. O sistema foi desenhado para que o impacto financeiro e logístico só ocorra depois de validações explícitas do operador e das regras de negócio.

A criação de uma encomenda formaliza a intenção de compra e materializa o estado inicial da máquina (`Rascunho`). Antes de persistir, o serviço valida a existência e atividade do fornecedor, a presença de linhas e a contratualização individual de cada produto, sobrescrevendo o preço de custo de cada linha com o preço vigente em `ProdutoFornecedor`.

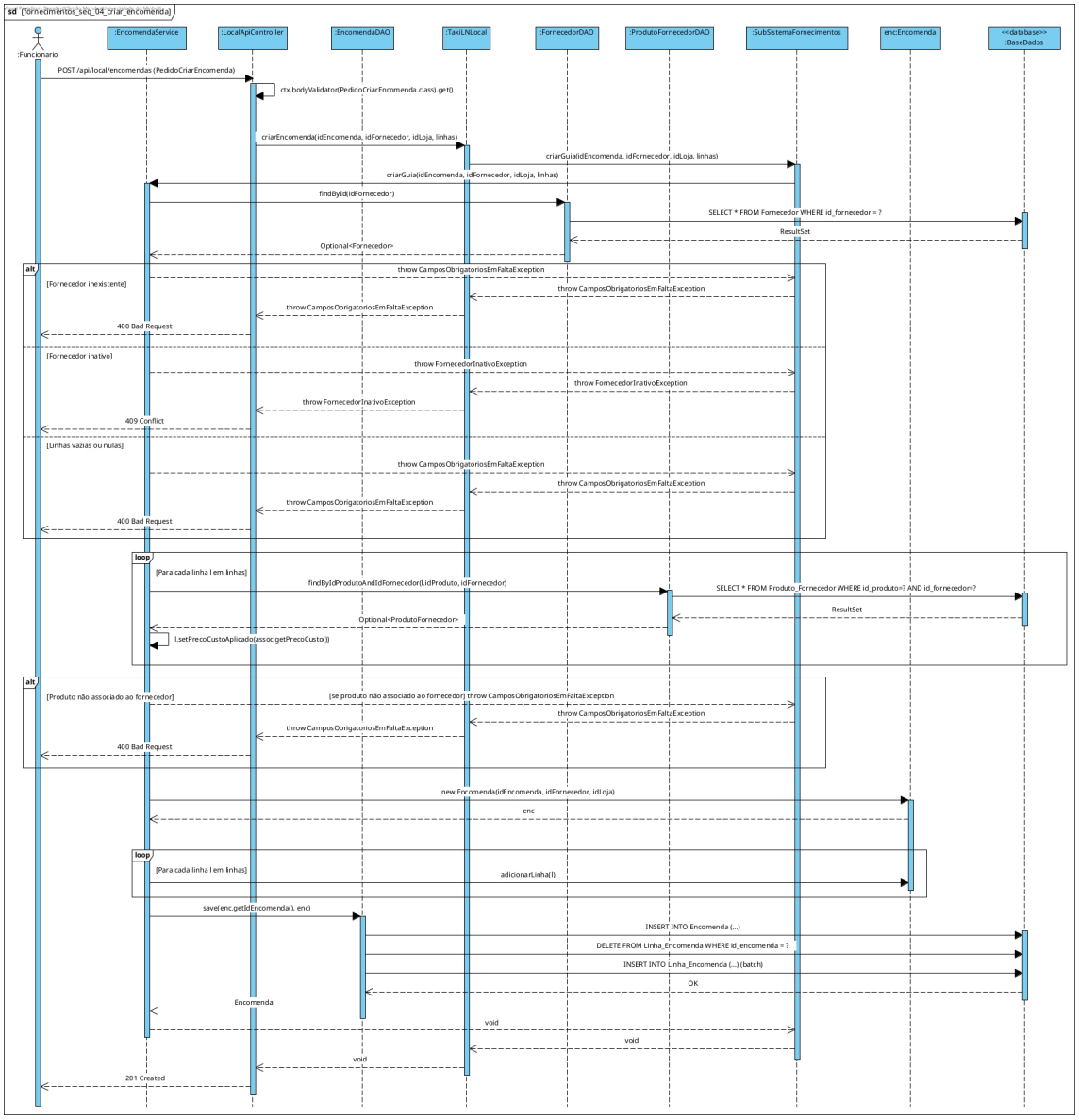


Figura 83: Diagrama de Sequência: Criar Encomenda a Fornecedor

Após a elaboração, a encomenda avança da fase pendente para o estado `Enviada`. Este fluxo evidencia o "selamento" dos dados — a partir deste momento, `podeModificarLinhas` devolve `false`, impedindo qualquer alteração às linhas enquanto a mercadoria não chegar à loja.

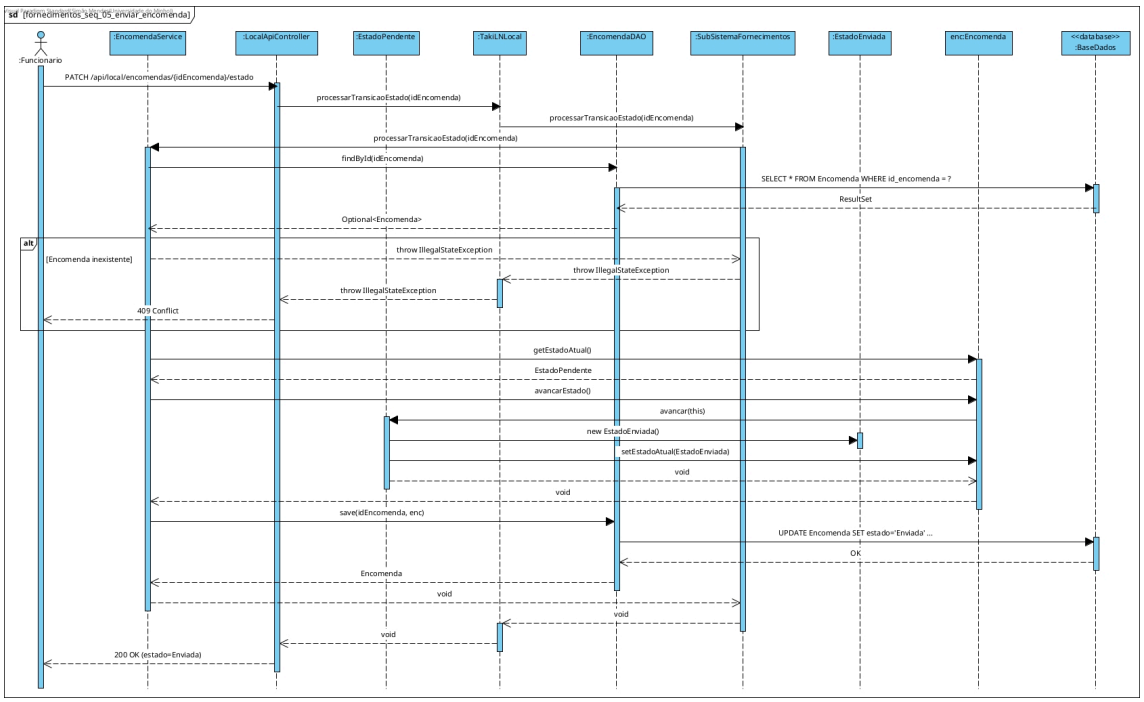


Figura 84: Diagrama de Sequência: Enviar Encomenda

No momento da receção, a encomenda transita para `Concluída`. Este processo regista a data de entrega e desencadeia, para cada linha, a chamada a `IInventarioService.processarEntradaEncomenda`, efetivando a entrada física da mercadoria no inventário da loja.

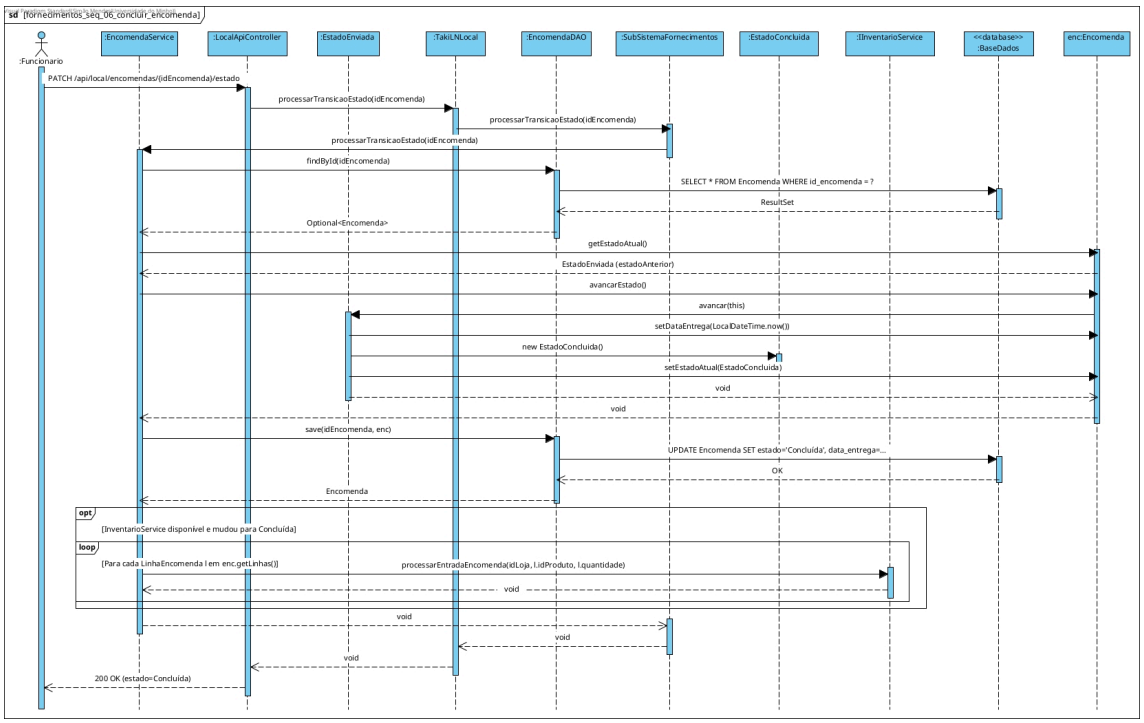


Figura 85: Diagrama de Sequência: Concluir Encomenda

Caso ocorra um imprevisto comercial, a máquina de estados permite o cancelamento seguro da encomenda — admissível apenas a partir de `Pendente`. A tentativa de cancelar a partir de `Rascunho` é rejeitada com a indicação de que o registo deve antes ser eliminado, e a partir de `Enviada`, `Concluída` ou `Cancelada` é igualmente recusada por inviabilidade da transição.

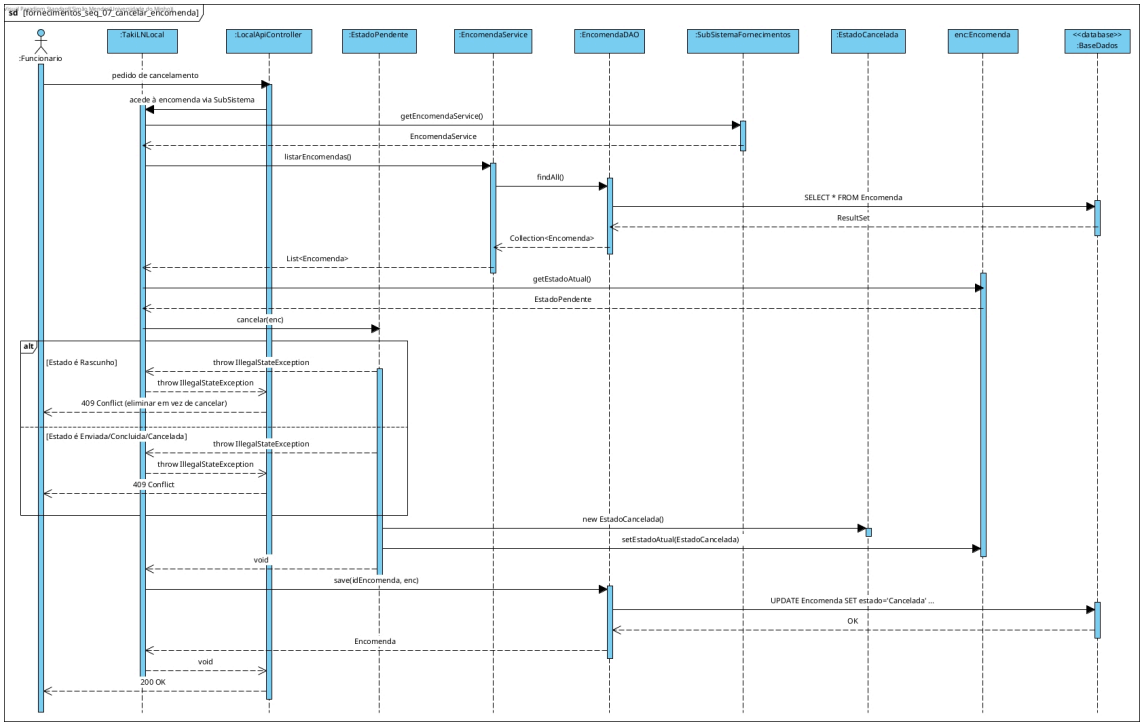


Figura 86: Diagrama de Sequência: Cancelar Encomenda

Paralelamente às mudanças de estado, o cálculo do total da guia é uma operação transversal e crítica para a validação financeira, sendo computado iterativamente sobre o agregado de `LinhaEncomenda` via `Encomenda.getValorTotal`.

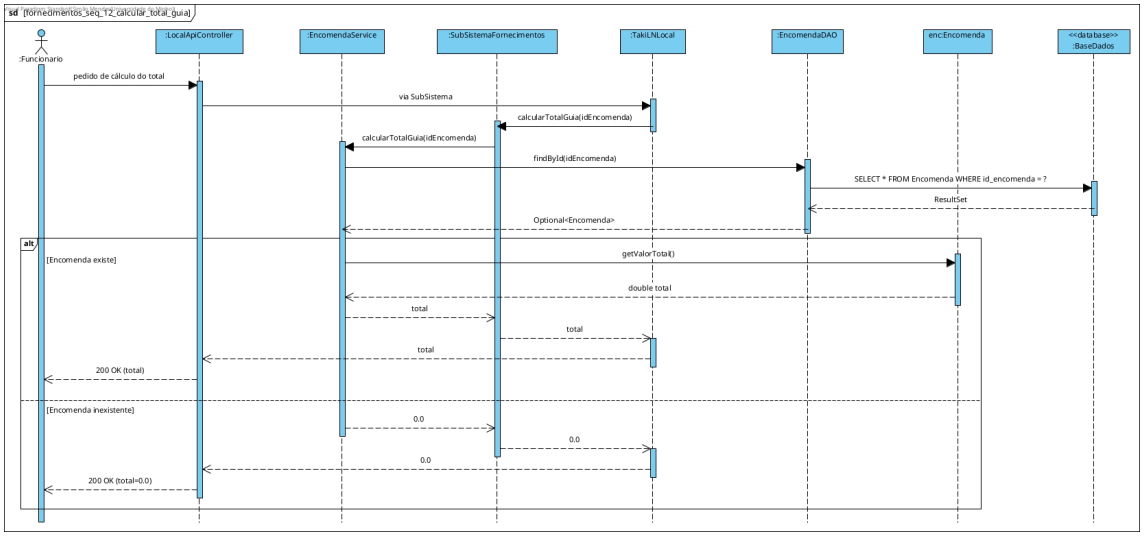


Figura 87: Diagrama de Sequência: Calcular Total da Guia

Desenho Comportamental: Vínculos Comerciais

Para impor o “Bloqueio Preventivo”, as compras assentam na premissa de que os produtos têm de estar contratualizados.

A associação de um produto a um fornecedor define o preço de custo da relação e autoriza (funciona como uma *whitelist*) que esse produto seja incluído em futuras encomendas para esse fornecedor. O fluxo contempla ainda a manutenção automática do atributo `preferencial`: se o preço da nova associação for estritamente inferior ao de outra associação preferencial pré-existente do mesmo produto, essa marca é rebaixada, mantendo coerente a noção de fornecedor preferencial por artigo.

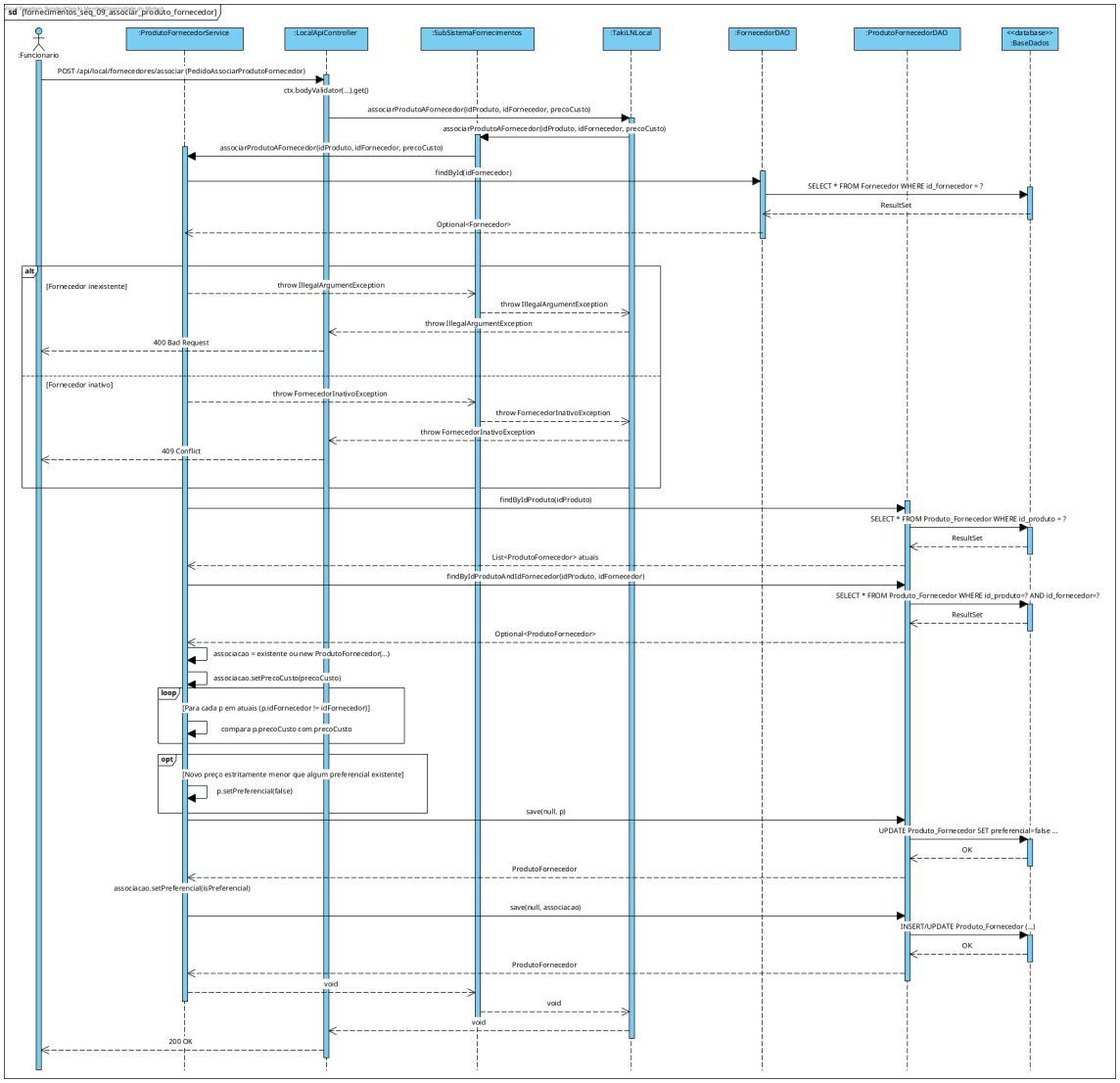


Figura 88: Diagrama de Sequência: Associar Produto a Fornecedor

Quando um acordo comercial cessa, a quebra do vínculo remove o produto do leque de opções de compra desse fornecedor, impossibilitando erros nas reposições futuras.

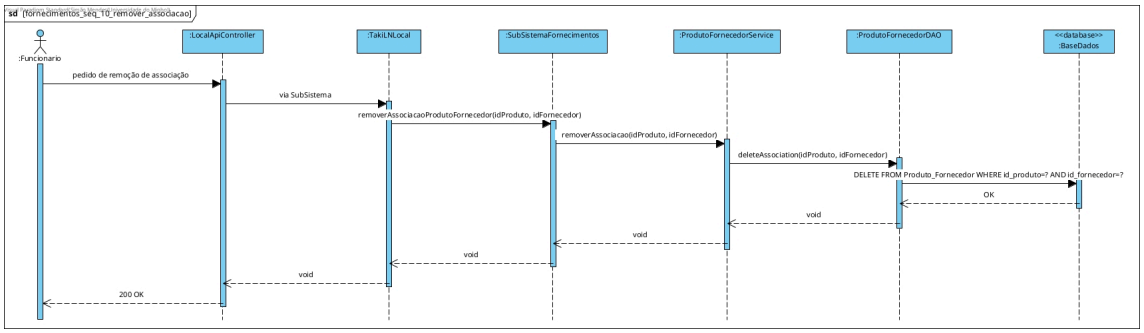


Figura 89: Diagrama de Sequência: Remover Associação Produto-Fornecedor

Por fim, no momento de abastecimento, o sistema interroga os vínculos para listar e filtrar rapidamente os produtos que o fornecedor em questão está autorizado a fornecer à loja.

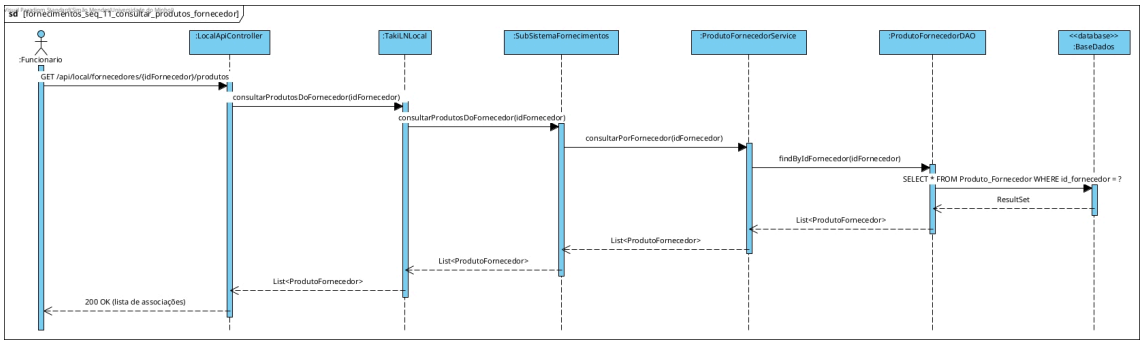


Figura 90: Diagrama de Sequência: Consultar Produtos de um Fornecedor

3.2.7. Ponto de Venda e Transações

Desenho Arquitetural: Vendas, Devoluções e Promoções (POS)

O subsistema de Vendas forma o núcleo transacional de alta disponibilidade da plataforma `Taki`. A sua conceção orquestra o processamento exato de pagamentos físicos com a avaliação e aplicação em tempo real de incentivos comerciais.

Arquitetura Orientada a Eventos (Padrão *Observer*): A operação de fecho de caixa recorre a um modelo assíncrono de difusão de notificações. Esta estratégia arquitetural introduz um desacoplamento profundo entre a transação económica central e os processos secundários:

- **Isolamento de Performance**: A arquitetura desenhada autoriza a execução de reações periféricas independentes (como a chamada a serviços de faturação externa ou a atualização de métricas analíticas) sem qualquer grau de penalização para a velocidade de resposta na frente de caixa.

Motor Hierárquico de Resolução de Promoções: O apuramento de benefícios comerciais apoia-se num algoritmo de travessia otimizado, o qual inspeciona a totalidade da topologia em árvore do catálogo:

- **Avaliação Abrangente**: O motor promocional analisa, num único ciclo computacional, as campanhas diretamente associadas aos artigos e as campanhas inferidas por herança hierárquica. Em seguida, processa as regras de prioridade e aplica a oferta economicamente mais vantajosa para o consumidor final;
- **Uniformidade Sistémica**: Este mecanismo centralizado certifica uma aplicação estritamente uniforme e isenta de ambiguidades no que respeita às políticas de desconto vigentes na infraestrutura da franquia.

Abstração Periférica e Latência Crítica: A estrutura encarrega-se de segmentar o processamento financeiro gerido por entidades externas, com a missão de blindar a fluidez do atendimento e a resiliência das operações através das seguintes diretrizes:

- **Camada Anticorrupção**: A plataforma instancia interfaces de abração estritas para mediar o diálogo com os terminais físicos de pagamento (TPA). Esta barreira de isolamento defende as regras de negócio nucleares contra interrupções de serviço ou anomalias de *hardware*;
- **Restrições Temporais**: O desenho técnico decreta limites de tolerância temporal inflexíveis para cada fase da finalização de compra. A adoção desta métrica previne bloqueios de *threads* prolongados, mitigando filas de espera e quebras sistémicas no atendimento ao público.

Faturação Encadeada e Conformidade Legal (`SAF-T PT`): A certificação fiscal exige que cada transação de venda resulte na emissão imediata de um documento estruturado e criptograficamente selado, forçando a implementação dos seguintes controlos arquiteturais:

- **Assinatura `RSA` e Encadeamento**: O sistema foi desenhado com base num algoritmo de *hashing* encadeado em que a assinatura de cada fatura depende intrínseca e irreversivelmente da assinatura da fatura imediatamente anterior. Este padrão de imutabilidade previne inserções ou remoções cirúrgicas de faturas no repositório, garantindo o total cumprimento das rigorosas diretrizes da Autoridade Tributária;
- **Isolamento Fiscal**: A entidade concetual associada à faturação opera num silo isolado do núcleo da venda, assegurando que o documento legal apenas adquire existência matemática após a consolidação bancária da transação, mantendo uma barreira estrita entre as operações comerciais (caixa) e as obrigações estatais.

Desenho Estrutural

A representação estrutural denota o agregado da transação de venda e os componentes responsáveis pela avaliação das lógicas comerciais e de pagamento.

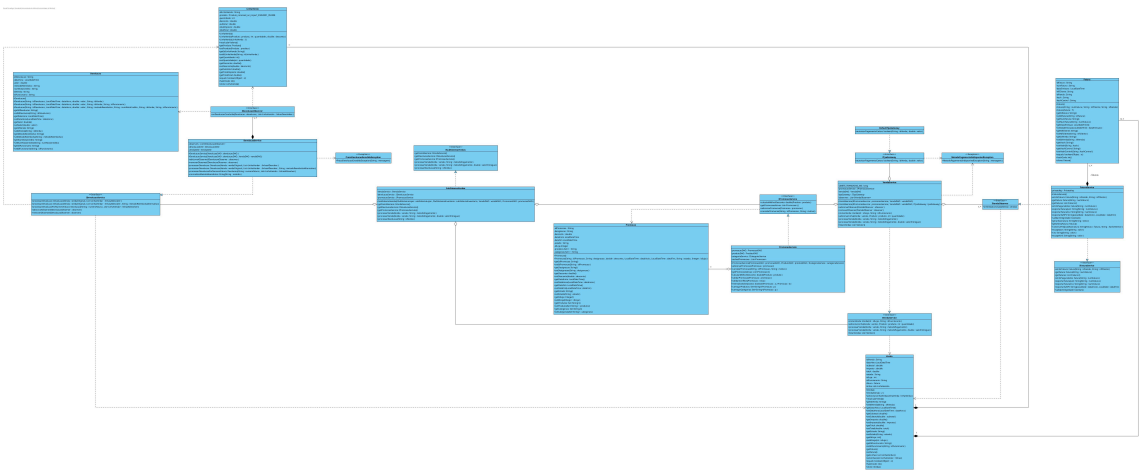


Figura 91: Diagrama de Classes: Subdomínio de Vendas (POS)

Desenho Comportamental

Os modelos de comportamento para a finalização de transações e para a resolução de campanhas estipulam a execução rigorosa de todas as validações num intervalo de tempo enquadrado com os requisitos de baixa latência de um Ponto de Venda. Os fluxogramas seguintes dissecam estas interações.

O **diagrama de sequência de registo de venda (Figura 92)** ilustra o fluxo completo de uma transação, desde a obtenção do serviço de vendas pelo funcionário até à conclusão da venda. O processo envolve a inicialização de uma nova venda, seguida de um ciclo de adição de produtos com cálculo automático de descontos e recálculo do total. De seguida, ocorre o processamento do pagamento, que pode ser eletrónico

(com autorização via *gateway* TPA) ou em numerário (com cálculo do troco). Finalmente, a notificação dos observadores, que desencadeia a emissão e assinatura da fatura através do `FaturaService`. **Este último processo, detalhado no segundo diagrama (Figura 93)**, abrange a criação do objeto Fatura, a construção do *payload* de assinatura com base no *hash* da fatura anterior, e o armazenamento da fatura assinada, garantindo assim a integridade e rastreabilidade do documento fiscal.

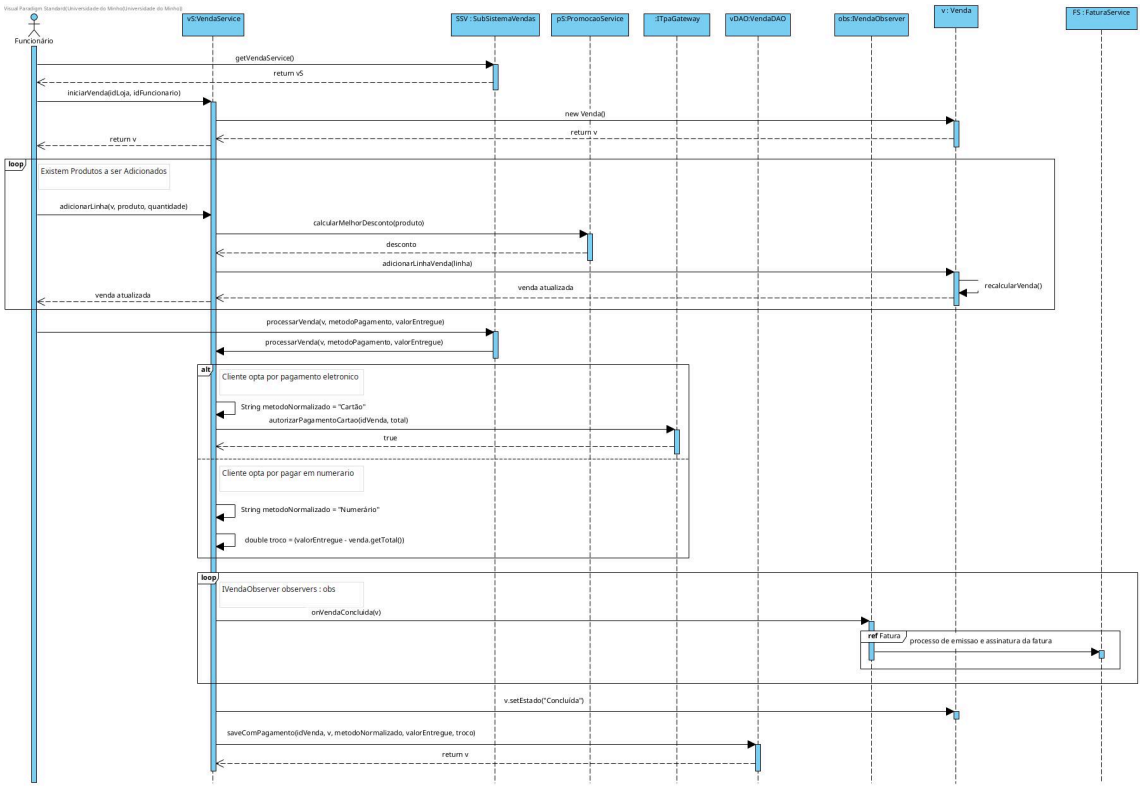


Figura 92: Diagrama de Sequência: Registar Venda

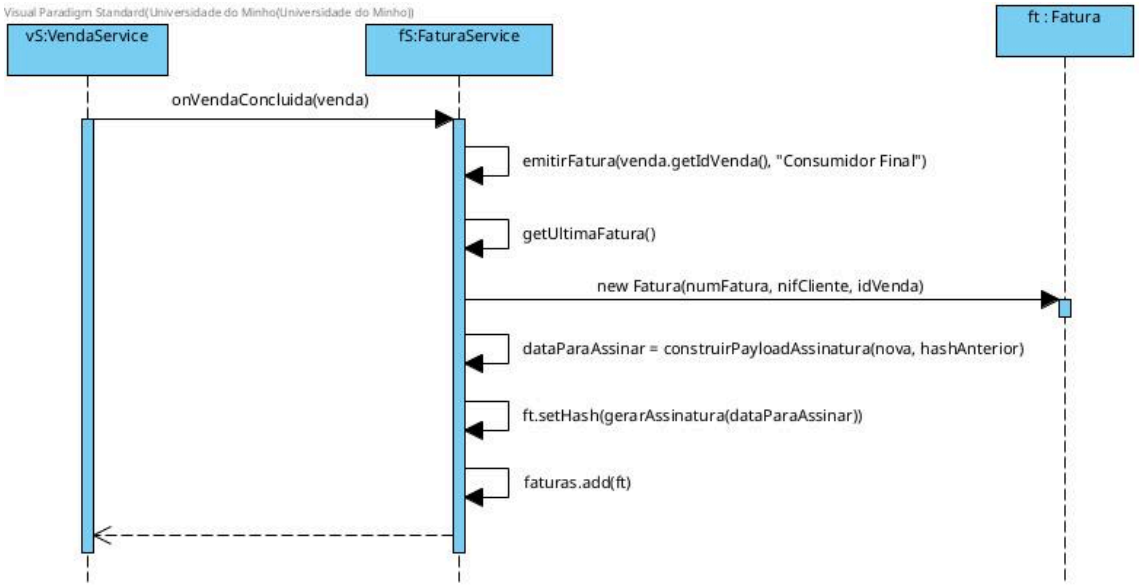


Figura 93: Diagrama de Sequência: Emissão da Fatura (referido no diagrama anterior)

O **diagrama de sequência de processamento de devolução (Figura 94)** ilustra o fluxo iniciado pelo funcionário, que invoca o método com a venda original, as linhas a devolver e um eventual método de reembolso alternativo. O serviço consulta o `VendaDAO` para recuperar o método de pagamento original, cria um novo

objeto `Devolucao`, persiste-o juntamente com as linhas a devolver, e, por fim, notifica todos os observadores registados. Isto permitirá que componentes externos reajam ao evento, tal como a reintegração de *stock* ou o acionamento do reembolso, antes de retornar o objeto de devolução ao funcionário.

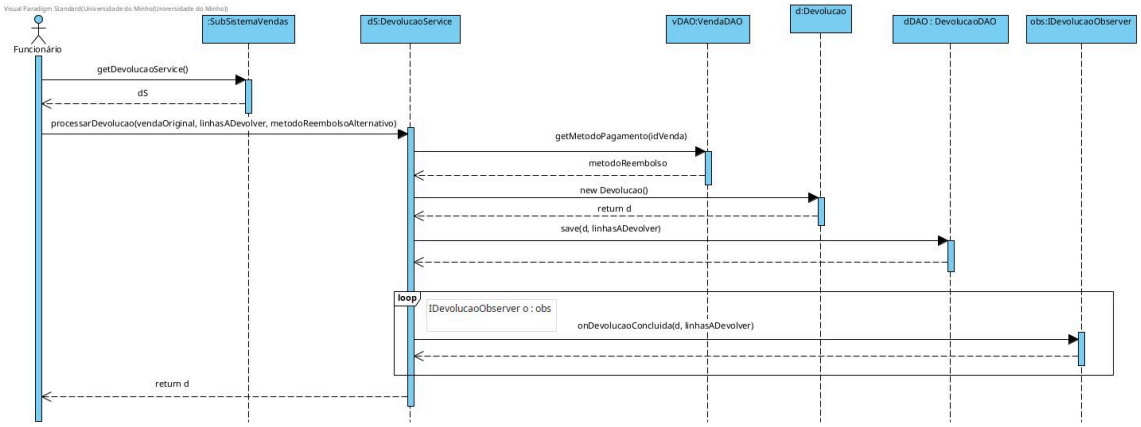


Figura 94: Diagrama de Sequência: Processar Devolução

O **diagrama de sequência de criação de promoção (Figura 95)** ilustra o fluxo iniciado pelo administrador, que invoca p método com os dados da nova campanha. O serviço executa internamente a validação da promoção e a verificação de conflitos com promoções existentes antes de persistir o registo. Após o armazenamento, dois blocos opcionais tratam da associação de âmbito:

- caso a promoção tenha produtos definidos, percorre-os em ciclo invocando `addProduto` para cada um;
- de forma análoga, caso existam categorias associadas, itera sobre elas chamando `addCategoria`.

Assim é garantida uma ligação granular entre a promoção e os artigos ou grupos de artigos aos quais se aplica.

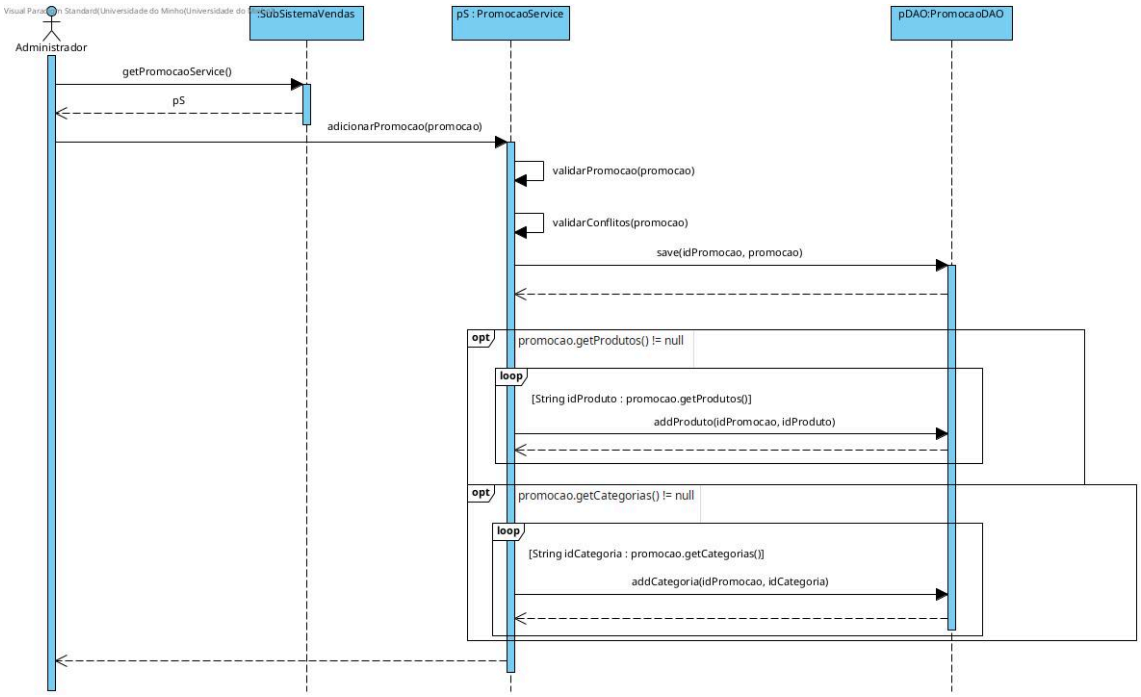


Figura 95: Diagrama de Sequência: Criar Política de Promoção

O **diagrama de sequência de cancelamento de promoção (Figura 96)** ilustra um fluxo simples iniciado pelo administrador, que invoca o método com o identificador da promoção e o motivo do cancelamento. O serviço recupera o objeto correspondente, atualiza o seu estado para “Cancelada” diretamente no objeto

Promocao, e persiste a alteração de volta na base de dados, concluindo o processo sem necessidade de notificações adicionais ou lógica de compensação.

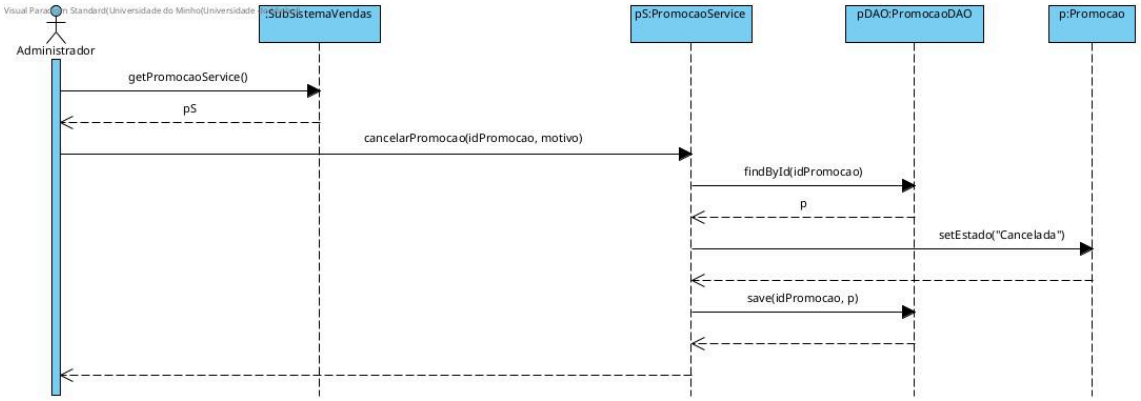


Figura 96: Diagrama de Sequência: Cancelar Política de Promoção

3.2.8. Estatísticas e Relatórios

Desenho Arquitetural: Modelo de Estatísticas e Analítica

O subsistema de Estatísticas constitui o módulo de inteligência de negócio da plataforma Taki. A sua arquitetura foi delineada para extrair e agregar dados operacionais, garantindo o fornecimento de métricas exatas para o apoio à decisão estratégica, com base no histórico consolidado das operações comerciais e da gestão de armazém.

Agregação em Memória e Abstração de Dados: O apuramento de relatórios e dos Indicadores-Chave de Desempenho (KPIs) recorre a uma estratégia de extração controlada e processamento dinâmico na camada aplicacional, dispensando o uso de lógicas pesadas na base de dados:

- **Cálculos Dinâmicos e Filtragem na Camada Lógica**: O serviço de estatísticas coordena a recolha das transações e recorre aos fluxos de dados funcionais para a aplicação de filtros em memória (por loja, por janela temporal ou por categoria de produto). Esta abordagem arquitetural isola a base de dados de lógicas de negócio complexas, centralizando o cálculo matemático e a sumarização no motor aplicacional;
- **Dependência Estrita de Abstrações (DAO)**: O modelo analítico não interage com os repositórios físicos diretamente, delegando a responsabilidade de acesso e mapeamento objeto-relacional aos *Data Access Objects* (VendaDAO, InventarioDAO, ProdutoDAO). Esta barreira protetora garante que as regras de extração estatística permanecem imunes a eventuais migrações da tecnologia de persistência.

Motor de Correlacionamento de Dados Multidomínio: A formulação de estatísticas rigorosas exige uma interligação estrutural entre diversos domínios da aplicação, suportada por interfaces de agregação resilientes:

- **Visão Holística da Franquia**: O subsistema assume a responsabilidade de cruzar informações isoladas, congregando o fluxo financeiro do serviço de vendas e o impacto gerado pelas campanhas promocionais. Esta fusão de dados origina indicadores de eficácia (KPI's) unificados em toda a rede comercial;
- **Avaliação Flexível e Granularidade**: O desenho garante o suporte a algoritmos de filtragem dinâmica através de múltiplos eixos analíticos (janelas temporais ou categorias de inventário), permitindo análises que viajam desde a tendência macroeconómica da franquia até ao escrutínio minucioso de um artigo específico.

Desenho Estrutural

A representação estrutural elucida as entidades agregadoras e as interfaces de serviço especializadas na compilação, indexação e cálculo de dados históricos e operacionais.

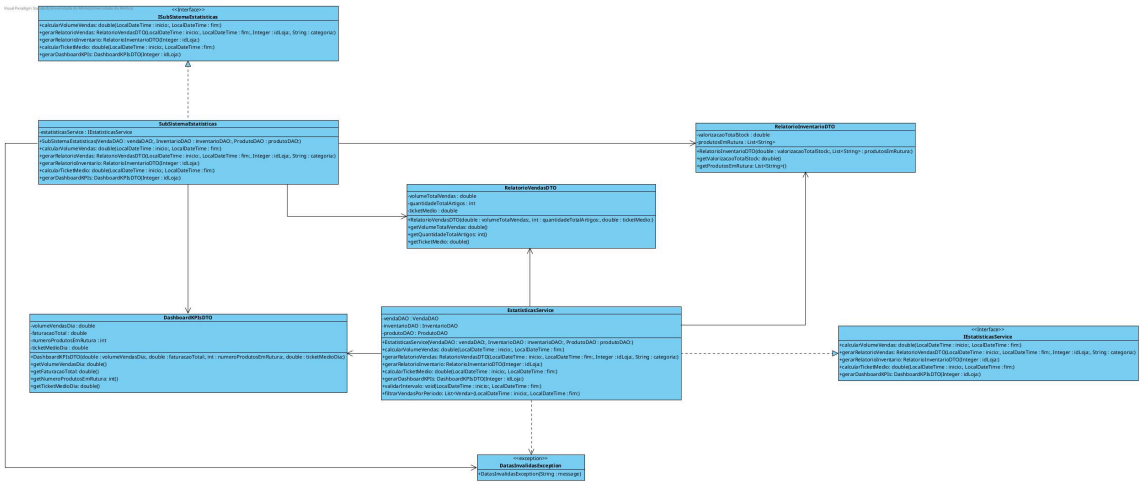


Figura 97: Diagrama de Classes: Subdomínio de Estatísticas

Desenho Comportamental

Os modelos de comportamento elucidam as iterações do sistema perante pedidos de compilação analítica, demonstrando o fluxo de recolha em massa de dados e a subsequente filtragem.

O **diagrama de sequência de geração do relatório de vendas (Figura 98)** ilustra o fluxo iniciado pelo gestor, que solicita a produção do relatório com um intervalo temporal, uma loja e uma categoria como parâmetros. O serviço valida o intervalo, filtra as vendas por período e por loja, e percorre, em dois ciclos aninhados, cada venda e cada linha de venda, consultando o `ProdutoDAO` para obter as categorias de cada produto e acumulando as métricas relevantes. Após a iteração completa, instancia um `RelatorioVendasDTO` com o volume total, a quantidade de artigos e o *ticket* médio, retornando-o ao gestor.

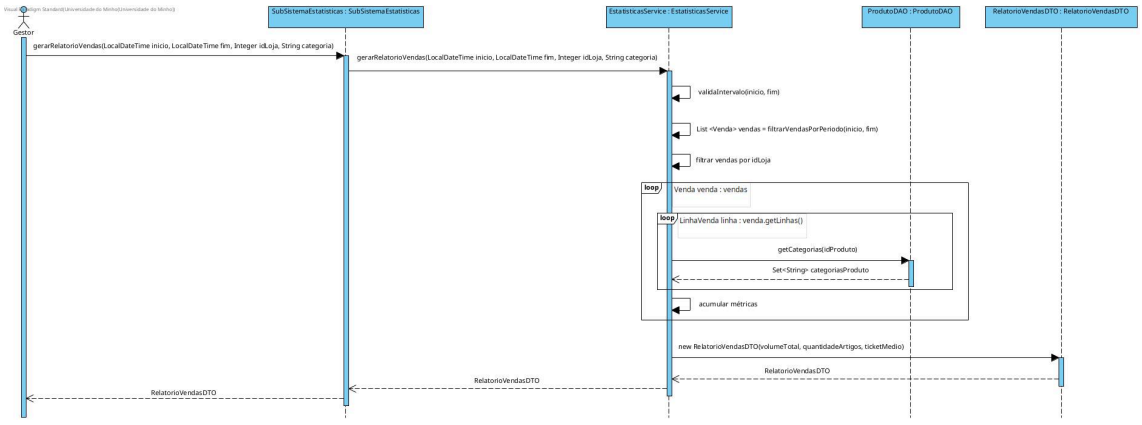


Figura 98: Diagrama de Sequência: Gerar Relatório de Vendas

O **diagrama de sequência de geração do relatório de inventário (Figura 99)** ilustra o fluxo iniciado pelo gestor, que solicita o relatório para uma determinada loja. O serviço obtém todos os registos de inventário, filtra-os pelo identificador da loja, e itera sobre cada entrada consultando o `ProdutoDAO` para recuperar os dados do produto correspondente, calculando a valorização de cada item pela multiplicação da quantidade pelo preço de custo. Após o ciclo, verifica quais os produtos em situação de rutura com base na quantidade mínima definida, e instancia um `RelatorioInventarioDTO` com a valorização total e as situações de rutura identificadas, retornando-o ao gestor.

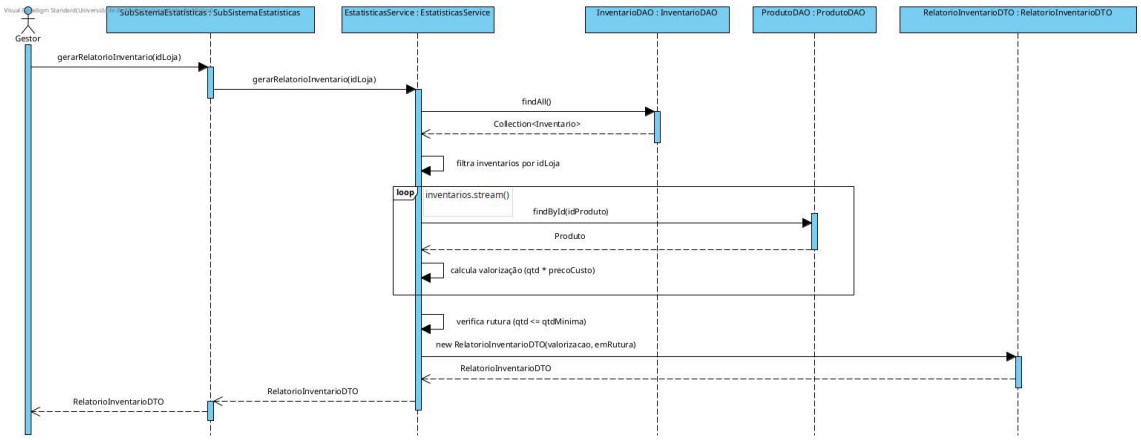


Figura 99: Diagrama de Sequência: Gerar Relatório de Inventário

3.2.9. Sistema Local

Desenho da Arquitetura: Fachada Local (TakiLNLocal)

A classe `TakiLNLocal` constitui o núcleo de coordenação operacional das instâncias de loja da plataforma `Taki`. A sua conceção estabelece um ponto de entrada único para a totalidade das operações locais, desde a autenticação de funcionários até à sincronização com a sede, através da orquestração de cinco subsistemas independentes sob uma interface contratual estrita.

Domínios e Subsistemas Coordenados: A fachada representa o ponto de convergência de cinco áreas de domínio distintas, cada uma com responsabilidades estritas:

- **Subsistema de Vendas**: Gere o ciclo de vida transacional da venda, o apuramento de promoções e o processamento de devoluções;
- **Subsistema de Lojas**: Assegura a gestão do catálogo de produtos, a hierarquia de categorias e o controlo de acessos dos funcionários;
- **Subsistema de Inventário**: Monitoriza os níveis de *stock*, o registo de movimentos de mercadoria e a emissão de alertas de rotura;
- **Subsistema de Fornecimentos**: Orquestra o diálogo com entidades externas para a gestão de encomendas e o acompanhamento de fornecedores;
- **Subsistema de Sincronização**: Garante a persistência local de eventos e a sua transmissão resiliente para a infraestrutura central (Sede).

Padrão Fachada e Desacoplamento Estrutural: A adoção do padrão *Facade* impõe uma fronteira arquitetural rígida entre a camada de apresentação e os subsistemas internos de negócio. Esta estratégia confere à plataforma as seguintes propriedades:

- **Encapsulamento Total**: O acesso direto a subsistemas individuais por clientes externos é inexistente. A mediação integral da comunicação pela fachada garante que as evoluções internas nos módulos não propaguem efeitos para o exterior da camada de negócio;
- **Núcleo de Composição**: A fachada atua como a entidade coordenadora exclusiva para a criação e interligação de todos os subsistemas. Esta centralização no controlo de dependências anula a existência de ligações diretas entre os diferentes módulos funcionais;
- **Isolamento para Validação**: A interface `ITakiLNLocal` estipula o contrato estrito de serviços da plataforma. A imposição desta fronteira abstrata viabiliza a injeção de componentes simulados (*mocks*) durante as fases de teste, o que preserva a integridade das camadas superiores.

Desenho Estrutural

A representação estrutural expõe a fachada `TakiLNLocal`, a interface de contrato e a coordenação dos subsistemas funcionais.

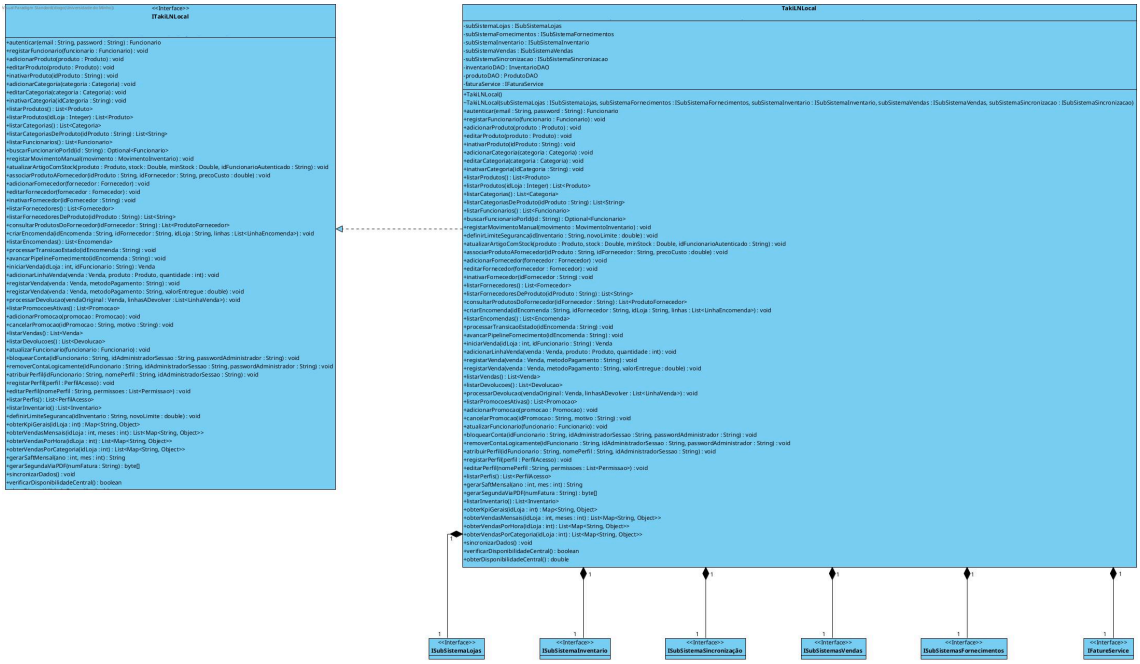


Figura 100: Diagrama de Classes: Fachada Local e Subsistemas

Desenho Comportamental

O modelo seguinte detalham a dinâmica de interação e a delegação de responsabilidades.

O **diagrama de sequência de autenticação** detalha o processo de validação de credenciais iniciado pelo utilizador através do endpoint `POST /api/local/auth/login` com o endereço de e-mail e palavra-passe. O `LocalApiController` invoca o método `autenticar(email, password)` na fachada `TakiLNLocal` através da interface `ITakiLNLocal`. A fachada acede ao `SubSistemaLojas` para obter o `FuncionarioService`, que coordena o processo de autenticação consultando o `FuncionarioDAO` para recuperar o funcionário pelo e-mail. Caso o utilizador não seja encontrado, o serviço lança uma `CredenciaisInvalidasException`, resultando numa resposta HTTP `401 Unauthorized`. Se o funcionário for encontrado, o serviço verifica o estado da conta através de `verificarEstadoConta(funcionario)`, lançando uma `ContaBloqueadaException` (com resposta `403 Forbidden`) se a conta estiver bloqueada. De seguida, valida a palavra-passe usando `passwordHasher.matches(password, hash)`; se a validação falhar, lança novamente `CredenciaisInvalidasException`. Após a validação bem-sucedida, o serviço devolve um clone do objeto `Funcionario` ao controlador, que gera um token JWT através de `gerarTokenJwt(funcionario)` e o envia num *cookie* HttpOnly chamado `auth_token`, respondendo com `200 OK` e o objeto `Funcionario` serializado em JSON, garantindo assim a segurança do token contra ataques XSS e permitindo ao utilizador aceder aos recursos protegidos do sistema.

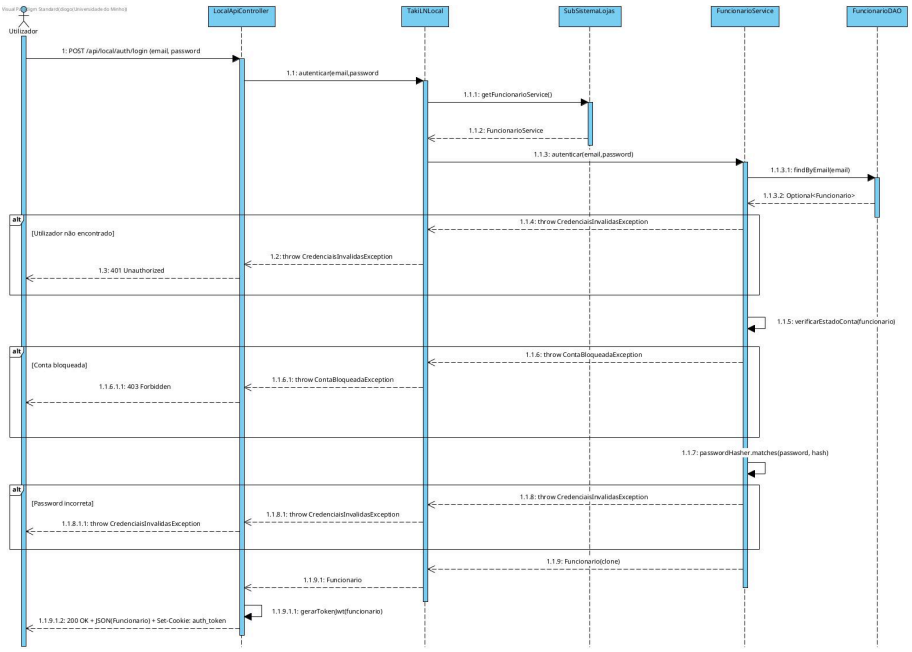


Figura 101: Diagrama de Sequência: Efetuar Autenticação

3.2.10. Sistema Central

Desenho Arquitetural: Fachada Principal (TakiLNGlobal)

O motor central da plataforma `Taki` está consolidado no componente global da Lógica de Negócio (LN). A sua arquitetura foi concetualizada não para albergar algoritmos complexos, mas sim para atuar como o maestro de todo o ecossistema, assegurando a intercomunicação entre módulos dispersos.

Ponto Único de Acesso (Padrão *Facade*)

O desenho da arquitetura decreta que a camada de apresentação está estritamente proibída de invocar diretamente os submódulos da aplicação. Para garantir este isolamento, instituiu-se o padrão estrutural *Facade* (Fachada).

Esta entidade atua como a **única fronteira de comunicação** de toda a aplicação. Do ponto de vista concetual, isto blinda a camada visual contra o complexo organograma de classes interno. Se o subsistema de estatísticas ou de inventário sofrer uma refatorização topológica no futuro, a camada exterior permanecerá imune, visto que o seu único elo de ligação e conhecimento é a Fachada central.

Fragmentação de Domínio e Prevenção do *God Object*

Numa aplicação de gestão à escala empresarial, a aglomeração de todas as regras num único bloco ditaria o colapso da manutenção (o antipadrão *God Object*). Como tal, a arquitetura orquestrou a divisão em quadrantes de alta coesão:

- **Lojas e Inventário**: Responsável pelo catálogo e recursos humanos;
- **Fornecimentos**: Gestor da cadeia de abastecimento;
- **Estatísticas e `KPIs`**: Entidade passiva de leitura e inteligência de negócio;
- ***View***: Interface de sincronização visual.

A componente global atua como o agregado concetual destes quadrantes, mapeando as intenções do utilizador para o subsistema com a autoridade correta para o executar.

Desenho Estrutural

O diagrama estrutural espelha a topologia de agregação, provando que a componente global não depende das classes físicas de negócio, mas sim de portas (interfaces) que definem os contratos dos subsistemas.

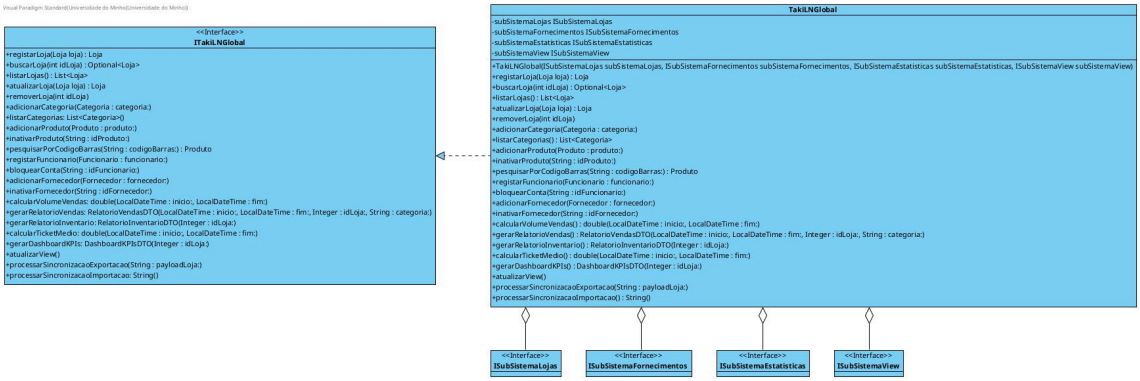


Figura 102: Diagrama de Classes: Fachada Principal da Lógica de Negócio

3.2.11. Arquitetura da API

A arquitetura da API do sistema `Taki` é desenhada de acordo com princípios de modularidade e segregação de responsabilidades. Encontra-se otimizada para um ambiente de execução distribuído entre um **Nó Central** (*Central*) e **Nós Locais** (*Local*).

Modelo de Execução Híbrido e Funções de Nó

- A API utiliza um sistema de funções (`NodeRole`) que determina o comportamento do servidor em tempo de execução através da variável de ambiente *APP_MODE*;
- **Nó Local (LOCAL):** Inicia apenas a `LocalApiApp`, focada em operações de ponto de venda, inventário e gestão da loja física;
- **Nó Central (CENTRAL):** Inicia apenas a `GlobalApiApp`, o que permite a gestão agregada da cadeia de lojas e o acesso centralizado aos dados de todas as unidades.

Camada de *Middleware* e Segurança Partilhada A arquitetura inclui uma "Camada Partilhada" configurada pelo `ApiBootstrapHelper`, que garante a consistência em ambas as APIs. Esta camada inclui:

- **Segurança (`JWT`):** Autenticação baseada em *tokens* `RS256`, com validação estrita de perfis de acesso (RBAC);
- **Tratamento de Erros:** Um `TratadorGlobalExcecoesApi` que padroniza as respostas de erro para o formato JSON estruturado;
- **Modo Read-Only:** Um *middleware* específico que, quando o nó é CENTRAL, bloqueia operações de escrita em determinados *endpoints* para proteger a integridade dos dados que devem ser geridos exclusivamente a nível local.

Mapeamento de Endpoints (API Local) A `API Local` expõe os recursos necessários para a operação diária de uma filial.

Método	Endpoint	Descrição Funcional
POST	/api/local/auth/login	Autentica o utilizador e emite o token JWT de sessão.
POST	/api/local/auth/logout	Encerra a sessão ativa do utilizador e invalida o token JWT.
GET	/api/local/vendas	Recupera o histórico de transações de venda realizadas na loja local.
POST	/api/local/vendas	Regista uma nova transação de venda, o que aciona o respetivo abate de stock.
POST	/api/local/vendas/{id}/devolucao	Processa a devolução de uma venda e repõe o respetivo stock.
GET	/api/local/inventario	Lista o inventário atual da loja com as respetivas quantidades físicas e os alertas.
POST	/api/local/inventario/movimentos	Regista um ajuste manual de stock (ex: quebras, furtos) e o respetivo motivo.
GET	/api/local/inventario/alertas	Obtém a lista de produtos cujo stock se encontra abaixo do limiar de alerta configurado.
GET	/api/local/produtos	Consulta o catálogo de produtos disponíveis na loja para venda, com a apresentação dos preços e promoções ativas.
GET	/api/local/produtos/{id}	Apresenta o detalhe completo de um produto específico com base no seu identificador.
GET	/api/local/fornecimentos	Lista as encomendas a fornecedores (guias de remessa e estado de receção).
POST	/api/local/fornecimentos	Cria uma nova nota de encomenda a um fornecedor local.
PUT	/api/local/fornecimentos/{id}/estado	Atualiza o estado de uma encomenda (ex: de Pendente para Enviada ou de Enviada para Concluída).

Tabela 79: Principais Endpoints da API Local

Mapeamento de Endpoints (API Global) A `API Global` é inicializada apenas no Nó Central e foca-se na administração da rede e na consolidação analítica.

Método	Endpoint	Descrição Funcional
GET	/api/global/estatisticas/vendas	Gera o relatório agregado de faturação de toda a cadeia de lojas.
GET	/api/global/estatisticas/inventario	Devolve o relatório global consolidado de inventário, o que inclui a avaliação financeira.
GET	/api/global/estatisticas/kpi	Devolve os indicadores-chave de desempenho (ticket médio, total de vendas, top produtos).
GET	/api/global/lojas	Devolve a lista de todas as filiais (Nós Locais) registadas no sistema central.
GET	/api/global/produtos	Obtém a lista de todos os produtos do catálogo central da franquia.
POST	/api/global/produtos	Adiciona um novo produto ao catálogo central, para posterior sincronização com as filiais.
PUT	/api/global/produtos/{id}	Altera os dados de um produto existente (ex: preço, descrição).
DELETE	/api/global/produtos/{id}	Inativa (soft-delete) um produto no catálogo central.
GET	/api/global/categorias	Lista a árvore de categorias hierárquicas que agrupam os produtos.
POST	/api/global/categorias	Cria uma nova categoria organizativa de produtos no sistema.
GET	/api/global/promocoes	Lista todas as regras promocionais ativas na rede de lojas.
POST	/api/global/promocoes	Cria uma nova política de promoção (desconto) aplicável a determinados produtos ou categorias.
PUT	/api/global/promocoes/{id}/cancelar	Força o cancelamento de uma campanha promocional antes da sua data de fim prevista.
GET	/api/global/fornecedores	Recupera a lista de entidades parceiras de aprovisionamento registadas a nível central.
POST	/api/global/fornecedores	Regista uma nova entidade fornecedora.
DELETE	/api/global/fornecedores/{id}	Remove logicamente (soft-delete) a associação a um fornecedor no nó central.
GET	/api/global/utilizadores	Lista as contas de utilizadores autorizados a nível global (empregados, administradores).
POST	/api/global/utilizadores	Regista as credenciais de um novo funcionário e define os seus perfis de acesso.
PUT	/api/global/utilizadores/{id}/bloquear	Invalida o acesso de uma conta temporária ou permanentemente.
GET	/api/global/sincronizacao/status	Verifica o estado de replicação de dados entre os Nós Locais e o Nó Central.
POST	/api/global/sincronizacao/trigger	Despoleta um pedido imediato de sincronização (*pull/push*) de eventos de outbox.

Tabela 80: Principais Endpoints da API Global

3.3. *User-Interface*

A *User-Interface* representa o principal ponto de contacto entre os colaboradores da `Taki` e o sistema de gestão. Num ambiente de retalho de conveniência a operar constantemente, a eficiência, a clareza e a rapidez de interação não são apenas preferências estéticas, mas sim fatores críticos de sucesso para o negócio. Por conseguinte, o design da interface foi tratado como uma prioridade no ciclo de desenvolvimento, visando garantir transações ágeis e reduzir a curva de aprendizagem de novos operadores para um tempo inferior a duas horas.

Para materializar a visão do sistema e validar as decisões de arquitetura de informação antes da fase de codificação, foi concebido um protótipo de alta fidelidade recorrendo à plataforma de design colaborativo **Figma**. A construção deste protótipo permitiu à equipa mapear visualmente os Casos de Uso, testar fluxos de navegação e desenhar ecrãs distintos e otimizados para os diferentes perfis de acesso, nomeadamente a frente de loja (Ponto de Venda/POS) e a retaguarda (*Backoffice* para Gestores e Administração).

Por forma a assegurar que o protótipo cumpria os mais elevados padrões de usabilidade e respondia rigorosamente aos Requisitos Não Funcionais do projeto, todo o processo de conceção visual e interativa foi estritamente guiado pelas 10 Heurísticas de Usabilidade de `Jakob Nielsen`. A adoção destes princípios consagrados da Engenharia de *Software* e do *design* de interação garantiu a criação de um sistema robusto, focado na prevenção de erros, no controlo do utilizador e na utilização de uma linguagem visual familiar ao contexto do retalho (refletindo de forma pragmática o lema da organização: “Não inventes, Taki!”).

Nas subsecções seguintes, são apresentados os principais ecrãs do protótipo desenvolvido, acompanhados da respetiva justificação sobre o modo como as heurísticas de Nielsen foram aplicadas na prática para otimizar a operação diária da cadeia de lojas.

Protótipo

Com o intuito de materializar os requisitos do sistema e validar as decisões de arquitetura de informação antes do início da fase de implementação técnica, foi concebida uma maquete interativa de alta fidelidade com recurso à ferramenta de design colaborativo `Figma`.

A prototipagem assumiu um papel central no ciclo de desenvolvimento do Sistema de Gestão da `Taki`, funcionando como uma prova de conceito visual. Esta etapa permitiu à equipa simular os fluxos de navegação dos vários Casos de Uso e garantir que a interface responde de forma rigorosa ao paradigma de usabilidade exigido por um ambiente de retalho de conveniência (transações rápidas e curva de aprendizagem mínima).

O desenho das interfaces reflete a aplicação prática das heurísticas de usabilidade previamente estabelecidas e respeita a identidade pragmática da organização (“Não inventes, Taki!”). O protótipo evidencia a separação clara entre as duas grandes dimensões do sistema:

- **Frente de Loja (POS):** Um ambiente de operação rápida, otimizado para a leitura de códigos de barras e interação ágil, destinado prioritariamente aos Funcionários;
- **Retaguarda (*Backoffice*):** Um ambiente analítico e de gestão de dados, com foco no controlo de acessos, gestão de catálogo e análise de métricas, destinado aos perfis de Gestor e Administrador.

Nas figuras seguintes, são apresentados os principais ecrãs que compõem a interface da plataforma. A exposição ilustra a jornada visual e lógica dos utilizadores, estruturada no seguinte fluxo operacional:

1. **Acesso e Autenticação:** O ecrã de entrada unificado, onde a validação de credenciais determina o perfil e as permissões do utilizador (UC10);
2. **Frente de Loja (Ponto de Venda):** O terminal de vendas (POS), ilustrando o ambiente de caixa rápido com grelha visual de produtos e carrinho de compras (UC1);
3. **Retaguarda (Catálogo e Fornecedores):** As interfaces de gestão diária da loja, englobando o painel de inventário, a listagem de fornecedores e a configuração de campanhas promocionais (UC2 a UC7);

4. **Retaguarda (Administração e Segurança):** A área restrita de topo, desenhada para a configuração de contas de utilizador, atribuição de lojas e gestão de acessos (UC11 a UC16).

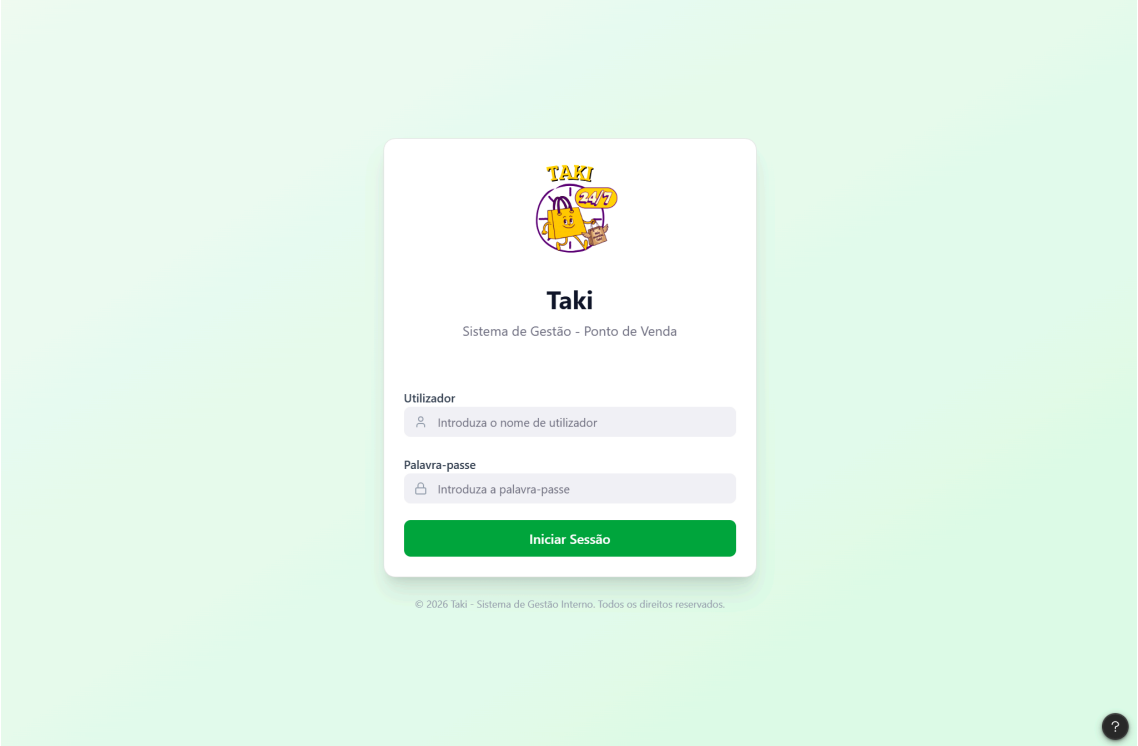


Figura 103: Login à plataforma

Este é o ponto de entrada único e centralizado da plataforma. Seguindo a heurística de `Design Estético e Minimalista`, o ecrã apresenta uma interface limpa, desprovida de distrações visuais, contendo apenas o logótipo da Taki e o formulário de credenciais. É a partir deste ecrã que o sistema valida a identidade do utilizador e aplica o Controlo de Acessos Baseado em Perfis (RBAC), determinando se o utilizador será encaminhado para a Frente de Loja ou se terá acesso aos separadores restritos de gestão.

Figura 104: Opções de lojas da empresa

Para garantir uma gestão centralizada e eficiente de toda a cadeia, o sistema implementa um **Seletor de Contexto** localizado estrategicamente na parte superior da barra lateral. Este menu `dropdown` funciona

como um filtro global, permitindo ao utilizador (com as devidas permissões, o Proprietário ou Administrador) alternar rapidamente entre a visão de uma loja específica ou a visão consolidada de "Todas as Lojas".

- **Principais características:**
  - **Ubiquidade e Persistência:** Por estar posicionado acima das abas de navegação na barra lateral, o seletor está sempre visível e acessível. Isto permite que o utilizador troque o contexto da análise sem ter de interromper a tarefa atual ou navegar para fora da página onde se encontra;
  - **Isolamento de Dados:** Ao selecionar uma loja específica (ex: Taki Famalicão), o sistema filtra automaticamente todas as informações apresentadas, desde os níveis de *stock* e vendas no Dashboard até à lista de funcionários e relatórios, garantindo que os dados visualizados pertencem exclusivamente a essa unidade;
  - **Visão Global:** A opção "Todas as Lojas" oferece uma perspetiva macro da organização, consolidando métricas e indicadores de desempenho de todos os pontos de venda num único ecrã, facilitando a comparação de resultados e a tomada de decisões estratégicas;
  - **Integração com RBAC (Role-Based Access Control):** A visibilidade e funcionalidade deste menu respeitam a hierarquia de acesso (RBAC). Enquanto um Gerente de Loja visualiza apenas a sua unidade, os cargos de gestão de topo usufruem da flexibilidade total de alternância de contexto.

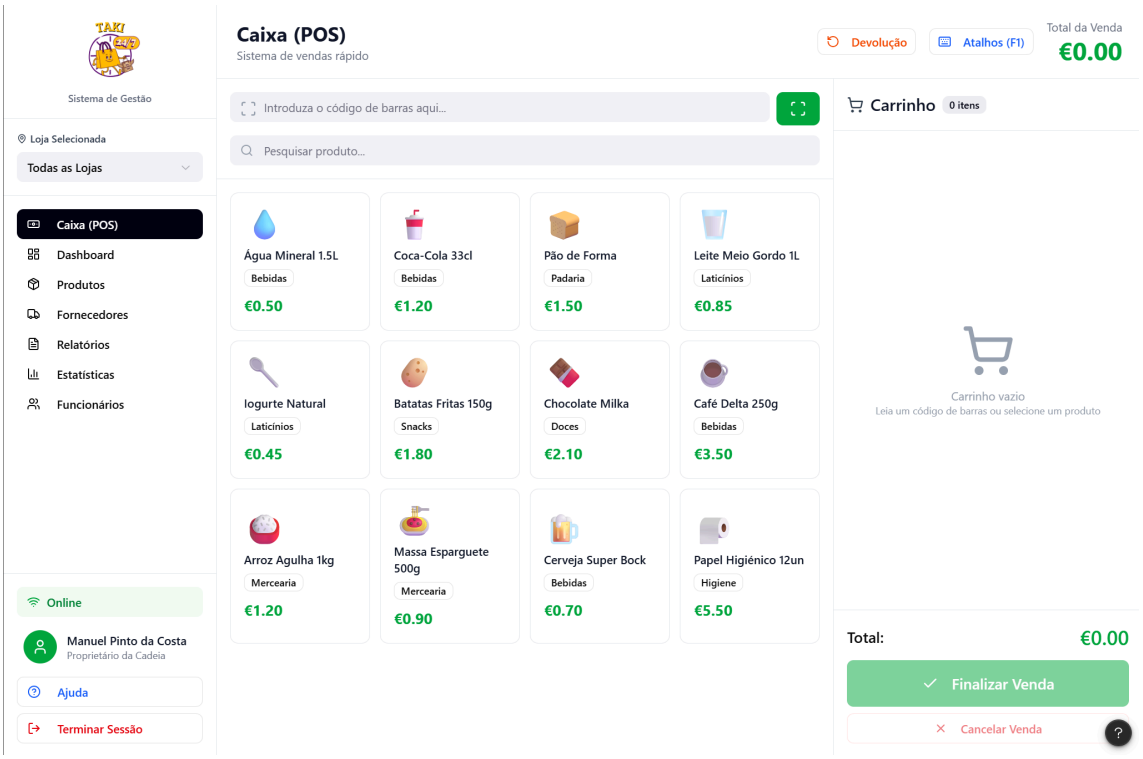


Figura 105: Aba da Caixa de Vendas (POS)

O ecrã do Ponto de Venda (POS) representa o núcleo operacional da Taki, desenhado para maximizar a velocidade de atendimento ao cliente num ambiente de retalho acelerado. A área principal da interface exibe todos os artigos disponíveis para comercialização e os respetivos preços. Esta disposição aplica diretamente a heurística de Reconhecimento em vez de Memorização, isentando o operador de caixa de decorar listas de códigos numéricos.

No topo da interface, o sistema disponibiliza uma barra dedicada à leitura de códigos de barras e, imediatamente abaixo, uma barra de pesquisa manual de produtos. Esta dupla abordagem garante a Flexibilidade e Eficiência de Utilização, permitindo que as transações ocorram numa questão

de segundos através de um leitor ótico, mas oferecendo um mecanismo de busca de contingência caso a etiqueta de um produto esteja danificada.

O terço direito do ecrã é inteiramente dedicado ao estado da transação atual. No topo direito, o valor total da venda é exibido de forma proeminente, assegurando a `Visibilidade do Estado do Sistema em tempo real`. Abaixo do total, encontra-se a listagem dinâmica do carrinho de compras.

Na base do painel lateral direito situam-se as ações de fecho: os botões **"Finalizar Venda"** e **"Cancelar Venda"**. A inclusão de um botão de cancelamento imediato e perfeitamente visível reflete o princípio de `Controlo e Liberdade do Utilizador`, fornecendo ao operador uma "saída de emergência" rápida num único clique, caso o cliente desista da compra ou ocorra um erro no registo dos artigos, sem necessidade de navegar por menus complexos.

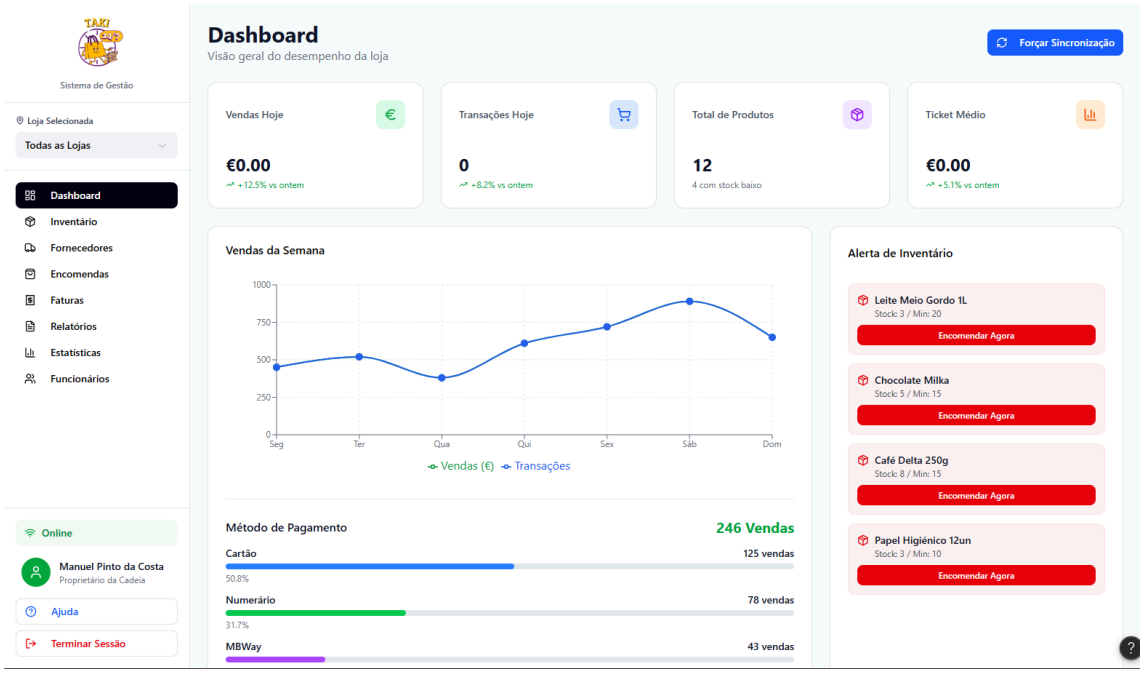


Figura 106: DashBoard da plataforma

O *Dashboard* funciona como a sala de controlo da plataforma. Este ecrã aplica diretamente a heurística de `Visibilidade do Estado do Sistema`, fornecendo à gerência uma visão global e em tempo real sobre a operação da cadeia de lojas. Apresenta os principais Indicadores-Chave de Desempenho (KPIs) de forma resumida, permitindo ao Gestor e os cargos acima deste detetar imediatamente anomalias no volume de vendas diário ou produtos que necessitem de atenção urgente, sem ter de navegar por menus complexos.

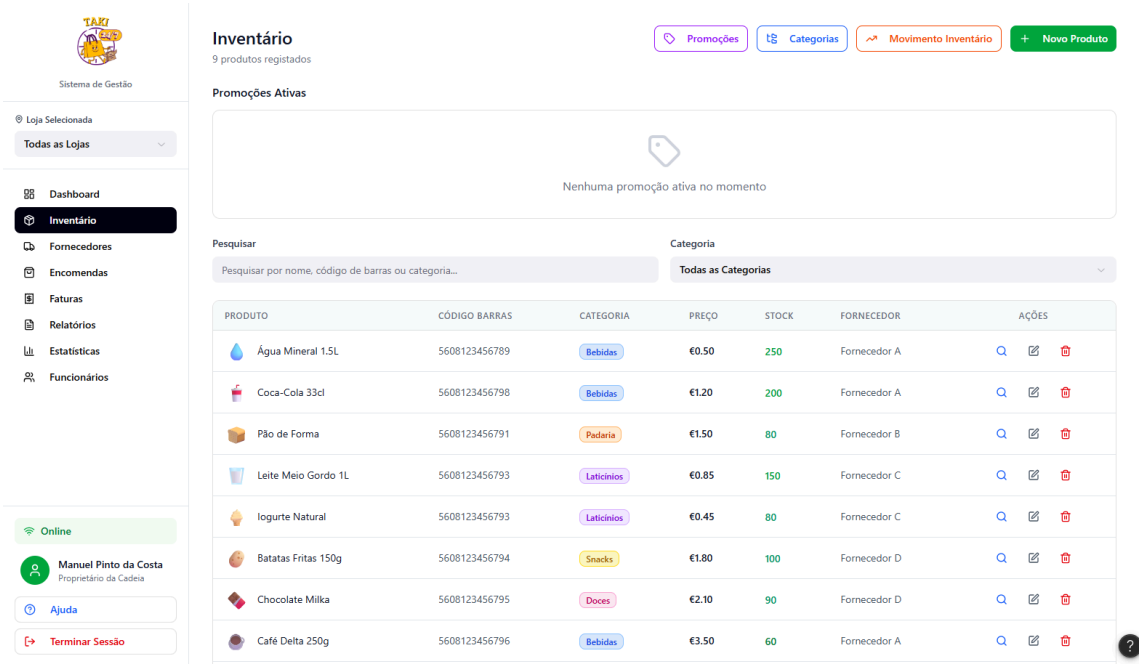


Figura 107: Aba da Gestão do Inventário da empresa

Este ecrã oferece uma visão tabular detalhada de todo o catálogo de artigos comercializados na Taki. A listagem contém métricas cruciais de cada produto, como a categoria, o preço e a quantidade de *stock* disponível. A estrutura em tabela permite uma leitura rápida e a identificação imediata de necessidades de reposição, garantindo que a informação visualizada mapeia perfeitamente o ambiente físico do retalho.

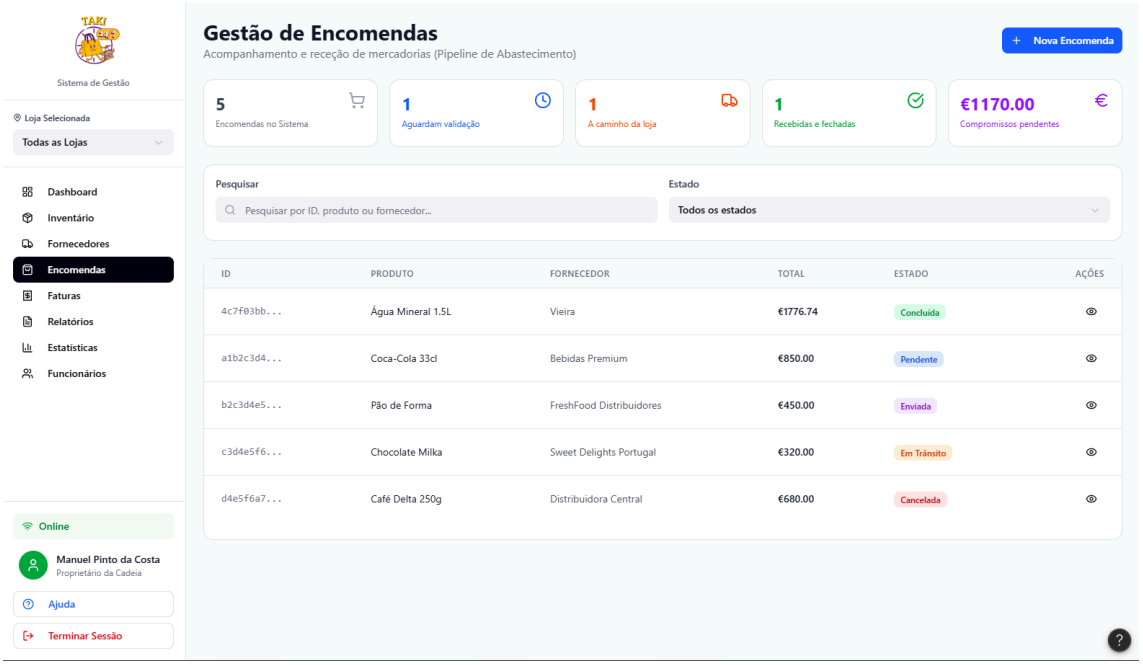


Figura 108: Aba da Gestão de Encomendas da Empresa

Este ecrã constitui o centro da pipeline de abastecimento da Taki, proporcionando uma monitorização em tempo real de todo o fluxo de mercadorias, desde a intenção de compra até à receção física em loja. Através de um conjunto de indicadores de desempenho no topo, o utilizador obtém uma radiografia instantânea das operações, identificando encomendas que aguardam validação, remessas em trânsito e o volume total de compromissos financeiros pendentes com fornecedores.

A interface tabular detalha cada transação, associando artigos específicos aos seus respetivos fornecedores e estados logísticos. A utilização de sinalética visual cromática (etiquetas de estado) permite distinguir

rapidamente entre pedidos pendentes, enviados ou concluídos, facilitando a gestão de prioridades. Esta estrutura organizada garante uma coordenação precisa entre as necessidades de reposição detetadas no inventário e a execução efetiva das compras, assegurando a continuidade operacional e a eficiência da cadeia de abastecimento.

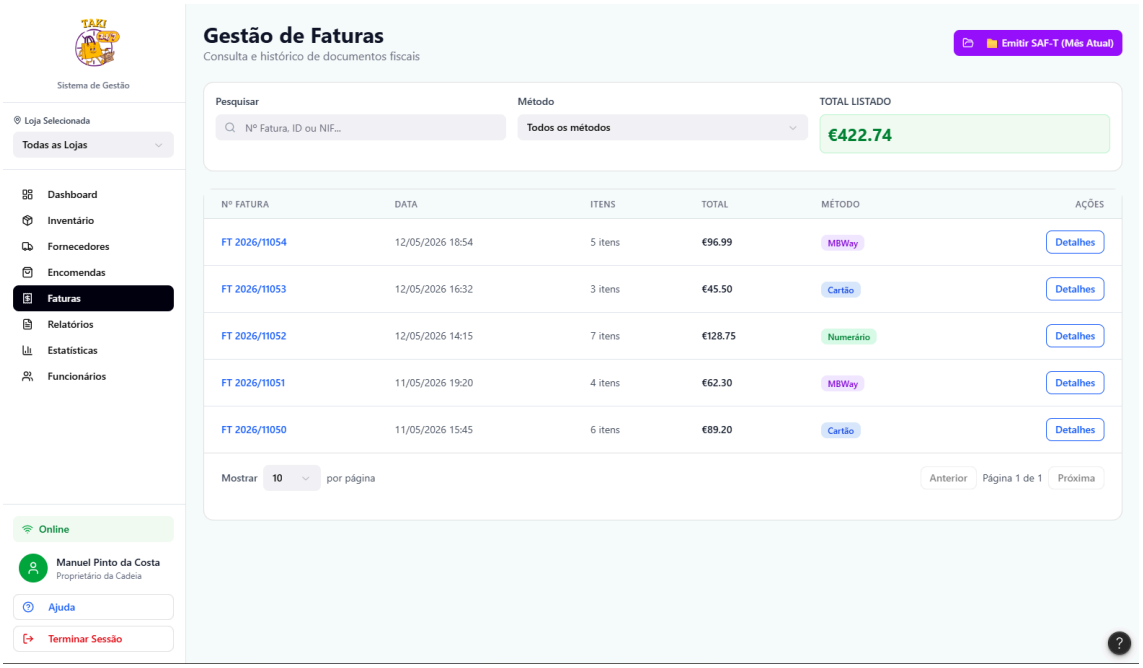


Figura 109: Aba da gestão de Faturas da Empresa

Este ecrã centraliza o arquivo digital de faturas da Taki para a consulta e histórico de documentos fiscais. Através de uma interface intuitiva, o utilizador pode filtrar transações por número de fatura, NIF ou método de pagamento, obtendo uma visão clara dos fluxos financeiros gerados em loja. O destaque para o montante total listado permite um controlo imediato da faturação no período selecionado, assegurando a transparência e a conformidade legal necessária ao ambiente de retalho.

A listagem tabular organiza cada documento com métricas precisas, incluindo a data e hora exata da transação, o volume de itens por venda e o valor total faturado. A identificação visual dos métodos de pagamento (como MBWay, Cartão ou Numerário) e o acesso direto aos detalhes de cada fatura facilitam processos de devolução ou esclarecimento de dúvidas. Adicionalmente, a presença de uma ferramenta dedicada à emissão do ficheiro SAF-T (PT) garante que a exportação de dados para a Autoridade Tributária seja realizada de forma rápida e segura, consolidando a gestão documental da empresa. Também é possível (através do botão Detalhes na coluna Ações), exportar uma fatura pretendida em PDF garantindo que clientes e Administradores possam obter as faturas.

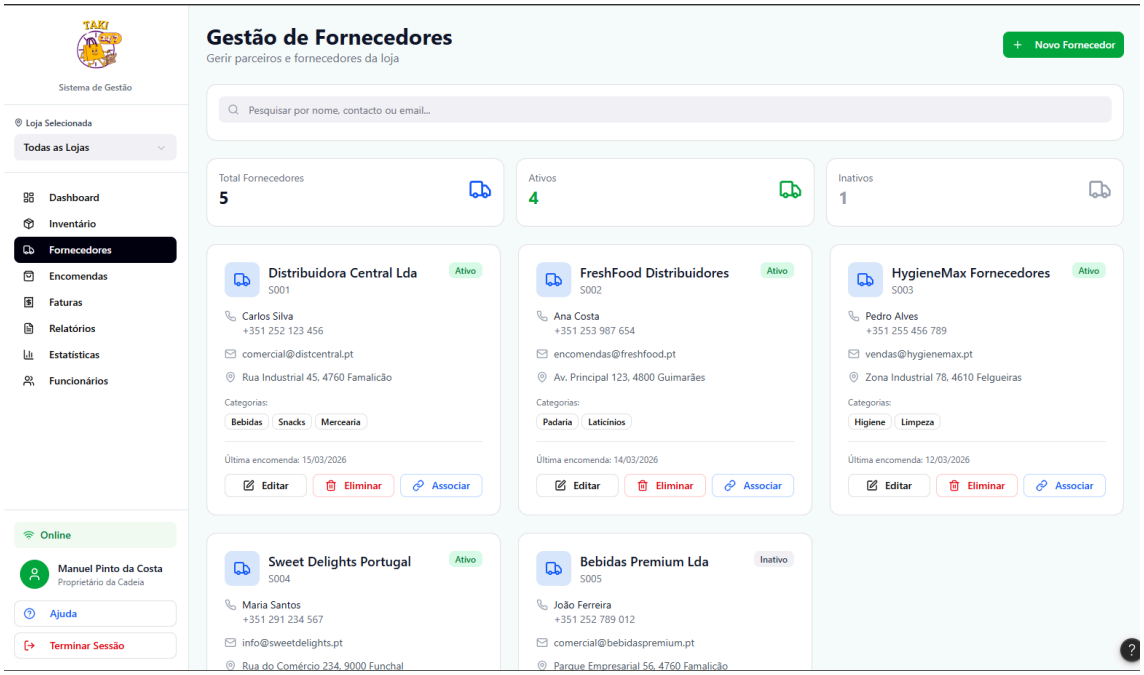


Figura 110: Aba da Gestão dos Fornecedores da empresa

Dedicado à gestão da cadeia de abastecimento, este ecrã centraliza a lista de todos os parceiros comerciais da organização. A tabela exibe os dados vitais de contacto e faturação (Nome, NIF, E-mail, Telefone) e as categorias que fornecem à organização. A interface garante que a equipa consegue facilmente auditar, adicionar ou inativar entidades fornecedoras, preparando a informação necessária para a futura emissão de notas de encomenda de forma ágil e estruturada.

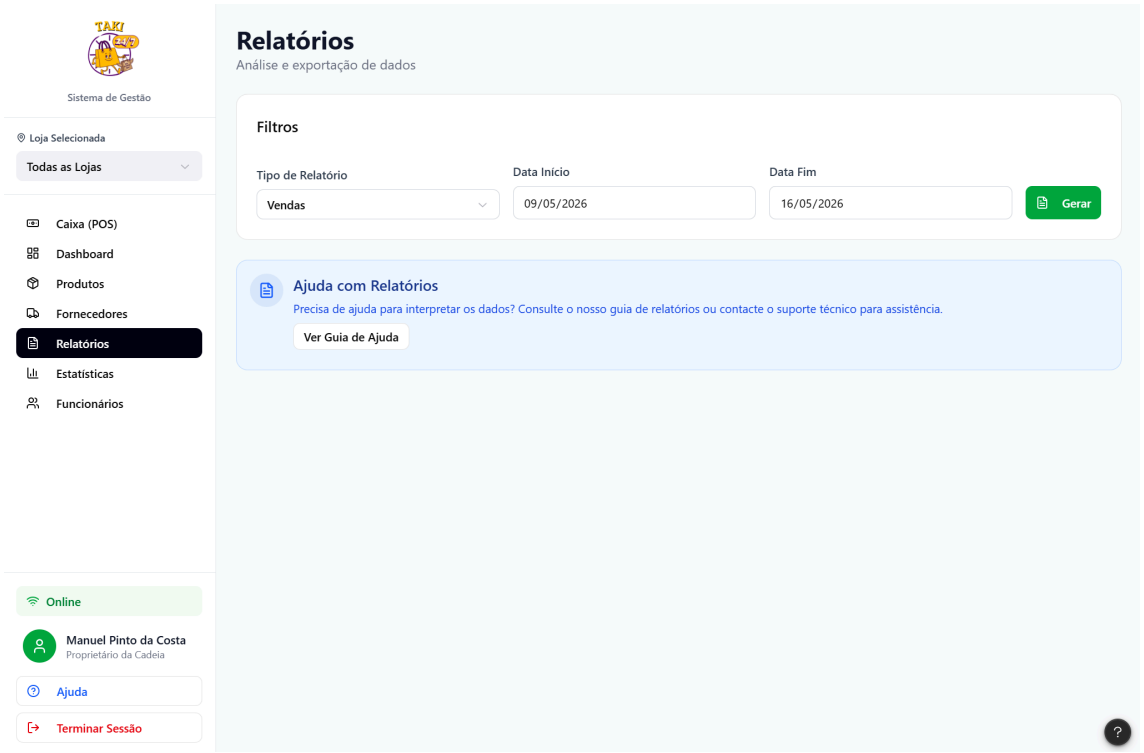


Figura 111: Aba da Geração de Relatórios da empresa

Esta secção é fundamental para a conformidade contabilística e fiscal da `Taki`. O ecrã permite à gerência extrair listagens estruturadas e detalhadas com base em filtros temporais e categóricos. A interface foi

desenhada para facilitar a exportação de dados (como listagens de faturação e valorização de *stock*), minimizando o risco de erros na comunicação de informações aos departamentos financeiros.

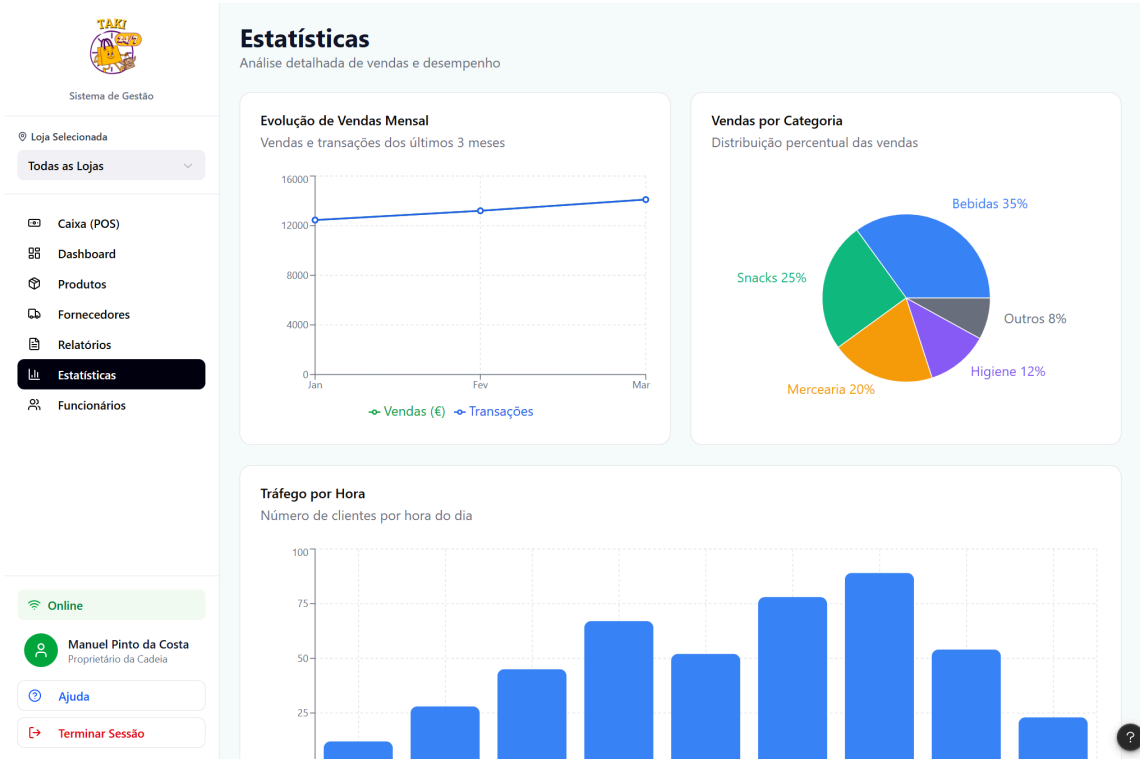


Figura 112: Aba da Estatísticas da empresa

A secção de Estatísticas traduz os dados brutos em gráficos e representações visuais. Ao aplicar a heurística de `Reconhecimento em vez de Memorização`, o sistema liberta o Gestor de analisar longas tabelas numéricas, permitindo-lhe reconhecer tendências de vendas, picos de afluência e desempenho por categorias num relance visual rápido, o que acelera drasticamente a tomada de decisão estratégica.

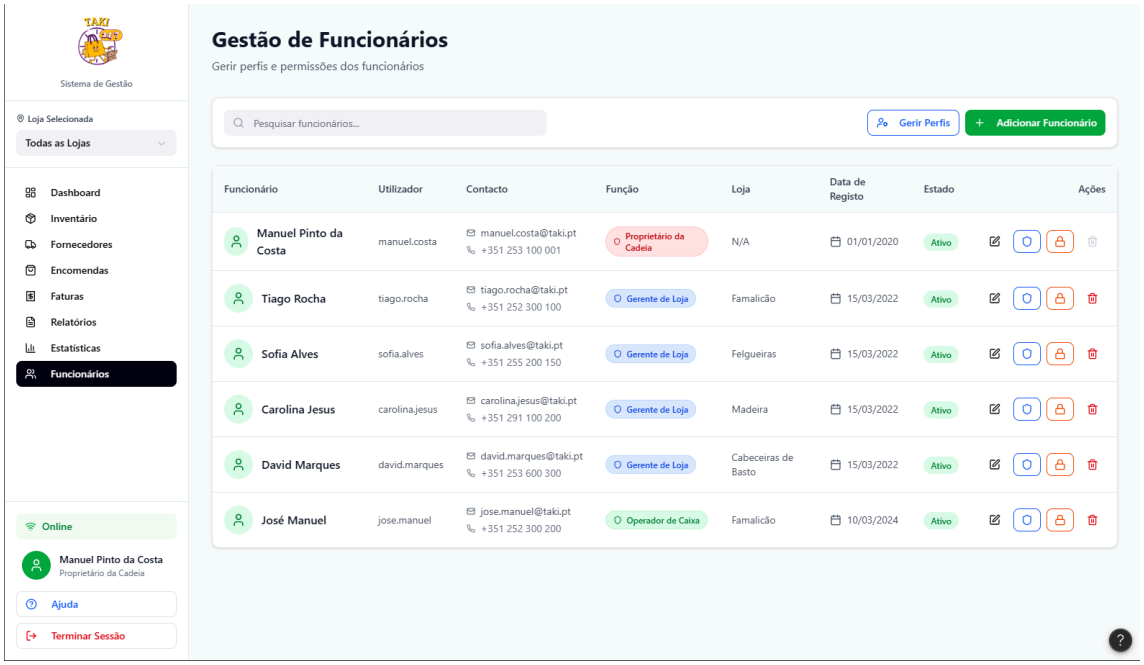


Figura 113: Aba da Gestão de Funcionários da empresa

Este ecrã lista a equipa de colaboradores ativos no sistema, identificando os seus dados, a loja a que estão afetos e o seu respetivo cargo (perfil de acesso). É através desta interface que a administração gere o ciclo de vida das contas, adicionando novos operadores, editando permissões ou bloqueando acessos. A disposição clara dos botões de ação garante o Controlo e Liberdade do Utilizador ao administrar elementos sensíveis da segurança da plataforma.

3.4. Utilização de Inteligência Artificial

Nesta fase crucial do projeto, focada na definição da arquitetura e do *design* do *software*, a equipa efetuou ajustes à sua abordagem relativamente à utilização de Inteligência Artificial. Inicialmente, o projeto recorria ao modelo *Gemini* 3.1 *Pro*, integrado através do *Gemini CLI*. Contudo, a equipa deparou-se com um período de instabilidade acentuada desta plataforma que se prolongou por cerca de 5 semanas. Durante este período, os tempos de resposta (*prompts*) atingiram demoras de várias horas, o que tornou a utilização do *Gemini CLI* inviável e comprometeu severamente a fluidez do processo de desenho arquitetural. Face a este bloqueio, a equipa viu-se obrigada a migrar as suas operações para a plataforma *Copilot*, baseada nos modelos da família Claude.

A transição iniciou-se com a adoção do modelo **Claude Opus** $\mathbf{4.5}$, que demonstrou uma capacidade analítica superior na resolução de problemas complexos de concorrência e de estruturas de dados. Posteriormente, já na fase final de consolidação desta secção, o modelo **Claude Opus** $\mathbf{4.5}$ deixou de estar abrangido pela nossa licença académica da plataforma *Copilot*. Como consequência, a equipa operou uma nova transição e adotou o modelo **Claude Sonnet** $\mathbf{4.6}$. Este modelo preservou um nível de qualidade e de raciocínio lógico perfeitamente alinhado com as exigências do projeto.

3.4.1. Diretrizes de Sistema e a Ativação das Regras de Andrej Karpathy

Com o objetivo de maximizar o valor acrescentado da utilização destes modelos e de mitigar o risco de soluções sobre-projetadas (*over-engineering*) ou de código especulativo, a equipa recorreu à instalação e ativação direta das **Competências de Andrej Karpathy** (*Andrej Karpathy Skills*) no ambiente da plataforma. Em vez de uma mera inspiração metodológica ou técnica manual de *prompting*, tratou-se da execução de um comando específico que integrou estas regras estritas no *Copilot*, forçando a IA a operar sob a seguinte matriz de comportamento:

1. **Exposição Antecipada de Assunções:** Obrigou o modelo a declarar explicitamente todas as suas premissas de raciocínio antes de gerar qualquer proposta arquitetural ou de código;
2. **Prioridade Absoluta à Simplicidade:** Condicionou a IA a rejeitar abstrações complexas desnecessárias, privilegiando sempre a solução funcional mais direta;
3. **Modificações Cirúrgicas:** Restringiu a atuação da IA rigorosamente ao alvo do pedido, bloqueando liminarmente qualquer formatação ou alteração colateral em lógicas não relacionadas;
4. **Verificação e Orientação a Objetivos:** Determinou que qualquer solução proposta fosse obrigatoriamente acompanhada por critérios claros de sucesso e verificação empírica.

Em simultâneo com a integração destas restrições técnicas e estruturais de *software*, a equipa configurou o comportamento linguístico e de negócio da IA através de um ficheiro próprio, denominado `copilot_instruction.md`. Este documento da nossa autoria definiu as regras intrínsecas ao projeto (como a obrigatoriedade do tom académico, a proibição absoluta da utilização de gerúndios e o contexto do domínio da Taki). O conteúdo integral destas instruções encontra-se disponível na secção de Anexos do relatório (ver Anexo 8).

3.4.2. Interações e *Prompts* Chave na Arquitetura

O processo de modelação arquitetural exigiu instruções explícitas e rigorosas para garantir que os artefactos gerados (decisões de base de dados, diagramas de Lógica de Negócio e avaliações de usabilidade) respeitavam a visão inicial da equipa e o escopo académico. Apresentam-se em seguida exemplos dos *prompts* fundamentais aplicados a cada subcamada do sistema.

Camada de Dados (*Data Layer* - BD)

Para o desenho físico da base de dados e gestão da concorrência multiutilizador, a equipa orientou o modelo para o aproveitamento de mecanismos nativos do SGBD, com a delegação da responsabilidade de implementar soluções de integridade não bloqueantes. > "Atua como o Engenheiro de Base de Dados Principal. Com base no nosso Modelo de Domínio, elabora a estratégia de implementação das tabelas de Inventário e Vendas no PostgreSQL. Precisamos de garantir a proteção das quantidades em *stock* perante compras simultâneas por múltiplos clientes, mas sem bloquear a aplicação globalmente. Em vez de *locks* na camada de Java, propõe uma solução arquitetural que tire partido de Triggers do SGBD e do mecanismo MVCC do PostgreSQL. O teu *output* deve explicar a decisão arquitetural com foco na latência mínima."

Adicionalmente, para assegurar a inviolabilidade e rastreabilidade das transações financeiras, procurou-se a melhor estratégia de estruturação das tabelas de registo: > "No âmbito do registo de faturas, o sistema necessita de implementar uma cadeia de imutabilidade (semelhante ao exigido pelo SAF-T). Sugere a arquitetura da tabela no PostgreSQL e os tipos de dados mais adequados para armazenar assinaturas criptográficas (Hashes RSA) de forma a que cada registo dependa inequivocamente do anterior. Foca a resposta na eficiência de leitura estrutural."

Lógica de Negócio (LN)

Na estruturação do núcleo da aplicação, as interações deram primazia à separação de responsabilidades e à implementação formal de Padrões de Desenho da Engenharia de *Software*. > "Aplica as tuas diretrizes de sistema de simplicidade e clareza ('Karpathy Skills'). Para o subsistema de Fornecimentos, desenha a arquitetura de transição de estados do ciclo de vida de uma encomenda (Pendente, Em Processamento, Entregue, Cancelada). Sugere a melhor abordagem arquitetural para implementar o padrão *State* em Java, com o objetivo de assegurar que as regras de negócio de bloqueio de edição nos estados selados fiquem inteiramente isoladas das entidades de domínio simples. Descreve o fluxo pretendido sem escrever implementações profundas de código."

O planeamento do Catálogo e Gestão de Lojas exigiu particular atenção à otimização da leitura de dados em árvore, com o intuito de prevenir constrangimentos de *performance* conhecidos: > "No domínio do Catálogo, possuímos uma hierarquia profunda de Categorias de Produtos. Se iterarmos individualmente sobre cada categoria na base de dados para procurar os seus descendentes, incorremos no problema estrutural de 'N+1 queries'. Apresenta uma estratégia arquitetural em Java que efetue uma única leitura plana (um único comando *SELECT*) e construa de forma recursiva a árvore hierárquica em memória. Especifica as estruturas de dados recomendadas."

No que concerne ao Ponto de Venda (POS), a equipa recorreu à IA para delinear um acoplamento fraco entre o ato de transação e as operações logísticas inerentes: > "Pretendemos que, ao finalizar e validar uma venda no POS, o sistema notifique o módulo de Inventário para desencadear os procedimentos de abate. Contudo, exigimos um baixo acoplamento entre estes dois domínios na arquitetura Java. Formula uma solução baseada no padrão de desenho *Observer*, onde o módulo de Vendas atua como Publicador e o Inventário como Subscritor. Elabora a assinatura da *interface* abstrata e descreve a injeção da dependência sem detalhar o código de acesso a dados."

***User Interface* (UI)**

No que diz respeito à camada de apresentação, os pedidos dirigidos à IA tiveram como propósito validar a ergonomia dos esboços visuais criados pela equipa e certificar o alinhamento do protótipo com os padrões da indústria. > "Comporta-te como um Especialista Sénior de UI/UX. Analisa a estrutura e disposição conceptual do nosso ecrã de Ponto de Venda (POS). Redige uma avaliação crítica desta arquitetura visual de acordo com as Heurísticas de Nielsen. A resposta deve ser elaborada em formatação de texto *Typst* e deve apresentar uma lista das heurísticas respeitadas pelo nosso *design* e daquelas onde este apresenta falhas de usabilidade. Deves concluir a tua avaliação com propostas estruturais concretas para melhorar a visibilidade do estado do sistema durante o ato de pagamento."

Através da convergência entre a notável capacidade analítica dos modelos da família Claude e a disciplina incutida pelas diretrizes de Andrej Karpathy, a equipa conseguiu escalar as suas capacidades de tomada de decisão. Esta combinação metodológica permitiu antever potenciais bloqueios e afinar atempadamente as fundações de *software* do projeto.

3.4.3. Prompts da LN

No desenvolvimento do sistema Taki, a Inteligência Artificial foi integrada como um recurso estratégico para validar decisões arquiteturais e garantir a adoção de melhores práticas de engenharia de software desde a fase de desenho. A utilização de modelos de linguagem permitiu explorar diferentes padrões de desenho e estruturas de serviços, servindo como um mecanismo de revisão consultivo que auxiliou na definição da lógica de negócio e na separação de responsabilidades entre as camadas do sistema.

Lógica de Negócio (LN):

- Desenha a lógica de validação de tokens JWT para o controlo de acesso baseado em papéis (RBAC). Como deve a camada de negócio processar a distinção entre um 'Gestor de Loja' e um 'Administrador Global' ao aceder a endpoints de exportação de dados SAF-T, garantindo que as políticas de segurança são aplicadas antes da execução de qualquer query?
- Como estruturar a lógica de cálculo de promoções acumuladas (ex: descontos de categoria + descontos sazonais) na camada de negócio, garantindo que o motor de regras seja extensível para novos tipos de campanhas sem violar o Princípio de Aberto/Fechado (Open/Closed Principle)?
- Explica como o padrão Service pode ser estruturado no backend para gerir a lógica de atualização de stock de forma independente da API. Como garantir que a reserva de stock seja tratada como uma operação atómica na camada de negócio para evitar inconsistências em vendas simultâneas?

4. Implementação e Desenvolvimento Assistido por LLM

Este capítulo tem como objetivo descrever como as decisões arquiteturais e os requisitos definidos nas fases anteriores foram concretizados em código funcional, pronto a ser executado no ambiente alvo. Neste contexto, são apresentadas as tecnologias, linguagens, *frameworks* e bibliotecas adotadas, bem como a organização dos módulos, componentes e interfaces que compõem a solução desenvolvida. Abordam-se ainda as convenções adotadas para garantir a legibilidade e a consistência do código e os mecanismos de gestão de dependências e de configuração do ambiente.

4.1. Configuração do Ambiente de Trabalho

A materialização do Sistema de Gestão da Taki exigiu a definição de um ambiente de desenvolvimento reprodutível, isolado e consistente entre todos os membros da equipa. Para alcançar este objectivo, a equipa adoptou o **Docker** como tecnologia de contenção e orquestração de serviços, eliminando a dependência de configurações manuais e garantindo que cada elemento da equipa operou sobre uma infraestrutura tecnicamente idêntica.

4.1.1. Orquestração com Docker Compose

A gestão dos múltiplos *containers* foi centralizada num ficheiro `docker-compose.yml`, que declara e orquestra os serviços necessários ao ciclo de desenvolvimento e teste. A topologia adoptada replica fielmente a arquitectura distribuída alvo: arrancam cinco instâncias de PostgreSQL — uma representativa da **base de dados central** (`db_central`) e quatro representativas das **bases de dados locais das filiais** (`db_local_1`, `db_local_2`, `db_local_3` e `db_local_4`), correspondentes às lojas de Famalicão, Felgueiras, Madeira e Cabeceiras. Esta decisão permite simular e testar integralmente o paradigma de sincronização central/local e validar a complexidade da rede de replicação desde o ambiente de desenvolvimento, garantindo que eventuais problemas de concorrência ou consistência sejam detetados precocemente.

Cada *container* de base de dados é inicializado de forma automática com os *scripts* SQL da solução, montados como volumes por ordem de execução numérica explícita: em primeiro lugar o `1-schema.sql` (DDL das tabelas e relações), seguido do `2-logica_ativa.sql` (*triggers* e funções de domínio), do `3-sincronizacao.sql` (mecanismo de *outbox* e replicação), do `4-roles.sql` (controlo de acesso baseado em papéis) e, por fim, do `5-indices.sql` (otimização de *performance* através de índices compostos e parciais).

4.1.2. Isolamento e Replicabilidade do Ambiente

A adopção desta abordagem de contenção traz vantagens directas para a qualidade do desenvolvimento. O ambiente é **perfeitamente replicável**: qualquer membro da equipa executa um único comando (`docker compose up`) para obter, em segundos, uma rede de bases de dados operacional e idêntica à dos restantes. Esta característica eliminou a classe de erros do tipo *"funciona na minha máquina"*, acelerou a integração de novos membros e simplificou a execução da bateria de testes de integração, que exige a disponibilidade simultânea das cinco instâncias de base de dados.

4.1.3. Configuração da Aplicação por Variáveis de Ambiente

A aplicação Java não embute quaisquer credenciais ou *strings* de ligação no código-fonte. Toda a configuração de infraestrutura é injectada no arranque através de variáveis de ambiente, definidas em ficheiros `.env` geridos localmente por cada programador e excluídos do controlo de versões via `.gitignore`. A variável `TAKI_DB_URL` aponta para a instância de base de dados adequada ao contexto de execução — local ou central —, enquanto `TAKI_DB_USER` e `TAKI_DB_PASSWORD` definem as credenciais de acesso (em desenvolvimento, `admin` e `password123`; em produção, credenciais geridas por *secrets management*). Esta decisão arquitectural garante que o mesmo artefacto binário compilado opera, sem modificações, em qualquer dos ambientes previstos: desenvolvimento local, testes de integração e produção em *cloud*.

4.1.4. Ferramentas e Controlo de Versões

O código-fonte do projecto foi gerido através do **Git**, com um repositório centralizado que segrega os módulos da aplicação (`/src`), os *scripts* de base de dados (`/database`) e os ficheiros de infraestrutura na raiz do projecto. Cada membro da equipa utilizou o ambiente de desenvolvimento da sua preferência, com suporte para projectos **Gradle**, ferramenta utilizada para a gestão de dependências e o ciclo de *build* da aplicação. A imagem base adoptada no Docker — `gradle:8.6-jdk21` — confirma a utilização de **Java 21 LTS** como plataforma de execução. As dependências críticas do projecto — nomeadamente o *driver* JDBC do PostgreSQL, a biblioteca de testes **JUnit 5** e o *framework* **Javalin** para a camada de API REST — são declaradas no ficheiro `build.gradle` e resolvidas pelo Gradle em todos os ambientes de desenvolvimento.

4.2. *Data Layer*

Nesta secção documenta-se a concretização física do modelo relacional desenhado nos capítulos anteriores. O processo de implementação da base de dados *PostgreSQL* engloba desde a definição da infraestrutura de tabelas e permissões, até à otimização do armazenamento, indexação e codificação de rotinas ativas (*triggers*). Documenta-se igualmente o mecanismo fulcral de consolidação de dados distribuídos.

4.2.1. Criação do Esquema Físico

Após a conclusão do desenho lógico, a equipa de desenvolvimento iniciou a implementação física do *Data Layer*. O processo consistiu na tradução do esquema relacional para a linguagem de definição de dados específica do SGBD *PostgreSQL*.

O *script* final (`schema_postgres.sql`) foi estruturado em três fases operacionais: a configuração da sessão, a criação do esquema de isolamento e a instanciação das tabelas com as respetivas restrições de integridade.

Configuração Inicial do Ambiente No *PostgreSQL*, ao contrário de outros SGBDs como o *MySQL*, a gestão de verificação de chaves é robusta e não necessita de ser desativada temporariamente. Nesta implementação, os `UUID` são gerados na camada aplicacional (Java) e passados explicitamente para as instruções `INSERT`.

Criação do Esquema (*Schema*) De forma a isolar o ecossistema da 'Taki' de outras bases de dados que possam coexistir no mesmo servidor, os objetos lógicos da solução foram desenhados para execução num contexto dedicado. Na versão final entregue, o `schema_postgres.sql` cria as tabelas no `public`, mantendo compatibilidade direta com o arranque dos ambientes Docker e com os testes de integração.

```
CREATE SCHEMA IF NOT EXISTS taki;
```

Criação de Tabelas e Restrições A fase final do *script* DDL consiste na criação das tabelas de negócio e suporte operacional necessárias aos requisitos. A sintaxe `CREATE TABLE` define não só as colunas e os seus

tipos de dados físicos (mapeados na secção 3.1.3), mas também as restrições de integridade que protegem a lógica de negócio.

A tabela `Venda` serve como um excelente exemplo da aplicação das várias decisões arquiteturais:

- O atributo `id_venda` é definido como Chave Primária do tipo `UUID`, garantindo a sua unicidade na rede;
- O campo `data_hora` é obrigatório (`NOT NULL`) e registado pela aplicação no momento de conclusão da venda;
- As colunas `id_loja` e `id_funcionario` são marcadas como Chaves Estrangeiras (`FOREIGN KEY`), criando uma ligação referencial explícita às respetivas tabelas-mãe.

```
CREATE TABLE Venda (
    id_venda UUID PRIMARY KEY,
    data_hora TIMESTAMP NOT NULL,
    subtotal NUMERIC(10,2) NOT NULL,
    imposto NUMERIC(10,2) NOT NULL,
    total NUMERIC(10,2) NOT NULL,
    estado VARCHAR(15) NOT NULL,
    id_loja INT NOT NULL,
    id_funcionario UUID NOT NULL,
    CONSTRAINT fk_venda_loja FOREIGN KEY (id_loja) REFERENCES Loja(id_loja),
    CONSTRAINT fk_venda_func FOREIGN KEY (id_funcionario) REFERENCES
Funcionario(id_funcionario)
);
```

Esta metodologia foi aplicada de forma transversal a todas as tabelas do sistema, resultando num esquema físico robusto e fiel ao modelo lógico.

Na revisão final da implementação, foram removidas estruturas fora de escopo (monitorização e auditoria genérica), mantendo apenas objetos diretamente ligados a RF/RNF e casos de uso.

4.2.2. Gestão de Acessos e Utilizadores

A gestão de acessos na rede `Taki` é implementada com uma arquitetura de **Defesa em Profundidade**, onde a segurança é garantida em duas camadas distintas e complementares: a Base de Dados e a Aplicação.

Perfis na Base de Dados (Papéis Técnicos)

Ao nível do motor PostgreSQL, não são criados utilizadores individuais para cada funcionário. Em vez disso, a estratégia consiste em criar um número muito reduzido de **perfis técnicos** com âmbitos bem definidos. Os funcionários humanos nunca se ligam diretamente à base de dados.

Foram implementados os três perfis seguintes, cujo código de criação se encontra no ficheiro `user_roles.sql`. As palavras-passe no referido *script* são representadas pelo seu *hash* seguro (SHA-512) para fins de documentação, demonstrando que estas nunca são armazenadas em texto legível.

1. **`taki_app_user` (Perfil da Aplicação Local):** Este é o único utilizador que a aplicação Java de cada loja usa para comunicar com a sua base de dados local. As suas permissões são de `SELECT`, `INSERT` e `UPDATE`, sendo-lhe expressamente revogada a permissão de `DELETE` para forçar o uso de *Soft Deletes* (alteração do campo `estado`), como definido nas Decisões Arquiteturais;
2. **`taki_admin` (Super-Administrador Central):** Um super-utilizador com privilégios totais (`ALL PRIVILEGES`) sobre a base de dados central. A sua utilização é restrita a tarefas de desenvolvimento, manutenção e execução de *scripts* de sincronização;
3. **`taki_analytics` (Perfil de Leitura Central):** Um utilizador de acesso exclusivo à base de dados central, destinado às ferramentas de *Business Intelligence* e *Dashboards* da Direção. Possui apenas permissões de `SELECT`, garantindo que a análise de dados agregados nunca pode, por acidente, corromper ou eliminar dados históricos.

Perfis na Aplicação (Papéis de Negócio)

A diferenciação entre o que um `operador_caixa`, um `gestor_armazem` e um `gestor_loja` podem fazer é gerida inteiramente pela **Lógica de Negócio na aplicação Java**.

Após um funcionário efetuar o *login* na interface web, a aplicação consulta a tabela `Funcionario` para verificar o valor do seu atributo `cargo`. Com base nesse valor, o código Java decide dinamicamente se deve renderizar um botão, permitir o acesso a um determinado menu ou autorizar a execução de uma certa operação. Este modelo simplifica a gestão da base de dados, centraliza as regras de negócio no *backend* e aumenta a segurança ao abstrair os utilizadores finais do contacto direto com o SGBD.

Na revisão final, o controlo de acessos na camada de negócio passou a incluir explicitamente o ciclo de gestão de perfis exigido pelos casos de uso: criação de perfil, edição de permissões e atribuição de perfil a utilizador existente. Estas operações são aplicadas no `PerfilAcessoService` e no `FuncionarioService`, mantendo o mapeamento para o campo `cargo` em `Funcionario`.

Adicionalmente, tentativas de operações administrativas sem permissão válida são bloqueadas e registadas na camada de negócio (*logging* de segurança), reforçando a rastreabilidade exigida pelos requisitos de controlo de acesso.

4.2.3. Povoamento da Base de Dados

A validação de um *Data Layer* robusto exige a sua submissão a um volume de dados representativo da carga operacional esperada em ambiente de produção. A inserção manual de milhares de registos é impraticável, pelo que a equipa desenvolveu um *script* de povoamento automatizado em **Python**, denominado `populate.py`, cujo código integral se encontra disponível nos anexos do relatório.

Estratégia de Geração de Dados O *script* utiliza a biblioteca **Faker** (com a localização `pt_PT`) para gerar dados sintéticos com um elevado grau de realismo, desde nomes e moradas portuguesas a números de identificação fiscal válidos.

A geração dos dados respeita a arquitetura distribuída do sistema 'Taki', simulando o estado da rede num determinado ponto no tempo através de uma sequência de 4 passos lógicos:

1. **Geração de Dados-Mestres:** São criados os registos que têm uma génese central e única, nomeadamente as quatro Lojas e as contas de utilizador dos Gestores e do Dono da empresa, com base nos dados recolhidos na fase de análise;
2. **Geração de Catálogos Comuns:** O *script* gera um catálogo base com 100 produtos, 10 categorias e 15 fornecedores. Estes dados são considerados pré-sincronizados e são carregados de forma idêntica para a base de dados central e para cada uma das quatro bases de dados locais;
3. **Geração de Dados Transacionais Locais:** Para cada loja, o *script* simula de forma isolada a sua atividade diária, gerando milhares de `Vendas`, `Pagamentos`, `Faturas` e `Movimentos de Inventário`. Todas estas transações são corretamente associadas ao `id_loja` e aos funcionários da respetiva filial;
4. **Geração de Dados Consolidados:** Por fim, o *script* agrega todos os dados transacionais gerados para as lojas locais num único conjunto de ficheiros, que representa o estado final da base de dados central após a sincronização noturna.

Formatos de Saída e Carregamento O *script* `populate.py` adapta-se ao nó de destino (central ou loja local específica). Em vez de exportar ficheiros `.csv` separados, o *script* compila diretamente um ficheiro `SQL` completo (ex.: `populate_central.sql` ou `populate_loja_1.sql`) guardado na diretoria `population_output/sql`. Este ficheiro contém todas as instruções genéricas e locais num único fluxo de inserções (`INSERT INTO`), simplificando imenso a execução e a depuração.

Para agilizar o processo e evitar intervenção manual, a equipa disponibilizou um *script* auxiliar em *bash* (`load_data.sh`). Este utilitário recebe o alvo pretendido (ex: `./load_data.sh 1` ou `./load_data.sh central`), executa o Python internamente se o *dataset* não estiver pré-gerado, limpa previamente a base de dados de destino com `TRUNCATE CASCADE`, e invoca o utilitário `psql` injetando diretamente o *script* `SQL` gerado no *container* Docker correto, simplificando drasticamente a preparação de qualquer ambiente de testes.

4.2.4. Cálculo do Espaço e Crescimento

O planeamento da capacidade de armazenamento é uma disciplina crucial na arquitetura de bases de dados, visando garantir a escalabilidade e prevenir custos inesperados ou falhas de serviço por exaustão de disco. Para o sistema Taki, esta análise é particularmente importante devido à natureza transacional do negócio, que implica um crescimento constante e rápido do volume de dados.

Nesta secção, detalha-se o cálculo do espaço em disco necessário, dividindo a análise em duas partes: a **dimensão inicial** mínima para a estrutura da base de dados e a **taxa de crescimento anual** projetada.

Dimensão Inicial

A dimensão inicial representa o espaço mínimo necessário para armazenar o "esqueleto" da base de dados, ou seja, um registo por cada tabela. Este cálculo permite validar a adequação dos tipos de dados escolhidos (`UUID`, `NUMERIC`, etc.) e compreender o peso estrutural de cada entidade.

Tabela	**Tamanho por Registo (bytes)**
Loja	234
Categoria	69
Produto	455
Fornecedor	298
Funcionario	408
Produto_Fornecedor	32
Promocao	81
Promocao_Produto	32
Promocao_Categoria	32
Produto_Categoria	32
Encomenda	68
Linha_Encomenda	64
Inventario	50
Movimento_Inventario	174
Venda	83
Linha_Venda	78
Pagamento	58
Fatura	88
Devolucao	48
Linha_Devolucao	60
Alerta_Stock	120
Total (Estrutura)	**2564**

Tabela 81: Dimensão inicial mínima da estrutura completa da base de dados.

Como a tabela Tabela 81 demonstra, o espaço base para a estrutura completa da base de dados é de aproximadamente $\mathbf{2.4}$ **KB**, um valor que continua a validar a eficiência dos tipos de dados selecionados.

Taxa de Crescimento Anual

A projeção de crescimento anual é o cálculo mais importante para o planeamento de capacidade. A estimativa baseia-se num indicador de negócio fundamental: o número de vendas diárias.

- **Métrica de Crescimento:** Assume-se um valor conservador de $\mathbf{150}$ **vendas por dia, por cada uma das** $\mathbf{4}$ **lojas**;
- **Registos Anuais (Vendas):** `150 * 365 * 4 = 219.000` novas vendas por ano.

As outras tabelas transacionais crescem em função deste número, como detalhado na tabela seguinte, que agora inclui todas as entidades relevantes.

Entidade	Fator de Crescimento	Registos/Ano	Tamanho (bytes/registo)	Espaço Anual (KB)
Venda	1.0	219000	83	18177.0
Linha_Venda	3.0	657000	78	51246.0
Pagamento	1.0	219000	58	12702.0
Fatura	1.0	219000	88	19272.0
Movimento_Inventario	3.2	700000	174	121800.0
Devolucao	0.05	11000	48	528.0
Linha_Devolucao	0.075	16500	60	990.0
Encomenda	N/A	400	68	27.2
Linha_Encomenda	N/A	4000	64	256.0
Produto	N/A	200	455	91.0
Categoria	N/A	20	69	1.38
Fornecedor	N/A	50	298	14.9
Produto_Fornecedor	N/A	100	32	3.2
Produto_Categoria	N/A	400	32	12.8
Funcionário	N/A	10	408	4.08
Promocao	N/A	100	81	8.1
Promocao_Produto	N/A	150	32	4.8
Promocao_Categoria	N/A	50	32	1.6
Inventario	N/A	800	50	40.0
Alerta_Stock	0.02	4380	120	525.6
Total Anual Estimado				**225.7 MB**

Tabela 82: Crescimento anual estimado para a Base de Dados Central, incluindo todas as entidades.

Conclusão

A análise completa, abrangendo as 21 entidades, torna a projeção mais precisa. A tabela `Movimento_Inventario`, anteriormente omitida, revela-se como a maior contribuinte para o crescimento do disco. O novo total anual estimado é de aproximadamente $\mathbf{225.7}$ **MB**. Este valor, embora superior, continua a ser perfeitamente comportável pela arquitetura definida (servidor central *on-premise*), validando que o sistema tem capacidade para escalar durante vários anos sem necessidade de reinvestimento em *hardware*.

4.2.5. Lógica Ativa: Vistas, Funções e *Triggers*

Na persecução das boas práticas de Engenharia de *Software* preconizadas por Ian Sommerville, e com o intuito de garantir um sólido desacoplamento entre a persistência dos dados e a lógica aplicacional em Java, a equipa implementou a "Lógica Ativa" diretamente no motor PostgreSQL.

Esta secção abrange três pilares fundamentais: ***Triggers*** (para impor a atomicidade física estrita), ***Views*** (para as agregações estruturais e analíticas) e **Funções Utilitárias** (para o encapsulamento de código e matemática avançada).

Funções de Gatilho (Triggers)

Os Gatilhos instalados agem como a "polícia estática" da Base de Dados. Estes executam silenciosamente à margem da aplicação para assegurar os bloqueios físicos exigidos pelos Requisitos. Não obstante do terminal Ponto de Venda ou ferramenta administrativa na qual o dado é originado, os Triggers impõem sempre um rigor matemático.

1. **Abatimento Automático de Stock (trg_venda_abater_stock)**: Acionado instantaneamente `AFTER INSERT` sobre a entidade temporal `Linha_Venda`. Cobre a validação dos requisitos e a **User Story 1** e **User Story 2**. No momento em que o retalhista passa um produto no leitor ótico da caixa e a base de dados submete a linha transacional, este trigger subtrai matematicamente a quantidade vendida à tabela subjacente de `Inventario`. Em simultâneo, introduz de forma silenciosa uma entrada formal ao **histórico** de auditoria ("Saída") na tabela `Movimento_Inventario`;
2. **Reposição de Stock por Devolução (trg_devolucao_repor_stock)**: Em clara oposição ao gatilho anterior, este atua num bloco temporal de Pós-Venda (`AFTER INSERT ON Linha_Devolucao`). Responsabiliza-se por somar ativamente ao stock de inventário a referida mercadoria que foi reposta na loja física, para salvaguardar os registos da contabilidade física;
3. **Bloqueador Estrito de Stock Negativo (trg_prevencao_stock_negativo)**: Atua como mecanismo primordial para a salvaguarda da consistência estrutural dos dados, em prol dos requisitos estritos do negócio. Dispara sempre num ciclo `BEFORE UPDATE ON Inventario`. Se a dedução matemática originada por concorrência atirar o stock de armazém de uma loja abaixo do literal "0" (-1, −2, etc), este gatilho emite uma Exceção Crítica nativa `RAISE EXCEPTION`. Isto aborta toda a transação na base de dados e protege o negócio de vendas fantasmas sem cobertura logística real;
4. **Reposição de Stock na Entrega de Encomenda (trg_encomenda_entregue_repor_stock)**: Acionado em `AFTER UPDATE ON Encomenda`, executa apenas quando existe transição efetiva de estado para `Entregue`. O gatilho percorre as linhas da encomenda e adiciona as quantidades ao `Inventario` da loja, registando simultaneamente um movimento de entrada em `Movimento_Inventario` para auditoria;
5. **Gestão Automática de Alertas de Stock (trg_inventario_alerta_stock)**: Atualiza a tabela `Alerta_Stock` sempre que o inventário muda, criando, atualizando ou resolvendo alertas (`STOCK_BAIXO`, `STOCK_CRITICO`, `RUTURA`) de forma automática no motor relacional;
6. **Bloqueio de Inativação de Fornecedor com Pendências (trg_fornecedor_inativacao_sem_pendentes)**: Impede a transição para estado inativo quando existem encomendas por concluir, reforçando a integridade do ciclo de fornecimentos.

Exemplo de Implementação (Stock Negativo):

```
CREATE OR REPLACE FUNCTION trg_func_prevencao_stock_negativo() RETURNS TRIGGER AS $$
BEGIN
    IF NEW.quantidade < 0 THEN
        RAISE EXCEPTION 'Operação Invalida: Stock não pode ser negativo.';
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_prevencao_stock_negativo
BEFORE UPDATE ON Inventario FOR EACH ROW
EXECUTE FUNCTION trg_func_prevencao_stock_negativo();
```

Exposição em Vistas (*Views*)

A "Retaguarda" da Taki obriga a que os quadros da Gestão absorvam métricas de milhões de transações do POS numa fração de milissegundos. Para aliviar a memória dinâmica dos nossos micro-serviços *backend*, a equipa delegou estas consolidações algébricas em exclusivo ao interior do motor relacional através da criação de ***Virtual Views*** ("Vistas").

1. **Dashboard Executivo (View_Dashboard_Kpi_Diario)**: Traduz numericamente os "Cartões Visuais Resumo" expostos na secção do Protótipo Visual (Dashboard). Este componente filtra as métricas operacionais das transações concluídas no dia correspondente (`CURRENT_DATE`), e apura estritamente os totais faturados em euros e o respetivo número nominal de vendas agrupadas por filial;
2. **Sentinela de Inventário (View_Alertas_StockCritico)**: Consiste na materialização estrita do **Requisito Funcional RF10** e da ***User Story* 5** (alertas para ruína logística temporal). Recolhe nativamente a junção transversal de todos os Produtos catalogados e extrai, em isolamento para o ecrã do administrador do *Backoffice*, o registo dos artigos cuja `quantidade` desceu até à baliza limite, ou diretamente abaixo da `quantidade_minima`;
3. **Fatiação Departamentada de Negócio (View_Desempenho_Categorias)**: Dá suporte vital ao Gráfico ***Pie Chart*** requisitado para a analítica visual da Taki. A sua sintaxe resolve internamente toda a sobreposição temporal pesada com a ligação matricial das associações N:M (`Produto_Categoria`) estruturadas em cruzamento com todas as saídas consolidadas de fecho (`Linha_Venda` / `Venda`). Esta vista transmite limpidamente ao ecossistema Java qual departamento retalhista obteve a exata rendibilidade de tesouraria numa determinada Loja.

Exemplo de Implementação (Painel de Métricas):

```
CREATE OR REPLACE VIEW View_Dashboard_Kpi_Diario AS
SELECT
    v.id_loja,
    COUNT(v.id_venda) AS contagem_vendas_dia,
    COALESCE(SUM(v.total), 0) AS total_faturado_dia
FROM Venda v
WHERE DATE(v.data_hora) = CURRENT_DATE AND v.estado = 'Concluída'
GROUP BY v.id_loja;
```

Funções Utilitárias (Functions)

As **Funções** operam ao nível da base de dados como utilitários algébricos independentes (com capacidade de devolver valores de retorno à lógica superior). O recurso a funções independentes puramente matemáticas permite mitigar a ocorrência estrutural de eventuais blocos de "Spaghetti Code" dentro de Gatilhos complexos. Computam valores com segurança sem prejudicar o padrão arquitetural Data Access Object do Java.

1. **Mecanismo Reutilizável de Avaliação (func_valorizacao_estoque)**: Preenche fidedignamente o RF08, correspondente aos processos fiscais e relatórios imobilizados exigidos pela Administração e Contabi-

lidade. Em modo macro, esta função multiplica constantemente a flutuação instantânea estipulada no armazém da filial (`Inventario.quantidade`) com a sua constante unitária associativa de custo de compra primário (`Produto.preco_custo`). Em suma, projeta para as áreas de administração a conversão precisa de todos os bens vitais em valor de capital estanque (Euro).

Exemplo de Implementação (Valorização de Inventário):

```
CREATE OR REPLACE FUNCTION func_valorizacao_estoque(p_id_loja INT) RETURNS NUMERIC AS $$
DECLARE
    total_valor NUMERIC(15,2);
BEGIN
    SELECT COALESCE(SUM(i.quantidade * p.preco_custo), 0) INTO total_valor
    FROM Inventario i
    JOIN Produto p ON i.id_produto = p.id_produto
    WHERE i.id_loja = p_id_loja;
    RETURN total_valor;
END;
$$ LANGUAGE plpgsql;
```

4.2.6. Indexação e Otimização

A escalabilidade comercial pretendida para a plataforma Taki assume que o **Data Layer** processará elevadas densidades de transações simultâneas de Ponto de Venda. Por esta via, o varrimento genérico (**Sequential Scan**) das tabelas resultaria no colapso da RAM aos finais de mês.

Posto isto, implementámos árvores "B-Tree" estruturais sobre chaves externas (*Foreign Keys*) e eixos lógicos críticos, garantindo aos nossos Gatilhos e Views tempos de pesquisa algorítmica de $O(\log n)$. Assumimos como pressuposto fulcral que o PostgreSQL já instancia nativamente índices sobre *Primary Keys* e colunas `UNIQUE` automáticas, focando-nos meramente em Índices Compostos utilitários:

1. **`idx_inventario_produto_loja` sobre `Inventario(id_produto, id_loja)`**: O índice de performance mais vital para a consistência da loja física. Uma vez que toda a nossa Lógica Ativa e os **Triggers** efetuam uma máscara `WHERE id_produto = X AND id_loja = Y` centenas de vezes por minuto, cruzar estas duas *Foreign Keys* num só índice B-Tree assegura um acesso sub-milissegundo para os abates de invetário;
2. **`idx_linha_venda_venda` sobre `Linha_Venda(id_venda)`**: Dada a ausência de indexações nas `FOREIGN KEY` do PostgreSQL por norma, aplicámos este índice explícito para viabilizar as extrações fulminantes e massivas de faturas robustas, cruzando os artigos passados na caixa e o seu respetivo descritivo transacional;
3. **`idx_linha_venda_produto` sobre `Linha_Venda(id_produto)`**: Acelera expressamente os pesados cálculos do Gráfico `View_Desempenho_Categorias`, para varrer unicamente os produtos específicos em vez de todas as vendas mortas da Base de Dados;
4. **`idx_venda_dashboard` sobre `Venda(id_loja, DATE(data_hora), estado)`**: Um índice puramente "Funcional". Pré-calcula e enxuga a data temporal ao nível matemático de modo a que a *query* sumária de Dashboard descole estritamente a agregação final do dia de trabalho;
5. **`idx_movimento_inv_inventario` sobre `Movimento_Inventario(id_inventario)`**: Permite às auditorias de contabilidade o rastreio da flutuação linear sem pesquisar a grelha logarítmica nacional da Taki.

Exemplo de Implementação (Índice Composto Crítico):

```
CREATE INDEX idx_inventario_produto_loja ON Inventario USING btree (id_produto,
id_loja);
```

Para além da indexação elementar, a equipa avaliou otimizações físicas adicionais. Na versão final da entrega, o ***tuning*** explícito de `autovacuum` não ficou fixado no DDL (`schema_postgres.sql`), para manter portabilidade entre ambientes e evitar parametrizações rígidas fora do escopo funcional.

4.2.7. Mecanismo de Sincronização e Consolidação

O modelo global arquitetural da **Taki** dita que a infraestrutura se ramifique num formato em árvore: possuímos um Servidor Central consolidado, suportado por inúmeras ramificações autónomas (lógicas de Lojas Locais). Pelas diretrizes de Fiabilidade exigidas (como o estrito requisito de Tolerância a Falhas em situações de interrupções de serviço de internet locais), a base de dados do ***Data Layer*** requer mecanismos robustos, isolados e assíncronos para a transferência de inteligência e consolidação financeira diária das lojas para a base de dados nacional.

Em vez da abordagem intrusiva que acoplaria matrizes binárias ou *timestamps* de validação a todos os nossos elegantes Modelos Conceptuais (corrompendo os mapeamentos base da Base de Dados), decidimos implementar o venerável padrão corporativo *Outbox Pattern* (Tabela de Caixa de Saída Exclusiva).

Arquitetura do *Outbox Pattern* (Tabela Isolada)

A mecânica de comunicação unilateral diária (*Push Local-To-Central*) é integralmente operada por detrás do pano pelos motores de PostgreSQL Locais, abstraindo o Java de lógicas desnecessárias:

1. **`Fila_Sincronizacao` (Tabela Correio)**: Uma tabela puramente física `(id_fila, nome_tabela, id_entidade, operacao)` invisível ao utilizador e aos painéis de gerência. Desenhada de forma volátil, não possui *Foreign Keys* para não bloquear eliminações da BD;
2. **Agentes de Captação Pós-Transacional**: Foi enxertada uma série de Gatilhos silenciosos de modo `AFTER INSERT` e `AFTER UPDATE` nas tabelas núcleo geradoras de dinheiro da rede de lojas, a realçar: **Vendas**, **Produto**, **Movimentos de Inventário** e **Alertas de *Stock***.

Mecânica de Consolidação: Sempre que o POS regista um guião de compra, o PostgreSQL clona sub-repticiamente o `UUID` da entidade gerada e enfia o seu ponteiro na `Fila_Sincronizacao`. À meia-noite restrita da loja física, instigado por uma *CronJob* nativa do ambiente Java Aplicacional de persistência, um *Endpoint* de Sincronização fará `SELECT * FROM Fila_Sincronizacao`. Todo o lote novo empilhado ao longo das últimas $24H$ no disco será lido, transformado num objeto unificado, e remetido ao **Warehouse** Central em requisições de máxima segurança (TLS 1.3). Após a receção do validador `HTTP 200 OK` por parte da API da central, a *Application Layer* dispara internamente sobre a Loja o comando destruidor `TRUNCATE / DELETE`, limpando toda a Fila até ao dia imediato e poupando GigaBytes nas drives da base de dados local.

Exemplo de Implementação (Ponteiro de Fila e *Trigger*):

```
CREATE TABLE Fila_Sincronizacao (
    id_fila SERIAL PRIMARY KEY,
    nome_tabela VARCHAR(50) NOT NULL,
    id_entidade UUID NOT NULL,
    operacao VARCHAR(10) NOT NULL CHECK (operacao IN ('INSERT', 'UPDATE', 'DELETE')),
    data_registo TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE OR REPLACE FUNCTION func_sync_venda() RETURNS TRIGGER AS $$
BEGIN
    INSERT INTO Fila_Sincronizacao (nome_tabela, id_entidade, operacao)
    VALUES ('Venda', NEW.id_venda, TG_OP);
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_sync_venda
AFTER INSERT OR UPDATE ON Venda
FOR EACH ROW EXECUTE FUNCTION func_sync_venda();
```

Na versão final, o mecanismo ficou implementado com funções específicas por tabela (`func_sync_venda`, `func_sync_produto`, `func_sync_movimento`, `func_sync_alerta_stock`), evitando uma função genérica dinâmica que dificultava a rastreabilidade e manutenção.

4.3. Lógica de Negócio

Nesta subsecção, detalha-se a implementação física da camada de acesso a dados em linguagem **Java**, concretizando a arquitetura conceptual previamente definida.

4.3.1. Camada de Acesso a Dados (`DAO`)

Gestão Dinâmica de Conexões e Infraestrutura (`ConnectionManager`)

Para assegurar uma comunicação estável, segura e altamente adaptável entre a camada lógica (Java) e o Sistema de Gestão de Bases de Dados (PostgreSQL), foi desenhado um módulo isolado com o padrão estrutural *Singleton*, denominado `ConnectionManager`. Este componente abstrai totalmente as lógicas de estabelecimento de sessão, oferecendo um ponto único de entrada para todas as transações da plataforma.

De forma a garantir o elevado desempenho e a escalabilidade exigidos pelo cenário de múltiplas transações simultâneas (especialmente nos Pontos de Venda), o `ConnectionManager` implementa **Connection Pooling** através da biblioteca **HikariCP**. A utilização do HikariCP assegura que as ligações à base de dados são mantidas ativas e reutilizadas de forma extremamente eficiente, minimizando o pesado *overhead* da criação constante de novas ligações TCP para cada operação e garantindo tempos de resposta consistentes abaixo da barreira dos 10 segundos exigida nos requisitos não funcionais.

A grande inovação arquitetural neste módulo prende-se com a **injeção dinâmica de dependências via Variáveis de Ambiente (ficheiros `.env`)**. Ao invés de embutir ("*hardcode*") *strings* de ligação no código-fonte, o sistema lê a sua configuração no arranque. Esta decisão permite que o **exatamente mesmo código binário compilado (.jar)** opere em dois contextos estruturalmente distintos:

- **Contexto de Loja (Autónomo):** O sistema arranca com a variável `TAKI_DB_URL` apontada para a máquina local (`jdbc:postgresql://db_local:5432/taki_db`) e autentica-se obrigatoriamente com o perfil técnico `taki_app_user`. Como definido na Secção de Gestão de Acessos, este papel relacional não possui permissões de *DELETE*, garantindo a preservação absoluta do histórico transacional local;
- **Contexto de Sede (Central):** O sistema arranca no servidor consolidado com a `TAKI_DB_URL` apontada para o nó central (`jdbc:postgresql://db_central:5432/taki_central_db`) e autentica-se através de perfis privilegiados ou de *Analytics*, dependendo do serviço em execução.

A orquestração destes parâmetros, em ambos os cenários, é gerida de forma transparente pelas configurações de infraestrutura da rede Docker associada. Esta arquitetura descentralizada elimina qualquer risco de exposição de credenciais num repositório de código e facilita a expansão e o *deploy* massivo para novas filiais da franquia.

A Fundação: O DAO Genérico (`AbstractDAO`)

A espinha dorsal da persistência da aplicação é garantida pela classe abstrata `AbstractDAO<K, V>`. Esta classe implementa a interface genérica `DAO<K, V>`, estabelecendo um contrato rigoroso para as operações padrão de Criação, Leitura, Atualização e Eliminação (CRUD) sobre qualquer entidade do sistema. Ao centralizar o método `getConnection()` — que invoca o `ConnectionManager` para obter uma sessão limpa e transacional da base de dados —, o `AbstractDAO` assegura que todos os componentes de acesso a dados partilham a mesma infraestrutura, evitando a dispersão de lógica de configuração e unificando o tratamento de exceções (tais como as `SQLException`).

Mapeamento Relacional Direto (JDBC)

Para garantir o rigor no cumprimento do requisito não funcional de tempo de resposta (inferior a 10 segundos por transação) e minimizar o *overhead* computacional, a equipa optou pela utilização de **JDBC puro** em detrimento de ORMs (*Object-Relational Mappers*) pesados ou repletos de abstrações lentas. Os DAOs concretos, tais como o `ProdutoDAO` ou o `VendaDAO`, herdam de `AbstractDAO` e implementam as

suas próprias *queries* SQL nativas e altamente otimizadas, permitindo o aproveitamento total dos índices criados previamente na BD.

A segurança sistémica, particularmente a mitigação do vetor número um de ataque cibernético a plataformas de gestão, a *SQL Injection*, é assegurada através da adoção inegociável de *Statements* Pré-Compilados (`PreparedStatement`). A injeção de estado é concretizada por *bind variables* diretamente do Java. Eis um exemplo prático desta abordagem de abstração de injeção, com a formatação natural dos `UUID` do sistema:

```
// Exemplo ilustrativo de proteção SQL num DAO
String sql = "SELECT * FROM Produto WHERE id_produto = ?";
try (Connection conn = getConnection();
     PreparedStatement ps = conn.prepareStatement(sql)) {

    // O driver JDBC garante que a string injetada é imaculadamente
    // tratada como valor literal UUID e nunca como código relacional
    ps.setObject(1, UUID.fromString(key));
    ResultSet rs = ps.executeQuery();

    // ...
```

Esta combinação de Java de baixo nível com *PreparedStatements* confere à `Taki` uma latência minúscula, crucial para agilizar o atendimento nas caixas de Ponto de Venda (POS).

Atualização Final da Camada DAO (Estado Implementado)

Na versão final da solução, a camada DAO ficou alinhada com o esquema efetivamente em produção e com os requisitos funcionais, sem introduzir tabelas auxiliares fora de escopo. O conjunto de DAOs ativos cobre os agregados operacionais da plataforma:

- `LojaDAO`, `FuncionarioDAO`, `CategoriaDAO`, `ProdutoDAO`, `FornecedorDAO`;
- `ProdutoFornecedorDAO`, `EncomendaDAO`, `PromocaoDAO`, `VendaDAO`, `DevolucaoDAO`, `FaturaDAO`;
- `InventarioDAO`, `OutboxDAO`, `StatisticsDAO`.

Notas de Arquitetura (DAO + Lógica de Negócio)

1. A persistência de perfis técnicos de base de dados (roles PostgreSQL) é tratada em `user_roles.sql`; já os perfis de negócio da aplicação são geridos na lógica (`PerfilAcessoService`) e aplicados sobre as contas de `Funcionario` via campo `cargo`;
2. O ciclo de vida de contas e perfis na camada lógica inclui registo, edição, bloqueio, remoção lógica, atribuição de perfil a utilizador e edição de permissões de perfil;
3. A sincronização distribuída usa `OutboxDAO` para leitura/limpeza da `Fila_Sincronizacao`, mantendo o desacoplamento entre a lógica aplicacional e os gatilhos de captura de eventos no PostgreSQL;
4. A entidade de domínio `MovimentoInventario` mantém paridade estrutural com a tabela `Movimento_Inventario` (`id_movimento`, `tipo`, `quantidade`, `data_registo`, `motivo`, `id_inventario`, `id_funcionario`), evitando atributos transitórios não persistidos.

4.3.2. Consumo da Lógica Ativa e Sincronização

A integração da Lógica de Negócio com a Lógica Ativa da base de dados está implementada através de DAOs e serviços que consomem os efeitos de *triggers*, funções e vistas do PostgreSQL.

No fluxo operacional local, os serviços de vendas e inventário não duplicam regras críticas que já são garantidas no SGBD (abatimento/reposição de stock, prevenção de *stock* negativo e gestão de alertas), limitando-se a persistir os eventos de negócio e a ler o estado consolidado.

A sincronização usa o padrão Outbox com `OutboxDAO`, que lê e limpa a `Fila_Sincronizacao` após confirmação de envio, mantendo o acoplamento mínimo entre Java e SQL ativo.

Para análise e dashboard, o `StatisticsDAO` consome vistas/funções agregadas no servidor relacional, preservando consistência entre relatórios e dados transacionais.

4.3.3. Implementação do Catálogo (Lojas)

A implementação do subsistema de Lojas (Catálogo) foca-se em resolver problemas complexos de estrutura e segurança. A lógica de negócio foi desenhada para proteger o sistema contra inconsistências e vulnerabilidades, centralizando o controlo nos serviços desenvolvidos em Java.

Injeção de Dependências e Segurança das Passwords

A gestão de funcionários e das suas credenciais foi construída com a segurança como prioridade. Em vez de acoplar um algoritmo de encriptação diretamente nas classes de negócio, a plataforma usa o padrão de Injeção de Dependências através da interface `IPasswordHasher`. No arranque do sistema, é injetada a implementação `BCryptPasswordHasher` no `FuncionarioService`. Esta abordagem garante que as passwords são processadas usando um algoritmo forte (BCrypt) que inclui um *salt* aleatório, o que impede ataques de "Força Bruta" ou "Rainbow Tables".

```
// Excerto da injeção de dependências para segurança (SubSistemaLojas.java)
public SubSistemaLojas(ConnectionManager connectionManager) {
    this.funcionarioDAO = new FuncionarioDAO(connectionManager);
    // Injeção da estratégia de hashing que implementa a interface IPasswordHasher
    this.funcionarioService = new FuncionarioService(
        funcionarioDAO,
        new BCryptPasswordHasher(),
        new PerfilAcessoService()
    );
}
```

Resolução de Árvores Hierárquicas em Memória (`CategoriaService`)

O catálogo de uma loja não é uma lista simples, mas sim uma estrutura em árvore com múltiplos níveis de categorias. Para evitar fazer várias consultas à base de dados para obter o caminho de uma categoria (problema N+1 *queries*), foi implementado um algoritmo recursivo em Java (`CategoriaService.obterCaminhoHierarquico()`). Este algoritmo navega em memória, a partir de uma categoria-filha, usando a referência `idCategoriaPai` até chegar à raiz. Este processo permite reconstruir rapidamente o "caminho" completo de um produto (ex: "Têxtil > Vestuário > Calças") e é fundamental para aplicar promoções que são herdadas.

Isolamento de Regras Fiscais (`TaxaIva`)

Para evitar erros humanos e garantir que os valores de impostos são sempre os corretos, a lógica das taxas de IVA foi isolada numa Enumeração (`TaxaIva`). A conversão entre o valor numérico guardado na base de dados e a taxa de imposto correspondente é feita pelo método `fromValor()`. Este mecanismo força a que as classes `Produto` e `LinhaVenda` calculem os preços usando valores de IVA pré-definidos e válidos, de acordo com as leis fiscais.

Validação de Códigos de Barras (`EAN`)

O sistema não confia apenas nas restrições da base de dados (SQL `UNIQUE`) para garantir que os códigos de barras são únicos. O `ProdutoService` faz uma verificação prévia, procurando ativamente pelo código EAN no sistema sempre que um produto é inserido ou modificado. Se for encontrado um código duplicado, a operação é interrompida e é lançada a exceção `ProdutoExistenteException`. Este bloqueio antecipado na camada de negócio previne erros da base de dados e permite enviar uma mensagem de erro clara para a interface do utilizador.

4.3.4. Implementação de Inventário

Implementação: Subsistema de Inventário e Auditoria

A concretização do subsistema de inventário no código Java distribui responsabilidades entre três serviços especializados: o `InventarioService`, o `MovimentoInventarioService` e o `AlertaStockService`. A camada aplicacional foca-se na validação de regras de negócio e na orquestração de movimentos manuais, delegando ao PostgreSQL, através de *triggers*, a execução automática das atualizações de stock despoletadas por vendas, devoluções e encomendas.

Gestão de Movimentos Manuais e Fluxo de Exceções (InventarioService)

O `InventarioService` implementa as interfaces `IInventarioService`, `IVendaObserver` e `IDevolucaoObserver`. Os métodos `onVendaConcluida` e `onDevolucaoConcluida` estão intencionalmente vazios — o abate e a reposição de stock são tratados exclusivamente pelos triggers `trg_venda_abater_stock` e `trg_devolucao_repor_stock` do PostgreSQL, sem qualquer intervenção da camada Java.

O método `registarMovimentoManual` trata os ajustes manuais iniciados por operadores. Começa por consultar o registo de inventário via `inventarioDAO.findById()`, lançando `ArtigoNaoEncontradoException` se o artigo não existir. Para movimentos de saída ou quebra, verifica se a quantidade disponível é suficiente, lançando `StockInsuficienteException` se não for. A atualização é persistida via `inventarioDAO.save()` e o movimento registado via `inventarioDAO.addMovimento()`, associando-o ao produto e ao funcionário responsável.

```
// InventarioService.java — ajuste manual de stock
public void     public void registarMovimentoManual(MovimentoInventario movimento)
throws StockInsuficienteException, ArtigoNaoEncontradoException {
        // Consulta o estado atual do inventario
        Optional<Inventario> inventarioOpt =
this.inventarioDAO.findById(movimento.getIdInventario());

        if (inventarioOpt.isEmpty()) {
            throw new ArtigoNaoEncontradoException(...);
        }

        Inventario inventario = inventarioOpt.get();
        double quantidadeAnterior = inventario.getQuantidade();
        double novaQuantidade = quantidadeAnterior;

        // Regras de negocio para movimentos manuais
        if (movimento.getTipo() == TipoMovimento.ENTRADA) {
            novaQuantidade += movimento.getQuantidade();
        } else {
            // Saida ou Quebra: Verifica stock disponivel
            if (inventario.getQuantidade() < movimento.getQuantidade()) {
                throw new StockInsuficienteException(...);
            }
            novaQuantidade -= movimento.getQuantidade();
        }

        // Atualiza o inventario
        inventario.setQuantidade(novaQuantidade);
        this.inventarioDAO.save(inventario.getIdProduto(), inventario);

        if (movimento.getDataRegisto() == null) {
            movimento.setDataRegisto(LocalDateTime.now());
        }
        this.inventarioDAO.addMovimento(inventario.getIdProduto(), movimento);
    }
```

Validação de Auditoria (MovimentoInventarioService)

O `MovimentoInventarioService` é responsável pela validação das regras de negócio aplicadas aos movimentos antes da sua persistência. O método `registarMovimento` aplica três validações em cadeia: rejeita quantidades não positivas com `QuantidadeInvalidaException`; rejeita datas futuras com `DataInvalidaException`; e exige motivo preenchido para movimentos do tipo `QUEBRA`, lançando `MotivoObrigatorioException` se estiver ausente. A consulta do histórico é feita através de `inventarioDAO.getMovimentos()`, devolvendo os registos ordenados por data descendente.

```
// MovimentoInventarioService.java — validações em cadeia
    public void registarMovimento(MovimentoInventario movimento) throws
QuantidadeInvalidaException, DataInvalidaException, MotivoObrigatorioException {

        // 1. Valida quantidade estritamente positiva
        if (movimento.getQuantidade() <= 0) {
            throw new QuantidadeInvalidaException("A quantidade do movimento deve ser
estritamente superior a zero.");
        }

        // 2. Valida se a data do registo nao e futura
        if (movimento.getDataRegisto() != null &&
movimento.getDataRegisto().isAfter(LocalDateTime.now())) {
            throw new DataInvalidaException("Nao e possivel registar movimentos com data
futura.");
        }

        // 3. Valida obrigatoriedade de motivo para Quebra
        if (movimento.getTipo() == TipoMovimento.QUEBRA &&
            (movimento.getMotivo() == null || movimento.getMotivo().isBlank())) {
            throw new MotivoObrigatorioException("Os movimentos de Quebra manual exigem
uma justificacao.");
        }

        // Os movimentos sao gerados por triggers na base de dados (conforme
InventarioDAO)
    }
```

Monitorização de Alertas via View (AlertaStockService)

O `AlertaStockService` não executa cálculos nem persiste alertas diretamente — essa responsabilidade pertence ao trigger `trg_inventario_alerta_stock` do PostgreSQL. O serviço Java limita-se a consultar a view `View_Alertas_StockCritico` através do `StatisticsDAO`, que agrega os produtos cujo stock atual não excede o limite mínimo. O método `verificarAlertasLoja` mapeia cada linha da view para um objeto `AlertaStock`, enriquecendo-o com a data de consulta e uma mensagem formatada para apresentação na interface de gestão.

```
// AlertaStockService.java — consulta da view de alertas
    public List<AlertaStock> verificarAlertasLoja(int idLoja) {
        // Consulta a vista da base de dados através do DAO de estatísticas
        List<java.util.Map<String, Object>> alertasRaw =
this.statisticsDAO.getAlertasStockCritico(idLoja);

        // Processa e enriquece os alertas
        return alertasRaw.stream()
            .map(row -> {
                AlertaStock alerta = new AlertaStock();
                alerta.setIdLoja((Integer) row.get("idLoja"));
                alerta.setCodigoBarras((String) row.get("codigoBarras"));
                alerta.setNomeProduto((String) row.get("produto"));
                alerta.setQuantidadeAtual(((Number)
row.get("stockAtual")).doubleValue());
```

```
                alerta.setLimiteMinimo(((Number)
row.get("limiteMinimo")).doubleValue());
                alerta.setDataAlerta(LocalDateTime.now());
                alerta.setMensagem(String.format("Aviso: O produto [%s] atingiu o limite
critico. Stock atual: [%.2f]", alerta.getNomeProduto(), alerta.getQuantidadeAtual()));
                return alerta;
            })
            .collect(Collectors.toList());
    }
```

4.3.5. Implementação de Fornecimentos e Encomendas

A implementação da logística interna repousa sobre a exigência primária da imutabilidade legal de faturas de compra a fornecedores, o que exige que as transações fiquem seladas e incorruptíveis no instante exato da sua submissão técnica. Esta garantia não se confia à interface gráfica, mas sim ao nível basilar do código da linguagem *backend*.

Modelação Ativa do Padrão Estrutural *State*

O percurso negocial de cada fornecimento rege-se imperativamente pelo *Design Pattern* denominado *State*. A infraestrutura define uma interface elementar e rigorosa, `IEstadoEncomenda`, da qual emanam classes de estado individualizado que operam como nós estáticos na máquina de estados (como por exemplo `EstadoRascunho`, `EstadoPendente`, ou `EstadoEnviada`).

A modelação arquitetural exprime-se na abstração do bloqueio mecânico exposta no método abstrato `podeModificarLinhas()`. O serviço condutor (`EncomendaService`), quando mandatado a acrescentar um item ao manifesto comercial, interroga direta e invariavelmente a máquina de estados que governa a entidade de domínio nesse momento. Caso a instância corrente, transacionada para `EstadoPendente`, devolva uma afirmação lógica negativa, o código Java interrompe fulminantemente o fluxo e aciona uma `IllegalStateException`, proibindo a manipulação e rejeitando o *input* antes sequer da tentativa de gravação física.

```
// Barreira arquitetural nas entidades de domínio logístico (Encomenda.java)
public void adicionarLinha(LinhaEncomenda linha) {
    if (!this.estadoAtual.podeModificarLinhas()) {
        throw new IllegalStateException(
            "Recusa legal: Inviável alterar métricas de transações seladas."
        );
    }
    this.linhas.add(linha);
}
```

Transições de Estado Monolíticas e Condicionadas

A par do bloqueio de edição interna, a progressão cronológica do aprovisionamento também é rigorosamente submetida à máquina de estados. As invocações de mudança, como o `avancarEstado()` presente no ciclo de negócio, obrigam o sistema a escrutinar se a transição pedida é legal ou ilícita face à etapa corrente. A título ilustrativo, o código impossibilita categoricamente a alteração de uma `Encomenda` recém-iniciada em `EstadoRascunho` diretamente para um `EstadoConcluida`, contornando de forma imperativa as normas de conformidade. Esta topologia direcional de transições força a passagem ininterrupta pelas métricas de pendência comercial e envio.

Inibição de Risco Logístico com Isolamento Funcional

No decurso das complexas interações entre os departamentos (Loja e Central), o núcleo de gestão `FornecedorService` consolida e analisa o contexto global dos fornecedores antes do processamento de trâmites comerciais. Esta gestão previne, proativamente, a abertura cega de propostas a parceiros económicos banidos do ecossistema informático por via de uma inativação administrativa (*Soft Delete* via injeção

local da flag de estado). Para materializar a regra sem falhas de coerência, sempre que o funcionário invoca os catálogos logísticos, a lógica isolada lança de imediato uma excepção de domínio especializada (`FornecedorInativoException`).

Esta barreira ergue-se como corolário final do modelo concetual em linguagem orientada a objetos: restringe e canaliza ativamente a máquina, recusando delegar no motor de base de dados aquilo que pertence legitimamente à avaliação restrita do próprio modelo de gestão na classe correspondente.

4.3.6. Implementação do Ponto de Venda e Transações

O Ponto de Venda (POS) inclui os processos mais críticos do sistema, tanto a nível legal como de performance. A implementação foca-se no cumprimento rigoroso da norma `SAF-T PT` e na garantia de que as vendas e as devoluções são operações atómicas.

Transações Atómicas e Notificações Assíncronas (`VendaDAO` e Padrão *Observer*)

A gravação de uma venda tem de ser num bloco único e indivisível. No `VendaDAO`, a funcionalidade de `auto-commit` do `JDBC` é desligada de propósito. A inserção de dados acontece em cascata: primeiro regista-se a Venda, depois todas as `Linhas de Venda` associadas, os dados do `Pagamento` (`TPA`) e, por fim, a informação legal na `Fatura`. Apenas se tudo correr bem é que a transação é confirmada com um `commit`. Se ocorrer qualquer erro, o sistema faz `rollback` de tudo, impedindo que a base de dados fique num estado corrompido.

Depois de a transação ser concluída com sucesso, o sistema muda para um modelo reativo, usando o padrão *Observer*. Através da interface `IVendaObserver`, a classe de vendas emite um evento (`onVendaConcluida`). Esta separação de responsabilidades permite que outras partes do sistema, como o módulo de Inventário (`InventarioDAO`), reajam à venda e atualizem o *stock*, sem que o código de faturação precise de saber ou de se preocupar com isso.

Conformidade Fiscal e Assinatura Digital (SAF-T PT)

O módulo de faturas (`FaturaService`) foi implementado para cumprir os critérios rigorosos da Autoridade Tributária para a exportação do ficheiro `SAF-T`. A classe `FaturaService` é responsável por gerar a assinatura digital `RSA` (usando uma chave pública e privada) e por verificar se a numeração e a data das faturas seguem uma ordem cronológica correta.

Usando o algoritmo `SHA1withRSA`, o sistema cria uma "impressão digital" única para cada fatura. Esta impressão digital é calculada com base em quatro elementos: a data de emissão, a data/hora exata, o número da fatura e o ***hash*** da fatura anterior. O resultado é encriptado com a chave privada, criando uma assinatura que não pode ser falsificada. Este mecanismo de encadeamento impede que uma fatura seja apagada ou alterada do sistema, pois isso quebraria toda a sequência de assinaturas, garantindo a integridade dos dados fiscais.

```
// O modelo SAF-T garante um encadeamento irreversível com a fatura anterior
private String construirPayloadAssinatura(Fatura fatura, String hashAnterior) {
    return String.format("%s;%s;%s;%s",
            fatura.getDataEmissao().toLocalDate().toString(),
            fatura.getDataEmissao().toString(),
            fatura.getNumFatura(),
            hashAnterior != null ? hashAnterior : ""); // Elo de ligação
}
```

Posteriormente, o módulo gera o ficheiro `XML` no formato `SAF-T` (`exportarSaftPt()`), incluindo todas as vendas do período (`<SalesInvoices>`), fechando assim o ciclo de conformidade legal de forma transparente e robusta.

O Algoritmo do Motor de Promoções

Para aplicar descontos e campanhas de forma dinâmica, o `PromocaoService` não se limita a procurar promoções diretamente associadas a um produto. Quando uma venda está a decorrer, o sistema sobe recursivamente na árvore de categorias, a partir da categoria do produto. Por exemplo, se o produto for "Calças", o sistema verifica promoções para "Calças", depois para "Vestuário" e, finalmente, para "Têxtil". O motor de promoções recolhe todas as campanhas aplicáveis e, em caso de sobreposição, escolhe a que for mais vantajosa para o cliente.

4.3.7. Implementação das Estatísticas e Relatórios

Implementação: Modelo de Estatísticas e Analítica

O módulo de Estatísticas foi implementado para agregar e processar dados históricos de forma rápida, sem sobrecarregar a base de dados com pesquisas (`queries`) pesadas. A abordagem escolhida foca-se na extração simples e no cálculo dos indicadores diretamente em memória, blindando a comunicação com a interface através de objetos de transporte.

Processamento em Memória (`Java Streams API`)

Em vez de pedir à base de dados para fazer somatórios ou filtragens complexas através de `SQL`, o `EstatisticasService` pede os dados brutos através dos respetivos `DAOs` (`VendaDAO` e `InventarioDAO`) e encarrega-se da matemática em memória.

Recorrendo intensivamente à `Stream API` do `Java`, o sistema consegue filtrar vendas por data ou categoria, contar produtos e calcular totais em poucas linhas de código. Por exemplo, para obter as vendas de um determinado mês, o código cria uma *stream* de todas as vendas e aplica um `.filter()` para reter apenas as que encaixam nas datas pedidas.

```
// A filtragem temporal ocorre em memória através da Stream API
private List<Venda> filtrarVendasPorPeriodo(LocalDateTime inicio, LocalDateTime fim) {
    return vendaDAO.findAll().stream()
            .filter(v -> v.getDataHora() != null
                    && !v.getDataHora().isBefore(inicio)
                    && !v.getDataHora().isAfter(fim))
            .collect(Collectors.toList());
}
```

4.3.8. Implementação do Sistema Local

Implementação: Fachada Local (`TakiLNLocal`)

A transposição da arquitetura da fachada local para o código fonte assenta nos princípios de Inversão de Controlo e delegação estrita. A `TakiLNLocal` não contém lógica de negócio própria, limita-se a agregar os cinco subsistemas locais e a reencaminhar cada operação para o serviço especializado correspondente.

Construção e Composição dos Subsistemas

O construtor de produção instancia diretamente todos os DAOs necessários e compõe os cinco subsistemas: `SubSistemaLojas`, `SubSistemaInventario`, `SubSistemaFornecimentos`, `SubSistemaVendas` e `SubSistemaSincronizacao`. O `SubSistemaVendas` recebe referências aos subsistemas de lojas e de inventário já construídos, materializando a dependência cruzada entre subsistemas. A classe expõe um segundo construtor com visibilidade de pacote que recebe os cinco subsistemas como interfaces, permitindo a substituição por *mocks* nos testes unitários com Mockito.

```
// construtor de produção
this.subSistemaVendas = new SubSistemaVendas(this.subSistemaLojas,
this.subSistemaInventario,                          vendaDAO, promocaoDAO);


// construtor para testes (package-private)
TakiLNLocal(ISubSistemaLojas subSistemaLojas,
                ISubSistemaFornecimentos subSistemaFornecimentos,
                ISubSistemaInventario subSistemaInventario,
                ISubSistemaVendas subSistemaVendas,
                ISubSistemaSincronizacao subSistemaSincronizacao) { ... }
```

Delegação e Proteção das Listas

Todos os métodos do contrato `ITakiLNLocal` consistem numa única instrução que navega até ao serviço adequado dentro do subsistema e reencaminha a chamada. Os métodos de listagem envolvem a lista devolvida em `Collections.unmodifiableList`, impedindo que a camada superior modifique o estado interno dos subsistemas.

```
@Override
public Funcionario autenticar(String email, String password) throws
ContaBloqueadaException,  CredenciaisInvalidasException {
        return this.subSistemaLojas.getFuncionarioService().autenticar(email, password);
    }


@Override
public java.util.List<Produto> listarProdutos() {
        return
java.util.Collections.unmodifiableList(this.subSistemaLojas.getProdutoService().listarProdutos());
    }

@Override
public void sincronizarDados() throws FalhaSincronizacaoException
{
this.subSistemaSincronizacao.getSincronizacaoService().exportarLoteParaSede(java.time.LocalDateTime.now().
  }
```

4.3.9. Implementação do Sistema Global

Implementação: Orquestração Global (`TakiLNGlobal`)

A transposição da arquitetura de orquestração para o código fonte assentou em princípios de Inversão de Controlo e delegação estrita, resultando numa classe (`TakiLNGlobal`) limpa e isenta de lógicas algorítmicas próprias.

Injeção de Dependências e Contratos (`Interfaces`)

A instanciação do motor global recorre à técnica de Injeção de Dependências pelo Construtor. Em vez da classe utilizar a palavra-chave `new` para erguer fisicamente os subsistemas, esta exige que as peças do motor lhe sejam fornecidas no momento em que é criada.

A implementação impõe um acoplamento extremamente fraco: os parâmetros recebidos não são as classes reais (como `SubSistemaEstatisticas`), mas sim os contratos virtuais (`ISubSistemaEstatisticas`). Esta dependência polimórfica permite trocar instantaneamente qualquer peça do motor por *Mocks* durante os testes unitários (`JUnit`), garantindo o isolamento da classe.

Delegação Pura (Encaminhamento Funcional)

Na implementação dos métodos definidos pelo contrato `ITakiLNGlobal`, estes dispensaram lógicas de validação (estruturas `if/else`) ou processamento de ciclos, consistindo unicamente numa linha de instrução que invoca a função gémea no subsistema correspondente.

```
// Implementação baseada em delegação estrita e acoplamento a interfaces
@Override
public RelatorioVendasDTO gerarRelatorioVendas(LocalDateTime inicio, LocalDateTime fim,
Integer idLoja, String categoria) throws DatasInvalidasException {
    // A chamada e os respetivos argumentos são reencaminhados diretamente
    // para a interface do subsistema especializado em estatísticas
    return subSistemaEstatisticas.gerarRelatorioVendas(inicio, fim, idLoja, categoria);
}
```

4.3.10. Implementação da API

A implementação técnica utiliza a framework `Javalin` para a exposição de serviços REST, tirando partido de uma estrutura robusta de `DTOs` e serviços de lógica de negócio.

Gestão de Ciclo de Vida e Lançamento

O `NodeApiLauncher` atua como o ponto de entrada principal, utilizando o `NodeRoleConfig` para detetar o perfil do nó e iniciar os servidores `Jetty` em portas distintas (tipicamente 8080 para `Local` e 8081 para `Global`). O `ApiBootstrapHelper` auxilia na resolução dinâmica de portas e configuração de *headers* de segurança e cache.

Autenticação e Autorização (`JWT RSA256`)

A segurança é implementada de forma rigorosa no `JwtAuthUtils` e `MiddlewareSegurancaApi`:

- **Validação:** Os *tokens* são extraídos de cookies (`auth_token`) ou do cabeçalho `Authorization`;
- **Assinatura:** Utiliza-se chaves `RSA (públicas/privadas)` configuradas via variáveis de ambiente (`TAKI_JWT_PUBLIC_KEY` e `TAKI_JWT_PRIVATE_KEY`) para assinar e validar a autenticidade dos *tokens*;
- **Guardas de Rota:** A implementação utiliza "`Handlers`" de segurança que exigem papéis específicos (`exigirQualquerRole`) ou âmbitos de acesso (`exigirScopes`) antes de permitir a execução do controlador.

Comunicação via `DTOs` (*Data Transfer Objects*)

Para evitar o acoplamento direto com as entidades do domínio, a `API` utiliza `DTOs` para entrada e saída de dados:

- **Validação de Dados:** Os `DTOs` utilizam anotações de validação (como `NotBlank`, `Email`, `Positive`) que são processadas pelo `Javalin` antes de chegarem à lógica de negócio;
- **Mapeamento:** Os controladores (`LocalApiController` e `GlobalApiController`) são responsáveis por converter estes `DTOs` em objetos de domínio e vice-versa, garantindo que apenas a informação necessária é exposta.

Tratamento Global de Exceções

O `TratadorGlobalExcecoesApi` mapeia exceções específicas da lógica de negócio (ex: `ProdutoInexistenteException`, `CredenciaisInvalidasException`) para códigos de estado `HTTP` `padronizados` (404, 401, etc.). A resposta é sempre devolvida no formato `ErroApiResponse`, que inclui um código interno, uma mensagem e um timestamp.

4.4. *User-Interface*

A interface do sistema Taki foi desenvolvida utilizando o ecossistema **Vue.js 3**, tirando partido da `Composition API` para garantir um código modular, reativo e de fácil manutenção. A implementação seguiu os seguintes pilares técnicos:

1. Arquitetura de Gestão de Estado (Pinia)

Em vez de manipular dados diretamente nos componentes, utilizou-se o Pinia para a gestão de estado global. Esta abordagem permite que a informação (como o inventário, o estado da venda no POS ou a autenticação do funcionário) seja partilhada de forma consistente entre diferentes vistas, garantindo a integridade dos dados sem acoplamento excessivo.

2. *Design* de Componentes e Estilização

A interface foi construída com `Tailwind CSS`, permitindo um design responsivo e performativo.

- **Atomic Design:** Desenvolveram-se componentes de UI reutilizáveis (Input, Button, Dialog, Select) localizados em src/components/ui/;
- **Modularidade:** As vistas principais (Dashboard, POS, Inventário, Gestão de Funcionários) foram isoladas para facilitar a escalabilidade do sistema.

3. Comunicação e Integração (`REST API` & Persistência Real)

A interface consome dados em tempo real através de uma REST API desenvolvida em Java, utilizando o Axios como cliente HTTP centralizado.

- **Consumo Dinâmico:** Substituiu-se a estrutura de *mocking* inicial por chamadas assíncronas aos endpoints do *backend*, garantindo que todas as operações (vendas, gestão de inventário e autenticação) persistem diretamente nas bases de dados PostgreSQL (Central e Local);
- **Controlo de Concorrência:** A integração foi desenhada para lidar com acessos simultâneos, implementando lógica no frontend para processar respostas de conflito (HTTP 409) e garantir a integridade do *stock* durante operações críticas no POS;
- **Padronização de Dados:** Utiliza-se o formato JSON para o intercâmbio de informações, assegurando que os modelos de dados do Vue.js estão em total conformidade com os objetos de negócio do servidor Java.

4.5. Utilização de Inteligência Artificial

No âmbito da implementação do sistema de gestão 'Taki', a Inteligência Artificial assumiu um papel central para acelerar o ciclo de desenvolvimento, garantir a adoção das melhores práticas de engenharia de *software* e materializar os artefactos arquiteturais em código executável. Contudo, esta fase foi marcada por significativos desafios metodológicos e logísticos que forçaram a equipa a reavaliar as ferramentas utilizadas.

4.5.1. Transição Tecnológica: De Gemini e Copilot para Claude Code

Inicialmente, a equipa planeou conduzir a implementação através do *Gemini CLI* e do *GitHub Copilot*. No entanto, durante as fases mais intensas de codificação, deparámo-nos com constrangimentos técnicos severos. O *Gemini CLI* começou a apresentar tempos de resposta extremamente elevados, com as respostas a sofrer demoras de aproximadamente uma hora por *prompt*, o que quebrou por completo o ritmo de desenvolvimento. Em simultâneo, as políticas da plataforma *Copilot* sofreram alterações, situação que resultou numa clara diminuição dos limites de utilização e na remoção dos modelos mais avançados, o que deixou como alternativa principal um modelo substancialmente mais fraco (baseado em Codex 5.3).

Perante a degradação da qualidade do código gerado e a inviabilidade dos tempos de espera, a equipa tomou a decisão estratégica de adquirir uma subscrição *Claude Pro*. Esta aquisição viabilizou a adoção

do *Claude Code*, uma interface de linha de comandos (*CLI*) nativa da Anthropic. Esta transição revelou-se um ponto de viragem no projeto. O *Claude Code* proporcionou acesso direto aos modelos mais potentes da Anthropic, nomeadamente o **Claude Sonnet 4.6** e o **Claude Opus 4.7** (nas suas versões de raciocínio avançado). Estes modelos asseguraram não só um raciocínio lógico superior e uma compreensão profunda do contexto do projeto, mas também uma capacidade de interação direta com o sistema de ficheiros local. Esta autonomia contextual eliminou a necessidade de copiar e colar fragmentos de código de forma manual, e permitiu que a IA atuasse como um verdadeiro par de programação.

4.5.2. Metodologia de Implementação Assistida

Com o ambiente estabilizado no *Claude Code*, a equipa adotou uma metodologia iterativa de implementação. O processo não se baseou na geração cega de ficheiros inteiros, mas sim numa construção progressiva orientada por restrições arquiteturais. A IA recebeu instruções para ler o documento `CLAUDE.md` do projeto (o qual resultou de uma cópia e adaptação direta do ficheiro `GEMINI.md` original, para conter as mesmas regras de negócio e de formatação no formato reconhecido pelo Claude) antes de propor qualquer bloco de código.

Abaixo apresentam-se exemplos das interações fundamentais que guiaram a implementação prática nas diferentes camadas do sistema:

Interface de Utilizador (UI)

Na camada de apresentação (em Vue.js), a preocupação central foi a robustez e a ergonomia:

> "Atua como um especialista em UX/UI e programador Vue.js Sénior. Analisa a estrutura de navegação do *dashboard* da Taki. Com base nesta estrutura, propõe e implementa a otimização do fluxo de fecho de caixa no *frontend*. O objetivo é minimizar os erros de introdução manual por parte do operador, com a preservação de uma interface limpa e reativa. Fornece apenas o código do componente relevante, para assegurar o isolamento da lógica de estado."

Camada de Dados (DL)

Para a materialização da base de dados, a IA auxiliou na tradução do modelo conceptual para esquemas físicos otimizados no PostgreSQL:

> "Analisa o esquema relacional concebido para as tabelas de 'Venda', 'LinhaVenda' e 'Produto'. Escreve o código SQL (DDL) para a sua criação no PostgreSQL, com a aplicação das melhores práticas de indexação para otimizar consultas de relatórios de vendas anuais que envolvem milhões de registos. Assegura a integridade dos dados através da correta definição de *foreign keys* e *check constraints*."

Lógica de Negócio (LN)

A implementação das regras centrais em Java exigiu um alinhamento rigoroso com os padrões de desenho pré-estabelecidos:

> "Explica e codifica a implementação do padrão *Service* em Java para gerir a lógica de atualização de *stock*. O código deve garantir que, ao registar uma venda, a reserva de *stock* seja atómica e *thread-safe*. Implementa mecanismos que previnam condições de corrida (por exemplo, bloqueios otimistas ou pessimistas) quando múltiplos operadores realizam vendas do mesmo produto em simultâneo."

> "No contexto do modelo de domínio atual, como devemos estruturar e codificar a lógica de cálculo de promoções acumuladas (por exemplo: descontos de categoria somados a descontos sazonais) na camada de negócio? Apresenta a implementação em Java com capacidade para estender o motor de regras a novos tipos de campanhas, no estrito respeito pelo Princípio de Aberto/Fechado (*Open/Closed Principle*)."

> "Desenha e codifica a lógica de validação de *tokens* JWT para o controlo de acesso baseado em papéis (RBAC). Demonstra no código como a camada de negócio processa a distinção entre um 'Gestor de Loja' e um 'Administrador Global' ao aceder a *endpoints* críticos, como a exportação de dados SAF-T. Assegura que as políticas de segurança são aplicadas antes da execução de qualquer transação na base de dados."

5. Verificação, Validação e Avaliação da Qualidade do *Software* Produzido

Esta parte tem como propósito demonstrar como asseguramos que o sistema desenvolvido está em conformidade com os requisitos especificados, funciona corretamente no ambiente pretendido e apresenta níveis adequados de fiabilidade, eficiência e manutenibilidade. Enquanto a **verificação** foca-se em garantir que o sistema foi construído corretamente, ou seja, se segue as especificações, a **validação** assegura que o sistema foi bem construído, ou seja, se atende às reais necessidades dos utilizadores e do negócio. Adicionalmente, a **avaliação da qualidade** abrange dimensões como desempenho, segurança, usabilidade e robustez.

5.1. Verificação

O processo de verificação assegura a conformidade estrita do código face aos requisitos arquiteturais estipulados na fase de desenho. Esta etapa suporta-se em metodologias de teste automatizado e análise estática, com o propósito de detetar e corrigir anomalias de forma prematura no ciclo de desenvolvimento.

5.1.1. Testes unitários (LN)

A validação da Lógica de Negócio (LN) constitui uma etapa basilar no ciclo de desenvolvimento do sistema 'Taki', a qual assenta na execução de testes unitários exaustivos. A equipa desenvolveu esta suíte com recurso à *framework* `JUnit 5`. Para garantir o isolamento absoluto dos componentes em avaliação, adotou-se a biblioteca `Mockito`. Esta abordagem metodológica permite a simulação rigorosa do comportamento da Camada de Dados (`DAO`s) através da injeção de dependências fictícias (*mocks*) nos diversos serviços da LN.

A estruturação de cada cenário de teste obedece, de forma inflexível, ao padrão AAA (*Arrange, Act, Assert*). Este paradigma divide o fluxo de verificação em três fases conceptuais estanques:

- **Preparação (*Arrange*)**: Estabelecimento do estado inicial do sistema e instanciação dos dados de teste requeridos para o cenário;
- **Execução (*Act*)**: Invocação da operação alvo ou do comportamento específico alvo de escrutínio;
- **Verificação (*Assert*)**: Confirmação empírica das asserções, a qual atesta a exatidão do resultado obtido face ao comportamento esperado.

Este nível de rigor arquitetural assegura a operação independente das regras de negócio relativamente ao mecanismo de persistência de dados. A suíte de testes não se limita a aferir o caminho nominal de sucesso. A estratégia de testes engloba de igual modo o tratamento de falhas e de casos limite. Entre os múltiplos aspetos cobertos, destacam-se:

- A verificação de limites numéricos nos cálculos matemáticos ligados ao IVA e ao valor global da transação;
- A interceção e o processamento de exceções específicas, tais como erros de autenticação ou anomalias relacionadas com falhas de inventário;
- A validação estrita de fluxos alternativos face a entradas de dados inesperadas, nulas ou logicamente incompletas.

Desta forma, a equipa certifica uma cobertura vasta e metódica de toda a camada funcional da aplicação, com o claro propósito de eliminar falhas conceptuais em fases prematuras do projeto.

5.1.2. Testes aos DAOs

Contrariamente à Lógica de Negócio, a verificação da Camada de Dados exige uma abordagem suportada em testes de integração direta. Os *Data Access Objects* (DAOs) recusam o uso de dependências fictícias de forma intencional. A validação destes componentes decorre através de ligações ativas a uma instância real e volátil do Sistema de Gestão de Bases de Dados *PostgreSQL*. A orquestração deste ambiente isolado efetua-se mediante a alocação de contentores *Docker*.

Esta estratégia técnica revela-se imperativa para a comprovação de múltiplos fatores de elevada complexidade:

- **Transações JDBC Manuais**: A validação empírica atesta a execução rigorosa e ininterrupta das instruções de `commit` e `rollback`. A atomicidade das operações financeiras e de gestão de *stock* depende na íntegra desta precisão técnica;
- **Consultas SQL Nativas**: A equipa certifica a adequação sintática e a eficácia das interrogações escritas diretamente na linguagem própria da base de dados;
- **Integração com a Lógica Ativa**: Os ensaios confirmam o acionamento e a fiabilidade irrepreensível de mecanismos autónomos. Esta validação inclui os *Triggers* desenhados para a atualização instantânea do inventário após o fecho de uma fatura, e as Vistas Materializadas que suportam a extração célere de relatórios estatísticos e de indicadores de desempenho diários.

Para salvaguardar o isolamento total entre os ensaios laboratoriais, o código aciona rotinas automáticas de higienização de dados de forma repetida. A invocação sistemática de instruções do tipo `TRUNCATE TABLE ... CASCADE` limpa o estado de cada tabela de forma integral entre execuções contíguas. Esta prática de engenharia elimina qualquer possibilidade de dados residuais transitarem entre os diferentes testes, o que afasta de forma peremptória o risco de resultados enviesados na avaliação do repositório de persistência.

5.1.3. Testes E2E da UI

Realizar os testes **E2E (End-to-End)** da UI é um processo crítico que valida a integridade funcional do sistema Taki, assegurando que o percurso do utilizador, desde o login até à gestão documental, ocorre sem falhas. Este procedimento foca-se em simular interações reais em ecrãs operacionais, como a **Gestão de Encomendas** e a **Gestão de Faturas**, para garantir que os dados apresentados nas tabelas refletem com precisão o estado das bases de dados e da lógica de negócio.

Para realizar este teste com sucesso, devem ser seguidos os seguintes passos fundamentais:

1. **Preparação da Infraestrutura e Dados**

Antes de interagir com a interface, o ambiente deve estar totalmente operacional e isolado. Isto implica:

- **Arranque dos Contentores:** Garantir que os serviços PostgreSQL e o Backend (Javalin) estão ativos e a comunicar;
- **População de Dados:** Executar os scripts de carregamento (`load_data.sh`) para garantir que existem entidades, como o utilizador `dono@taki.pt` ou encomendas específicas, para serem visualizadas na UI.

2. **Execução do Fluxo de Autenticação**

O teste começa obrigatoriamente pela validação do acesso. O sistema deve processar o pedido de login via `API Global` ou `Local`, emitindo o cookie *auth_token* e redirecionando o utilizador para o `Dashboard principal` conforme o seu perfil. A UI deve refletir corretamente o estado "Online" e o perfil do utilizador (ex: Proprietário da Cadeia) no menu lateral.

3. **Validação das Operações de Negócio (Cenários Críticos)**

Uma vez dentro do sistema, o teste foca-se na manipulação de dados em ecrãs tabulares:

- **Monitorização do Abastecimento:** Na aba de Encomendas, deve verificar-se se os cartões de KPI (encomendas a aguardar validação ou compromissos pendentes) são atualizados dinamicamente após a criação de uma nova encomenda.
- **Histórico Documental:** Na aba de Faturas, o teste valida se a pesquisa por NIF ou número de fatura devolve os resultados esperados e se o botão `"Emitir SAF-T"` despoleta a chamada correta ao endpoint da API para gerar o ficheiro XML do mês atual.

4. **Verificação de Resiliência e Erros**

O realizar deste teste exige também testar o "caminho infeliz". Deve observar-se como a UI reage a falhas, garantindo que o `TratadorGlobalExcecoesApi` envia mensagens claras para o utilizador em caso de erro de rede ou permissões insuficientes, em vez de expor *logs* técnicos do servidor.

Este ciclo de testes assegura que o sistema Taki não é apenas uma coleção de APIs funcionais, mas uma ferramenta de gestão robusta e fiável para o ambiente de retalho.

5.1.4. Análise estática

A análise estática é o processo de examinar o código-fonte e os artefactos de um projeto **sem o executar**. O seu objetivo é identificar, através da inspeção direta dos ficheiros, padrões que indiquem problemas de qualidade, manutenibilidade ou segurança, **antes que estes se manifestem em falhas em tempo de execução**. Ao contrário da análise dinâmica (testes), a análise estática é realizada sobre o texto do programa: a sua **estrutura, organização, documentação e a presença de *code smells***, isto é, idiomas reconhecidamente problemáticos que, mesmo sem causar erros imediatos, tornam o código mais difícil de compreender, testar e evoluir. Neste relatório, o projeto `Taki` é avaliado nas três dimensões fundamentais desta análise: **(1)** a qualidade e presença de documentação, **(2)** a organização e arquitetura do código, e **(3)** a identificação de ***code smells*** concretos.

Documentação

A **documentação do código Java é um dos pontos fortes** do projeto. As interfaces públicas, que constituem o contrato entre camadas, estão todas documentadas com blocos `Javadoc` completos. A `ITakiLNLocal`, por exemplo, documenta cada operação com descrição, incluindo notas de arquitetura sobre a autonomia do nó local e o ciclo de sincronização. Outro exemplo é a `IFaturaService` que garante conformidade com `SAF-T` (PT) e a política de assinatura `RSA` encadeada.

Os **comentários *inline* presentes no código são do tipo explicativo**, ou seja, são comentários que clarificam decisões de *design* e não apenas o que o código faz. No `ConnectionManager.java`, por exemplo, é explicada a estratégia de configuração por variáveis de ambiente e a razão pela qual o mesmo `JAR` pode correr tanto num nó local como na sede sem recompilação.

Organização

- **Estrutura de Pacotes (*Backend*)**

A organização dos pacotes `Java` segue uma **arquitetura em camadas bem definida e consistente**, que é um dos pontos mais fortes do projeto:

```
pt.uminho.taki
├── api/
│   ├── global/     → Controller, Routes, App para a API Global (porta 8081)
│   ├── local/      → Controller, Routes, DTOs, App para a API Local (porta 8080)
│   └── shared/     → Segurança (JWT, Middleware), Erros comuns
├── dao/            → Todos os DAOs (camada de acesso a dados)
└── ln/             → Lógica de Negócio
    ├── ITakiLNLocal.java / TakiLNLocal.java   → Fachada local
    ├── ITakiLNGlobal.java / TakiLNGlobal.java → Fachada global
    ├── vendas/
    ├── inventario/
    ├── fornecimentos/
    ├── lojas/
    ├── fatura/
    ├── estatisticas/
    ├── sincronizacao/
    └── view/
```

Esta estrutura materializa corretamente o padrão de **Fachada** (***Facade***): `TakiLNLocal` e `TakiLNGlobal` são o único ponto de entrada da `API` para a lógica de negócio, delegando internamente para **subsistemas** (`ISubSistemaVendas`, `ISubSistemaInventario`, etc.), que por sua vez delegam para serviços especializados (`VendaService`, `InventarioService`, etc.). A separação entre o módulo `api` e o módulo `ln` é nítida e respeitada.

A separação entre as APIs `local` (operações da loja) e `global` (operações da sede) através do mecanismo `TAKI_NODE_ROLE` é uma decisão arquitetural madura e corretamente implementada em `NodeApiLauncher.java` e `NodeRoleConfig.java`.

- **Organização dos Ficheiros (*Frontend*)**

O frontend Vue.js segue uma estrutura razoável e convencional (`views/`, `components/`, `stores/`, `services/`, `router/`, `utils/`). As ***views*** refletem os casos de uso do sistema, o que torna a navegação no código intuitiva.

- **Organização dos Ficheiros `SQL` e `Docker`**

O diretório `database/` organiza os scripts `SQL` de forma **com responsabilidades bem separadas**: `schema_postgres.sql` (`DDL`), `logica_ativa_postgres.sql` (gatilhos e funções), `sincronizacao_postgres.sql` (mecanismo de *outbox*) e `user_roles.sql` (`RBAC` na base de dados). Esta separação facilita a manutenção e a inicialização dos *containers*, como evidenciado no `docker-compose.yml` onde os volumes são montados por ordem numérica explícita.

Code Smells

- **`LocalApiController` Extenso**

O `LocalApiController.java` concentra, num único ficheiro de 1003 linhas, tanto a orquestração dos pedidos `HTTP` como a lógica de mapeamento de entidades para `JSON`, a validação de *input* e a construção dos *tokens* `JWT`. Esta **concentração é compreensível num projeto com prazos curriculares** e o facto de o

código estar funcional e bem testado mostra que as **responsabilidades estão claramente identificadas internamente**.

Uma **evolução natural** seria isolar os mapeamentos numa classe `LocalApiMapper` e a lógica `JWT` num `JwtTokenBuilder` independente, o que facilitaria testes unitários mais granulares e a reutilização desses componentes noutros contextos.

- ***Password Default* em `ConnectionManager`**

O `ConnectionManager.java` define um valor de *fallback* para a *password* da base de dados diretamente no código, para os casos em que a variável de ambiente `TAKI_DB_PASSWORD` não está configurada. Esta abordagem tem a **vantagem de simplificar o arranque do ambiente de desenvolvimento** sem configuração adicional, mas tem como contrapartida a exposição da credencial no histórico do repositório `Git`.

Uma alternativa simples seria **substituir o *fallback* por uma exceção informativa**,como um `throw new IllegalStateException("TAKI_DB_PASSWORD não configurada")`, complementada por um ficheiro `.env.example` com instruções claras, já presente na raiz do projeto.

- **Estados de Domínio como `String`**

O estado de entidades como Venda (“Pendente”, “Concluída”) e Produto (“Ativo”, “Inativo”) é **representado como `String` em vez de um tipo `Enum` dedicado**.

Para um projeto desta dimensão, a abordagem é pragmática e funciona corretamente no contexto atual. No entanto, a introdução de `Enums` próprios, como `EstadoVenda` ou `EstadoProduto`, teria a vantagem de tornar os valores possíveis explícitos e verificáveis pelo compilador, eliminando a possibilidade de erros tipográficos silenciosos (ex.: “Concluida” sem acento) e facilitando refactorizações futuras.

5.2. Validação

A etapa de validação empírica do sistema atesta a capacidade da solução em responder com eficácia às necessidades reais e aos fluxos de trabalho dos operadores de negócio. Através da execução de cenários práticos e de provas de conformidade, a equipa confirma a entrega do valor tecnológico previamente acordado.

5.2.1. Álgebra Relacional (DL)

A construção de um modelo lógico de dados não fica totalmente validada apenas pela sua conformidade com as regras de normalização. É estritamente necessário provar que a estrutura de tabelas desenhada é capaz de responder aos requisitos operacionais do negócio e extrair a informação solicitada pelos utilizadores finais.

Para formalizar esta prova, recorre-se à **Álgebra Relacional**, uma linguagem teórica e processual que dita como a base de dados vai combinar, filtrar e projetar os dados para responder a perguntas reais. Se conseguirmos traduzir as necessidades dos utilizadores da ‘Taki’ para expressões algébricas viáveis, confirmamos definitivamente que o nosso modelo de dados suporta o *software*.

Abaixo apresentam-se 5 interrogações cruciais do sistema, validadas através da respetiva expressão algébrica:

1. Listagem de Catálogo Ativo Pergunta de Negócio: “Quais são os nomes e os códigos de barras de todos os artigos que estão atualmente disponíveis para venda?” **Objetivo:** Provar a extração simples com filtro e seleção de colunas.

$$\pi_{\text{nome, codigo_barras}}(\sigma_{\text{estado = Ativo}}(\text{Produto}))$$

Justificação: A operação de Seleção (σ) filtra a tabela `Produto` para reter apenas as linhas operacionais. De seguida, a operação de Projeção (π) "corta" a tabela verticalmente para devolver apenas os dois atributos exigidos pelo operador.

2. Consulta de *Stock* Específico Pergunta de Negócio: "Qual é a quantidade de inventário disponível para o produto com o código de barras '560123' na loja número 1?" **Objetivo:** Provar a junção entre o catálogo de produtos e os dados logísticos de uma filial.

$$\pi_{\text{quantidade}}\big(\sigma_{\text{codigo_barras} = 560123 \ \text{ and } \ \text{id_loja} = 1}(\text{Inventario} \bowtie \text{Produto})\big)$$

Justificação: Através da Junção Natural ($\bowtie$), o sistema cruza as chaves primárias e estrangeiras entre o `Produto` e o `Inventario`. Aplica-se então a dupla restrição lógica para isolar a loja e o artigo pretendido.

3. Auditoria de Operadores de Caixa Pergunta de Negócio: "Quais os nomes e cargos dos funcionários que registaram transações de venda no dia 20 de maio de 2026?" **Objetivo:** Provar o rastreio da intervenção humana nas vendas.

$$\pi_{\text{nome, cargo}}\big(\sigma_{\text{data_hora} \geq \text{2026-05-20} \ \text{ and } \ \text{data_hora} < \text{2026-05-21}}(\text{Funcionario} \bowtie \text{Venda})\big)$$

Justificação: O modelo permite descobrir a autoria dos atos transacionais unindo a tabela `Venda` com a tabela `Funcionario` que efetuou a mesma. A Projeção esconde as credenciais (`password_hash`) por motivos de segurança, revelando apenas a identidade visual.

4. Pesquisa de Fornecedores por Categoria Pergunta de Negócio: "Quais são os parceiros comerciais (nomes das empresas) que nos abastecem com artigos da secção 'Laticínios'?" **Objetivo:** Provar a robustez das tabelas associativas ($N : M$) para saltar múltiplos contextos na base de dados.

$$\pi_{\text{nome}}\big(\sigma_{\text{designacao} = \text{Laticinios}}(\text{Categoria} \bowtie \text{Produto} \bowtie \text{Produto_Fornecedor} \bowtie \text{Fornecedor})\big)$$

Justificação: Esta é uma extração complexa que confirma a eficácia estrutural da base de dados. A travessia começa na `Categoria`, une-se ao `Produto`, passa pela tabela intermédia de cardinalidade e desagua na entidade `Fornecedor`, permitindo à Direção analisar facilmente as suas fontes logísticas.

5. Verificação de Descontos Aplicáveis Pergunta de Negócio: "Que campanhas de promoção ativas incidem diretamente sobre o produto "Água 1L" (código "12345")?" **Objetivo:** Provar o motor de preços do Ponto de Venda (POS).

$$\pi_{\text{designacao, desconto}}\big(\sigma_{\text{estado} = \text{Ativa} \ \text{ and } \ \text{codigo_barras} = 12345}(\text{Promocao} \bowtie \text{Promocao_Produto} \bowtie \text{Produto})\big)$$

Justificação: Para que a caixa registadora desconte valores automaticamente, ela necessita de unir os dados da tabela mestre `Promocao` à tabela `Produto` através da associativa `Promocao_Produto`. A expressão prova que o modelo suporta a identificação imediata da taxa de desconto em vigor.

5.2.2. Testes unitários (cenários reais do cliente)

A equipa transladou as *User Stories* originais, arquitetadas na fase embrionária de conceção, para o formato de fluxos automatizados de teste. Esta conversão técnica teve como propósito primordial a replicação exata e ininterrupta dos cenários reais de utilização experienciados pelos operadores e gestores do negócio. A literatura classifica estes ensaios como testes de ponta-a-ponta (*End-to-End* ou E2E), uma vez que simulam o percurso transacional do utilizador no sistema desde a primeira interação até à conclusão de um ciclo funcional completo.

Através da instanciação direta da fachada unificadora (`TakiLN`), a suíte de testes E2E orquestra e avalia as seguintes operações encadeadas de elevada criticidade:

- **Gestão do Catálogo e Inventário**: Valida-se o registo orgânico de um funcionário com perfil de Gestor. Este ator prepara a entrada inicial de *stock* na base de dados, cria categorias de produtos e adiciona novos artigos ao inventário, com taxas de IVA associadas;
- **Transação em Ponto de Venda (POS)**: A simulação passa o controlo para um Operador de Caixa que acede ao sistema com as suas credenciais. O teste encadeia a abertura de uma venda, a leitura simulada do código de barras dos produtos, a adição ao carrinho e o apuramento milimétrico do preço total com impostos calculados em tempo real;
- **Tratamento de Exceções em Ambiente Produtivo**: A bateria de testes não simula apenas caminhos de sucesso. Forçam-se cenários de rutura de *stock*, nos quais o cliente tenta adquirir um volume de produtos superior ao limite físico existente na loja. O ensaio confirma que a aplicação deteta a falha na barreira imposta pelo gatilho lógico (*Trigger*) do *PostgreSQL* e trava a transação comercial com eficácia;
- **Logística de Devoluções**: Testa-se a lógica de reembolso mediante a devolução de um artigo nas horas posteriores à compra original. O código de validação confirma que a plataforma anula os lucros operacionais e emite de imediato um comando atómico para a reposição exata das unidades de volta à prateleira.

A execução contínua e sem sobressaltos desta prova empírica atesta de forma inequívoca que a solução concebida não apresenta anomalias de integração estrutural. O sucesso desta validação atesta a capacidade dos variados módulos do sistema em comunicarem entre si de forma articulada e fluída, para entregarem o valor de negócio perspetivado no caderno de encargos original.

5.2.3. Heurísticas de Nielsen

Para garantir que o Sistema de Gestão da Taki cumpre o rigoroso requisito de usabilidade — permitindo que um novo operador de loja aprenda a utilizar a plataforma em menos de duas horas —, o desenho da interface de utilizador (UI) e a experiência de navegação (UX) foram orientados pelas 10 Heurísticas de Usabilidade de Jakob Nielsen.

A aplicação destes princípios globais garante uma interação fluida, minimiza a curva de aprendizagem e reduz drasticamente a taxa de erro operacional na frente de loja e no Backoffice:

1. Visibilidade do estado do sistema:

A plataforma deve manter o utilizador continuamente informado sobre os processos em curso. No contexto da Taki, isto traduz-se na exibição clara do estado da ligação (sincronização com o servidor central versus modo offline local), na identificação do operador com sessão iniciada e na atualização em tempo real do subtotal do carrinho de compras no Ponto de Venda (POS).

2. Correspondência entre o sistema e o mundo real:

A linguagem da interface foi desenhada em português de Portugal (pt-PT) simples e direto, erradicando jargões técnicos de base de dados ou arquitetura de *software*. A iconografia reflete o ambiente físico de retalho (ex: recurso a ícones de códigos de barras, caixas de inventário e recibos).

3. Controlo e liberdade do utilizador:

Os operadores de caixa necessitam de mecanismos rápidos para reverter ações não intencionais num ambiente de ritmo acelerado. O sistema prevê "saídas de emergência" acessíveis, como botões de cancelamento de transação, remoção rápida de artigos do carrinho e opções de "Voltar" a um máximo de 2 a 3 cliques de distância.

4. Consistência e normas:

O design do sistema obedece a padrões estritos de localização e cor. As ações principais de submissão (ex: "Pagar" ou "Guardar") encontram-se padronizadas no canto inferior direito com uma paleta de cores de confirmação (verde), enquanto ações destrutivas (ex: "Cancelar" ou "Remover Fornecedor") partilham a mesma iconografia de alerta (vermelho) em todos os ecrãs.

5. Prevenção de erros:

Para mitigar falhas críticas que afetem a faturação ou o stock, a interface previne ações erróneas antes que estas ocorram. São exigidas validações (janelas de confirmação) para operações destrutivas, como a anulação de uma venda já registada ou a descontinuação de um produto do catálogo.

6. Reconhecimento em vez de memorização:

O sistema diminui a carga cognitiva dos funcionários ao dispensar a memorização de códigos numéricos de artigos (SKUs). O terminal POS disponibiliza uma grelha visual rápida com categorias, cores e fotografias dos produtos de alta rotação.

7. Flexibilidade e eficiência de utilização:

A arquitetura da interface atende tanto a novos utilizadores como a gestores experientes. No ecrã de vendas, a interação principal é otimizada para a leitura rápida via scanner de códigos de barras, mas oferece atalhos de teclado configuráveis para registos ágeis, garantindo transações em menos de 10 segundos.

8. Estética e design minimalista:

Seguindo o lema da organização ("Não inventes, Taki!"), as interfaces são desprovidas de elementos decorativos ou informações secundárias irrelevantes. O dashboard e o POS maximizam o espaço em branco, focando a atenção visual estritamente nos dados vitais para a tarefa em execução.

9. Ajudar os utilizadores a reconhecer, diagnosticar e recuperar de erros:

Sempre que uma exceção é gerada (ex: tentar adicionar um artigo sem stock ou falha na comunicação de pagamento), o sistema apresenta mensagens de erro em linguagem natural e propõe uma via construtiva de resolução (ex: "Produto esgotado. Deseja verificar alternativa no catálogo?"), em detrimento de códigos de erro técnicos.

10. Ajuda e documentação:

Apesar do foco na intuitividade e aprendizagem autónoma, o sistema inclui auxílios contextuais subtis, tais como pequenas descrições (*tooltips*) que surgem ao passar o cursor sobre ícones estatísticos no dashboard do gestor, facilitando o enquadramento de novos colaboradores sem necessidade de consulta a manuais externos extensos.

5.2.4. Reunião de Aceitação

A Reunião de Aceitação constituiu um ponto de controlo fundamental no ciclo de desenvolvimento do Sistema de Gestão da *Taki*, momento que reuniu a equipa ***SegFault*** e a administração da empresa. O principal objetivo desta sessão consistiu na avaliação do progresso alcançado e na validação da arquitetura de *software* projetada.

Durante a apresentação, a estrutura e a solução propostas foram recebidas de forma muito positiva pelo cliente, que confirmou o pleno alinhamento do desenho técnico com os objetivos de negócio e operacionais das lojas. Contudo, o processo de validação permitiu identificar uma dificuldade de interpretação por parte dos *stakeholders* em relação aos diagramas UML (classes e sequência), devido à sua natureza estritamente técnica.

Para colmatar esta falha de comunicação, a equipa adotou uma postura ágil e comprometeu-se a enriquecer a documentação com uma componente textual simplificada e descritiva, elaborada para traduzir a notação técnica em linguagem de negócio. Adicionalmente, estabeleceram-se diretrizes para que os futuros reportes de estado do projeto adotem um formato mais executivo. A reunião culminou com a aprovação formal da arquitetura e com o parecer favorável ao avanço para a fase de implementação.

Sistema de Gestão da *Taki* - Reunião de Aceitação
Local: Sede da *Taki*
Data: 07 de maio de 2026
Duração: 1h e 30 minutos
Início: 10h00 | **Término:** 11h30

Participantes:

- Sr. Manuel Pinto da Costa (Sócio-Gerente) e Sócios Gestores (Tiago Rocha, Sofia Alves, Carolina Jesus, David Marques)
- Diogo Campos, João Delgado, Nelson Mendes, Simão Mendes, Tomás Machado (Equipa *SegFault*)

Pauta da reunião e Conclusões:

- Apresentação do estado atual do projeto e detalhe do desenho da arquitetura de *software*;
- Identificação de dificuldades de interpretação dos diagramas técnicos por parte dos *stakeholders*;
- Decisão de desenvolver uma componente textual descritiva para traduzir a notação técnica para os intervenientes não-técnicos;
- Definição de um formato de relatório mais focado no negócio para futuras atualizações;
- Aprovação formal da arquitetura e parecer favorável para o avanço da implementação.

Figura 114: Resumo da Ata de Reunião de Aceitação. A versão integral do documento encontra-se documentada no **Anexo 10**.

5.3. Avaliação da Qualidade

A aferição da qualidade do produto transcende o mero sucesso funcional e foca-se na avaliação dos pilares de segurança, de resiliência sob pressão e de manutenibilidade da base de código. Esta bateria de auditorias não funcionais certifica a robustez arquitetural da plataforma num ambiente operacional exigente.

Implementámos uma suite de testes de desempenho para o sistema Taki, utilizando ferramentas de simulação para avaliar a robustez das APIs REST sob condições de utilização intensiva. Para facilitar a execução técnica, foi integrada a task `performanceTest` no ficheiro `build.gradle`, permitindo a execução isolada via terminal: `./gradlew performanceTest .`

5.3.1. Testes de carga (eficiência)

O teste de carga, implementado em `LoadPerformanceTest.java`, visou validar a estabilidade do sistema perante um cenário de utilização nominal elevada.

- **Objetivo:** Simular a atividade simultânea de 50 operadores de caixa;
- **Fluxo de Teste:** Cada utilizador virtual realiza a autenticação seguida de múltiplas consultas ao catálogo de produtos através do endpoint `/api/local/produtos`;
- **Correção Técnica:** Para garantir a estabilidade, o mecanismo de autenticação nos testes foi atualizado para utilizar o header `Authorization: Bearer <token>`, assegurando a persistência da sessão independentemente da gestão de *cookies* do cliente de teste.

Cenário: 50 utilizadores concorrentes a realizar um total de 1000 pedidos.

Configuração: Utilização do header `Authorization: Bearer` para garantir a persistência da sessão.

Sucesso	Throughput	Latência Média	Status
100%	10.96 req/s	4527 ms	**ESTÁVEL**

- **Resultados:** O sistema demonstrou total estabilidade com uma taxa de sucesso de $\mathbf{100\%}$ e latência controlada (3.5s para o fluxo completo).

5.3.2. Testes de stress (fiabilidade)

O teste de stress visou identificar a curva de degradação do sistema até aos 500 utilizadores simultâneos.

Metodologia: Incremento progressivo de carga para observar o comportamento do pool de ligações e a gestão de exceções.

Utilizadores	Sucesso	Tempo Total	Status
50	100%	8222 ms	SATURATED
250	100%	24599 ms	SATURATED
500	100%	42460 ms	SATURATED

Análise de Saturação:

Os resultados revelam que o sistema é extremamente resiliente, mantendo uma **taxa de erro de 0%** mesmo sob pressão máxima. Contudo, o estado de **Saturação** é atingido precocemente devido à contenção na `HikariPool` (gestão de ligações à BD) e ao limite de processamento paralelo do servidor `Jetty`.

A latência escala linearmente com o número de utilizadores: enquanto com 50 utilizadores o sistema responde em cerca de 8 segundos, com 500 utilizadores esse tempo sobe para 42 segundos.

Apesar deste decréscimo de performance sob cargas extremas, é de salientar que a uma simulação de 50 utilizadores simultâneos o sistema satisfaz integralmente o Requisito Não Funcional 5 que estipula o processamento da transação num limite inferior a 10 segundos. Este cenário de carga é mais do que suficiente para a realidade da Taki, visto que não se prevê a operação simultânea de mais de 2 a 3 caixas registadoras por loja.

5.3.3. Testes de segurança

A salvaguarda da informação central da rede de franquias e a proteção da identidade dos funcionários representam pilares inegociáveis na arquitetura do 'Taki'. Para atestar a fiabilidade dos mecanismos defensivos desenhados, a equipa implementou um conjunto exaustivo de testes direcionados para as falhas de segurança mais comuns em aplicações distribuídas. O resultado prático consubstancia-se numa suíte rigorosa de avaliações algorítmicas, divididas em três frentes essenciais:

- **Criptografia de Credenciais e Proteção contra Força-Bruta**: A plataforma recusa o armazenamento em texto limpo (*plain text*). Todas as credenciais de acesso sujeitam-se ao algoritmo de cifra de sentido único *BCrypt*, enriquecido com a junção de *salts* (sementes aleatórias dinâmicas). De forma empírica, a bateria de testes submete senhas idênticas de dois utilizadores distintos e valida que os *hashes* matemáticos resultantes na base de dados são radicalmente diferentes. Esta validação anula de base a viabilidade de ataques via *Rainbow Tables*. Adicionalmente, os testes de integração do Sistema de Caixa validam que intrusões continuadas com senhas erróneas desencadeiam o bloqueio orgânico e imediato do operador;

- **Blindagem de Sessões Baseadas em JWT**: A gestão de autenticação e de transição de estado da API Global exige uma camada de verificação sem falhas. O código fonte sujeita a utilidade de processamento de *Tokens* (JWT) a diversas tentativas de intrusão forçada. Injetam-se assinaturas assimétricas falsificadas (*spoofing* de chaves RS256), fornecem-se pacotes temporais já ultrapassados (expirados) e adulteram-se as secções de escopo (*scopes* de permissão) para tentar forçar uma elevação de privilégios não autorizada. O sistema, em todos os cenários testados com sucesso, recusa o processamento do pedido e emite um alerta de violação de acesso;
- **Imunidade a Injeções de Código (*SQL Injection*)**: A defesa contra ataques de injeção direta de comandos arbitrários na base de dados atinge-se através de um *design* estrutural restritivo. A camada de acesso aos dados obriga o uso compulsivo e universal de *Prepared Statements*. Os ensaios de integração comprovam que mesmo a introdução de carateres perigosos (como aspas simples ou fechos de instrução com ponto e vírgula) em campos de texto, tais como as descrições de produtos ou os nomes dos fornecedores, não causa a quebra semântica das consultas SQL subjacentes, comportando-se o sistema de forma segura.

5.3.4. Métricas de Qualidade (ISO/IEC 25010)

A norma ISO/IEC 25010 define um modelo de qualidade que permite avaliar o *software* de forma multidimensional. O sistema Taki foi avaliado segundo este modelo, garantindo que as necessidades dos utilizadores finais e os requisitos técnicos de engenharia são satisfeitos.

1) Funcionalidade

O *software* cumpre os objetivos de negócio propostos:

- **Completitude:** Implementa o ciclo completo de venda (POS), gestão de inventário com alertas de segurança, e a componente fiscal (emissão de faturas SAF-T);
- **Correção:** A lógica de cálculo financeiro (impostos, subtotais e promoções) é validada por testes unitários exaustivos.

2) Confiabilidade

O sistema demonstra maturidade na gestão de falhas:

- **Tolerância a falhas:** O uso do padrão *Outbox* na sincronização garante que, em caso de falha de rede, os dados locais não se perdem e são reenviados assim que a ligação é restabelecida;
- **Recuperabilidade:** A gestão manual de transações no VendaDAO assegura que operações complexas (venda + baixa de *stock*) são atómicas: ou ocorrem na totalidade ou é feito um *rollback* completo.

3) Usabilidade

O *frontend* desenvolvido em Vue.js prioriza a experiência do utilizador:

- **Operacionalidade:** O ecrã de POS (POSScreen.vue) foi desenhado para rapidez de operação. A organização em componentes reutilizáveis mantém a consistência visual em todo o sistema;
- **Apreensibilidade:** A interface segue padrões modernos de *dashboarding*, facilitando a aprendizagem por novos funcionários.

4) Eficiência de Desempenho

A escolha da *stack* tecnológica favorece a performance:

- **Comportamento temporal:** O uso de `Javalin` e `JDBC` direto resulta numa `API` de baixíssima latência e arranque quase instantâneo;
- **Utilização de recursos:** A sincronização periódica ou noturna evita picos de processamento durante as horas de maior movimento na loja, preservando os recursos para as vendas.

5) Segurança

A proteção de dados é uma prioridade evidente no código:

- **Confidencialidade:** Utilização de *tokens* `JWT` assinados com `RS256` (`RSA-256`), garantindo que a identidade dos utilizadores não pode ser forjada. Para além disso, as *passwords* são armazenadas com *salting* e *hashing* (`BCrypt`);
- **Integridade:** O sistema de faturas utiliza assinaturas encadeadas, onde cada fatura contém a *hash* da anterior, impedindo a adulteração de registos históricos;
- **Responsabilização:** O Controlo de Acesso Baseado em Perfis (`RBAC`) restringe operações sensíveis a perfis autorizados (ex.: GERENTE ou ADMIN).

6) Compatibilidade

O sistema foi desenhado para coexistir em ambientes distribuídos:

- **Coexistência:** A arquitetura de duas `APIs` independentes (Local e Global) permite que o *software* corra em simultâneo no mesmo servidor físico se necessário, sem conflitos de recursos;
- **Interoperabilidade:** A exposição de serviços via `REST JSON` garante que qualquer cliente (*Web*, *Mobile* ou integração externa) possa comunicar com o *backend* da `Taki`.

7) Manutenibilidade

- **Modularidade:** Divisão clara entre api, ln e dao;
- **Testabilidade:** Cobertura robusta com 40 ficheiros de teste e uso de *Testcontainers*;
- **Analizabilidade:** Interfaces públicas totalmente documentadas com `Javadoc`.

8) Portabilidade

A facilidade de transferência de ambiente é garantida pela contentorização:

- **Adaptabilidade:** O uso de `Docker` e `Docker Compose` permite que todo o ecossistema (Base de Dados + *Backend* + *Frontend*) seja instanciado em segundos em qualquer sistema operativo;
- **Facilidade de Instalação:** A configuração via variáveis de ambiente (`TAKI_NODE_ROLE`) permite que o mesmo binário `Java` se comporte como um nó de Loja ou como a Sede, bastando alterar uma *flag* de configuração.

Em suma, o sistema `Taki` apresenta um **equilíbrio notável entre as 8 características da `ISO/IEC` 25010**. Destaca-se especialmente na Segurança e na Manutenibilidade, que são os pilares de um sistema de gestão empresarial crítico, garantindo que o *software* é não só robusto contra ataques, mas também resiliente e fácil de evoluir perante novas necessidades de negócio.

5.3.5. Síntese dos Resultados da Verificação e Validação

A fase de **Verificação e Validação (V&V)**, complementada pela **Avaliação de Qualidade**, provou de forma inequívoca a robustez, a segurança e o alinhamento do Sistema de Gestão da *Taki* com os requisitos de negócio estipulados. O recurso a metodologias de teste exaustivas permitiu consolidar a arquitetura e eliminar potenciais anomalias antes da transição para um ambiente produtivo.

No âmbito da **Verificação**, a execução de **testes unitários à Lógica de Negócio**, efetuada com recurso a **`JUnit 5`** e **`Mockito`**, assegurou a precisão dos cálculos financeiros e das regras operacionais num ambiente isolado. De igual modo, os testes de integração aplicados aos ***Data Access Objects (DAOs)***, orquestrados através de contentores **`Docker`** com **`PostgreSQL`**, atestaram a atomicidade das transações e a fiabilidade da comunicação com a lógica ativa, onde se incluem *triggers* e vistas materializadas. A consolidar este esforço, a análise estática ao código e as baterias preliminares de testes *End-to-End* (E2E) aplicadas à Interface de Utilizador (UI) validaram a ausência de vulnerabilidades e confirmaram a estabilidade estrutural dos componentes visuais.

No que concerne à **Validação**, o sistema demonstrou total aderência aos cenários operacionais exigidos pela rede de lojas. A adequação da base de dados relacional ficou matematicamente provada através de expressões de **Álgebra Relacional**, que demonstraram a capacidade do modelo para responder a perguntas reais da administração. A **simulação das *User Stories*** em formato E2E certificou o correto funcionamento dos fluxos críticos de trabalho, nomeadamente a gestão de catálogo, o processamento atómico de vendas no POS, a logística de devoluções e a emissão de alertas de stock. Adicionalmente, a conformidade da interface com as **Heurísticas de Nielsen** assegura uma experiência de utilização intuitiva e de rápida aprendizagem para os novos operadores de loja. Este alinhamento técnico e funcional culminou na satisfação dos *stakeholders*, formalizada na fase de aceitação.

Por fim, a **Avaliação da Qualidade** confirmou o grau de excelência exigido pelo referencial **ISO/IEC 25010**. As auditorias de segurança provaram a **imunidade do sistema contra injeções SQL**, a resiliência na **proteção de credenciais** com algoritmos irreversíveis de *hashing* dinâmico e a segurança na transmissão de dados via *tokens* `JWT`. Os testes de carga e de stress atestaram o comportamento temporal, a eficiência de desempenho e a tolerância a falhas do sistema sob elevada pressão transacional.

Em suma, os resultados obtidos atestam que o *software* produzido se encontra tecnicamente maduro e apto a suportar as operações vitais da `Taki`, de forma a salvaguardar a proteção da informação e a autonomia plena das respetivas filiais.

5.4. Utilização de Inteligência Artificial

A fase de Verificação e Validação (V&V) é, indiscutivelmente, a etapa mais crítica para assegurar a fiabilidade e a robustez de um sistema de gestão retalhista como o `Taki`. Nesta fase, a utilização de Inteligência Artificial ultrapassou a simples assistência à escrita, e transformou-se num mecanismo ativo de auditoria e inspeção de código.

5.4.1. O Impacto de Modelos Avançados na Validação

Ao contrário da fase de prototipagem, na qual a IA é frequentemente tolerada a gerar abstrações genéricas, a validação de *software* exige um grau de precisão milimétrico. A capacidade de um modelo de linguagem para inferir casos limite (*edge cases*), detetar vulnerabilidades de segurança e simular condições de corrida depende diretamente do seu poder de raciocínio. A transição metodológica para os modelos mais avançados da família Claude (através do *Claude Code*) fez uma diferença tremenda neste aspeto da engenharia.

Modelos com arquiteturas mais fracas tendem a sugerir testes unitários lineares, focados apenas no "caminho feliz" (*happy path*), e ignoram anomalias complexas que surgem em ambientes concorrenciais (como a plataforma de sincronização entre lojas e o servidor central). Por oposição, a elevada janela de contexto e a acuidade lógica do *Claude Code* viabilizaram uma análise holística do código-fonte. O modelo demonstrou capacidade para ler o código de implementação Java, cruzar essa informação com os requisitos definidos em Typst e produzir baterias de testes JUnit 5 e Mockito para assegurar a cobertura de injeções de falhas, exceções de base de dados e cenários de carga.

A IA operou como um auditor de segurança e analista de qualidade, de forma a garantir que o código não só cumpria os requisitos funcionais, mas também resistia a ataques ou utilizações indevidas.

5.4.2. Metodologia de Validação e Prompts Chave

O recurso à IA nesta etapa focou-se em três eixos centrais: a auditoria preventiva de segurança, a otimização de consultas críticas de base de dados e a expansão da cobertura de testes automatizados. Apresentam-se em seguida as diretrizes fornecidas ao modelo para atestar o rigor do processo de validação:

Verificação de Segurança e Integridade

A segurança dos dados operacionais e das identidades dos utilizadores motivou uma análise profunda aos métodos criptográficos e de acesso:

> "Atua como um Auditor de Segurança de Aplicações. Explica a diferença prática entre armazenar uma palavra-passe com o recurso a algoritmos SHA256 versus Bcrypt no contexto do nosso sistema de gestão de retalho. Analisa o nosso código atual de autenticação e indica como devemos implementar a validação técnica no *backend* para proteger as rotas administrativas e assegurar que as assinaturas criptográficas (*hashes*) se mantêm resistentes a ataques de força bruta e *rainbow tables*."

Para garantir a coerência dos dados no patamar mais baixo do sistema, a IA serviu o propósito de inspecionar os esquemas relacionais de forma defensiva:

> "Inspeciona este esquema SQL referente à ligação entre a tabela de 'Funcionario' e 'Loja'. Avalia as potenciais quebras de integridade referencial. Quais são as *constraints* e validações que devemos adicionar imperativamente ao esquema para garantir que a eliminação de um funcionário numa loja não corrompe o histórico de faturas, e que o estado do funcionário reside estritamente confinado aos valores 'Ativo', 'Inativo' ou 'Bloqueado'?"

Validação da Camada de Apresentação e Dados

No âmbito da usabilidade e performance, o modelo atuou na validação dos fluxos criados para atestar a sua viabilidade em produção:

> “Atua como um engenheiro de *Quality Assurance* (QA) especializado em *frontend*. Avalia o componente Vue.js responsável pelo fecho de caixa. Quais são os cenários de falha na introdução manual de valores que este código não acautela? Propõe cenários de teste automatizado (com Vitest) para assegurar que a interface reage corretamente a dados inválidos e a falhas de rede durante a submissão.”

> “Para validar a resiliência da base de dados, analisa a instrução SQL responsável pela extração do relatório anual de vendas. Dado que a tabela ‘LinhaVenda’ terá milhões de registos, este comando irá provocar uma clara degradação de performance. Apresenta as instruções de validação de métricas com o comando ‘EXPLAIN ANALYZE’ no PostgreSQL e propõe os índices adequados para baixar o custo de execução da consulta para valores aceitáveis num ambiente de produção.”

Com esta abordagem, a IA não gerou apenas código de teste, mas moldou a cultura de validação do projeto, ao forçar a equipa a pensar proativamente na falha antes da sua ocorrência no sistema em produção.

6. Conclusões e Trabalho Futuro

O desenvolvimento do **sistema `Taki` demonstra um compromisso rigoroso com os princípios da Engenharia de *Software***, o que resulta numa plataforma de gestão de franquias que é simultaneamente funcional, segura e tecnicamente avançada.

6.1. Pontos Fortes

A **arquitetura desenvolvida é modular e foi pensada para ser escalável** desde a sua conceção. A adoção do padrão *Facade*, aliada a uma separação clara do código em camadas bem definidas: API, lógica de negócio (LN) e acesso a dados (DAO), permitiu que **subsistemas complexos fossem desenvolvidos em paralelo**, sem que houvesse interferência ou dependências excessivas entre os desenvolvedores. Além disso, a **dualidade de papéis entre os modos Local e Global**, resolvida através de uma simples configuração, demonstra que o *design* foi planeado para suportar a escalabilidade horizontal de forma elegante e prática.

No que diz respeito à **segurança e à integridade dos dados**, o sistema mostrou estar a um elevado patamar de maturidade profissional. A implementação de assinaturas `RS256` para os *tokens* `JWT` garante a autenticidade e a não-repudiação das comunicações. Mais ainda, o **encadeamento de *hashes* nas faturas, pensado para estar em conformidade com os requisitos do `SAF-T`**, eleva o projeto a um patamar de maturidade profissional, para assegurar que a informação financeira não pode ser adulterada sem deteção.

A **cultura de testes é igualmente robusta**. Com 40 ficheiros de teste, incluindo testes de integração que recorrem a bases de dados reais geridas pelos *Testcontainers*, o sistema ganha uma estabilidade única. Este rigor permite que novas funcionalidades sejam introduzidas com grande confiança e com um risco reduzido de regressões, o que facilita a evolução contínua do produto.

Por fim, a **documentação técnica** foi tratada como um elemento estratégico. A opção de documentar de forma exaustiva todas as interfaces públicas garante que o contrato entre as diferentes camadas da aplicação é claro, inequívoco e facilmente compreensível. Isto não só agiliza a integração de novos membros na equipa, como também reduz o atrito na manutenção e evolução do código por diferentes pessoas ao longo do tempo.

6.2. Pontos A Melhorar

Apesar das inegáveis qualidades do sistema, identificam-se também alguns pontos críticos que merecem reflexão e melhoria contínua.

Em primeiro lugar, verifica-se uma acumulação de responsabilidades na camada `API`. O `LocalApiController`, em particular, **tornou-se um ponto de congestão de lógica**, com a concentração em si de funções de mapeamento entre objetos, validação de dados de entrada e orquestração de fluxos de negócio. **Esta abordagem, embora prática e eficiente, representa uma fragilidade significativa no que toca à separação de preocupações a longo prazo.** A tendência natural para o crescimento do controlador poderá levar a fragilidades na separação de preocupações a longo prazo.

Em segundo lugar, o **sistema evidencia uma dependência excessiva de literais**, uma abordagem que poderíamos designar como *stringly-typed*. O uso frequente de cadeias de caracteres para representar estados internos, papéis de utilizador ou tipos de entidades introduz um risco real de erros silenciosos, pois um

pequeno erro num literal, como uma maiúscula ou um espaçamento, pode passar despercebido durante o desenvolvimento e manifestar-se apenas em tempo de execução, com consequências potencialmente graves. Este risco poderia ser amplamente **mitigado com a adoção de tipos mais fortes, como `Enums`** ou classes especializadas, que trariam maior expressividade e segurança ao código.

Por último, a gestão de credenciais merece uma nota de atenção. **A presença de *passwords* de *fallback* diretamente no código-fonte, ainda que justificada pela necessidade de garantir um arranque rápido** e autónomo do ambiente de desenvolvimento, é uma **prática que não deve ser transposta** para cenários de produção. Em contextos reais, a exposição inadvertida destas credenciais constitui uma vulnerabilidade de segurança evitável. Assim sendo, a **adoção de mecanismos de configuração externa**, como variáveis de ambiente ou serviços de segredos, iriam mitigar este aspeto.

6.3. Expansão de Funcionalidades

Para dar continuidade ao crescimento da plataforma, sugerem-se três linhas de evolução que respondem a necessidades reais do negócio, melhoram a experiência dos utilizadores e aproximam o sistema de um modelo mais inteligente e reactivo.

- **Interface Móvel para Inventário**

Uma das evoluções mais imediatas e de maior retorno prático consiste no desenvolvimento de uma aplicação móvel leve, desenhada especificamente para operações de inventário no terreno. Esta **aplicação consumiria a `API Local` já existente**, o que permite que os colaboradores realizassem contagens de *stock* e registassem quebras de produtos diretamente no corredor da loja, com recurso a apenas um dispositivo móvel com câmara. A leitura de códigos de barras, combinada com uma interface simples e focada na tarefa, elimina a necessidade de folhas de papel ou terminais dedicados, com redução de erros humanos e o que acelera significativamente o processo de inventariação. Esta evolução transforma a `API Local` num verdadeiro ponto de apoio para o chão de fábrica (ou corredor da loja), com a extensão do valor do sistema para além da gestão administrativa.

- ***Dashboard* Analítico Mais Avançado**

Paralelamente, a vertente analítica do sistema pode ser elevada a um patamar superior, **com a evolução do atual `EstatisticasService` para incluir algoritmos de previsão de procura**. Com base no histórico de vendas acumulado, seria possível gerar modelos preditivos que antecipam, para cada produto e para cada período, a quantidade provável de vendas futuras. Estes sinais de previsão ajudam os lojistas a tomar decisões mais informadas no momento de encomendar *stock* aos fornecedores, com a redução de ruturas e e mitigação de excessos de inventário. A apresentação destas previsões num *dashboard* analítico avançado, com gráficos, indicadores de confiança e cenários alternativos, transforma o sistema num aliado estratégico da gestão, e não apenas num repositório de dados históricos.

- **Sincronização em Tempo Real**

Por último, com o crescimento da rede de franquias, a necessidade de visibilidade centralizada do *stock* torna-se crítica. O mecanismo de sincronização atual, de natureza periódica, pode ser evoluído para um modelo baseado em eventos, com recurso a tecnologias como *Websockets* ou *Server-Sent Events*. Nesta nova arquitetura, **qualquer alteração de *stock* registada numa loja dispara imediatamente um evento** que é propagado à Sede. Esta visão em tempo real permite à gestão central monitorizar a saúde do inventário em toda a rede, detetar anomalias assim que ocorrem e até desencadear ações automáticas, como por exemplo, alertas de rutura iminente ou transferências rápidas entre lojas. Esta evolução aproxima o sistema de um modelo *real-time first*, onde a latência deixa de ser um compromisso e passa a ser um requisito fundamental.

7. Referências Bibliográficas

1. Sommerville, I 2016, **Software Engineering**, 10th edn, Pearson, Boston.
2. PostgreSQL Global Development Group 2024, **PostgreSQL 16 Documentation**, viewed 13 May 2026, https://www.postgresql.org/docs/16/.
3. Oracle Corporation 2024, **The Java Tutorials: Java SE 21**, viewed 13 May 2026, https://docs.oracle.com/en/java/javase/21/.
4. Docker Inc. 2024, **Docker Documentation**, viewed 13 May 2026, https://docs.docker.com/.
5. Docker Inc. 2024, **Docker Compose Documentation**, viewed 13 May 2026, https://docs.docker.com/compose/.
6. Javalin 2024, **Javalin - A lightweight Java and Kotlin web framework**, viewed 13 May 2026, https://javalin.io/documentation.
7. Gradle Inc. 2024, **Gradle Build Tool Documentation**, viewed 13 May 2026, https://docs.gradle.org/current/userguide/userguide.html.
8. Vite 2024, **Vite - Next Generation Frontend Tooling**, viewed 13 May 2026, https://vitejs.dev/guide/.
9. JUnit Team 2024, **JUnit 5 User Guide**, viewed 13 May 2026, https://junit.org/junit5/docs/current/user-guide/.
10. Git 2024, **Git Documentation**, viewed 13 May 2026, https://git-scm.com/doc.
11. Campos, JC & Ribeiro, AN 2025, **DSS-T03-T04-Modelação do Domínio**, Desenvolvimento de Sistemas de Software, Universidade do Minho.
12. Campos, JC & Ribeiro, AN 2025, **DSS-T05-T06-Use Cases**, Desenvolvimento de Sistemas de Software, Universidade do Minho.
13. Campos, JC & Ribeiro, AN 2025, **DSS-T07-T08-Diagramas de Use Case**, Desenvolvimento de Sistemas de Software, Universidade do Minho.
14. Campos, JC & Ribeiro, AN 2025, **DSS-T09-Diagramas de Actividade**, Desenvolvimento de Sistemas de Software, Universidade do Minho.
15. Campos, JC & Ribeiro, AN 2025, **DSS-T10-APIs e subsistemas**, Desenvolvimento de Sistemas de Software, Universidade do Minho.
16. Campos, JC & Ribeiro, AN 2025, **DSS-T11-T12-Modelação Estrutural**, Desenvolvimento de Sistemas de Software, Universidade do Minho.
17. Campos, JC & Ribeiro, AN 2025, **DSS-T13-T14-Diagramas de Sequência**, Desenvolvimento de Sistemas de Software, Universidade do Minho.
18. Campos, JC & Ribeiro, AN 2025, **DSS-T15-T16-OCL**, Desenvolvimento de Sistemas de Software, Universidade do Minho.
19. Campos, JC & Ribeiro, AN 2025, **DSS-T17-T18-ORM**, Desenvolvimento de Sistemas de Software, Universidade do Minho.
20. Campos, JC & Ribeiro, AN 2025, **DSS-T19-Diagramas de Package**, Desenvolvimento de Sistemas de Software, Universidade do Minho.

21. Campos, JC & Ribeiro, AN 2025, **DSS-T19-T20-Modelação Comportamental - Máquinas de Estado**, Desenvolvimento de Sistemas de Software, Universidade do Minho.

22. Campos, JC & Ribeiro, AN 2025, **DSS-T21-T22-Diagramas de Instalação-Fecho**, Desenvolvimento de Sistemas de Software, Universidade do Minho.

8. Lista de Siglas e Acrónimos

BD Base de Dados
DW *Data Warehouse*
OLTP *On-Line Analyical Processing*
RF Requisito Funcional
RFN Requisito Não Funcional
FN Forma Normal
API *Application Programming Interface*
BD Base de Dados
KPIs *Key Performance Indicators* (Indicadores-Chave de Desempenho)
NIF Número de Identificação Fiscal
POS *Point of Sale* (Ponto de Venda)
RBAC *Role-Based Access Control* (Controlo de Acesso Baseado em Papéis)
SAF-T (PT) *Standard Audit File for Tax Purposes - Portuguese version*
SLA *Service Level Agreement* (Acordo de Nível de Serviço)
TLS *Transport Layer Security*
TPA Terminal de Pagamento Automático
DAO *Data Access Object*
DL *Data Layer*
DML *Data Manipulation Language*
E2E *End-to-End*
ER *Entity-Relationship*
IA Inteligência Artificial
IAG Inteligência Artificial Generativa
ISO/IEC *International Organization for Standardization / International Electrotechnical Commission*
IVA Imposto sobre o Valor Acrescentado
JDBC *Java Database Connectivity*
JWT *JSON Web Token*
LLM *Large Language Model*
LN Lógica de Negócio
LTS *Long-Term Support*
MVCC *Multi-Version Concurrency Control*
RGPD Regulamento Geral sobre a Proteção de Dados
RS256 *RSA Signature with SHA-256*
RSA Rivest–Shamir–Adleman
SGBD Sistema de Gestão de Base de Dados
SQL *Structured Query Language*
UI *User Interface*
UML *Unified Modeling Language*
UUID *Universally Unique Identifier*
1NF Primeira Forma Normal
2NF Segunda Forma Normal
3NF Terceira Forma Normal
CLI *Command-Line Interface*
CSS *Cascading Style Sheets*
CSV *Comma-Separated Values*

DDL *Data Definition Language*
EAN *European Article Number*
ENUM *Enumerated Type*
FK *Foreign Key* — Chave Estrangeira
FT Fatura (prefixo legal SAF-T)
HTTP *Hypertext Transfer Protocol*
JSON *JavaScript Object Notation*
KB *Kilobyte*
LI4 Laboratórios de Informática IV
MB *Megabyte*
PDF *Portable Document Format*
PK *Primary Key* — Chave Primária
RAM *Random Access Memory*
RF Requisito Funcional
RNF Requisito Não Funcional
SHA *Secure Hash Algorithm*
URL *Uniform Resource Locator*
UX *User Experience*
XML *eXtensible Markup Language*

9. Anexos

Anexo 1: Ata de Reunião de Aprovação do Projeto (*Kick-off*)

ATA DA REUNIÃO

DATA: 26 DE FEVEREIRO DE 2026
HORA DE INÍCIO: 13H30

Participantes

- Sr. Manuel Pinto da Costa - Sócio-Gerente da Taki
- Diogo Campos - Eng. de Software da SegFault
- João Delgado - Eng. de Software da Seg Fault
- Nelson Mendes - Eng. de Software da Seg Fault
- Simão Mendes - Eng. de Software da Seg Fault
- Tomás Machado - Eng. de Software da Seg Fault

Descrição da reunião

- Foi realizada uma reunião com o Sr. Manuel Pinto da Costa, sócio-gerente da Taki, com o objetivo de apresentar a arquitetura, rever o cronograma e aprovar o arranque do novo Sistema de Gestão da Taki. Durante a reunião, foram discutidos os seguintes pontos:

Itens de Ação

- **1. Paradigma de Operação Arquitetural**
  - Confirmada a necessidade de bases de dados locais em cada filial (Felgueiras, Madeira e Cabeceiras de Basto) para garantir o funcionamento contínuo, mesmo em caso de falhas de rede.
  - Aprovação da sincronização diária dos dados de vendas e stock com o servidor central sediado em Famalicão.
- **2. Experiência de Utilização e Requisitos de Negócio**
  - Reforçada a importância de um sistema altamente responsivo e de fácil usabilidade para os funcionários, alinhado com a filosofia "Não inventes, Taki!".
  - Foco na rapidez de transação em caixa e na precisão rigorosa do controlo de stock distribuído.
- **3. Revisão do Cronograma de Execução**
  - Validação das fases de desenvolvimento, milestones de entrega e prototipagem.
  - Concordância quanto à obrigatoriedade do cumprimento dos prazos estipulados para assegurar a prontidão do sistema antes da época alta de vendas.
- **4. Aprovação e Transição de Fase**
  - Parecer positivo e aprovação formal do Estudo de Viabilidade arquitetural e orçamental.
  - Autorização oficial para o avanço imediato para a fase de Levantamento e Análise de Requisitos.

Comunicados Finais

- Após a validação das propostas, o Sr. Manuel Pinto da Costa reforçou a confiança na equipa e a expectativa por um sistema robusto e descomplicado.
- A equipa da Seg Fault iniciará de imediato a engenharia de requisitos e a modelação do sistema.
- A reunião foi encerrada às 15h15.

Anexo 2: Ata de Reunião de Levantamento de Requisitos

ATA DA REUNIÃO

DATA: 05 DE MARÇO DE 2026
HORA DE INÍCIO: 13H30

Participantes

- Sr. Manuel Pinto da Costa (Sócio-Gerente da Taki)
- Tiago Rocha (Sócio e Gestor de Famalicão)
- Sofia Alves (Sócia e Gestora de Felgueiras)
- Carolina Jesus (Sócia e Gestora da Madeira)
- David Marques (Sócio e Gestor de Cabeceiras de Basto)
- Equipa SegFault: Diogo Campos, João Delgado, Nelson Mendes, Simão Mendes, Tomás Machado

Descrição da reunião

- Foi realizada uma sessão de trabalho exaustiva com os sócios da Taki, com o propósito de mapear os processos operacionais diários de cada loja e as dores de negócio atuais. O foco central incidiu na identificação de estrangulamentos na frente de caixa e nas falhas do controlo de inventário manual.

Itens de Ação

- **1. Autonomia das Frentes de Loja**
  - Os sócios confirmaram a necessidade absoluta de os Pontos de Venda (POS) operarem de forma totalmente independente do servidor central ao longo do dia.
  - Esta abordagem garante imunidade a falhas de conectividade, com a sincronização de dados a ocorrer exclusivamente no fecho de cada turno.
- **2. Módulo de Vendas e Faturação**
  - O Sr. David Marques sublinhou a exigência de o sistema calcular automaticamente as taxas de IVA e suportar pagamentos em numerário e cartão (TPA).
  - O Ponto de Venda deve assegurar a emissão imediata e sem atrasos do respetivo recibo, para evitar a formação de filas.
- **3. Gestão de Inventário e Logística**
  - O Sr. Tiago Rocha destacou a urgência de o software descontar o stock em tempo real após cada venda e emitir alertas de reposição.
  - A interface local deve permitir a criação autónoma de encomendas aos fornecedores a partir de cada loja, para evitar prateleiras vazias.
- **4. Usabilidade e Controlo de Acessos**
  - A D. Sofia Alves apelou a uma interface minimalista com o intuito de reduzir drasticamente a curva de aprendizagem de novos operadores.
  - Acordou-se a criação de perfis de acesso distintos para proteger os dados sensíveis da gestão e impedir a execução de erros operacionais.

Comunicados Finais

- A administração validou todas as necessidades operacionais descritas ao longo da sessão.
- A equipa SegFault assumiu o compromisso formal de traduzir estes pedidos em Requisitos Funcionais e Não Funcionais rigorosos.
- A reunião foi encerrada às 16h00.

Anexo 3: Ata de Reunião de Validação dos Requisitos

ATA DA REUNIÃO

DATA: 15 DE MARÇO DE 2026
HORA DE INÍCIO: 10H00

Participantes

- Sr. Manuel Pinto da Costa - Sócio-Gerente da Taki
- Diogo Campos - Eng. de Software da SegFault
- João Delgado - Eng. de Software da Seg Fault
- Nelson Mendes - Eng. de Software da Seg Fault
- Simão Mendes - Eng. de Software da Seg Fault
- Tomás Machado - Eng. de Software da Seg Fault

Descrição da reunião

- Reunião de caráter executivo com o propósito de rever, debater e aprovar o documento final de Especificação de Requisitos. A sessão serviu para assegurar que as traduções técnicas elaboradas pela equipa de engenharia correspondem de forma exata às expectativas de negócio da cadeia Taki.

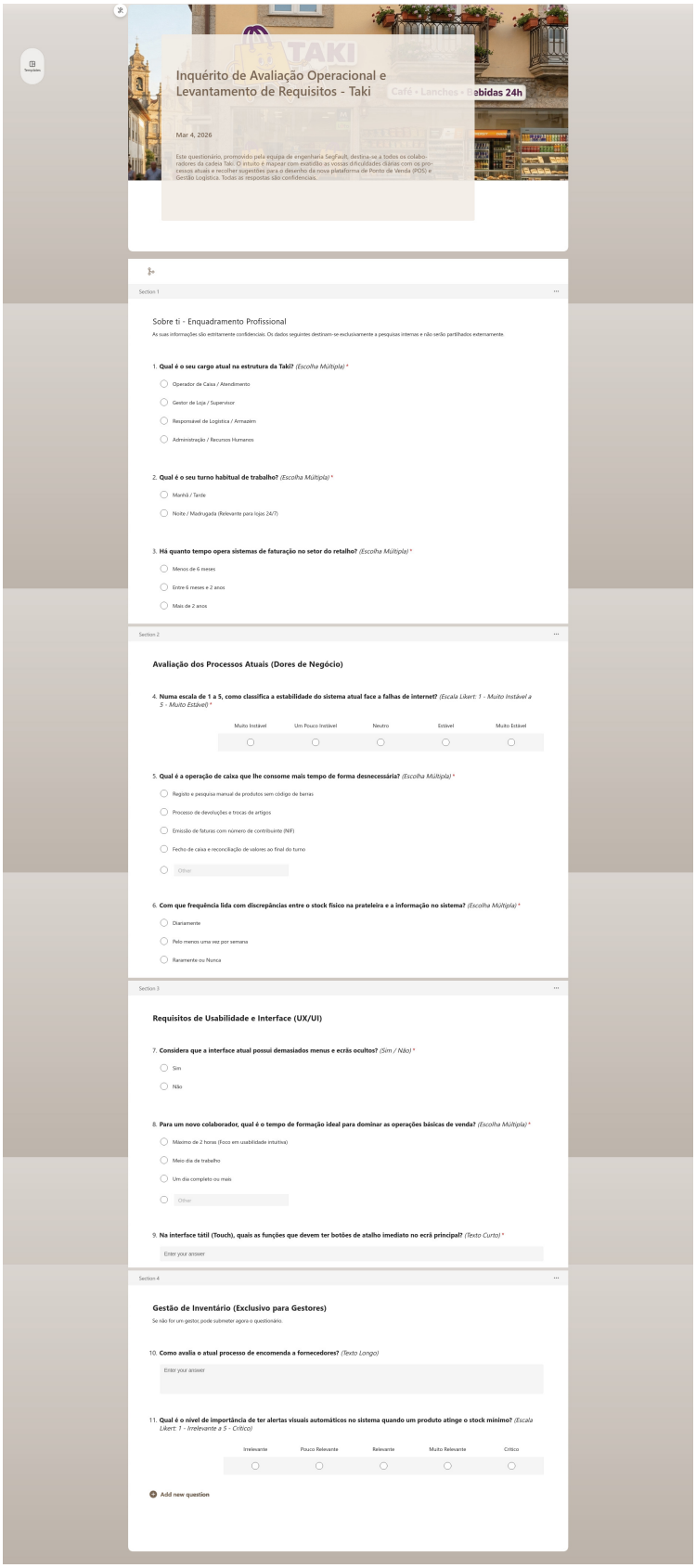
Itens de Ação

- **1. Revisão dos Requisitos Funcionais**
  - A equipa leu e explicou detalhadamente os 20 requisitos funcionais propostos para a operação diária das lojas e do servidor central.
  - O Sr. Manuel confirmou que os processos descritos para a categorização de produtos, devoluções e relatórios de inteligência de negócio refletem o modelo de operação idealizado.
- **2. Conformidade Legal e Segurança**
  - O cliente aprovou a obrigatoriedade de exportação regular da faturação através do formato SAF-T (PT), para garantir a conformidade com a Autoridade Tributária.
  - Houve uma validação do modelo de segurança, com ênfase na encriptação de dados e na execução de backups automáticos cifrados locais e centrais.
- **3. Padrões de Desempenho e Usabilidade**
  - A SegFault assegurou que o sistema processará transações comuns em menos de 10 segundos, para garantir a fluidez nas caixas.
  - Este parâmetro de eficiência, aliado à interface intuitiva, obteve a total concordância da gerência para a continuidade do projeto.
- **4. Transição de Fase de Projeto**
  - Face ao alinhamento total entre a documentação e os objetivos do cliente, o documento de requisitos obteve o parecer positivo e definitivo.
  - A equipa de engenharia recebeu autorização formal para iniciar o desenho da arquitetura, a modelação da base de dados e a prototipagem de interfaces visuais.

Comunicados Finais

- O Sr. Manuel Pinto da Costa expressou grande satisfação com o rigor técnico demonstrado pela equipa.
- A SegFault obteve autorização formal para iniciar o desenho da arquitetura, a modelação da base de dados e a prototipagem de interfaces visuais.
- A reunião foi encerrada às 11h30.

Anexo 4: Inquérito de Avaliação Operacional e Levantamento de Requisitos



Inquérito de Avaliação Operacional e Levantamento de Requisitos - Taki

Mar 4, 2026

Este questionário, promovido pela equipa de engenharia SegFault, destina-se a todos os colaboradores da cadeia Taki. O intuito é mapear com exatidão as vossas dificuldades diárias com os processos atuais e recolher sugestões para o desenho da nova plataforma de Ponto de Venda (POS) e Gestão Logística. Todas as respostas são confidenciais.

Section 1

Sobre ti - Enquadramento Profissional

As suas informações são estritamente confidenciais. Os dados seguintes destinam-se exclusivamente a pesquisas internas e não serão partilhados externamente.

1. **Qual é o seu cargo atual na estrutura da Taki?** *(Escolha Múltipla)* *
   - Operador de Caixa / Atendimento
   - Gestor de Loja / Supervisor
   - Responsável de Logística / Armazém
   - Administração / Recursos Humanos

2. **Qual é o seu turno habitual de trabalho?** *(Escolha Múltipla)* *
   - Manhã / Tarde
   - Noite / Madrugada (Relevante para lojas 24/7)

3. **Há quanto tempo opera sistemas de faturação no setor do retalho?** *(Escolha Múltipla)* *
   - Menos de 6 meses
   - Entre 6 meses e 2 anos
   - Mais de 2 anos

Section 2

Avaliação dos Processos Atuais (Dores de Negócio)

4. **Numa escala de 1 a 5, como classifica a estabilidade do sistema atual face a falhas de internet?** *(Escala Likert: 1 - Muito Instável a 5 - Muito Estável)* *

Muito Instável	Um Pouco Instável	Neutro	Estável	Muito Estável
○	○	○	○	○

5. **Qual é a operação de caixa que lhe consome mais tempo de forma desnecessária?** *(Escolha Múltipla)* *
   - Registo e pesquisa manual de produtos sem código de barras
   - Processo de devoluções e trocas de artigos
   - Emissão de faturas com número de contribuinte (NIF)
   - Fecho de caixa e reconciliação de valores ao final do turno
   - Other

6. **Com que frequência lida com discrepâncias entre o stock físico na prateleira e a informação no sistema?** *(Escolha Múltipla)* *
   - Diariamente
   - Pelo menos uma vez por semana
   - Raramente ou Nunca

Section 3

Requisitos de Usabilidade e Interface (UX/UI)

7. **Considera que a interface atual possui demasiados menus e ecrãs ocultos?** *(Sim / Não)* *
   - Sim
   - Não

8. **Para um novo colaborador, qual é o tempo de formação ideal para dominar as operações básicas de venda?** *(Escolha Múltipla)* *
   - Máximo de 2 horas (Foco em usabilidade intuitiva)
   - Meio dia de trabalho
   - Um dia completo ou mais
   - Other

9. **Na interface tátil (Touch), quais as funções que devem ter botões de atalho imediato no ecrã principal?** *(Texto Curto)* *

   Enter your answer

Section 4

Gestão de Inventário (Exclusivo para Gestores)

Se não for um gestor, pode submeter agora o questionário.

10. **Como avalia o atual processo de encomenda a fornecedores?** *(Texto Longo)*

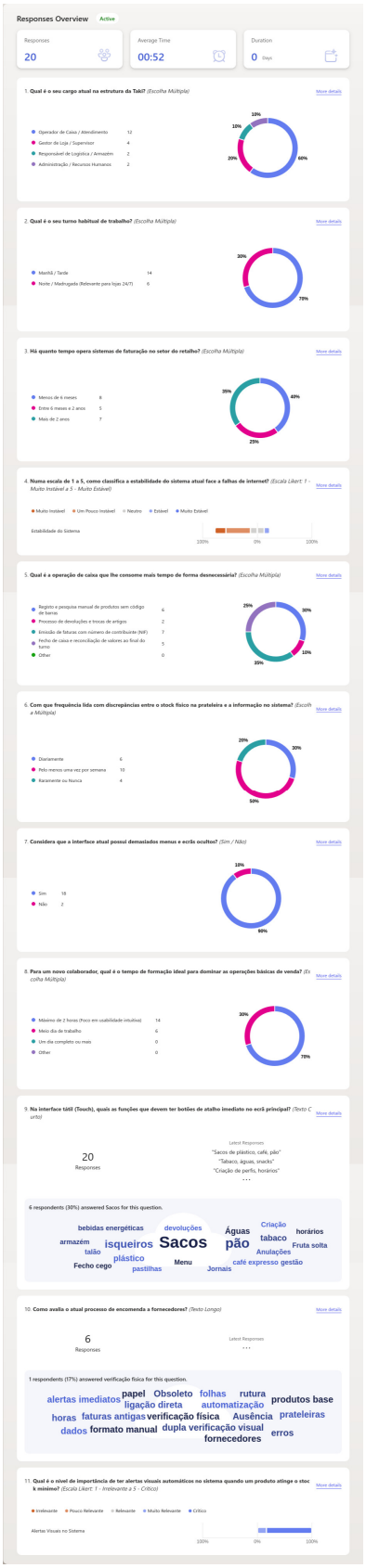
    Enter your answer

11. **Qual é o nível de importância de ter alertas visuais automáticos no sistema quando um produto atinge o stock mínimo?** *(Escala Likert: 1 - Irrelevante a 5 - Crítico)*

Irrelevante	Pouco Relevante	Relevante	Muito Relevante	Crítico
○	○	○	○	○

Add new question

Anexo 5: Resultados do Inquérito de Avaliação Operacional e Levantamento de Requisitos



Anexo 6: Relatório de Pesquisa Externa

Auditoria a Sistemas POS, Arquiteturas Distribuídas e Legislação Fiscal no Retalho

ELABORADO POR:

SEGFAULT SOFTWARE SOLUTIONS

INTRODUÇÃO E METODOLOGIA

1. Introdução

O desenvolvimento de um Sistema de Gestão Integrada para o setor do retalho rápido exige uma compreensão profunda do panorama tecnológico e regulamentar atual. Com o intuito de fundamentar as opções arquiteturais da plataforma Taki, a SegFault conduziu uma investigação externa rigorosa. O foco incidiu na análise de plataformas concorrentes (sistemas de Ponto de Venda líderes no mercado de conveniência), na exploração de arquiteturas resilientes a falhas de conectividade e no mapeamento exaustivo das obrigações fiscais da Autoridade Tributária.

2. Metodologia

Para garantir o rigor técnico deste relatório, a equipa recorreu às seguintes abordagens:

- **Benchmarking Tecnológico:** Avaliação de software de retalho de referência para identificar os padrões da indústria em termos de fluidez de transação (objetivo sub-10 segundos).
- **Análise Documental Legal:** Consulta de diplomas legislativos, com especial destaque para a Portaria n.º 302/2016 e as diretrizes de certificação de software de faturação da Autoridade Tributária e Aduaneira.
- **Auditoria de Cibersegurança:** Revisão das recomendações do Centro Nacional de Cibersegurança (CNCS) aplicadas a redes de retalho distribuídas.

3. Resultados da Pesquisa

3.1. Paradigmas de Arquitetura no Retalho (Edge Computing vs. Cloud)

O estudo demonstrou que as cadeias de conveniência mais eficientes rejeitam a dependência exclusiva de servidores na nuvem (Cloud) para as operações de caixa. A pesquisa valida a adoção de um modelo híbrido:

- **Autonomia Local:** A instalação de uma base de dados em cada filial elimina a latência de rede. O operador de caixa comunica apenas com o servidor físico da loja, para garantir o registo ininterrupto de vendas face a qualquer quebra do fornecedor de internet.
- **Sincronização Assíncrona:** Em vez de transmitir cada venda em tempo real para a sede, os sistemas modernos empacotam os dados e realizam a sincronização no fecho do turno. Esta estratégia de consolidação central reduz a carga nos servidores principais e diminui os custos de infraestrutura.

3.2. Inteligência de Negócio e Gestão de Stock

O mercado dita que a gestão de inventário reativa conduz a quebras de lucro. Os sistemas de topo incorporam módulos proativos. Detetou-se a necessidade de o sistema emitir alertas automáticos de rutura iminente (com base no stock de segurança) e de permitir a emissão direta de notas de encomenda a fornecedores a partir de cada terminal local.

3.3. Conformidade Legal e Fiscal (SAF-T)

Qualquer sistema comercial em Portugal opera sob escrutínio estatal. A pesquisa confirmou a obrigatoriedade dos seguintes mecanismos de conformidade:

- **Exportação SAF-T (PT):** O software deve gerar o ficheiro Standard Audit File for Tax Purposes na sua versão mais recente (1.04_01), compilando de forma imaculada todos os documentos de venda e faturas-recibo.
- **Assinatura Digital de Documentos:** A arquitetura terá de contemplar um algoritmo de controlo (chave pública/privada RSA) para gerar assinaturas únicas para cada fatura, para impossibilitar a adulteração dos valores após a impressão do talão.

3.2. Segurança de Informação e Controlo de Acessos

Face à ameaça crescente de ataques informáticos (ransomware) e à necessidade de proteção de dados (RGPD), as melhores práticas do mercado impõem as seguintes normas:

- **Criptografia de Transportes:** É mandatório o uso do protocolo TLS 1.3 para encriptar os pacotes de dados durante a sincronização entre as filiais e a sede corporativa em Famalicão.
- **Proteção de Credenciais:** As senhas dos operadores nunca devem constar na base de dados em formato de texto limpo (plain text). O sistema deverá aplicar algoritmos de hashing irreversíveis com salt dinâmico.

4. Conclusão

A pesquisa externa consolida a premissa de que a estabilidade do negócio da Taki depende de uma fundação tecnológica descentralizada. A análise ao mercado ditou que a adoção de bases de dados locais com consolidação noturna não é apenas uma preferência funcional, mas sim o padrão-ouro da indústria para prevenir perdas financeiras em cenários de falha de rede. Adicionalmente, os resultados deste estudo serviram de alicerce para a redação dos Requisitos Não Funcionais do projeto. Ao incorporar nativamente as exigências fiscais da Autoridade Tributária e os protocolos de cifra de topo, a SegFault assegura a entrega de um sistema robusto, eficiente e em total conformidade com a lei.

Anexo 7: Diretrizes de Sistema para o *Gemini* (`GEMINI.md`)

Gemini Workspace Configuration: LI4 Franchise Management System

1. Role & Persona

As your Gemini assistant for this project, I will operate as a **Senior Software Engineering Lead**. My primary function is to guide and assist in the development of a comprehensive management solution for a retail franchise, following a structured and rigorous software engineering methodology.

My capabilities extend beyond mere code generation. I am here to assist with:

- **Requirements Engineering:** Eliciting, analyzing, and documenting requirements.
- **System & Architectural Design:** Creating high-level architectural blueprints, detailed design diagrams (UML, ERD, etc.), and data models.
- **Implementation:** Writing clean, maintainable, and well-documented code.
- **Verification & Validation:** Developing test plans, writing automated tests, and ensuring the final product meets the specified requirements.
- **Project Documentation:** Authoring and maintaining all necessary project documentation in the specified `Typst` format.

2. Project Vision & Context

Vision: To engineer a complete software solution for a chain of general-purpose franchise stores. This system will manage core business operations, including but not limited to inventory, sales, customer management, and inter-franchise coordination.

Context: This is a formal academic project for LI4, emphasizing a structured development process. The final output is not just the software itself, but also the comprehensive documentation that chronicles the engineering journey, from initial conception to final validation. The existing file structure and `.typ` files indicate a strong focus on this process.

3. Guiding Principles & Workflow

We will adhere to a structured, phased development lifecycle that resembles a Waterfall model, ensuring that each stage is deliberately and thoroughly completed.

1. **Phase-Based Progression:** All tasks will be contextualized within the current phase of the project (e.g., Requirements Gathering, Analysis, Design, Implementation, Testing). We will not proceed to implementation without a solid foundation in analysis and design.
2. **Documentation as a Deliverable:** All artifacts—user stories, use case diagrams, domain models, architectural diagrams—are critical deliverables. I will assist in creating and refining these documents in the project's `Typst` format. When requested, I can generate diagram descriptions or code (e.g., for PlantUML, Mermaid) to be embedded into the documentation.

3. **Formal & Rigorous Approach:** We will strictly follow the methodologies and principles outlined in Ian Sommerville's "Software Engineering" textbook, located in the project folder. This book will be our primary guide for process and documentation.
4. **Clarity and Traceability:** Every requirement must be traceable through the design, implementation, and test cases.
5. **Collaborative Architectural Decisions & Permissions:** I will NOT make any unilateral decisions regarding architecture, design, or implementation. All such decisions MUST pass through you and be explicitly approved. I will actively propose solutions and point out improvements (even if I think your current decisions or code are weak), but I will NEVER take action on them without your explicit permission.
6. **Test-Driven Approach (Mandatory):** Whenever generating or modifying implementation code, I MUST concurrently provide the corresponding unit tests (using JUnit 5 and Mockito). Code is not considered complete without its accompanying tests, which must follow the Arrange, Act, Assert (AAA) pattern.
7. **Clean Code:** I will prioritize writing code that is clean, modular, properly abstracted, and self-documenting. I will actively avoid overly complex code, deep nesting, and repetitive boilerplate, favoring clear intent and SOLID principles.
8. **Completion Mandate:** I MUST always ensure my responses are complete and fully address the user's request. I will not stop or end a response prematurely, especially not at internal markers like `_end_thought_`.

4. Technical & Documentation Standards

- **Documentation Format:** For any content intended for `.typ` files, I will provide the raw text for you to place in the web-based Typst editor.
- **Technology Stack & Architecture:**
  - The implementation will be centered around **Java** for the core business logic and backend.
  - We use a **Dual REST API Architecture**:
    - **Local API (port 8080):** Serves standard store operations like POS and inventory. Runs in all store profiles (`TAKI_NODE_ROLE=BRANCH` or `HUB`).
    - **Global API (port 8081):** Serves administrative actions and global catalog management. Runs exclusively in the Hub store profile (`TAKI_NODE_ROLE=HUB`).
  - We use **Gradle (`gradlew`)** to build, compile, and manage dependencies for the Java backend. All build instructions should utilize the local wrapper (e.g., `./gradlew build`).
  - The data layer utilizes **PostgreSQL**.
  - The entire ecosystem is orchestrated using **Docker** containers (including local and central databases) and `docker-compose`. Environment variables are the preferred method for injecting dynamic configurations (such as DB credentials and node roles).
  - **Synchronization:** The Hub Global API coordinates the sync flow (outbox pattern) with the Central Server to propagate data chain-wide.

    - **Security & APIs:** Web Sessions are strictly maintained via `HTTPOnly` Cookies (mitigating XSS). JWTs must be signed using asymmetric `RS256` keys. All DTOs transitioning through the Javalin REST APIs MUST be heavily annotated using Bean Validation (JSR 380) rules to guarantee structural integrity.

- **Implementation Rule:** I MUST read and thoroughly understand the corresponding `.typ` requirement files (Functional, Non-Functional, Use Cases) BEFORE writing or suggesting any implementation code. Code must strictly reflect the approved requirements.
- **Code Style & Conventions:** All code will be written with a strong emphasis on readability, maintainability, and consistency, adhering strictly to idiomatic Java conventions.

5. My Commitment

I will act as a proactive partner in this project. I will challenge assumptions, suggest improvements, and ensure that we maintain a high standard of engineering excellence throughout the lifecycle of the project. My goal is to help you deliver a successful project that is not only functional but also well-engineered and thoroughly documented.

6. Communication Style

- **Primary Language:** All communication and generated content will be in English.
- **Translations:** If you request a translation into Portuguese, I will use the Portugal-specific dialect (pt-PT).
- **Grammar:** I will use direct and active language, and strictly avoid continuous tenses (e.g., "-ing" forms) in English, and the "gerúndio" when speaking in Portuguese, unless absolutely necessary.

Anexo 8: Instruções de Contexto para o *Copilot* (`copilot-instructions.md`)

Copilot instructions for LI4_2526

Build, test, and run commands

- Backend (Gradle wrapper):
  - `cd backend && ./gradlew build`
  - `cd backend && ./gradlew test`
  - Single test class: `cd backend && ./gradlew test --tests pt.uminho.taki.ln.vendas.VendaServiceTest`
  - Single test method: `cd backend && ./gradlew test --tests "pt.uminho.taki.ln.vendas.VendaServiceTest.testMethodName"`
- Frontend (Vite):
  - `cd frontend && npm install`
  - `cd frontend && npm run dev`
  - `cd frontend && npm run build`
- Local infrastructure:
  - `docker compose up -d db_local db_central`

No dedicated lint script is currently configured in `frontend/package.json` or `backend/build.gradle`.

High-level architecture

- The repository models a distributed retail system with **local** and **central** data contexts:
  - `docker-compose.yml` runs two PostgreSQL instances: `db_local` (store-side) and `db_central` (headquarters).
  - SQL bootstrapping in `database/` applies shared schema/business logic plus synchronization triggers.
- Backend is organized as a layered Java application exposing two independent REST APIs (Dual API Architecture):
  - **Local API (Port 8080):** Handles store operations (POS, inventory, local supply, sync execution). Runs in all store profiles. Integrates with `pt.uminho.taki.ln.TakiLNLocal`.
  - **Global API (Port 8081):** Handles chain-wide management (global catalog, centralized suppliers, global reports/KPIs). Runs **only** on the Hub store profile. Integrates with `pt.uminho.taki.ln.TakiLNGlobal`.
- **Topology & Deployment Rules:** Controlled by the `TAKI_NODE_ROLE` environment variable:
  - `HUB`: Starts both Local API and Global API. Centralizes global management.

    - `BRANCH`: Starts only Local API. Functions independently for daily operations, syncing with the central server.
- Synchronization uses an **outbox pattern**:
    - Database triggers in `database/sincronizacao_postgres.sql` write changed entity IDs into `Fila_Sincronizacao`.
    - `OutboxDAO` is read/delete oriented (`save` is intentionally unsupported).
    - `SincronizacaoService` exports/imports batches against central HTTPS endpoints and enforces TLS 1.3. Global API coordinates this flow with the Central Server.
- Frontend is Vue 3 + Pinia (`frontend/src/stores/*`), communicating via `apiClients.js` to target `/api/local` or `/api/global` proxies depending on the action.

Key repository conventions

- Domain language is Portuguese (pt-PT) across entities, services, exceptions, and most user-facing labels; preserve existing naming and terminology.
- Treat `ITakiLNLocal` / `ITakiLNGlobal` as application boundaries. New integrations should call these facades/subsystems instead of bypassing rules directly in UI-facing code.
- DAO implementations follow `AbstractDAO<K,V>` + `DAO<K,V>` contracts (`save/findById/findAll/exists/delete/count/clear`). Keep new DAOs consistent with this shape.
- Database connectivity is environment-driven through `ConnectionManager` (`TAKI_DB_URL`, `TAKI_DB_USER`, `TAKI_DB_PASSWORD`).
- Requirements traceability is expected by project docs (`README.md`, `GEMINI.md`): align behavior changes with documented functional/non-functional requirements before implementation.
- Security conventions already present in code:
    - backend sync endpoints must be HTTPS/TLS 1.3,
    - frontend auth store uses role hierarchy (`Proprietário da Cadeia` > `Administrador do Sistema` > `Gerente de Loja` > `Operador de Caixa`) for access gating.

Mandatory requirement traceability (LI4-report.pdf)

- Always map code changes to the report requirements before implementation and before considering a task done.
- Functional coverage baseline:
    - `RF01-RF04`: POS, payments, receipts, returns.
    - `RF05-RF10`: inventory movements, restocking orders, stock visibility, low-stock alerts.
    - `RF11-RF15`: products/categories/suppliers, product-supplier links, pricing/promotions.
    - `RF16-RF18`: sales/inventory reporting and global KPI dashboard.
    - `RF19-RF20`: user lifecycle, profile-based authorization.

- Non-functional constraints that must be explicitly validated in development:
  - `RNF01` : strict RBAC + denied-access logging.
  - `RNF02` : daily local-to-central sync, atomic transfer, retry/alert on failure.
  - `RNF03` : legal/fiscal compliance requirements (including SAF-T and document integrity).
  - `RNF04` : encrypted communications using TLS 1.3+ only.
  - `RNF05` : POS performance target (<10s for common sale flow).
  - `RNF06` : usability and pt-PT clarity in UI flows.
  - `RNF07` : central service availability target (99.5%) and resilience.
  - `RNF08` : secure password storage (strong salted hashing, never plain text).

Anexo 9: Diretrizes de Sistema para o *Claude* (`CLAUDE.md`)

CLAUDE.md

This file provides guidance to Claude Code (claude.ai/code) when working with code in this repository.

Project context

Taki is an academic project (UMinho LI4 / Engenharia de Software) that models a distributed retail-franchise management system. Domain language is **Portuguese (pt-PT)** across entities, services, exceptions, DB tables, and UI labels — preserve the existing naming when extending code.

The implementation is subordinate to the requirements documented in `LI4-report.pdf` (RF01–RF20, RNF01–RNF08). Non-trivial behavior changes should be mapped to the corresponding requirement before being considered done.

Build, test, and run

Backend (Java 21, Gradle wrapper, Javalin REST):

```
cd backend && ./gradlew build
cd backend && ./gradlew test
cd backend && ./gradlew run            # launches NodeApiLauncher honoring APP_MODE
# Single class: ./gradlew test --tests pt.uminho.taki.ln.vendas.VendaServiceTest
# Single method: ./gradlew test --tests "pt.uminho.taki.ln.vendas.VendaServiceTest.metodo"
```

The `test` and `run` Gradle tasks both load the repo-root `.env` into system properties / environment before executing — so `TAKI_DB_URL`, `TAKI_JWT_*`, `APP_MODE`, etc. are picked up automatically when running through Gradle.

Frontend (Vue 3 + Vite + Pinia + Tailwind):

```
cd frontend && npm install
cd frontend && npm run dev             # dev server with proxy to /api/local + /api/global
cd frontend && npm run build
```

Vite reads env from the **repo root** (`envDir: '../'`) and proxies `/api/local` → `TAKI_LOCAL_API_PORT` (default 8080) and `/api/global` → `TAKI_GLOBAL_API_PORT` (default 8081).

Infrastructure:

```
docker compose up -d                   # 1 central DB + 4 local store DBs (ports 5432-5436)
./database/load_data.sh central        # populate central DB
./database/load_data.sh 1               # populate store 1 DB (also: 2, 3, 4)
```

`load_data.sh` invokes `database/populate.py` if the SQL dump under `population_output/sql/` is missing. `population_output/` is gitignored.

There is no lint script configured in either `frontend/package.json` or `backend/build.gradle`.

Architecture

Dual REST API by node role. `NodeApiLauncher` reads the `APP_MODE` env var (resolved by `NodeRoleConfig`) and starts:

- `APP_MODE=LOCAL` (default if unset/blank) → only the **Local API** on port 8080 (POS, local inventory, sync execution). Backed by `pt.uminho.taki.ln.TakiLNLocal` / `ITakiLNLocal`.
- `APP_MODE=CENTRAL` → Local API **and** the **Global API** on port 8081 (chain-wide catalog, suppliers, employees, KPIs/reports). Backed by `pt.uminho.taki.ln.TakiLNGlobal` / `ITakiLNGlobal`. If the Global API fails to start, the launcher tears down the Local API to keep the node consistent.

  Note: `.github/copilot-instructions.md` references an older variable name (`TAKI_NODE_ROLE`) and values (`HUB`/`BRANCH`). The code uses `APP_MODE` with `LOCAL` / `CENTRAL`. Trust the code

(`NodeRoleConfig`, `.env.example`, `docker-compose.yml`).

Layering.

- `pt.uminho.taki.api` — Javalin entry points, controllers, routes, DTOs, and shared infrastructure (`shared/erros` global exception handler, `shared/seguranca` JWT auth + middleware + auth context).
- `pt.uminho.taki.ln` — business/logic layer (`TakiLNLocal`, `TakiLNGlobal`, plus per-domain packages: `vendas`, `inventario`, `fornecimentos`, `lojas`, `fatura`, `estatisticas`, `sincronizacao`, `report`, `view`). UI/API code should call the `ITakiLN*` facades rather than reaching into individual services.
- `pt.uminho.taki.dao` — JDBC DAOs over `ConnectionManager` (env-driven: `TAKI_DB_URL`, `TAKI_DB_USER`, `TAKI_DB_PASSWORD`). All DAOs implement the `DAO<K,V>` contract via `AbstractDAO<K,V>` (`save / findById / findAll / exists / delete / count / clear`); preserve this shape when adding DAOs.

Synchronization (outbox pattern). Triggers in `database/sincronizacao_postgres.sql` enqueue changed entity IDs into `Fila_Sincronizacao`. `OutboxDAO` is read/delete-only (`save` is intentionally `UnsupportedOperationException` — writes happen via DB triggers, not from the app). `SincronizacaoService` ships batches over **HTTPS / TLS 1.3** to the central node; the Global API on the central node coordinates the inbound side.

Frontend. Vue 3 SPA. State per domain in `src/stores/*` (Pinia). HTTP wired through `src/services/apiClients.js` + `httpClient.js`, hitting `/api/local` or `/api/global` depending on the action. Auth store enforces role hierarchy: `Proprietário da Cadeia` > `Administrador do Sistema` > `Gerente de Loja` > `Operador de Caixa`.

Conventions and constraints

- **Sessions/JWT.** Web sessions are kept in `HttpOnly` cookies. JWTs are signed with asymmetric **RS256**; the keypair is loaded from `TAKI_JWT_PUBLIC_KEY` / `TAKI_JWT_PRIVATE_KEY` (Base64-encoded PEM). Never log or commit them.
- **Validation.** REST DTOs use Jakarta Bean Validation (Hibernate Validator). New request DTOs going through Javalin must carry the appropriate JSR-380 annotations — the global handler in `api/shared/erros` relies on this.
- **Passwords.** Stored with jBCrypt (salted hash); never accept or store plaintext (RNF08).
- **Sync transport.** All sync endpoints must be HTTPS / TLS 1.3 (RNF04, RNF02).
- **Tests.** JUnit 5 + Mockito; integration tests under `src/test/java/.../ln/*IntegrationTest.java` use Testcontainers (PostgreSQL) — Docker must be running locally for them.
- **Docker DB topology.** `db_central` (5432) + `db_local_1..4` (5433–5436). The `taki_local_db_<n>` databases are independent stores; pick the right port when connecting from the host.

Anexo 10: Ata de Reunião de Aceitação

ATA DA REUNIÃO

DATA: 07 DE MAIO DE 2026
HORA DE INÍCIO: 10H00

Participantes

- Sr.Manuel Pinto da Costa (Sócio-Gerente da *Taki*)
- Tiago Rocha (Sócio e Gestor de Famalicão)
- Sofia Alves (Sócia e Gestora de Felgueiras)
- Carolina Jesus (Sócia e Gestora da Madeira)
- David Marques (Sócio e Gestor de Cabeceiras de Basto)
- Equipa *SegFault:* Diogo Campos, João Delgado, Nelson Mendes, Simão Mendes, Tomás Machado

Descrição da reunião

- Reunião de acompanhamento e validação com o propósito de avaliar o progresso do desenvolvimento do projeto até ao momento e garantir a aprovação, por parte do cliente, da arquitetura de *software* sugerida para a infraestrutura da *Taki*.

Itens de Ação

- **1. Avaliação do Progresso e Arquitetura:**
  - A equipa da *SegFault* apresentou o estado atual do projeto e detalhou o desenho da arquitetura planeada para o sistema. O Sr. Manuel Pinto da Costa e os restantes stakeholders fizeram uma avaliação muito positiva do trabalho desenvolvido, confirmando que a solução estrutural vai de encontro aos objetivos de negócio.
- **2. Dificuldade de Interpretação:**
  - Durante a análise da documentação, os *stakeholders* apontaram um único ponto de fricção: demonstraram não estar a compreender totalmente a representação visual e o fluxo de informação exposto nos diagramas de classes e nos diagramas de sequência, por serem artefactos demasiado técnicos.
- **3. Resolução e Adaptação:**
  - Para colmatar esta falha de comunicação, a equipa de desenvolvimento decidiu por unanimidade que irá desenvolver e aprofundar a componente textual descritiva que antecede os diagramas. Esta documentação adicional servirá para "traduzir" a notação técnica, assegurando uma compreensão correta e completa da arquitetura por parte de todos os intervenientes não-técnicos.
- **4. Alinhamento de Comunicação:**
  - No seguimento da dificuldade sentida com a leitura dos diagramas, a equipa e os stakeholders aproveitaram para afinar a forma como o estado do projeto será reportado futuramente. Ficou estabelecido que as próximas atualizações adotarão um formato de relatório mais focado no negócio, acompanhadas de descrições simplificadas. Foi também validado o canal de comunicação direto entre a equipa *SegFault* e a gerência da *Taki,* garantindo um acompanhamento fluido e contínuo do projeto sem engessar o processo de desenvolvimento.

Comunicados Finais

- O cliente expressou satisfação com a capacidade de adaptação da equipa e deu parecer favorável para o avanço da implementação da arquitetura.
- A SegFault comprometeu-se a entregar a documentação reforçada nos próximos dias.
- A reunião foi encerrada às 11h30.

Anexo 11: Manual instalação

1. Pré-requisitos

Instalar os seguintes componentes em **todas as máquinas** onde o sistema será executado:

Componente	Versão mínima	Verificar com
Docker Engine	24.x	`docker --version`
Docker Compose	2.x (plugin)	`docker compose version`
Git	2.x	`git --version`

> **Nota:** JDK 21 e Node.js **não são necessários** nas máquinas — o Docker trata de tudo. São apenas necessários se quiser correr o projeto fora de containers.

2. Obter o Código

Em **cada máquina** (servidor central e cada servidor de loja):

```
git clone https://github.com/SirLordNelson/LI4_2526.git
cd LI4_2526
```

3. Gerar as Chaves RSA (JWT)

As chaves RSA são usadas para assinar e verificar os tokens JWT. Devem ser **geradas uma vez** e partilhadas por todas as máquinas.

```
# Gerar par de chaves RSA 2048-bit
openssl genrsa -out private_key.pem 2048
openssl rsa -in private_key.pem -pubout -out public_key.pem

# Converter para Base64 (formato aceite pelo sistema)
base64 -w 0 public_key.pem  > public_key.b64
base64 -w 0 private_key.pem > private_key.b64

# Mostrar os conteúdos — serão usados no .env
cat public_key.b64
cat private_key.b64
```

> ⚠️ **Importante:** O mesmo par de chaves deve ser usado no servidor central **e** em todos os servidores de loja para que os JWTs sejam mutuamente reconhecidos.

4. Configurar o Ficheiro `.env`

Copiar o template e preencher:

```
cp .env.example .env
```

Editar `.env` com os valores corretos. Os campos obrigatórios são:

```
TAKI_JWT_PUBLIC_KEY=<conteúdo de public_key.b64>
TAKI_JWT_PRIVATE_KEY=<conteúdo de private_key.b64>
```

5. Cenários de Instalação

O sistema suporta três modos de arranque, cada um com o seu ficheiro `docker-compose`:

Ficheiro	Modo	Quando usar
`docker-compose.yml`	Loja (BRANCH)	Servidor de cada loja — API local na porta 8080
`docker-compose-central.yml`	Central (CENTRAL)	Servidor da sede — API local + global
`docker-compose-dev.yml`	Desenvolvimento	Ambiente local completo numa só máquina

6. Instalar o Servidor Central (Sede)

Executar na **máquina da sede** (IP: `192.168.1.82`).

6.1. Arrancar a infraestrutura

```
docker compose -f docker-compose-central.yml down -v --remove-orphans
docker compose -f docker-compose-central.yml up -d --build
```

O comando levanta os seguintes serviços:

- `li4_db_central` — PostgreSQL na porta `5432`
- `li4_backend_central` — Javalin nas portas `8080` (API Local) e `8081` (API Global)
- `li4_frontend_central` — Vite na porta `5173`

6.2. Popular a base de dados central

```
# Aguardar a BD inicializar (~5 segundos)
sleep 5
bash ./database/load_data.sh central
```

6.3. Configurar CORS e IP da Sede

O IP da máquina da sede é registado no ficheiro `lan_ip.txt` na raiz do repositório (valor atual: `192.168.1.82`). A variável `TAKI_CORS_ORIGIN` no `docker-compose-central.yml` controla quais origens o backend aceita:

```
# Opção A — restrito ao IP da sede (recomendado)
- TAKI_CORS_ORIGIN=http://192.168.1.82:5173

# Opção B — sem restrições (só para demonstração)
- TAKI_CORS_ORIGIN=*
```

6.4. Abrir portas na firewall (Linux)

```
sudo ufw allow 5173/tcp   # Frontend
sudo ufw allow 8080/tcp   # API Local
sudo ufw allow 8081/tcp   # API Global
sudo ufw status
```

6.5. Verificar

```
# Health check da API Global (sede)
curl http://localhost:8081/health
# Esperado: 200 OK

# Login do gestor central
curl -c cookies.txt -s -X POST http://localhost:8081/api/global/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email":"dono@taki.pt","password":"admin"}' | python3 -m json.tool
```

6.6. Acesso de outros dispositivos na rede

Com o servidor central a correr em `192.168.1.82`, qualquer dispositivo na mesma rede acede em:

Serviço	URL
Frontend (sede)	`http://192.168.1.82:5173`
API Local	`http://192.168.1.82:8080`
API Global	`http://192.168.1.82:8081`

> O IP pode ser consultado a qualquer momento com `cat lan_ip.txt` ou `hostname -I | awk '{print $1}'`

7. Instalar um Servidor de Loja

Executar na **máquina de cada loja**. Repetir para cada loja, alterando o número N.

7.1. Configurar CORS e sincronização com a sede

No ficheiro `docker-compose.yml`, dois valores precisam de ser ajustados:

CORS — deve apontar para o IP do frontend desta loja:

```
environment:
  - TAKI_CORS_ORIGIN=http://<IP_DO_FRONTEND_LOJA>:5173
```

Sincronização com a sede — adicionar ao `.env` da loja o endereço da sede:

```
URLS_SEDE_CSV=http://192.168.1.82:8081/api/v1
```

> Para múltiplos endpoints de redundância, separar por vírgula:
> `URLS_SEDE_CSV=http://192.168.1.82:8081/api/v1,http://backup.taki.pt/api/v1`

7.2. Arrancar a infraestrutura da loja

```
docker compose -f docker-compose.yml down -v --remove-orphans
docker compose -f docker-compose.yml up -d --build
```

O comando levanta:

- `li4_db_local_1` — PostgreSQL na porta `5433`
- `li4_backend` — Javalin na porta `8080` (API Local) e `8081` (API Global)
- `li4_frontend` — Vite na porta `5173` (acessível de qualquer máquina da rede)

7.3. Popular a base de dados da loja

```
sleep 5
bash ./database/load_data.sh 1   # substituir 1 pelo número da loja (1–4)
```

7.4. Verificar

```
# Health check da API Local (loja)
curl http://localhost:8080/health
# Esperado: 200 OK

# Login do gerente da loja 1
curl -c cookies.txt -s -X POST http://localhost:8080/api/local/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email":"tiago.rocha@taki.pt","password":"admin"}' | python3 -m json.tool
```

8. Aceder ao Frontend de uma Loja a partir de outro Terminal

Como o Vite arranca com `--host 0.0.0.0`, qualquer dispositivo na mesma rede pode aceder à interface:

```
http://<IP_DO_SERVIDOR_LOJA>:5173
```

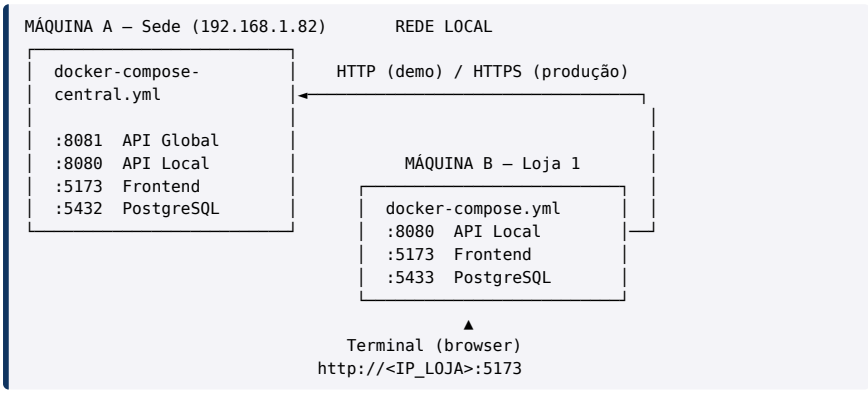
Para descobrir o IP do servidor de loja:

```
hostname -I | awk '{print $1}'
```

9. Credenciais de Demonstração

Utilizador	Email	Password	Papel	Loja
Gestor Central	`dono@taki.pt`	`admin`	GESTOR_CENTRAL	Central
Gerente Loja 1	`tiago.rocha@taki.pt`	`admin`	GESTOR_LOJA	1
Operador Loja 1	`operador1@taki.pt`	`admin`	OPERADOR_CAIXA	1
Gerente Loja 2	`sofia.alves@taki.pt`	`admin`	GESTOR_LOJA	2
Operador Loja 2	`operador2@taki.pt`	`admin`	OPERADOR_CAIXA	2
Gerente Loja 3	`carolina.jesus@taki.pt`	`admin`	GESTOR_LOJA	3
Operador Loja 3	`operador3@taki.pt`	`admin`	OPERADOR_CAIXA	3
Gerente Loja 4	`david.marques@taki.pt`	`admin`	GESTOR_LOJA	4
Operador Loja 4	`operador4@taki.pt`	`admin`	OPERADOR_CAIXA	4

10. Diagrama de Infraestrutura



11. Resolução de Problemas

Sintoma	Causa provável	Solução
`CORS error` no browser	`TAKI_CORS_ORIGIN` errado	Atualizar o IP do frontend no `docker-compose.yml` e reiniciar

Sintoma	Causa provável	Solução
Backend não arranca	Chaves JWT em falta	Verificar `TAKI_JWT_PUBLIC_KEY` e `TAKI_JWT_PRIVATE_KEY` no `.env`
BD não inicializa	Porta já em uso	`docker compose down -v` e verificar `ss -tlnp | grep 5432`
Frontend não acede ao backend	IP hardcoded	Verificar `VITE_LOCAL_API_TARGET` — deve usar `http://backend:8080` dentro do Docker
`401 Unauthorized`	Cookie não enviado	Garantir `withCredentials: true` no proxy Vite
Sincronização falha	Endpoint central inacessível	Verificar `URLS_SEDE_CSV` no `.env` e conectividade de rede